RECONCILED

Enterprise Architecture for Financial Systems

Željko Obrenović

Reconciled: Enterprise Architecture for Financial Systems

Željko Obrenović

This book is available at https://leanpub.com/reconciled

This version was published on 2026-10-19

This is a Leanpub book. Leanpub empowers authors and publishers with the Lean Publishing process. Lean Publishing is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Željko Obrenović

Contents

RECONCILED: Enterprise Architecture for Financial Systems **1**

2. Unreconciled: Fourteen Ways Finance Architecture Goes Wrong **2**

Fourteen Patterns to Recognize 3

How the Patterns Reinforce Each Other 18

Using the Patterns 21

Sources and Evidence 21

3. Meet Northmere: Five Brands, One Set of Books **22**

The Group at a Glance 23

Brands and Business Models 24

Legal Entities and the Licensed Payments Entity 27

The Systems Landscape 27

Volumes 31

The People 32

Seven Problems Northmere Starts 2027 With 34

Program Keystone 35

The Recurring Worked Examples 37

How the Chapters Use Northmere 40

Sources and Evidence 43

FOUNDATIONS: How Money Moves Through an Enterprise **44**

The Learning Path 45

4. The Chief Architect's Job: Making Many Systems Tell One Story **48**

Setting the Scene: Dana's First Month at Northmere 50

What Enterprise Architecture Is For 51

Why Finance Architecture Is Its Own Discipline 53

Architecture as a Function and a Service 54

Operating Models: Centralized, Federated and Advisory 58

Decision Rights: Mandatory Standards and Advice 61

The Matrix with Brand Finance Engineering 68

Riding the Architect Elevator . . . 69
Dana's Answers to the Three Questions . . . 72
Measuring the Architecture Function . . . 74
Integration with the Rest of the Estate and the Book . . . 77
Failure Modes . . . 77
The Chief Architect's View . . . 78
Dana's First 90 Days . . . 78
Questions to Ask . . . 82
From Accountability to a Map . . . 82
Sources and Evidence . . . 82

5. Map What Finance Does Before What Runs It . . . 84
Setting the Scene at Northmere: The Question the Portfolio Could Not Answer . . . 86
Concepts: Capability, Process, Value Stream, Application, Organization Unit . . . 87
Architecture: A Three-Level Finance Capability Map . . . 91
In Practice: Heat-Mapping Northmere . . . 101
Architecture: From Capabilities to Bounded Contexts . . . 107
How Capability Maps and Bounded Contexts Complement Each Other . . . 114
Capabilities to Applications to Teams: Finding the Ownership Gaps . . . 115
Trade-offs: How Much Map, and Which Kind . . . 118
Running a Capability Mapping Exercise with Finance . . . 120
Integration with the Rest of the Estate . . . 122
Failure Modes . . . 123
The Chief Architect's View . . . 124
Questions to Ask . . . 125
Sources and Evidence . . . 125

6. Manage the Application Portfolio as an Investment . . . 127
Setting the Scene at Northmere: 214 Applications and One Question . . . 129
Concepts: What a Portfolio Decision Is Made Of . . . 130
Architecture: The Portfolio as a Decision Pipeline . . . 133
In Practice: Building a Fact Base You Can Reconcile . . . 134
In Practice: Scoring Fitness Without Theater . . . 137
Pace Layers in a Finance Estate . . . 141
The Full Cost of a Finance Application . . . 144
Technology ROI as Practiced, and Its Limits . . . 149
Contract and License Dates as Architecture Inputs . . . 150
Rationalization Waves . . . 151
Sunset Discipline: Decommissioning That Counts . . . 156
Trade-offs: How to Run the Portfolio . . . 159
Integration With the Rest of the Estate . . . 160
Failure Modes . . . 163
The Chief Architect's View . . . 164

Questions to Ask . . . 166
Sources and Evidence . . . 167

7. Anatomy of a Finance Estate: Layers, Planes and Boundaries . . . 169
Setting the Scene at Northmere: Four Teams, One Job . . . 171
What a Reference Model Is, and What It Is Not . . . 172
The Volume Gradient . . . 173
The Reference Model: Eight Layers and Two Planes . . . 177
Where Vendors Sit . . . 186
In Practice: Northmere Mapped onto the Model . . . 187
Alternative Decompositions . . . 190
Integration With the Rest of the Estate . . . 192
Failure Modes . . . 193
The Chief Architect's View . . . 194
Questions to Ask . . . 195
From Layers to Ledgers . . . 196
Sources and Evidence . . . 196

8. Accounting for Architects: Ledgers, Journals and Revenue . . . 199
Setting the Scene at Northmere: The Event That Could Not Be Booked . . . 201
Double Entry: The Invariant Behind Every Number . . . 201
Anatomy of a Journal Entry . . . 203
The Chart of Accounts and Its Dimensions . . . 204
Operational Records, Subledgers and the General Ledger . . . 206
Accrual Accounting and the Period . . . 208
Revenue Recognition: Five Steps, Translated Into System Requirements . . . 209
Principal or Agent: The Judgment That Decides What You Capture . . . 211
Three Kinds of Money That Are Not Revenue . . . 213
The Lisbon Booking, Entry by Entry . . . 215
The Same Stay Under the Agency Model . . . 220
When the Money Goes Back: A Refund and the €420 Chargeback . . . 222
Tablekin: One Invoice, Two Revenue Treatments . . . 225
Currency: Revaluation Is Not Translation . . . 228
Intercompany: The Booking Across Two Legal Entities . . . 230
Accounting Invariants Your Systems Must Preserve . . . 232
Where the Accounting Treatment Should Be Applied . . . 234
Integration With the Rest of the Estate . . . 236
Failure Modes . . . 236
The Chief Architect's View . . . 237
Questions to Ask . . . 238
Sources and Evidence . . . 239

9. Order to Cash, Procure to Pay, Record to Report . . . 241
Setting the Scene: Two Documents Dana Asked to Follow . . . 243

Processes, Capabilities and Hand-offs . . . 243
The Process Map of a Finance Estate . . . 246
Record-to-Report: Where Every Process Lands . . . 247
Procure-to-Pay: The Supplier Invoice Across Four Systems . . . 248
Order-to-Cash and Its Marketplace Variants . . . 252
Partner Payouts . . . 257
Treasury: Cash, FX and Bank . . . 258
Tax: Determine, Report, Pay . . . 259
Plan-to-Perform . . . 260
Acquire-to-Retire . . . 261
Hire-to-Retire: The Finance Touchpoints . . . 262
The Hand-off Matrix . . . 263
Both Documents, Across Every Process . . . 265
Trade-offs: Where Process Logic Lives and Who Owns It . . . 267
Integration With the Rest of the Estate . . . 268
Failure Modes . . . 268
The Chief Architect's View . . . 269
Questions to Ask . . . 269
Sources and Evidence . . . 270

10. Finance Data Architecture: Who Owns the Truth . . . 272
Setting the Scene at Northmere: The Usage Revenue That Went Missing . . . 274
Four Kinds of Finance Data . . . 275
The Financial Master Data Domains . . . 276
Management Styles: Registry, Consolidation, Coexistence, Centralized . . . 279
The Architecture of Financial Master Data . . . 282
Designing the Group Chart of Accounts . . . 284
In Practice: The Mapping Table as Master Data . . . 288
Governing Change to the Chart of Accounts . . . 291
The Partner Master: Coexistence in Practice . . . 294
Ownership and Stewardship . . . 296
Data Quality Rules . . . 297
Lineage and Data Contracts . . . 300
Retention and Privacy . . . 302
Trade-offs: Where to Put the Authority . . . 303
Integration with the Rest of the Estate . . . 304
Failure Modes . . . 304
The Chief Architect's View . . . 305
Questions to Ask . . . 306
Sources and Evidence . . . 306

11. Controls by Design, Not by Audit Season . . . 308
Setting the Scene: A Significant Deficiency on the Payout Engine . . . 309

Internal Control over Financial Reporting in One Page . . . 310
A Control Taxonomy for Architects . . . 312
Key Reports and Information Produced by the Entity . . . 316
Segregation of Duties and Sensitive Access . . . 316
Remediation as Architecture: Fixing the Payout Engine . . . 318
SaaS Platforms, SOC 1 Reports and the Controls They Hand Back . . . 324
Audit Trail Requirements for Custom Ledgers . . . 329
Evidence by Design . . . 330
Continuous Control Monitoring . . . 332
Working with Internal Controls and the External Auditor Early . . . 333
The Security Baseline for Finance Systems . . . 333
Trade-offs: Four Ways to Remediate a Control Finding . . . 336
Integration with the Rest of the Estate . . . 338
Failure Modes . . . 338
The Chief Architect's View . . . 339
Questions to Ask . . . 340
Sources and Evidence . . . 341

12. Files, Events and APIs: How Finance Systems Talk . . . 343
Setting the Scene: The Night January Was Posted One and a Half Times . . . 345
Five Integration Styles and What Each Is Good For in Finance . . . 346
Ten Requirements for an Interface That Carries Money . . . 350
Exactly-Once Is an Effect, Not a Delivery Guarantee . . . 356
Integration Platforms: iPaaS, ESB, Event Backbone or Code . . . 359
Vendor Integration Surfaces at a Glance . . . 362
Canonical Financial Events, Introduced . . . 365
Worked Example: Redesigning the SFP-to-S/4HANA Daily Posting . . . 366
Integration Governance: Catalog, Contracts, Versions and Retirement . . . 370
Failure Modes . . . 371
The Chief Architect's View . . . 372
Questions to Ask . . . 373
Next: Many Entities, Currencies and Jurisdictions . . . 374
Sources and Evidence . . . 374

13. Global by Design: Entities, Currencies, Jurisdictions and Brands . . . 376
Setting the Scene: Seventy-One Entities and Three Requests . . . 378
Five Dimensions That Must Not Collapse Into One . . . 378
The Legal Entity Model . . . 380
How Each ERP Represents a Legal Entity . . . 382
Intercompany by Design . . . 384
Worked Example: A Royalty Crosses Two Currencies . . . 386
The Multi-Currency Model . . . 390
Many GAAPs, One Set of Events . . . 393

Local Requirements That Reach Into the Ledger . . . 394
Indirect Tax at Platform Scale . . . 395
Pillar Two: The Group's Data Model Becomes a Tax Return . . . 398
Licensed Entities: Where the Group Cannot Simply Move Money . . . 399
Opening Brazil: A Preview of the Launch Checklist . . . 400
Trade-Offs . . . 402
Integration With the Rest of the Estate . . . 403
Failure Modes . . . 403
The Chief Architect's View . . . 404
Questions to Ask . . . 405
Sources and Evidence . . . 406

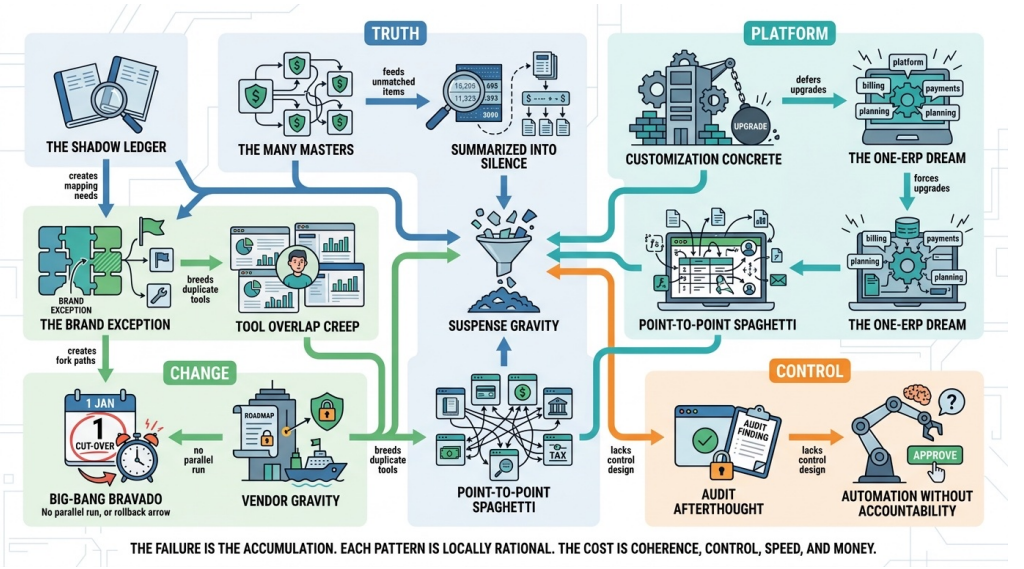
14. Build, Buy or Compose: Choosing What to Own . . . 408
Setting the Scene: Three Sourcing Decisions at Northmere . . . 410
The Sourcing Spectrum and When Each Choice Fits . . . 411
The Three Tests . . . 413
Cost of Ownership Over Seven to Ten Years . . . 418
Applying the Tests to Tablekin . . . 421
Two Earlier Decisions, Read Through the Method . . . 423
Fit-to-Standard and What It Asks of Finance . . . 424
A Selection Process That Finds the Truth . . . 426
A Weighted Scoring Model and Its Pitfalls . . . 430
Rationalization After M&A: Converge, Keep or Wrap . . . 431
Integration With the Rest of the Estate . . . 434
Failure Modes . . . 435
The Chief Architect's View . . . 435
Questions to Ask . . . 436
From Choosing to Changing . . . 437
Sources and Evidence . . . 437

RECONCILED: Enterprise Architecture for Financial Systems

Željko Obrenović

How a Chief Architect designs, integrates, governs and evolves the finance technology of a multinational enterprise — ERP, payments, close, consolidation and the custom platforms between them — as one coherent system. This is a living journal and a work in progress: I revise it as I learn more through my work.

2. Unreconciled: Fourteen Ways Finance Architecture Goes Wrong



IN THIS SECTION, YOU WILL: Learn fourteen named failure patterns that recur in enterprise finance estates, recognize their warning signs, understand the incentives that produce them and what they cost in control, money, speed and audit risk, and find the chapters whose designs prevent or repair each one.

KEY POINTS:

- **Each pattern is locally rational.** A brand team that keeps its own ledger, a controller who builds a mapping spreadsheet, a program that customizes the ERP to hit a go-live date: each made a sensible choice under its own constraints. The failure is the accumulation, and only someone accountable for the whole estate sees it.
- **The cost shows up in the close, the suspense account and the audit, rarely in the project budget.** These patterns are paid for in days to close, unreconciled balances, upgrade lead times and control findings, long after the project that created them was declared a success.

- **The patterns reinforce each other**. The Doppelgänger Record creates Spreadsheet Glue, which feeds The Suspense Swamp; The Permanent Exception breeds Capability Creep and Point-to-Point Spaghetti. Treating one symptom at a time rarely works; breaking the chain at its origin usually does.

A finance estate is the set of systems, data, integrations and controls through which an enterprise records what it owns, owes, earns and spends, and proves it. In a multinational, multi-brand group it includes the general ledger (the official book of account), subledgers (detailed records of one area, such as partner payables), payment platforms, close and consolidation tools, tax and treasury systems, and the custom platforms that brands build because no package fits their business model.

Such an estate rarely fails dramatically. What fails is **coherence**: the estate slowly loses the ability to show that its records agree with each other and with the money. Unmatched items accumulate. Mappings drift. Every upgrade becomes a project. The close takes nine days because people spend six of them reconciling things the architecture should have kept consistent.

This guide names the **fourteen failure patterns** I see most often. The names are shorthand for observable behavior, so a steering committee or design review can say "this is The Suspense Swamp" and move straight to the mechanism and the remedy. Each pattern is described the same way: a typical quote, what it looks like, why it happens, what it costs, a short illustration, the direction of the remedy, and the chapters that address it. The illustrations use **Northmere, the fictional group this book follows** (see Meet Northmere: Five Brands, One Set of Books); they are composites, not accounts of any real company.

None of these patterns implies incompetence. **Most are the residue of reasonable decisions** taken under deadline, by teams accountable for one part of the estate. That is exactly why they need a name and an owner.

Fourteen Patterns to Recognize

The patterns fall into four loose families: **truth** (the first five, which damage the ability to know which record is right), **platform** (the next four, which misuse the systems), **change** (how the estate grows and moves) and **control** (evidence and accountability).

Pattern	The warning sign
The Shadow Ledger	A brand or product system keeps its own books that nobody reconciles to the general ledger.
The Suspense Swamp	Unmatched items accumulate in clearing and suspense accounts faster than anyone clears them.
The Doppelgänger Record	The same entity — a legal entity, a partner, an account — is created and changed in several systems with no agreed authority.
Lost in Aggregation	Detail is aggregated before it reaches the ledger, and nobody can trace a balance back to its transactions.
Spreadsheet Glue	Mappings, controls and hand-offs that the close depends on live in spreadsheets on someone's laptop.
Customization Concrete	Custom code inside a vendor platform makes every upgrade a project and every release a risk.
The Swiss-Army ERP	One platform is expected to do everything, including what it was never designed to do at the required volume.
Point-to-Point Spaghetti	Every system talks directly to every other, in its own format, with its own error handling.
Capability Creep	Several platforms quietly acquire the same capability, and nobody decides which one owns it.
The Permanent Exception	Every brand, market or acquisition claims to be special, and each exception becomes a fork that never ends.
Big-Bang Bravado	Everything switches over at once, usually at year-end, with no way back.
The Borrowed Roadmap	The vendor's roadmap, packaging and sales cycle set the enterprise's architecture.
Audit Archaeology	Controls and evidence are retrofitted after the system is built, usually after an audit finding.
Nobody's Bot	Bots, scripts or AI agents post, approve or change financial data without an owner, a control or an evidence trail.

Every chapter in Parts I to IV opens with a callout naming the patterns it addresses, linked back here.

The Shadow Ledger

"Don't worry about the balances, our platform tracks all of that. Finance only needs the monthly totals."

What it looks like. A brand platform, a payout engine or a billing service holds balances that matter financially: amounts owed to partners, credits granted to customers, prepaid funds, accrued fees. It computes them, reports them on its own dashboards, and acts on them by paying out or netting. The general ledger receives a monthly figure, typed in or loaded from a file, and nobody proves that the platform's balances and the ledger's balances agree. Observable signs: finance cannot reproduce what the platform says a partner is owed; month-end adjustments are booked "to align with the platform".

Why it happens. Product engineering builds what the product needs. A payout engine needs a running balance per partner to decide what to pay, so it gets one; that balance is, functionally, a subledger, but nobody calls it one, so nobody designs it to the standards of one (double entry, immutability, period locks, reconciliation to the general ledger). Acquisitions add more: the acquired platform keeps working and reports its balances upward as totals.

What it costs. **Control**: an unreconciled balance is uncontrolled, so the reported liability rests on code nobody audits. **Cost**: every close investigates differences by hand. **Speed**: payout-rule changes become slow and fearful. **Audit risk**: a material balance computed outside the control environment pulls audit testing into the platform, usually at the worst moment.

An illustration. At Northmere, Wayfare runs its own payout engine for ground-transport suppliers. It tracks what each supplier is owed, nets cancellations and pays monthly. Oracle Fusion, Wayfare's ERP, receives one payables journal per supplier batch. In 2027 Hana Kowalski's team finds that the engine's open supplier balances exceed Fusion's accrued supplier liability by about **€1.3 million**, a figure nobody can explain without opening the engine's database, because cancellations after payout are netted in the engine but never posted as entries.

The remedy direction. Recognize any system that holds balances as a subledger, give it ledger discipline (entries, not mutable balances), and reconcile it to the general ledger at a defined frequency with an owner. Then decide whether it should be a subledger at all.

Where this book helps: When the ERP Isn't Enough: Custom Finance Platforms sets the boundary rules for what brand platforms may own; Never Update, Only Append: Designing Custom Ledgers shows how to build a subledger that behaves like one; Systems of Record: Ending the Doppelgänger Record decides who is authoritative for each balance; Reconciled by Design: Traceability and Audit Across Boundaries designs the subledger-to-GL reconciliation; Uber's Payments Platform: A Ledger at the Center of Money Movement shows a real payments ledger and where it must hand off to the book of record.

The Suspense Swamp

"It's just timing. Those items will clear next month."

What it looks like. A **clearing account** is a temporary account where money waits between two events, for example between a payment provider confirming a payment and the money arriving at the bank. A **suspense account** holds items that cannot yet be classified or matched. Both are legitimate and should net to near zero over a short, predictable period. The Suspense Swamp is the state where they do not: the balance grows, the aging profile lengthens, and the population becomes a mixture of genuine timing differences, unrecorded fees, missed refunds, duplicated postings and real losses that nobody can tell apart. Observable signs: the share of items older than 90 days rises month after month; the reconciliation of the account is "explained" by a list rather than a match; periodic write-offs are booked as a policy.

Why it happens. Suspense is where integration gaps go to hide. A new payment method launches before its settlement file can be matched automatically. A refund is processed in the payments platform but reaches the subledger late. A provider changes a report format. Each gap puts a small number of items into suspense; each item is individually immaterial, so nobody prioritizes it. Launching the payment method is somebody's target; clearing suspense is somebody else's chore.

What it costs. **Control**: the enterprise cannot assert that cash and liabilities are complete and accurate. **Cost**: old items cost far more to clear than new ones, because the context is gone, and some become write-offs. **Speed**: the close waits for the suspense review. **Audit risk**: aged suspense is a classic source of audit adjustments.

An illustration. At Northmere in 2026, on working day 3 of each close, about **1.9%** of payment-provider settlement lines for Stayloft's alternative payment methods (wallets, local bank transfers) are unmatched. Unreconciled balances in clearing and suspense accounts average **€14 million** at month-end, of which about **€2.1 million** is older than 90 days. Each new local payment method added a small, unmatched stream; together they made the largest open item on Leila Haddad's close agenda.

Figure 1: *The Suspense Swamp as sediment: small unmatched streams settle faster than they are cleared, and the oldest layer, €2.1 million over 90 days of Northmere's €14 million average balance, is the hardest to explain.*

The remedy direction. Treat suspense as a measured output of the architecture, not an accounting chore: matching rules designed before a payment method launches, settlement files treated as contracts with completeness checks, aging limits with owners, and the suspense balance reported as an engineering metric.

Where this book helps: Payments-to-Ledger: Draining the Suspense Swamp designs clearing accounts and three-way matching; BlackLine: Matching, Certifying and Closing covers high-volume transaction matching; Files, Events and APIs: How Finance Systems Talk covers completeness controls and error handling; Reconciled by Design: Traceability and Audit Across Boundaries provides the reconciliation catalog.

The Doppelgänger Record

"We created the new entity in SAP. Can someone set it up in the other systems?"

What it looks like. **Master data** is the shared reference data that transactions point to: legal entities, accounts, cost centers, customers, suppliers, partners, bank accounts. The Doppelgänger Record is the state where the same entity is created and maintained independently in several systems, each with slightly different attributes, and no system or team is agreed to be the authority. Observable signs: the same partner has different bank details in two systems; a legal entity exists in the ERP but not yet in the tax engine; the auditor's entity list takes a week to assemble.

Why it happens. Every platform ships with its own master data model and its own create screen, and the path of least resistance is to use it. Projects scope their own

system, not the authority question. Mergers bring complete parallel sets of master data. Master data management delivers no visible feature, so it is deferred.

What it costs. **Control**: approval over bank-detail changes means little if the detail can be changed in another system. **Cost**: every mapping table must be maintained and reconciled. **Speed**: opening an entity or onboarding a partner needs coordinated manual setup. **Audit risk**: reports whose population depends on unsynchronized masters are hard to rely on.

An illustration. Northmere has **71 legal entities in 38 countries**. Each must exist in SAP S/4HANA (or Oracle Fusion or Workday Financials), in OneStream, in Vertex, in Kyriba, in Workday HCM and in the payments platform, with consistent codes, currencies and ownership. When a dormant Asian entity was renamed in 2026, the change reached four systems within a month and the fifth only when the treasury team noticed a bank account mapped to the old name.

The remedy direction. Decide, per master data domain, which system is the authority, which systems subscribe, and who is the steward; then enforce it through integration (subscribers cannot create, only consume) rather than through policy alone.

Where this book helps: Finance Data Architecture: Who Owns the Truth covers master data domains, management styles and stewardship; Systems of Record: Ending the Doppelgänger Record builds the authority matrix; Global by Design: Entities, Currencies, Jurisdictions and Brands covers the legal-entity model; The Next Brand Should Be Boring: Onboarding as Configuration turns entity onboarding into configuration.

Lost in Aggregation

"The ledger can't handle that volume, so we post daily totals. The detail is in the platform somewhere."

What it looks like. High-volume businesses must summarize: putting every booking, click or payment into the general ledger would make it slow and unreadable. Summarization becomes a failure when the summary loses its link to the detail. Observable signs: a general-ledger line cannot be traced to the subledger lines that make it up; the summarization key changed and nobody kept the old one; the detail is retained for less time than the audit and statutory retention period; answering "which transactions make up this revenue figure for this property in March?" requires an engineer and a week.

Why it happens. Summarization is designed for performance by people who are not asked about investigation. The summarization key (which attributes are kept: entity, account, currency, date, business model) is set once, early, and then becomes invisible. Retention of detail is trimmed as a storage cost. Drill-back, going from a ledger line to its sources, is a requirement nobody states until an auditor asks.

What it costs. **Control**: a balance that cannot be traced can only be asserted, not demonstrated. **Cost**: every investigation becomes a data-engineering project. **Speed**:

errors in summaries take days to locate. **Audit risk**: auditors who sample transactions need a reliable path from the reported figure to the population; without it they expand testing.

An illustration. The Stayloft Finance Platform, Northmere's custom brand finance platform, posts about **38,000** general-ledger lines a day into SAP S/4HANA, summarized from about **3 million** subledger lines. In 2027 the external auditor asks Northmere to trace a March revenue balance for one Portuguese entity back to bookings. The summary lines carry a batch identifier, but for periods before 2025 the mapping from batch to subledger lines was kept only in a log store with a 13-month retention. The trace for those periods has to be rebuilt from raw events.

The remedy direction. Design summarization and drill-back together: a stable summarization key, a persisted link from every summary line to its constituent detail, retention aligned to audit and statutory requirements, and a tested drill-back path.

Where this book helps: Never Update, Only Append: Designing Custom Ledgers covers summarization and drill-back; Anatomy of a Finance Estate: Layers, Planes and Boundaries explains the volume gradient that forces summarization; The Life of a Booking: From €600 Payment to $97.38 of Revenue traces one transaction through every hop; Reconciled by Design: Traceability and Audit Across Boundaries designs the identifiers and evidence.

Spreadsheet Glue

"Only one person in group reporting knows how the mapping file works, and they are on leave during close."

What it looks like. **End-user computing** (EUC) is any spreadsheet, desktop database or personal script that performs a step a business process depends on. Spreadsheet Glue is the state where the close relies on EUCs for mappings, allocations, reconciliations, journal preparation or interface hand-offs. Observable signs: a journal is uploaded from a spreadsheet each month; account mappings between ledgers live in a workbook; a "macro" transforms a provider report before it is loaded; nobody can say how many such files exist or which are key to financial reporting.

Why it happens. Spreadsheets are the most flexible integration tool ever built, and controllers are skilled at them. When a new brand is acquired, the mapping of its chart of accounts to the group chart is needed by the first quarter close, long before a system could be configured; a workbook solves it in a week. The workbook then becomes permanent because it works.

What it costs. **Control**: reporting EUCs need versioning, access and logic checks; doing that well is expensive and doing it badly is a common deficiency. **Cost**: manual preparation every period. **Speed**: every change needs the one person who understands the file. **Audit risk**: auditors test spreadsheets used in controls as **information produced by the entity**, and uncontrolled ones invite findings.

An illustration. Northmere runs three charts of accounts (SAP group, Oracle for Wayfare, Workday for Tablekin) and maps them to the group structure only in OneStream, using mappings maintained in spreadsheets. Its 2027 interface inventory finds about **70** undocumented spreadsheets-in-the-loop among some 1,100 finance interfaces. When Tablekin added a new revenue account for usage fees, the mapping workbook was not updated, and a catch-all mapping rule put about **$41 million** of first-quarter usage revenue in "other revenue" at group level until the review caught it on working day 7.

The remedy direction. Inventory EUCs, rate them by reporting risk, control the ones that must stay, and replace the key ones with governed configuration in the system that owns the mapping or hand-off.

Where this book helps: Controls by Design, Not by Audit Season covers EUC and report controls; Finance Data Architecture: Who Owns the Truth covers governed mappings and the group chart of accounts; OneStream: Consolidating Actuals, Planning the Future covers consolidation mapping; Technical Debt on the Balance Sheet puts EUCs on the debt register.

Customization Concrete

"The standard doesn't fit how we work here, so we built a small enhancement."

What it looks like. Vendor platforms allow extension, and some extension is necessary. Customization Concrete is the state where custom code inside the vendor platform (modified standard objects, custom tables, user exits, bespoke reports) has grown until it governs the upgrade schedule. Observable signs: each upgrade needs months of regression testing; the custom object count is known only roughly; release notes are read for what they will break.

Why it happens. Each customization is approved in isolation, against a go-live date, by a project that will not run the system afterward. Clean-core disciplines (extend through published interfaces, outside the core) are cheaper over the system's life but more expensive in the project, and projects are measured on the project.

What it costs. Control: errors in custom posting logic hit the book of record directly. **Cost:** regression testing at every upgrade and scarce skills to maintain old code. **Speed:** vendor capability arrives late or never. **Audit risk:** custom posting logic draws IT general controls testing.

An illustration. Northmere's SAP S/4HANA core went live in 2021 as a greenfield implementation, chosen precisely to avoid carrying old customizations. It still accumulated about **2,300** custom objects from early design decisions, and upgrades now take **seven months**. Sven Aalto, the SAP platform lead, estimates that fewer than a third of those objects are used weekly.

The remedy direction. Make "standard first, extend outside the core" a mandatory standard with a waiver process, measure the custom object count and upgrade lead time

as fitness indicators, and retire unused objects deliberately.

Where this book helps: SAP S/4HANA: One Journal, Clean Core explains clean core and extensibility; Principles That Decide Things turns it into a principle and standard; Technical Debt on the Balance Sheet measures it; Changing the Engine in Flight: Modernization Patterns for Finance covers the paths out.

The Swiss-Army ERP

"Once everything is in one ERP, we'll have one version of the truth."

What it looks like. A single enterprise resource planning (ERP) platform is expected to absorb every finance capability: high-volume transaction processing, partner settlement, billing, payments, revenue allocation, planning and analytics. Observable signs: requirements for billions of events a year are written as ERP requirements; product teams wait for ERP configuration to launch a business model; the program's scope grows every quarter; "we will do it in the ERP" ends design discussions. The result is a Swiss-Army knife of finance: every blade present, none sharp enough at billions of events.

Why it happens. The ERP's original promise, one integrated database, is real and valuable for the accounting core. It is extended by analogy to everything financial. Vendors encourage it, because suite adoption is their growth model. It is also a reaction to the opposite failure: after years of Point-to-Point Spaghetti, one platform looks like the cure.

What it costs. **Control**: operational-volume detail degrades the ledger and makes the book of record harder to read. **Cost**: customization to make the ERP do product work, feeding Customization Concrete. **Speed**: business-model innovation waits for the ERP's release cycle. **Audit risk**: indirect; overruns tend to defer control design.

An illustration. In early 2027 a Keystone workshop at Northmere proposes moving Stayloft's merchant-model subledger, about 3 million lines a day, directly into the S/4HANA general ledger "so that there is one book". Dana Whitlock, the Chief Architect, asks the workshop to separate two goals: one book of record per entity (which Keystone needs), and one system processing every transaction (which it does not). The convergence decision is reframed as Central Finance versus an accounting-hub pattern, with high-volume detail staying in a subledger designed for it. In December 2027 the design authority chooses the accounting-hub pattern (Converge or Coexist: Living with More Than One ERP).

The remedy direction. Use the ERP as the accounting core it is designed to be, and place high-volume, product-coupled processing in layers designed for volume (subledgers, billing and settlement services), integrated through summarized postings with drill-back.

Where this book helps: Anatomy of a Finance Estate: Layers, Planes and Boundaries lays out the layers and the volume gradient; When the ERP Isn't Enough: Custom Finance Platforms sets the ERP boundary; Converge or Coexist: Living with More

Than One ERP compares convergence patterns; SAP S/4HANA: One Journal, Clean Core shows what the ERP core is designed to hold; Build, Buy or Compose: Choosing What to Own gives the decision criteria.

Point-to-Point Spaghetti

"We just need one more file from their system to ours. It's a small interface."

What it looks like. Systems integrate directly with each other, each pair with its own format, schedule, error handling and owner. Observable signs: an interface inventory nobody trusts; the same booking data transformed differently in four places; failures discovered by downstream users; several integration technologies, each with a different owner.

Why it happens. Each interface is the cheapest way to meet one project's need. Shared integration patterns require someone to fund and own them before any single project benefits. Acquisitions bring their own integration platforms. Finance-specific requirements, such as completeness proofs, control totals, period cut-off and replay, are not visible to the project until the first close.

What it costs. **Control**: without control totals, a missing file shows up only when a reconciliation fails. **Cost**: every source change ripples through every consumer. **Speed**: replacing a system means re-implementing all its interfaces. **Audit risk**: undocumented interfaces in the reporting path become findings.

An illustration. Northmere's 2027 inventory counts about **1,100** interfaces in the finance estate, of which about **240** are flat-file transfers. Three integration technologies coexist: SAP Integration Suite for SAP-centric flows, Kafka owned by Platform Engineering, and MuleSoft carrying legacy Wayfare and Tablekin flows. Lucía Romero, the lead integration architect, finds booking cancellations represented in five different ways across them.

The remedy direction. Introduce canonical financial events and a small number of governed integration styles, with completeness and control totals as standard, and migrate interfaces when systems change rather than in a big rewrite.

Where this book helps: Files, Events and APIs: How Finance Systems Talk covers styles, idempotency and control totals; Canonical Events: A Common Language for Money defines canonical events and adapters; Duplicates and Dependencies: The Hidden Costs of a Grown Estate classifies coupling; Architecture Governance That Speeds Decisions keeps new interfaces conformant; Uber's Payments Platform: A Ledger at the Center of Money Movement shows a public example of one narrow waist between many businesses and many providers.

Capability Creep

"We already pay for that module. Let's use it for our team's reconciliations."

What it looks like. Several platforms in the estate offer the same capability (reconciliation, billing, tax determination, payouts, planning), and different teams use different ones, each for good local reasons. Observable signs: two or three tools perform the same function for different entities; license renewals include modules nobody can name an owner for; the same metric is computed differently in two tools and the results disagree.

Why it happens. Enterprise suites grow sideways: each major release adds adjacent capability, often bundled at a marginal price. Teams adopt what is already licensed; acquired companies bring their own. Nobody holds the capability map that would show the overlap, or the decision right to resolve it.

What it costs. **Control**: one control run in several tools produces inconsistent evidence. **Cost**: duplicate licenses, integrations and skills. **Speed**: every process change is made several times. **Audit risk**: each tool's configuration is tested separately.

An illustration. In 2027 Northmere performs reconciliations in BlackLine, in a OneStream module bought in 2022 and rarely used, and in Oracle Fusion for Wayfare's intercompany balances. It runs three billing approaches (SFP commission billing, SAP FI-CA invoicing, Stripe Billing) and two tax engines for overlapping invoice flows (Vertex, and Oracle's tax engine in some Wayfare markets). Each choice was defensible when made.

The remedy direction. Map capabilities, not systems; assign one owning platform per capability per scope; record the rare legitimate duplications as explicit decisions with a review date; and use renewal dates to retire the rest.

Where this book helps: Map What Finance Does Before What Runs It builds the capability map; Manage the Application Portfolio as an Investment uses it to rationalize; Duplicates and Dependencies: The Hidden Costs of a Grown Estate runs the overlap register; Own Your Roadmap: Vendor Strategy Without Lock-In Panic uses contract timing as a lever.

The Permanent Exception

"Our business is different. The group standard doesn't work for us."

What it looks like. Every brand, market or acquired company argues for its own variant of a shared capability: its own payout logic, its own revenue calculation, its own chart of accounts, its own integration platform. Some exceptions are justified. The failure is that exceptions have no expiry, so each becomes a permanent fork. Observable signs: "temporary" exceptions older than two years; group standards with more waivers than adopters; new brands onboarded by copying the most similar existing brand's stack.

Why it happens. Brand leaders are measured on brand growth, and a shared standard is a dependency they do not control. The central function often cannot deliver the shared capability as fast as the brand needs it, so the exception is rational. Acquisitions are integrated lightly to protect the deal's value, and light integration hardens.

What it costs. **Control**: each fork needs its own documented and tested control design. **Cost**: duplicated engineering, licenses and operations. **Speed**: a new tax rule or reporting requirement is built once per fork. **Audit risk**: more variants, more testing.

An illustration. Northmere runs two partner-payout engines: the Stayloft Finance Platform's, and Wayfare's engine for ground-transport suppliers. When Wayfare was acquired in 2024, its engine was kept because ground-transport supplier contracts "work differently". By 2027 the differences that remain are a payment schedule and a netting rule, both of which the Stayloft engine could support as configuration. Retiring the duplicate is a Keystone Year 3 goal.

The remedy direction. Grant exceptions through a recorded decision with a reason, an owner and an expiry date; design shared capabilities with configuration points for the variations that are real; and fund the central capability well enough that the exception is not the faster path.

Where this book helps: The Next Brand Should Be Boring: Onboarding as Configuration turns brand and market variation into configuration; Architecture Governance That Speeds Decisions covers waivers with expiry; Global by Design: Entities, Currencies, Jurisdictions and Brands explains which variation is legally required; When the ERP Isn't Enough: Custom Finance Platforms judges when a fork is legitimate; Uber's Payments Platform: A Ledger at the Center of Money Movement shows a real platform that turned each new line of business into configuration.

Big-Bang Bravado

"We'll cut over on 1 January. Clean start, new year, new system."

What it looks like. A finance system replacement is planned as a single switch: all entities, all processes, all data, on one date, commonly the start of a fiscal year, with the old system decommissioned shortly after. Observable signs: no parallel run; rollback described as "not an option"; migration rehearsals scheduled late; a cut-over date set by the board calendar rather than readiness.

Why it happens. Big-bang plans look cheaper on paper: no interim interfaces, no dual maintenance, one training wave. The fiscal-year boundary is attractive because balances carry forward cleanly. Program leaders are rewarded for a date. Incremental patterns, such as moving one entity or one process at a time, require interim architecture that looks like waste to a steering committee.

What it costs. **Control**: a failed cut-over can leave the enterprise unable to close or evidence its controls. **Cost**: recovery costs far more than the interim architecture it avoided. **Speed**: everything waits for the slowest component. **Audit risk**: a year-end conversion lands in the audit's busiest weeks, and the completeness of migrated balances is tested closely.

An illustration. Keystone's Year 3 goal at Northmere is to migrate the Stayloft merchant subledger to the group finance core. An early integrator proposal schedules it for 1 January 2029, in one step. Leila Haddad rejects it: the merchant subledger carries deferred balances for bookings whose stays are months away, and a cut-over at year-end would put the migration and the US GAAP annual close on the same team in the same weeks. The plan becomes an entity-by-entity migration with parallel runs and reconciliations as stage gates.

The remedy direction. Plan migrations as a sequence of reversible steps, cut over at period boundaries away from quarter- and year-end, run old and new in parallel with reconciliations as gates, and keep a tested rollback for each step.

Where this book helps: Changing the Engine in Flight: Modernization Patterns for Finance catalogs strangler, parallel run and cut-over patterns; Modernizing Without Stopping the Close works through a migration step by step; The Middle Matters: Transition States and the Roadmap defines stable intermediate states; The Technology Is the Easy Part: Leading a Finance Transformation covers program design.

The Borrowed Roadmap

"The vendor says this is where the platform is going, so we should align our roadmap."

What it looks like. Architectural choices are made in the vendor's terms and on the vendor's timetable: a maintenance deadline drives a migration date, a license bundle drives the choice of module, a new product name drives a re-platforming. Observable signs: the target state reads like a vendor's product map; decisions cluster around renewal dates; nobody can say what the enterprise would do if the vendor changed prices or packaging.

Why it happens. Strategic vendors invest heavily in executive relationships and roadmap briefings, and their roadmaps are often good. Internal architecture functions may lack an independent target state, so the vendor's fills the gap. Maintenance deadlines are real and create genuine urgency. Commercial teams optimize the discount, not the dependency.

What it costs. **Control:** indirect; vendor-default control designs may not fit the enterprise's risks. **Cost:** lost negotiating power, and unused bundles that feed Capability Creep. **Speed:** the enterprise moves at the vendor's pace. **Audit risk:** low directly; the real risk is a concentration nobody chose.

An illustration. In 2027 Northmere must decide how to converge its brand ledgers: SAP Central Finance or an accounting-hub pattern. The SAP account team presents Central Finance alongside an expanded RISE with SAP renewal, with a discount that expires at quarter-end. Dana separates the two: the architecture decision is scheduled for Q4 2027 on its own criteria (volume, drill-back, coexistence with Fusion and Workday), and the commercial negotiation follows it rather than leading it. The decision, taken on 8

December 2027, adopts the accounting-hub pattern and rejects Central Finance; the RISE renewal is negotiated afterward (Own Your Roadmap: Vendor Strategy Without Lock-In Panic).

The remedy direction. Hold an enterprise-owned target state, tier vendors by strategic importance, maintain exit plans for critical ones, and sequence decisions so that architecture precedes commercial commitment.

Where this book helps: Own Your Roadmap: Vendor Strategy Without Lock-In Panic covers tiering, exit plans and contract levers; Build, Buy or Compose: Choosing What to Own covers the selection process; The Middle Matters: Transition States and the Roadmap provides the independent target; Built to Be Replaced: Evolving the Finance Estate Over a Decade treats vendor change as a stressor to design for.

Audit Archaeology

"Let's get the system live first. We'll sort out the controls before the auditors come."

What it looks like. Systems that affect financial reporting are designed and built without control requirements, and controls are added later, typically after an audit finding. Observable signs: internal controls learns about a new system at go-live; production access is broad "for support"; change approvals live in chat; evidence is assembled by hand before each audit. Each audit season becomes an excavation: teams dig through logs, tickets and chat history to reconstruct proof the system should have produced as a by-product.

Why it happens. Controls are perceived as compliance overhead, not as requirements. Product engineering teams building finance-relevant platforms often come from consumer software, where production access for developers is normal. Internal controls teams lack architecture vocabulary, and architects lack controls vocabulary, so neither engages early. Under the **Sarbanes-Oxley Act**, management must assess **internal control over financial reporting** (ICFR) and, for large filers, the auditor attests to it, yet the systems that carry those controls are designed without ICFR in mind.

What it costs. **Control**: deficiencies, graded under the US audit standard by severity. A **significant deficiency** is less severe than a material weakness but important enough to merit the attention of those overseeing financial reporting; a **material weakness** means there is a reasonable possibility that a material misstatement will not be prevented or detected on a timely basis (PCAOB AS 2201, Appendix A). **Cost**: retrofitting costs more than designing in. **Speed**: remediation freezes change. **Audit risk**: the highest of any pattern.

An illustration. Northmere's FY2026 audit raised a **significant deficiency** (not a material weakness) on the Stayloft payout engine: developers had production access, and payout rule changes lacked documented approval. Remediating it is a Keystone Year 1 goal. Mei Tanaka, head of internal controls, and Jonas Berg, who owns the platform,

find that the engine's deployment pipeline already records every change; what is missing is an approval step and the separation of who writes from who deploys.

The remedy direction. Treat control requirements as architecture requirements: involve controls specialists at design, build evidence capture into pipelines and platforms ("evidence by design"), and review the control impact of every change to a finance-relevant system.

Where this book helps: Controls by Design, Not by Audit Season covers ICFR, IT general controls and evidence by design; Reconciled by Design: Traceability and Audit Across Boundaries designs the auditor's walkthrough; Recover Before the Close: Resilience and Security for Finance covers access and incident handling; Never Update, Only Append: Designing Custom Ledgers builds immutability and audit trails into custom ledgers.

Nobody's Bot

"The bot handles those journals now. It's been running fine for months."

What it looks like. Robotic process automation, scheduled scripts, auto-matching rules and, increasingly, AI agents create, approve or change financial data. The automation itself may be sound; the failure is that it has no named owner, no control over its own changes, no evidence of what it did and why, and often runs under a shared or generic service account. Observable signs: journals posted by a user ID nobody recognizes; matching rules edited by whoever is on duty; an AI tool clearing reconciliation items with no record of its reasoning.

Why it happens. Automation is introduced bottom-up by operations teams under pressure to reduce manual work, often successfully. It does not pass through the architecture or controls processes designed for "systems". AI tools add a new twist: their behavior can change with a model update or a prompt edit, outside traditional change management. The savings are visible at once; the control gap is visible only when something goes wrong.

What it costs. **Control**: an unowned bot is an uncontrolled control. **Cost**: errors made at machine speed, investigated without logs. **Speed**: once trust is lost, the automation is switched off and manual work returns. **Audit risk**: generic service accounts and unlogged changes are typical findings.

An illustration. In 2027 Northmere's shared-service center runs a bot that posts monthly reclassification journals in SAP S/4HANA under a generic service account. When a cost-center hierarchy changes in Workday HCM, the bot continues to post with the old mapping for two months. Nobody owns the bot's logic; the person who built it has moved to another team. Keystone's Year 5 ambition of AI-assisted reconciliation now starts from a rule: every automated actor has an owner, its own identity, controlled changes and an evidence log.

The remedy direction. Register every automated actor that touches financial data, give each an owner and a distinct identity, put its logic under change control, log its decisions, and define where a human approves.

Where this book helps: Automate the Work, Not the Accountability covers agentic workflow controls; Controls by Design, Not by Audit Season covers automated controls and access; Who Owns What: Organizing Architecture Across Finance and Technology assigns ownership; Architecture Governance That Speeds Decisions brings automation into review.

How the Patterns Reinforce Each Other

The patterns rarely appear alone. Most estates show several, and they feed each other in recognizable chains. Seeing the chain matters because **fixing a downstream symptom while the upstream pattern continues produces the same symptom again**, usually within a year.

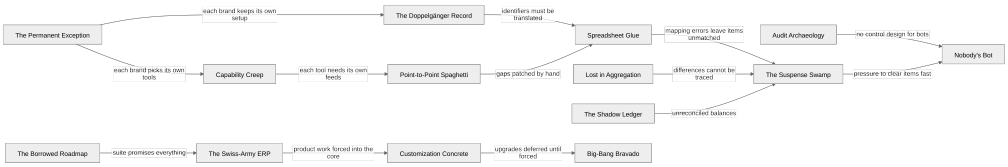


Figure 2: *The patterns form chains; most downstream symptoms, such as growing suspense balances, trace back to an upstream choice about authority, standards or vendors.*

Four chains appear most often.

The truth chain: **The Doppelgänger Record → Spreadsheet Glue → The Suspense Swamp**. When several systems create the same partner, entity or account, identifiers must be translated between them. The translation is first done in a spreadsheet, because that is fastest. The spreadsheet drifts from the systems it maps, so some transactions land against the wrong counterparty or account and fail to match. The unmatched items go to suspense. Finance then sees a suspense problem and staffs a clearing team, while the cause sits two steps upstream in master data authority. At Northmere, part of the €14 million suspense balance traces to partner records whose bank and contract identifiers differ between the Stayloft Finance Platform and SAP FI-CA. **Break the chain at the master**, not at the suspense account.

The fragmentation chain: **The Permanent Exception → Capability Creep → Point-to-Point Spaghetti**. Each brand exception brings its own tools; each tool needs its own interfaces; each interface carries its own format and error handling. Within a few acquisitions the estate has three integration technologies and five representations of a cancellation. The remedy is not a heroic integration rewrite but a governance change: exceptions expire, capabilities have owners, and new interfaces use canonical events.

The platform chain: The Borrowed Roadmap → The Swiss-Army ERP → Customization Concrete → Big-Bang Bravado. A vendor's suite story encourages putting everything in one platform; work the platform was not designed for is forced in through custom code; custom code makes upgrades so expensive that they are deferred until a maintenance deadline forces a large, single migration.

The control chain: Audit Archaeology → Nobody's Bot. An organization that designs controls late will also automate without control design, and pressure from The Suspense Swamp ("clear those items faster") accelerates it. The audit finding, when it comes, covers both.

Two patterns sit at the roots of most chains. **The Permanent Exception** is a governance failure: nobody has the decision right, or the will, to say no with an expiry date. **The Borrowed Roadmap** is a strategy failure: the enterprise lacks a target state of its own. That is why this book spends Part IV on governance, target states and vendor strategy, not only on technology.

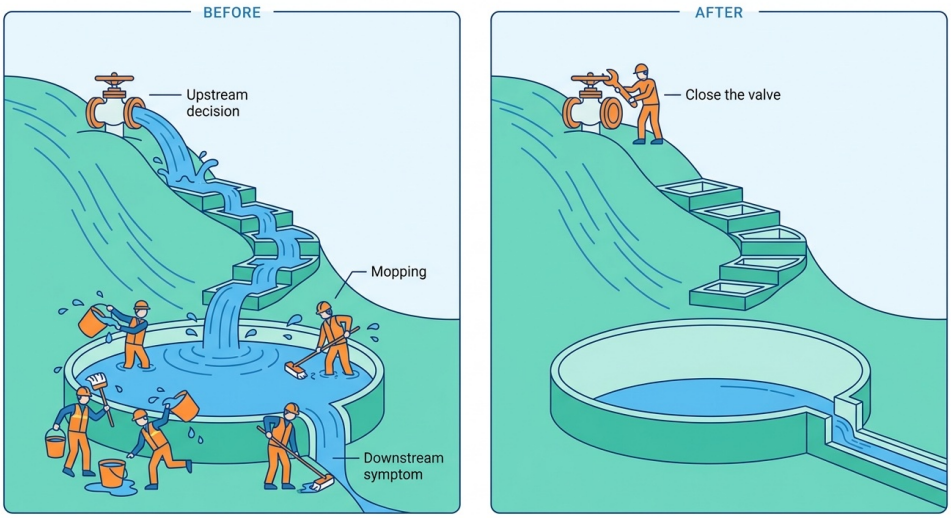


Figure 3: *Fixing a downstream symptom while the upstream pattern keeps flowing brings the symptom back; the cheaper fix is usually at the origin, a decision right, a standard or an expiry date.*

A diagnostic table

The patterns leave measurable traces. The indicators below are the ones I would ask for in a first review of any finance estate; each is cheap to obtain and hard to argue with.

Pattern	First indicator to ask for	Typical owner of the number
The Shadow Ledger	Balances held outside the GL with no scheduled reconciliation	Controller, platform owner
The Suspense Swamp	Suspense and clearing balances by aging bucket, trended over 12 months	Controller, payments operations
The Doppelgänger Record	Number of systems where each master data object can be created	Data governance lead
Lost in Aggregation	Time to trace a sampled GL line to its transactions	Finance systems, internal audit
Spreadsheet Glue	Count of key EUCs in the close, by risk rating	Internal controls
Customization Concrete	Custom object count and upgrade lead time	ERP platform owner
The Swiss-Army ERP	Share of program scope that is product or volume processing	Program design authority
Point-to-Point Spaghetti	Interfaces per system, share without control totals	Integration architecture
Capability Creep	Capabilities served by more than one platform	Enterprise architecture
The Permanent Exception	Open waivers and their age	Architecture review board
Big-Bang Bravado	Number of reversible steps in the migration plan	Program, design authority
The Borrowed Roadmap	Critical vendors without an exit plan	Chief Architect, procurement
Audit Archaeology	Control deficiencies by root cause (design vs operation)	Head of internal controls
Nobody's Bot	Automated actors without a named owner or distinct identity	Internal controls, operations

These are not targets in themselves. A suspense balance can be low because items are written off too quickly; a custom object count can fall because code moved to an unmanaged side system. **Read each indicator with the mechanism in mind**.

Using the Patterns

The names are meant to be used in rooms where decisions are made. In a design review, ask which pattern a proposal risks, and what would stop it. In a steering committee, ask which pattern a delay or overrun is a symptom of. In a portfolio review, ask which patterns the estate shows and which root (governance or target state) they trace back to.

Three habits keep the names useful rather than accusatory: **name the mechanism, not the person**; **pair every pattern with a measure** from the table above; and **look for the upstream pattern** before funding a downstream remedy, because the cheapest fix is usually a decision right, a standard or an expiry date, not a new tool.

Some duplication, customization and exception is legitimate. A second payment provider is a resilience choice, not Capability Creep; a market-specific tax process required by law is not The Permanent Exception. The test is whether the variation was **decided, owned and reviewed**, or simply accumulated.

The rest of the book works through the remedies. Before that, meet the group whose estate shows every one of these patterns in 2027, and the program that sets out to fix them: Meet Northmere: Five Brands, One Set of Books.

Sources and Evidence

Standards and regulation

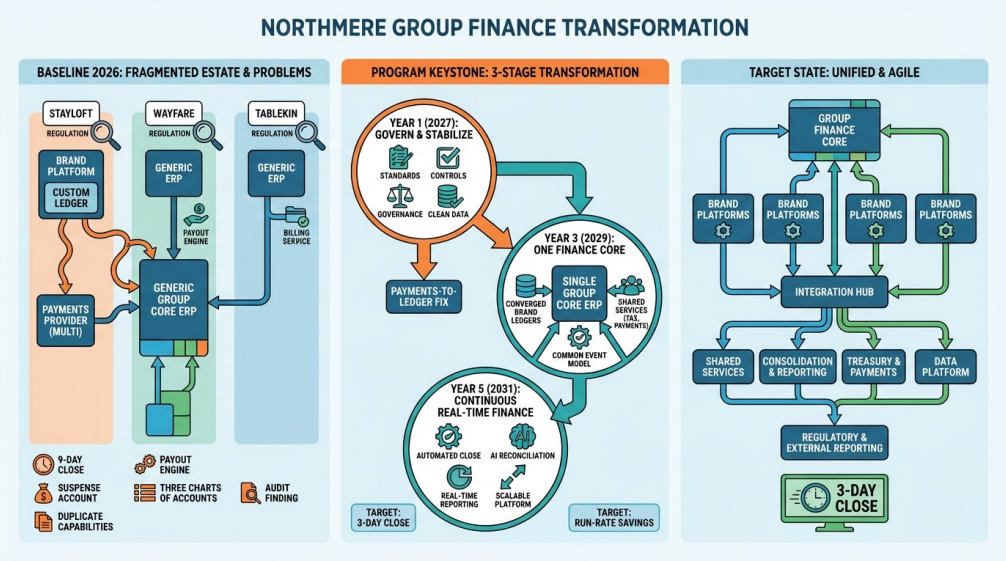
- AS 2201: An Audit of Internal Control Over Financial Reporting That Is Integrated with An Audit of Financial Statements, Public Company Accounting Oversight Board — definitions of material weakness and significant deficiency (Appendix A) and the auditor's testing of controls.

Practitioner and research sources

- Strangler Fig, Martin Fowler, 2024 (revised) — the incremental replacement pattern behind the remedy for Big-Bang Bravado.

The pattern names and their descriptions are the author's synthesis of common industry practice; Northmere is fictional, and the architectural recommendations are the author's judgment.

3. Meet Northmere: Five Brands, One Set of Books



IN THIS SECTION, YOU WILL: Meet Northmere Group, the fictional enterprise this book follows: its brands and business models, its legal entities and licensed payments entity, its systems landscape and volumes, the people who run it, the seven problems it starts 2027 with, the transformation program approved to solve them, and the worked examples the chapters reuse.

KEY POINTS:

- **Northmere is a fictional composite, not a disguised company.** It generalizes patterns common to multinational, multi-brand, technology-driven groups, so the book can show real architectural tensions with consistent numbers without describing any real enterprise's internals.
- **Its difficulty comes from variety, not size.** Four business models, three ERPs, two payment providers, two e-money institutions and 71 legal entities must produce one set of US GAAP statements and one auditable story of the money.

- **Every chapter starts from one of seven 2027 problems**. The nine-day close, the €14 million suspense balance, three charts of accounts, duplicate capabilities, an audit finding, a heavily customized ERP and a queue of new regulations give each chapter a concrete question to answer.

Architecture books often fail in one of two ways. Either their examples are toy systems too simple to show the trade-offs that matter, or they describe one real company in detail that does not transfer and cannot be verified. This book takes a third route: **one fictional enterprise, detailed enough to be realistic and consistent enough to be followed across fifty chapters**.

Northmere Group is fictional. Every name, figure, system choice and event on this page is invented for teaching. Its shape generalizes patterns that commonly appear in multinational, multi-brand, technology-driven groups: a marketplace business shifting from agency to merchant payments, acquired brands that arrive with their own ERPs, a licensed payments entity inside a commercial group, custom finance platforms built by product engineering, and a finance function trying to close faster while the business changes underneath it. It is **not a portrait of any real company**, and nothing here should be read as a claim about how any real enterprise is built.

The vendor platforms Northmere uses are real products, chosen because they commonly appear in such estates and because Part II explains each one in depth. Northmere's configuration of them, its problems with them and its decisions about them are invented.

The Group at a Glance

Northmere is a multinational digital commerce group, headquartered in Amsterdam, with major hubs in London, Dublin, Singapore, Austin and São Paulo. It is listed in New York, which makes it an SEC registrant reporting under US GAAP, with management's assessment and the auditor's attestation of internal control over financial reporting required under Section 404 of the Sarbanes-Oxley Act (see Controls by Design, Not by Audit Season).

Measure (fiscal 2026, calendar year)	Value
Gross transaction value (GTV), the total value customers transact through its platforms	about **$64 billion**
Revenue	about **$8.2 billion**
Employees	about **24,000**

Measure (fiscal 2026, calendar year)	Value
Legal entities	**71** in **38** countries
Currencies customers pay in	**46**
Currencies Northmere settles with partners in	**31**
Group reporting currency	US dollar (most European entities have euro functional currency)
Group GAAP	US GAAP; local GAAP or IFRS per entity for statutory accounts

The gap between GTV and revenue is the first thing an architect should notice. Most of the $64 billion is **other people's money**: payments that Northmere collects and passes on to properties, rail operators or restaurants, or that customers pay to those partners directly. Northmere's revenue is mainly the commissions and fees it earns along the way. The finance estate must track the whole flow, not just the revenue, because the money in transit is a liability, a regulatory obligation and a reconciliation problem.

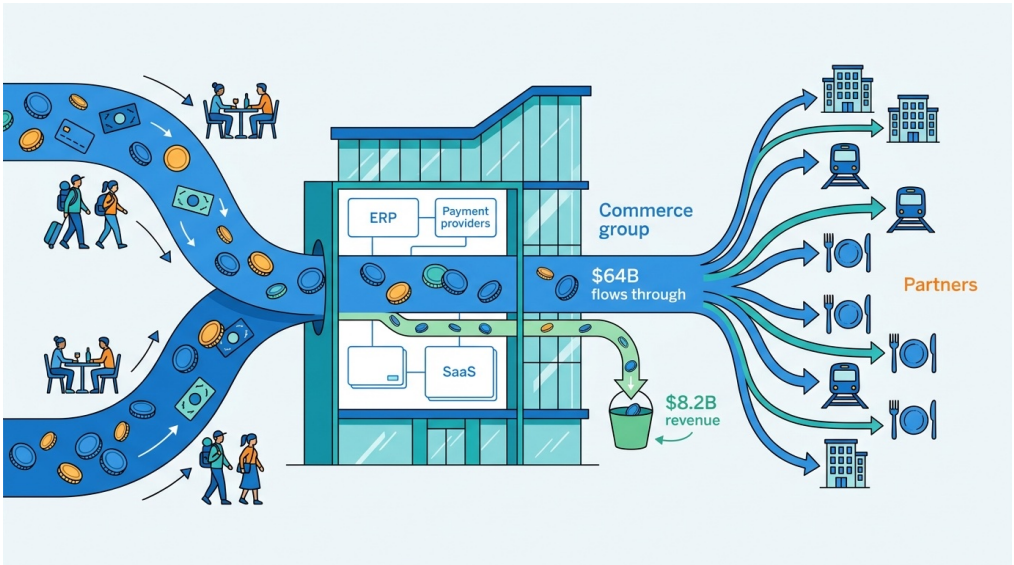


Figure 1: *Most of the $64 billion Northmere handles is other people's money in transit; its $8.2 billion of revenue is the smaller stream it keeps, but the estate must track the whole flow.*

Brands and Business Models

Northmere runs five businesses. Four earn revenue directly; one is the payments platform the others rely on.

Brand	What it does	Business model(s)	Share of 2026 revenue	Finance stack (2027 baseline)
Stayloft	Accommodation marketplace: hotels, apartments, homes	Agency and merchant; merchant share of GTV **64%** in 2026, up from 51% in 2023	~58%	**Stayloft Finance Platform (SFP)**, custom; posts daily summaries to SAP S/4HANA
Wayfare	Flights, rail and ground transport	Mostly agency for flights; merchant for rail and transfers	~19%	**Oracle Fusion Cloud ERP**; legacy Oracle E-Business Suite read-only until 2028
Tablekin	B2B software for restaurants and venues: reservations, table management, embedded payments	Subscription, usage fees and payments facilitation	~11%	**Workday Financial Management**, **Stripe Billing**, **Stripe Connect**
Northmere Ads	Advertising and sponsored listings across the brands	Cost-per-click and sponsored placement	~7%	Custom ad-billing service posting summarized receivables to S/4HANA

Brand	What it does	Business model(s)	Share of 2026 revenue	Finance stack (2027 baseline)
Northmere Pay	Internal payments platform and the licensed e-money institution	Payment acceptance, traveler-funds safeguard-ing, partner payouts	enabler; fees recharged	Custom platform; **Adyen** primary, **Stripe** secondary for Stayloft and primary for Tablekin; local payment-method providers in Brazil, India and Indonesia

The remaining ~5% of revenue comes from insurance commissions and other services, which the book uses only when needed.

The business models are what make Northmere's finance architecture hard, because each one creates different records, different timing and different obligations. In brief (the accounting is developed in Accounting for Architects: Ledgers, Journals and Revenue):

- **Agency model**. The traveler pays the property, usually at the stay. Northmere never holds the money; it invoices the property a commission monthly and collects it. The finance work is billing and receivables: invoices, dunning and partner balances, for hundreds of thousands of partners. At Stayloft this runs in SAP FI-CA, SAP's contract-accounting subledger for high-volume receivables.
- **Merchant model**. Northmere collects the traveler's payment at booking, holds it until after the stay, and pays the property the amount net of its commission. The finance work multiplies: payment acceptance, settlement reconciliation with the payment provider, refunds, chargebacks, a large liability for funds held, safeguard-ing obligations, and millions of partner payouts. Under US GAAP's principal-versus-agent assessment, Northmere treats itself as **agent** for the accommodation in both models, so its revenue is the commission (net presentation), even though the merchant model puts the full booking amount through its bank accounts.
- **Subscription and usage**. Tablekin charges venues a monthly or annual plan fee plus a fee per seated cover. Revenue is recognized as the service is delivered over the period; the finance work is rating usage, invoicing, collections and deferred revenue.

- **Marketplace payments facilitation**. Tablekin also processes diners' payments on behalf of venues through Stripe Connect, Stripe's product for platforms that route payments to connected sellers (Stripe: Every Movement Is a Balance Transaction covers its charge models). The venue is the seller; Tablekin earns a facilitation fee and must ensure every venue is paid correctly.
- **Advertising**. Northmere Ads bills advertisers for clicks and sponsored placements, a high-volume, rated-usage receivables flow.

Two trends shape the book's timeline. **Stayloft is shifting from agency to merchant**: the merchant share of its GTV rose from 51% in 2023 to 64% in 2026, and every point of that shift moves volume from invoicing into payments, settlement, safeguarding and payouts. And **Northmere grows by acquisition**: Tablekin joined in 2023 and Wayfare in 2024, each bringing a working finance stack that was never designed to fit the group's.

Legal Entities and the Licensed Payments Entity

Northmere's 71 legal entities include operating companies per brand and region, holding and financing companies, and the payments entities. Each legal entity keeps its own books, files its own statutory accounts and tax returns, and appears in group consolidation; the system representation of entities is one of the recurring themes of Global by Design: Entities, Currencies, Jurisdictions and Brands.

Two entities are special because they are regulated as financial institutions:

- **Northmere Payments B.V.** is an **e-money institution** (EMI), a firm licensed to issue electronic money and provide payment services, authorized in the Netherlands and supervised by De Nederlandsche Bank, and passported across the European Economic Area. It accepts payments for Stayloft merchant bookings and must **safeguard** traveler funds: keep them segregated from its own money in safeguarding accounts and reconcile them daily. As a financial entity it is subject to the EU's Digital Operational Resilience Act (DORA) and to PSD2, the payment services directive, with its successors PSD3 and the Payment Services Regulation when they apply.
- **Northmere Payments UK Ltd** is an FCA-authorized EMI serving UK customers, needed since the UK left the EU's passporting regime.

Northmere is not a bank, and the distinction matters. An EMI may hold customer funds only under safeguarding rules; it cannot lend them. The safeguarding reconciliation is a regulatory control that must be produced daily from data the finance estate can prove complete, which makes payments-to-ledger accuracy a licensing matter, not only an accounting one (Payments for Finance Architects: Eleven Reasons the Numbers Disagree and Payments-to-Ledger: Draining the Suspense Swamp develop this).

The Systems Landscape

Northmere's 2027 baseline estate has a recognizable shape: one group finance core on SAP, two brand ERPs from acquisitions, a custom platform for the brand that generates most of the volume, specialist platforms for close, consolidation, tax and treasury around the core, and three integration technologies connecting them.

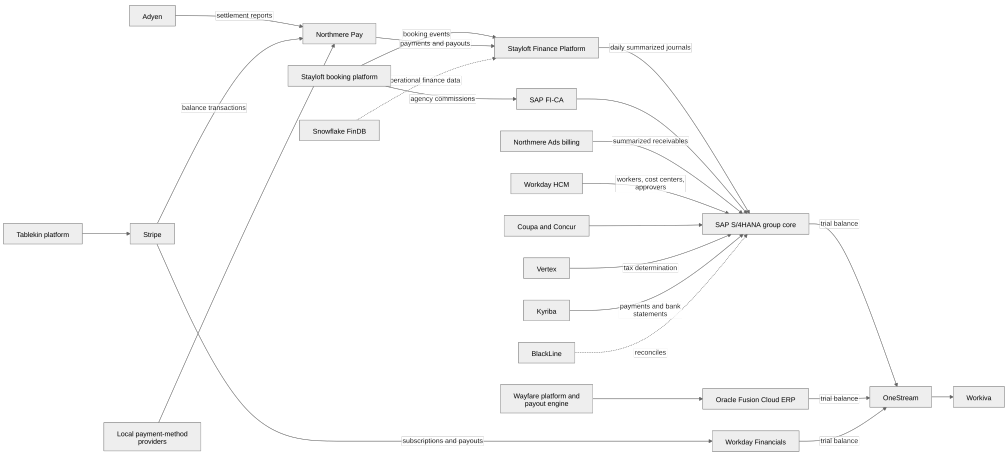


Figure 2: *Northmere's 2027 baseline: three ledgers meet only at consolidation in OneStream, and most of the volume flows through a custom platform before it reaches the group core.*

Area	System	What to know
Group finance core and GL	**SAP S/4HANA**, Private Edition under RISE with SAP	Greenfield implementation, live since 2021 (a system conversion from ECC 6.0 was rejected). General ledger, payables, receivables, assets, controlling and the group chart of accounts for Stayloft, Northmere Pay, Northmere Ads and corporate entities. About 2,300 custom objects. Central Finance is not used.

Area	System	What to know
High-volume receivables subledger	**SAP FI-CA**	Since 2025, for Stayloft agency partner accounts: commission invoices, dunning and balances for about 850,000 partner contract accounts.
Brand finance platform	**Stayloft Finance Platform** (custom)	Commission billing, the merchant-model subledger, partner payouts and traveler funds; owned by Stayloft Finance Engineering.
Brand ERPs	**Oracle Fusion Cloud ERP** (Wayfare), **Workday Financial Management** (Tablekin)	Both predate their acquisition; Wayfare moved from Oracle E-Business Suite to Fusion in 2022.
Human capital management	**Workday HCM**	Group-wide since 2019 for about 24,000 workers; source of organization structure, worker cost-center assignments and approvers; Workday Payroll in the US, UK and Canada.
Consolidation, planning, reporting	**OneStream**	Since 2022, replacing Oracle Hyperion HFM and a planning tool; loads trial balances from all three ledgers.
Close and reconciliation	**BlackLine**	Account reconciliations (about 6,800 balance-sheet accounts monthly), task management, journal entry and transaction matching of payment-provider settlements.

Area	System	What to know
Indirect tax	**Vertex**	Tax determination for S/4HANA and SFP commission invoicing; Wayfare uses Oracle's tax engine in some markets.
Treasury	**Kyriba**	Cash positioning, bank connectivity, FX hedging and the payment factory; about 410 bank accounts at 27 banks.
Procurement and expenses	**Coupa**, **SAP Concur**	Indirect spend and employee expenses, posting to S/4HANA; supplier payments executed through Kyriba.
Disclosure	**Workiva**	10-K and 10-Q drafting and inline XBRL, linked to OneStream outputs.
Finance data platform	**Snowflake** with dbt ("FinDB")	Operational finance data and governed metrics; tie-out to the ledger is a known gap.
Integration	**SAP Integration Suite**, **Kafka**, **MuleSoft**	SAP-centric flows, the event backbone owned by Platform Engineering, and legacy Wayfare and Tablekin flows respectively.
Architecture repository	**SAP LeanIX**	The application portfolio fact base.

The most important architectural fact in the table is a gap: **Wayfare and Tablekin are integrated with the group only at consolidation level**. Their ledgers send trial balances to OneStream, where accounts are mapped to the group structure. There is no shared chart of accounts, no common financial event, and no drill-back from a group number into a Wayfare or Tablekin transaction except through each brand's own system. Whether to close that gap with SAP Central Finance or with an accounting-hub pattern is an open decision for the fourth quarter of 2027 (Converge or Coexist: Living with

More Than One ERP discusses the options and records the December 2027 decision).

Volumes

The numbers below are 2026 figures, rounded. They explain why Northmere cannot simply put everything in its ERP.

Flow	Volume
Operational financial events (bookings, modifications, cancellations, payments, refunds, payouts, billed ad clicks)	~**1.1 billion** a year
Payment transactions through Northmere Pay	~**420 million** a year: Adyen ~68%, Stripe ~22%, local payment-method providers ~10%
Partner payouts (Stayloft merchant model and Tablekin venues)	~**9 million** a month
SFP subledger lines	~**3 million** a day
GL lines posted by SFP to S/4HANA after summarization	~**38,000** a day
Interfaces in the finance estate inventory (2027)	~**1,100**, of which ~240 flat-file transfers and ~70 undocumented spreadsheets-in-the-loop
Finance-relevant applications in LeanIX (2027)	**214**

The drop from 1.1 billion events to tens of thousands of daily ledger lines is the **volume gradient** that Anatomy of a Finance Estate: Layers, Planes and Boundaries uses to explain why finance estates have layers. Every step down that gradient is a summarization, and every summarization is a place where traceability can be lost.

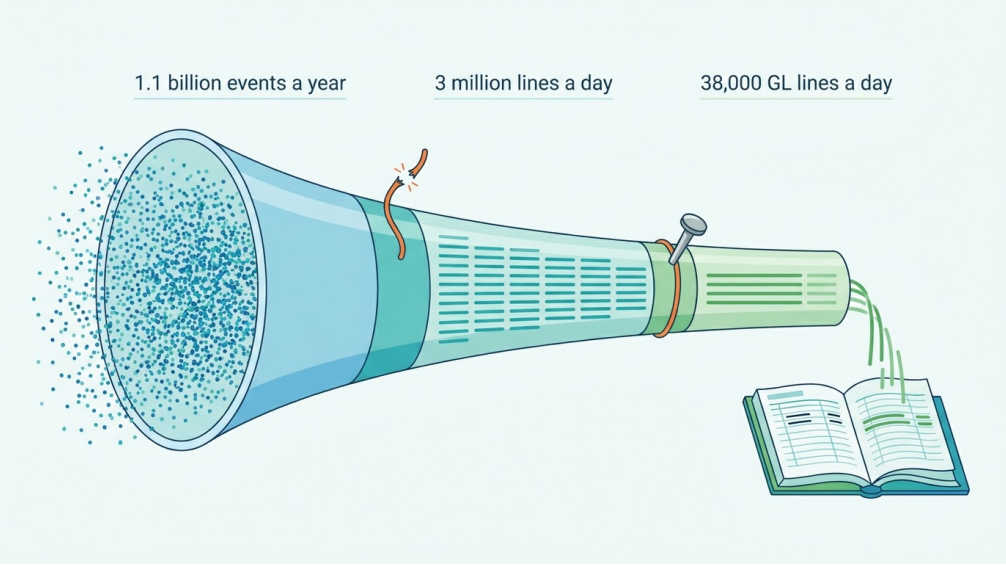


Figure 3: *The volume gradient: every step from 1.1 billion events to 38,000 ledger lines a day is a summarization, and every summarization is a place where traceability is either kept or lost.*

The People

The cast is small enough to remember. Each person owns something the architecture depends on, and most of the book's tensions are between their legitimate goals.

Name	Role	What they own and want
Dana Whitlock (she/her)	Chief Architect, Enterprise Applications & Financial Systems	Joined in September 2026 from a payments company. Reports to Ravi with a dotted line to Tomás; holds final decision rights on mandatory enterprise-application standards; leads about 30 architects and influences brand finance engineering through the matrix. The book's point of view.
Tomás Ferreira (he/him)	Group CFO	Sponsor of Program Keystone; wants a faster close and fewer surprises.
Leila Haddad (she/her)	Chief Accounting Officer and Group Controller	Owns record-to-report, accounting policy and the close; process owner for the general ledger.
Ravi Menon (he/him)	Group CTO	Owns engineering platforms (Kafka, cloud, identity); Dana's line manager.
Ingrid Solberg (she/her)	Managing Director, Northmere Pay	Owns the payments platform, provider relationships and the e-money entities.
Kofi Asante (he/him)	Group Head of Tax and Treasury	Owns Vertex, Kyriba and the Pillar Two program.
Mei Tanaka (she/her)	Head of Internal Controls (SOX)	Owns the ICFR program; liaison with the external auditor.
Jonas Berg (he/him)	Engineering Director, Stayloft Finance Engineering	Owns SFP and about 60 engineers.

Name	Role	What they own and want
Hana Kowalski (she/her)	Director of Finance Systems, Wayfare	Owns Oracle Fusion and Wayfare's payout engine.
Arjun Rao (he/him)	Head of Finance Systems, Tablekin	Owns Workday Financials and the Stripe integration.
Owen Mbeki (he/him)	Head of Finance Data	Owns FinDB and the governed metrics.
Lucía Romero (she/her)	Lead Enterprise Architect, Integration	Dana's lead for canonical financial events and the integration hub.
Marco Bellini (he/him)	CISO	Security architecture, PCI DSS scope, privileged access.
Sven Aalto (he/him)	SAP Platform Lead	Owns S/4HANA, FI-CA and Integration Suite operations.

Northmere's **external auditor** is an unnamed audit firm. It appears in several chapters as the party whose questions the architecture must be able to answer.

Seven Problems Northmere Starts 2027 With

When Dana arrives, the estate works: the books close, the statements are filed, partners are paid. But it works through effort, and the effort grows faster than the business. Seven problems define her starting position. Each is an instance of one or more patterns from Unreconciled: Fourteen Ways Finance Architecture Goes Wrong.

1. **The close takes nine working days.** The group target is **five** by the end of 2028 and **three** by 2030, as a step toward continuous close. Most of the nine days go to reconciliation, mapping and investigation, not to accounting judgment.
2. **Payments do not reach the ledger cleanly.** On working day 3 of each close, about **1.9%** of payment-provider settlement lines for Stayloft's alternative payment methods are unmatched. Unreconciled balances in clearing and suspense accounts averaged **€14 million** at month-end in 2026, with about **€2.1 million** older than 90 days. *(The Suspense Swamp.)*
3. **Three charts of accounts.** SAP's group chart, Oracle's Wayfare chart and Workday's Tablekin chart are mapped only in OneStream, with the mappings maintained in spreadsheets. *(Spreadsheet Glue, The Doppelgänger Record.)*

4. **Duplicate capabilities**. Two partner-payout engines (SFP and Wayfare's engine for ground-transport suppliers); three billing approaches (SFP commission billing, FI-CA invoicing, Stripe Billing); reconciliations performed in BlackLine, in a rarely used OneStream module bought in 2022, and in Oracle Fusion for Wayfare intercompany. *(Capability Creep, The Permanent Exception.)*
5. **An audit finding**. The FY2026 audit raised a **significant deficiency** (not a material weakness) on change management and access for the SFP payout engine: developers had production access, and payout-rule changes lacked documented approval. Remediation is under way. *(Audit Archaeology.)*
6. **A heavily customized ERP**. About 2,300 custom objects in S/4HANA, a low clean-core score, and upgrades that take seven months. *(Customization Concrete.)*
7. **Regulation on the horizon**. The EU's VAT in the Digital Age package (e-invoicing, digital reporting and deemed-supplier rules for platforms in short-term accommodation), Pillar Two's global minimum tax information return, national e-invoicing mandates in markets such as Poland, Belgium, France and Germany, PSD3 and the Payment Services Regulation, ongoing DORA obligations, and the planned Brazil launch. *(The Permanent Exception, if each is solved locally.)*

None of these is a technology problem alone. The close is slow because the estate cannot prove its records agree; the suspense balance grows because payment methods launch faster than matching rules; the audit finding exists because a product platform became a financial system without being designed as one.

Program Keystone

In **December 2026** Northmere's board approved **Program Keystone**, a multi-year finance transformation sponsored by Tomás, with a design authority chaired by Dana together with Leila. Its budget is about **$310 million** over five years (capital and operating spend), against a target of **$95 million** a year in run-rate savings by 2030 from retired systems, reduced manual effort and lower payment-provider and bank costs.

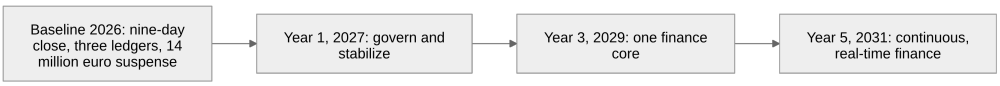


Figure 4: *Keystone moves through three plateaus; each must leave the estate stable and closable on its own, not only as a step toward the next.*

Horizon	Theme	Commitments
Year 1 (2027)	Govern and stabilize	One architecture decision record and review board; mandatory standards; payments-to-ledger completeness for cards and alternative payment methods; publish the financial-systems target state; portfolio return-on-investment baseline; remediate the SFP significant deficiency.
Year 3 (2029)	One finance core	Converge brand ledgers onto a single group finance core on S/4HANA (Central Finance versus an accounting-hub pattern to be decided in 2027 Q4); canonical financial events and a brand adapter kit; one group chart of accounts; tax determination and e-invoicing as shared services; retire the duplicate payout engine.
Year 5 (2031)	Continuous, real-time finance	Continuous close in three days; AI-assisted reconciliation and controls with auditable evidence; real-time views of cash and liabilities; new brands and markets onboarded as configuration.

Timeline anchors the chapters rely on: the decision record and review board launch in the first quarter of 2027 alongside version 1 of the capability map and the system-of-record matrix; the payments-to-ledger program runs from mid-2027, extending BlackLine transaction matching to alternative payment methods; the convergence decision falls in 2027 Q4; Brazil launches in 2028 with a new legal entity, local payment methods including PIX and local e-invoicing rules, the same year the Wayfare E-Business Suite archive is decommissioned; and in 2029 the group chart of accounts goes live and the SFP merchant subledger migrates, incrementally, to the group core.

Keystone also fixes the vocabulary of the integration layer. Its version-1 canonical financial events (`BookingConfirmed`, `PaymentCaptured`, `PaymentSettled`, `RefundIssued`, `ChargebackReceived`, `PartnerPayoutInstructed`, `SubscriptionInvoiced`, `UsageRated`, `AdClickBilled` and others) share an envelope that carries brand, legal entity, business model, source system, a correlation identifier such as the booking id, and an amount in minor units with currency. Files, Events and APIs: How Finance Systems Talk introduces them and Canonical Events: A Common Language for Money owns the model.

The Recurring Worked Examples

Four transactions recur throughout the book. Their amounts never change, so a reader can follow one booking from the traveler's card to the published statements across many chapters.

Example	Facts	Where it does most work
The Lisbon booking (Stayloft, merchant model)	A traveler from Germany books a three-night stay in a Lisbon apartment, total **€600.00** including VAT, paid by card through Adyen on **12 March 2027**; Adyen books the capture into settlement batch 1047 on **14 March 2027**, and the batch payout reaches Northmere Payments B.V.'s bank on **16 March 2027**. Stay **18–21 April 2027**. Commission at **15% = €90.00**, Stayloft's revenue as agent. Illustrative Adyen processing fee **€1.74**. Partner payout **€510.00** instructed on **22 April** and settled on **26 April 2027**.	Accounting for Architects: Ledgers, Journals and Revenue, Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Adyen: One Platform from Checkout to Settlement, The Life of a Booking: From €600 Payment to $97.38 of Revenue, Payments-to-Ledger: Draining the Suspense Swamp
The chargeback (Stayloft)	A traveler disputes a separate **€420** booking (stay 1–4 May 2027, commission €63.00, partner payout €357.00); the chargeback arrives on **13 June 2027**, **40 days after the stay**, long after the partner was paid, and is lost on 26 July. (Adyen: One Platform from Checkout to Settlement also follows an earlier, unrelated €420 chargeback.)	Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Payments-to-Ledger: Draining the Suspense Swamp, Reconciled by Design: Traceability and Audit Across Boundaries

Example	Facts	Where it does most work
The Tablekin invoice (subscription and usage)	An Austin venue on the "Pro" plan: **$249** a month plus **$0.25** per seated cover; 3,120 covers in March 2027 = **$780**; invoice **$1,029** before sales tax; billed through Stripe Billing, posted to Workday Financials.	Stripe: Every Movement Is a Balance Transaction, Workday: The Org Chart Behind the Ledger, Order to Cash, Procure to Pay, Record to Report
The Wayfare rail ticket (merchant)	A **€84.00** Paris–Lyon ticket; Wayfare is merchant of record but, under Northmere's policy, **agent** for the rail operator, so its revenue is the **€6.72** net margin, recognized at ticket issue; paid through Stripe as Wayfare's secondary payment provider; the rail operator is paid its €77.28 monthly.	Oracle Fusion: An Accounting Hub for High-Volume Events, Converge or Coexist: Living with More Than One ERP

The Lisbon booking is the book's spine. Within six weeks it touches the booking platform, Northmere Pay, Adyen, the e-money entity's safeguarding account, the SFP subledger, Vertex, S/4HANA, BlackLine and the partner's bank, and creates records on at least three clocks: when the money moved, when the service was delivered, and when the books were closed. Of the €600 that moves, only €90 is Northmere's revenue, and that revenue is earned in April, not in March when the cash arrives. Chapters simplify the VAT treatment of the commission when they need to, and say so.

How the Chapters Use Northmere

Each chapter opens with a scene set at Northmere in 2027, usually one of Dana's conversations with the cast, that poses the chapter's question. The chapter then works the answer through Northmere's numbers. The table maps the seven problems to the

chapters that do most of the work on each.

Northmere problem	Part I: Foundations	Part II: Platforms	Part III: Integrated	Part IV: Lead
Nine-day close	Order to Cash, Procure to Pay, Record to Report	BlackLine: Matching, Certifying and Closing, OneStream: Consolidating Actuals, Planning the Future	The Life of a Booking: From €600 Payment to $97.38 of Revenue	Technical Debt on the Balance Sheet
Payments-to-ledger gap and suspense	Files, Events and APIs: How Finance Systems Talk	Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Adyen: One Platform from Checkout to Settlement, Stripe: Every Movement Is a Balance Transaction	Payments-to-Ledger: Draining the Suspense Swamp, Reconciled by Design: Traceability and Audit Across Boundaries	Recover Before the Close: Resilience and Security for Finance
Three charts of accounts	Finance Data Architecture: Who Owns the Truth	OneStream: Consolidating Actuals, Planning the Future, SAP S/4HANA: One Journal, Clean Core	Converge or Coexist: Living with More Than One ERP, Systems of Record: Ending the Doppelgänger Record	The Middle Matters: Transition States and the Roadmap

Northmere problem	Part I: Foundations	Part II: Platforms	Part III: Integrated	Part IV: Lead
Duplicate capabilities	Map What Finance Does Before What Runs It, Manage the Application Portfolio as an Investment	Around the Core: Where Specialist Platforms Earn Their Place	Duplicates and Dependencies: The Hidden Costs of a Grown Estate	Own Your Roadmap: Vendor Strategy Without Lock-In Panic
SFP audit finding	Controls by Design, Not by Audit Season	When the ERP Isn't Enough: Custom Finance Platforms, Never Update, Only Append: Designing Custom Ledgers	Reconciled by Design: Traceability and Audit Across Boundaries	Architecture Governance That Speeds Decisions
Customized S/4HANA	Changing the Engine in Flight: Modernization Patterns for Finance	SAP S/4HANA: One Journal, Clean Core	Modernizing Without Stopping the Close	Technical Debt on the Balance Sheet
Regulation and new markets	Global by Design: Entities, Currencies, Jurisdictions and Brands	Around the Core: Where Specialist Platforms Earn Their Place	The Next Brand Should Be Boring: Onboarding as Configuration	Built to Be Replaced: Evolving the Finance Estate Over a Decade

Part IV follows Keystone itself: how Dana sets up governance in its first year, writes principles and standards, describes the target and transition states, negotiates with vendors, organizes teams and introduces AI under control. The Chief Architect's Job: Making Many Systems Tell One Story begins with her first ninety days.

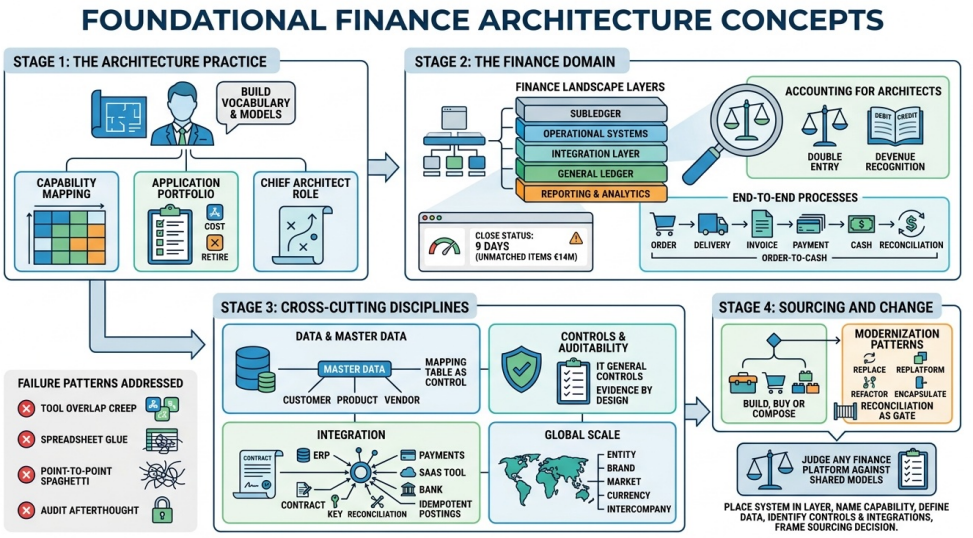
Three conventions hold throughout. Northmere is **always labeled fictional** the first time it appears in a chapter. Its facts come from one canonical record, so the €14 million, the 2,300 custom objects and the €600 booking mean the same thing everywhere. And when a chapter needs a new detail, a meeting, an interface or a metric, it adds it without contradicting what is established here.

With the group and its problems introduced, Part I begins with the role at the center of the book: what a Chief Architect is accountable for in a finance estate like this one. Continue with FOUNDATIONS: How Money Moves Through an Enterprise.

Sources and Evidence

Northmere Group, its brands, entities, people, figures and events are fictional and invented for teaching; they are not drawn from any single real company. The vendor products named are real; their capabilities are described, with sources, in the Part II chapters. Architectural observations on this page are the author's judgment.

FOUNDATIONS: How Money Moves Through an Enterprise



IN THIS PART, YOU WILL: Build the vocabulary and the models a Chief Architect needs before judging any finance platform: what the architecture function is for, what finance does, how a finance estate is layered, what accounting demands of systems, and how data, controls, integration, scale, sourcing and modernization shape every later decision.

FAILURE PATTERNS THIS PART ADDRESSES:

- **Capability Creep** — The capability map and the portfolio make duplicate capabilities visible and force an owner decision for each.
- **Spreadsheet Glue** — Treats mappings, hand-offs and control totals as governed master data and controls rather than files on a laptop.

- **Point-to-Point Spaghetti** — Chooses integration styles per job and gives every financial flow a contract, a key and a reconciliation.
- **Audit Archaeology** — Designs controls and evidence into systems from the start instead of retrofitting them after a finding.

In her first weeks at Northmere, the fictional group this book follows, Dana Whitlock is asked three questions by the CFO, Tomás Ferreira. The first is: **why does the close take nine working days?** The honest answer is not in any single system. It sits in a payments clearing account carrying €14 million of unmatched items, in three charts of accounts mapped through spreadsheets, in about 1,100 interfaces and in a payout engine whose change control drew an audit finding. Before she can say which platform to change, she needs a way to see the estate as one system.

This part provides that view. It describes no product; it sets out the concepts every later chapter relies on, so that a discussion about SAP, Workday, BlackLine or Adyen starts from shared definitions rather than vendor vocabulary.

Three terms carry most of the weight. A **finance estate** is the set of systems, data, integrations and controls through which an enterprise records what it owns, owes, earns and spends, and proves it. A **business capability** is something finance must be able to do, named independently of the system, team or process that does it today. A **subledger** is a detailed record of one area of activity (bookings, payouts, partner balances) whose summarized total must equal a control account in the **general ledger**, the official book of account. Much of this book is about keeping those totals provably equal as volume, brands and platforms multiply.

The chapters move from the architecture practice, through the finance domain and the disciplines that cut across every platform, to how systems are sourced and changed. Experienced finance-systems architects can skim the domain stage; experienced enterprise architects should not skip it, because it is where most architecture mistakes in finance begin.

The Learning Path

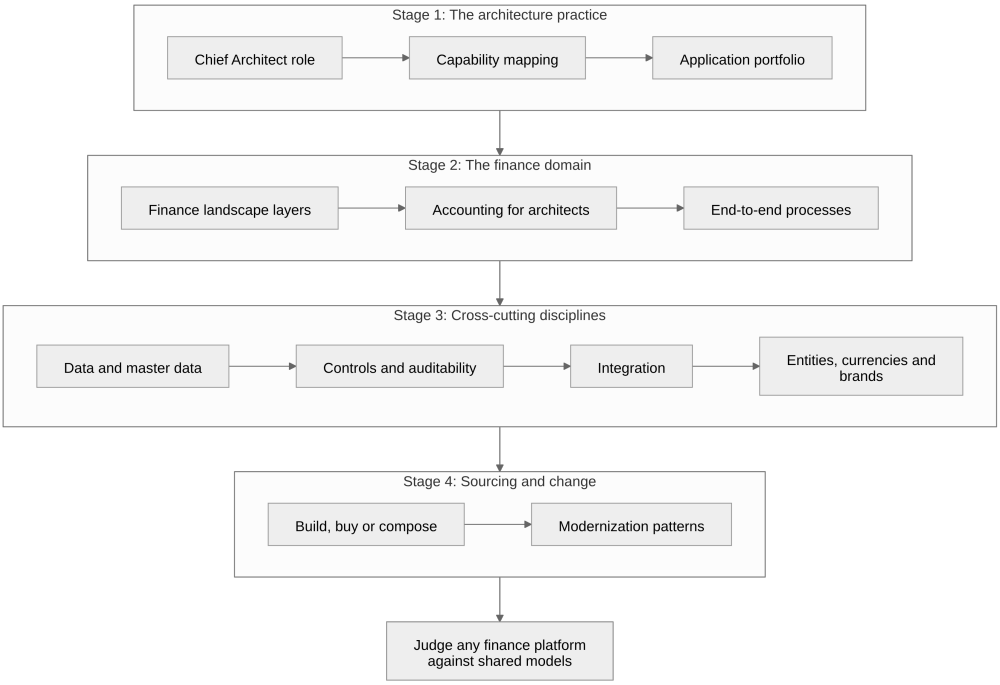


Figure 1: *Twelve chapters in four stages: first the practice, then the domain it serves, then the disciplines every platform must respect, and finally how systems are sourced and replaced.*

The architecture practice.

- The Chief Architect's Job: Making Many Systems Tell One Story — what the architecture function is for, which decisions it owns and how finance changes the job.
- Map What Finance Does Before What Runs It — how to name what finance does independently of systems, heat-map it, and draw bounded contexts.
- Manage the Application Portfolio as an Investment — how to build an inventory that reconciles, cost applications in full and retire them for real.

The finance domain.

- Anatomy of a Finance Estate: Layers, Planes and Boundaries — the layers of a finance estate, the volume gradient between them and the failures that occur at layer boundaries.
- Accounting for Architects: Ledgers, Journals and Revenue — double entry, revenue recognition and principal versus agent, worked through the €600 Lisbon booking.
- Order to Cash, Procure to Pay, Record to Report — the end-to-end processes as chains of hand-offs, including the marketplace variants of Order-to-Cash.

Cross-cutting disciplines.

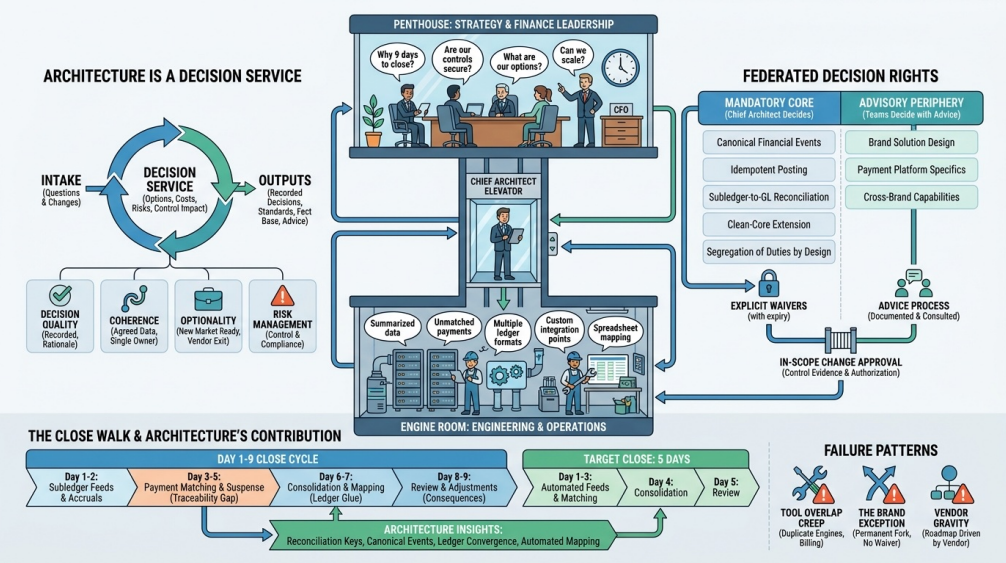
- Finance Data Architecture: Who Owns the Truth — who is authoritative for each master data domain and attribute, and why a mapping table is a control.
- Controls by Design, Not by Audit Season — IT general controls, remediation by architecture and evidence produced by the systems themselves.
- Files, Events and APIs: How Finance Systems Talk — integration styles chosen by job, idempotent postings and why a dead letter is not a suspense account.
- Global by Design: Entities, Currencies, Jurisdictions and Brands — legal entity, brand, market, business model and currency as separate dimensions, and inter-company by design.

Sourcing and change.

- Build, Buy or Compose: Choosing What to Own — sourcing capabilities rather than systems, costed over a decade and selected on the hardest scenarios.
- Changing the Engine in Flight: Modernization Patterns for Finance — choosing a modernization pattern by what must change, with reconciliation as the gate and retrievability as the end.

By the end of this part you should be able to place any finance system in a layer, name the capability it serves and the data it masters, say which controls and integrations it depends on, and frame a sourcing or modernization decision in terms a CFO and an auditor will both accept. You will not yet know how the individual platforms work; that is the next part. Begin with The Chief Architect's Job: Making Many Systems Tell One Story. PLATFORMS: What Each Is For, and What It Costs to Depend On then examines the platforms themselves.

4. The Chief Architect's Job: Making Many Systems Tell One Story



IN THIS CHAPTER, YOU WILL: Define what enterprise architecture is for in a finance estate, choose an operating model and a set of decision rights that fit a federated, multi-brand group, and see how a new Chief Architect turns a CFO's three blunt questions into a 90-day plan.

WHY FINANCE CARES: A CFO does not buy architecture; they buy a shorter close, numbers that tie out, controls that hold during change, and a transformation that actually switches systems off. The architecture function is the part of technology that can explain why those things are hard and which decisions would make them easier.

WHY ARCHITECTS SHOULD CARE: In finance, an architecture decision is often reversible only at the next period boundary, after an audit, or never. An architecture function without explicit decision rights either becomes a bottleneck or is ignored, and in a matrix with brand engineering the second happens quietly.

KEY POINTS:

- **Architecture is a decision service, not a document set**. The function exists to improve the quality, speed and coherence of decisions that are expensive to reverse. Its products are recorded decisions, standards with reference implementations, and a trusted fact base. It should be measured by those decisions' lead time and outcomes, not by how many diagrams it produces.
- **Decision rights must be explicit, and narrow where they are mandatory**. A short list of mandatory standards belongs to the Chief Architect, with expiring waivers. Accounting policy stays with the controller, control design with internal controls, and solution design inside the standards with the brands, under an advice process. For changes to in-scope financial systems, documented approval is a control and does not go away.
- **Finance changes the job**. Correctness, auditability, regulatory deadlines, period-end rhythms and money in flight make finance architecture its own discipline. The Chief Architect rides the elevator between the CFO's question ("why nine days?") and its causes in the engine room, then rides back up with costed options.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Capability Creep** — Duplicate payout engines, billing approaches and reconciliation tools are what accumulates when nobody holds the right to decide which platform owns a capability.
- **The Permanent Exception** — In a matrix without explicit decision rights and expiring waivers, every brand's exception becomes a permanent fork.
- **The Borrowed Roadmap** — An enterprise without an architecture function of its own lets each vendor's roadmap draw part of its target state.

A finance estate is the set of systems, data flows, controls and people that turn a company's operational activity into money moved, books kept and numbers reported. In a multinational, multi-brand group it is rarely designed. It is **accumulated**: an ERP chosen in one decade, a subledger built in another, brand ledgers that arrive with acquisitions, a consolidation tool that papers over the differences, and a few hundred spreadsheets carrying the mappings nobody had time to automate. Each piece was usually a sensible decision when it was made. The sum is rarely sensible.

Enterprise architecture is the discipline that treats that sum as a system. This book takes the view of a **Chief Architect** responsible for enterprise architecture, enterprise applications and financial systems. That person cannot run every platform and does not

own most of the engineers. They are nevertheless accountable for whether the estate holds together: whether two systems that describe the same money agree, whether a new brand can be onboarded without a new ledger, whether the next vendor renewal is a choice or a hostage negotiation.

This chapter sets out the job: what architecture is for in a finance estate, why finance makes it different, how the function is organized, where its decision rights stop, and how to tell whether it is any good. The rest of the book reuses its vocabulary.

Setting the Scene: Dana's First Month at Northmere

Northmere, the fictional multi-brand digital commerce group this book follows, is introduced in full in Meet Northmere: Five Brands, One Set of Books. In 2026 it reported about $8.2 billion of revenue on about $64 billion of gross transaction value. It has 71 legal entities in 38 countries and a New York listing that puts its financial reporting under the Sarbanes-Oxley Act. **Dana Whitlock** joined from a payments company in September 2026 as Chief Architect, Enterprise Applications and Financial Systems. She reports to Ravi Menon, the Group CTO, with a dotted line to Tomás Ferreira, the Group CFO, and leads about 30 architects. Her mandate letter gives her **final decision rights on mandatory enterprise-application standards**. The roughly 60 engineers who run the Stayloft Finance Platform, and the finance-systems teams at Wayfare and Tablekin, do not report to her.

In her first month she found an estate that every chapter of this book will return to:

- **Three ledgers**. SAP S/4HANA is the group finance core; Wayfare's Oracle Fusion Cloud ERP and Tablekin's Workday Financial Management came with acquisitions. The three meet only at consolidation, in OneStream, through chart-of-accounts mappings kept in spreadsheets. The **chart of accounts** is the list of accounts a ledger posts to.
- **A custom subledger doing ledger work**. The Stayloft Finance Platform (SFP) handles commission billing, the merchant **subledger** (the detailed record of individual bookings, payments and partner balances), and partner payouts. It posts about 38,000 summarized general-ledger (GL) lines a day into S/4HANA, condensed from about 3 million subledger lines.
- **Duplicate capabilities**. Two partner-payout engines; three billing approaches; reconciliations in BlackLine, a rarely used OneStream module and Oracle Fusion; indirect tax in Vertex and, for some Wayfare markets, Oracle's tax engine.
- **Plumbing**. About 1,100 interfaces, about 240 of them flat files and about 70 with spreadsheets in the loop; three integration technologies (SAP Integration Suite, Kafka, MuleSoft); 214 finance-relevant applications in SAP LeanIX.
- **Money not yet explained**. In 2026, unreconciled balances in payment-service-provider clearing and suspense accounts averaged €14 million at month-end, of

which about €2.1 million was older than 90 days. A **suspense account** is a holding account for amounts the books have received but cannot yet assign.

- **A control problem on a payout engine**. The external auditor's interim work had found that SFP developers had production access and that payout-rule changes lacked documented approval. It was later reported as a **significant deficiency** in internal control over financial reporting (ICFR) for FY2026: less severe than a material weakness, but important enough to merit the audit committee's attention (Controls by Design, Not by Audit Season).
- **An ERP that resists change**. About 2,300 custom objects in S/4HANA, and upgrades that take seven months.

Program Keystone, the multi-year finance transformation, was then a business case in draft, due to go to the board in December. In her third week Tomás asked her three questions. He said he had asked them before and been given slides.

1. **Why does the close take nine working days?** Competitors talk about four or five.
2. **Why do we have three ledgers?** Was that a decision anyone took, or something that happened to Northmere?
3. **What will Keystone actually change?** The draft case asks for about $310 million over five years. What would he be able to see in 2027 that he cannot see now?

None of the three is a technology question; each is answered by a chain of decisions, mostly unrecorded. That is the Chief Architect's job in a sentence: **make the decisions behind the estate explicit, make the next ones faster and better, and keep them coherent with each other**. Dana's answers come near the end of the chapter, after the tools she needed to produce them.

What Enterprise Architecture Is For

The definition of architecture that has served me best comes from Ralph Johnson, as Martin Fowler retells it: architecture is "the shared understanding that the expert developers have of the system design", and in the end it "is about the important stuff. Whatever that is." (Fowler, Software Architecture Guide). Enterprise architecture applies the same idea one level up. The system is the enterprise's whole application and data estate. The experts are spread across finance, engineering, security and several brands. "The important stuff" means the decisions that are expensive to reverse and that affect more than one team.

I use the following working definition throughout the book. **Enterprise architecture** is the function that makes cross-cutting, hard-to-reverse decisions about an enterprise's capabilities, applications, data and integration explicit, well-informed and coherent with

each other, and that keeps the enterprise able to make the next ones. Frameworks such as The Open Group's TOGAF Standard (10th Edition, published in 2022 and structured as Fundamental Content plus Series Guides) describe methods and content for doing this. They are useful reference material, not the purpose, which in a finance estate comes down to four things.

Purpose	What it means in a finance estate	What it looked like at Northmere in 2026
Decision quality	Decisions are taken with the options, costs, control impact and accounting consequences on the table, by people who hold the right to take them, and the reasoning is recorded	A cross-brand payouts decision had circled for months over "extend SFP" versus "use the Wayfare engine". The rationale for SFP's daily summarization level was nowhere written down
Coherence	Systems that describe the same money agree, or there is a designed way to prove that they agree. Each capability has one owner platform, or a deliberate reason for two	Three charts of accounts mapped in spreadsheets; two payout engines; reconciliation done in three tools
Optionality	The estate can absorb a new brand, market, regulation or vendor change without a rebuild. Exits from vendors are possible, if not cheap	Brazil (2028) would need a new entity, local payment methods and e-invoicing. Every vendor relationship was renewed by default
Risk	Architecture choices do not create control failures, regulatory breaches or concentration on one supplier that nobody chose	The SFP control finding; 70 interfaces with spreadsheets in the loop; three integration technologies with different error handling

Optionality deserves a word of caution, because it is the purpose most easily turned into a slogan. Gregor Hohpe describes architecture as selling options: the right, but not the obligation, to take a decision later, whose value rises with uncertainty (Architecture: Selling Options). A finance estate faces plenty of uncertainty: EU VAT in the Digital Age (ViDA), Pillar Two, the next payment-services regulation, the next acquisition. But an

option costs something to hold. A canonical event layer, an anticorruption layer in front of the ERP, or an exportable data model each has a price paid now. The rule I use is to buy an option only when I can name the decision it defers and roughly when that decision will fall due. "Keeping our options open" with no named decision is gold-plating.

Why Finance Architecture Is Its Own Discipline

Five traits of the finance domain change how familiar architecture techniques are weighed.

Trait	What it means	Architectural consequence	Where the book goes deeper
Correctness is binary at the boundary	A balance is right or it is wrong. "Eventually consistent" is acceptable inside a pipeline, never in the posted ledger at period end	Delivery guarantees are designed as at-least-once delivery plus idempotent effects plus reconciliation, never as a property bought from middleware	Files, Events and APIs: How Finance Systems Talk, Never Update, Only Append: Designing Custom Ledgers
Auditability	Every reported number must trace to the transactions behind it, and every change to a system that produces it must be authorized and evidenced	Audit trails, segregation of duties and change approval are design inputs, not retrofits. The auditor is a stakeholder in design reviews of in-scope systems	Controls by Design, Not by Audit Season, Reconciled by Design: Traceability and Audit Across Boundaries

Trait	What it means	Architectural consequence	Where the book goes deeper
Regulatory deadlines	Filing dates, e-invoicing mandates and tax reporting go-lives are external and fixed	The roadmap has immovable dates that technology did not choose, and architecture must leave room for them	Global by Design: Entities, Currencies, Jurisdictions and Brands, The Middle Matters: Transition States and the Roadmap
Period-end rhythm	Load, risk and organizational attention spike at month, quarter and year end	Change freezes, peak-capacity engineering and cut-overs aligned to period boundaries. A design that works on day 15 can fail on working day 2	Order to Cash, Procure to Pay, Record to Report, Recover Before the Close: Resilience and Security for Finance
Money in flight	At any moment, funds are between customer, payment service provider, bank, safeguarding account and partner, and some of them belong to someone else	The estate must show where every euro is and whose it is. For a licensed e-money institution, safeguarding reconciliation is a daily regulatory obligation	Payments-to-Ledger: Draining the Suspense Swamp, Payments for Finance Architects: Eleven Reasons the Numbers Disagree

These traits give finance architecture a different temperament from product architecture. The cost of a wrong decision often arrives months later, at an audit, a filing or a close, and lands on people who were not in the room. That delay is why decision quality and a written rationale matter more here than in most domains, and why finance teams can seem conservative to engineers: usually they are pricing a risk the engineers have not yet seen.

Architecture as a Function and a Service

Many enterprise architecture teams are judged by what they publish: a framework, a metamodel, current-state diagrams, a principles poster. They are easy to fund in year one and easy to cut in year three, because nothing changes when the documents improve. The alternative is to run architecture as a **function that delivers services**. The function has customers (finance leadership, brand engineering, the transformation program, internal controls), a small catalog of things it does for them, and service levels it can be held to.

Dana's catalog, as she drafted it in her second month, had six services.

1. **Decision service**. Intake of decisions that cross teams or touch mandatory standards; options with cost, risk, control impact and accounting consequences; a recorded decision with an owner and a review date; a published lead-time target. The mechanics of decision records and review forums belong to Architecture Governance That Speeds Decisions.
2. **Standards and patterns**. A short catalog of mandatory standards (for example idempotent posting, subledger-to-general-ledger reconciliation, clean-core extension of the ERP, segregation of duties by design) and a longer catalog of recommended patterns, each with a **reference implementation** a brand team can copy. Principles That Decide Things covers how to write them.
3. **Portfolio fact base**. The application inventory with owners, lifecycle, capability, cost, contract dates and audit relevance, kept accurate enough that people stop maintaining private copies (Manage the Application Portfolio as an Investment).
4. **Target state and dependency map**. Where the estate is going, the stable transition states on the way, and the cross-program dependencies that will block it if nobody owns them (The Middle Matters: Transition States and the Roadmap).
5. **Design authority for transformation**. For Keystone, a standing forum that rules on design questions with the authority of the program sponsor. Dana chairs it with Leila Haddad, the Chief Accounting Officer (The Technology Is the Easy Part: Leading a Finance Transformation owns the term).
6. **Embedded architecture**. Solution and integration architects who work inside brand and platform teams on their hardest problems. This is the service that earns the trust the other five spend.

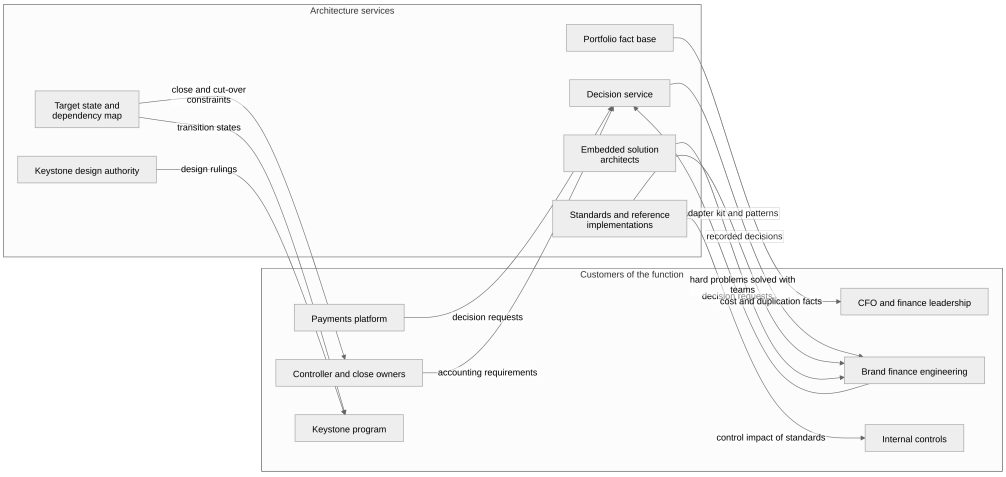


Figure 1: *The architecture function as a set of services with named customers. Each arrow is something a customer would notice if it stopped. A document set has readers; a function has customers who can complain.*

The service framing has two consequences that are easy to state and hard to keep. First, **every service needs a measure a customer would recognize**: lead time for decisions, the share of brand integrations built on the reference implementation, the accuracy of the portfolio when Finance uses it for a budget. Second, **the function must say no to work that has no customer**. A full current-state model of 214 applications to the level of every interface field is an example. It has no customer until a specific decision needs it.

Artifacts that earn their keep, and shelfware

Architecture produces artifacts, and some of them are indispensable. The test I apply is whether **someone outside the architecture team would notice within a quarter if the artifact stopped being maintained**. If nobody would notice, the artifact is shelfware: kept for its own sake, read by its authors.

Artifact	Earns its keep when	Becomes shelfware when	Northmere, December 2026
Decision records	Short, numbered, linked to the change they authorize, with options and consequences; searched when a decision is questioned	They are written after the fact to satisfy a gate, or run to twenty pages	None existed; Dana reconstructed eleven decisions from email and steering minutes
Mandatory standards with reference implementations	A brand team can copy the code, schema or test harness and pass conformance on the first attempt	They are prose rules with no example, or there are fifty of them	Proposed five; none yet in force
Application portfolio (LeanIX)	Finance uses it for budgets and contract renewals; owners update it because it drives their cost allocation	Architects update it once a year before an audit of the repository itself	214 applications, owners missing for about a fifth (Dana's estimate)
Capability map	It is used to find overlap and to assign each capability one owner platform	It is a poster with no applications mapped to it	Planned for Q1 2027 (Map What Finance Does Before What Runs It)
Systems-of-record matrix	It settles disputes about which system is authoritative for a legal entity, a partner or a rate	It lists systems without the data objects or the authority	Planned for Q1 2027 (Systems of Record: Ending the Doppelgänger Record)
Target and transition states	Each transition state is stable, fundable and shows what is switched off	It shows only the end state, with no route to it	Keystone draft showed year-five end state only

The pattern: **an artifact earns its keep when a decision depends on it**. Dana's rule was that no artifact is created unless a decision, a control or a program gate needs it, and whoever owns that dependency maintains it.

One caution about the repository itself. Northmere's portfolio lives in SAP LeanIX, which SAP acquired in November 2023. That says nothing against the product. It does mean that the enterprise's fact base about its SAP estate sits in a tool owned by the ERP vendor whose estate is being rationalized. Dana's response was not to replace it. She made sure the data could be exported on a schedule into the finance data platform, so the facts did not depend on the tool. This is a small, cheap instance of resisting **The Borrowed Roadmap**.

Operating Models: Centralized, Federated and Advisory

An **operating model** for architecture answers three questions: where the architects sit, who decides what, and how decisions are made across team boundaries. Three models recur, and most organizations blend them.

- **Centralized**. One function designs and approves solutions; standards are many and enforced through review.
- **Federated**. A small central team owns enterprise standards, the portfolio and the target state; architects sit in brand and platform teams with a professional line to the center.
- **Advisory**. The center offers guidance and patterns, but teams decide. Andrew Harmel-Law's **advice process** is the most developed version. Anyone may make an architectural decision, provided they first seek advice from everyone meaningfully affected and from people with relevant expertise, and record the advice they received, whether or not they followed it. The process is supported by decision records, a weekly open Architecture Advisory Forum, team-sourced principles and a technology radar (Harmel-Law, Scaling the Practice of Architecture, Conversationally).

	Centralized	Federated	Advisory
Who decides	The central function	Central team for enterprise standards; domain teams for the rest	The team doing the work, after seeking advice
Where architects sit	In one team, assigned to projects	Central core plus architects embedded in brands and platforms	Small central team; architecture is a skill spread across engineers
Coherence across the estate	High while the center keeps up	High on the few things standardized, variable elsewhere	Depends on culture and the quality of advice
Decision speed	Slow at scale; the center becomes the queue	Fast for local decisions; enterprise decisions need a forum with a lead-time target	Fastest; costs show up later as divergence
Brand buy-in	Low; architecture is something done to the brand	Medium to high, if standards come with reference implementations	High
Fit with financial controls	Good on paper; reviews create approval evidence	Good if the control gate for in-scope systems is explicit	Weak on its own. In-scope changes still need documented approval
Typical failure	Ivory tower and bottleneck; teams route around it	Central standards creep until it is centralized again; or the core is too thin to hold anything	**The Permanent Exception** at scale; nobody can say no

	Centralized	Federated	Advisory
When it is right	Small estates, or a short crisis period such as remediating a control failure	Multi-brand groups with a shared finance core: most of the readers of this book	High-trust engineering organizations with weak coupling between teams and little regulated scope

For a group like Northmere I recommend **federated, with a thin mandatory core and an advisory periphery**. The mandatory core is a short list of standards that protect the whole: how money events are published, how subledgers reconcile to the ledger, how the ERP is extended, how access is separated. Within that core, the Chief Architect decides. Outside it, teams decide using an advice process, and the architecture function competes for influence on the quality of its advice.

There is one finance-specific caveat, and I would not soften it. **The advice process does not remove documented change approval for systems in ICFR scope**. A brand team may decide its own design through advice. A change to a system that feeds the financial statements still needs authorization and evidence that satisfies the auditor, and that is what Northmere's SFP finding was about. The credible model runs advice for design and keeps a thin, explicit control gate for in-scope changes, owned jointly with internal controls. Where this recommendation does not hold: during the remediation of a significant deficiency, I would run the affected platform under a temporarily centralized model, with every change reviewed, and say in writing when that will end.

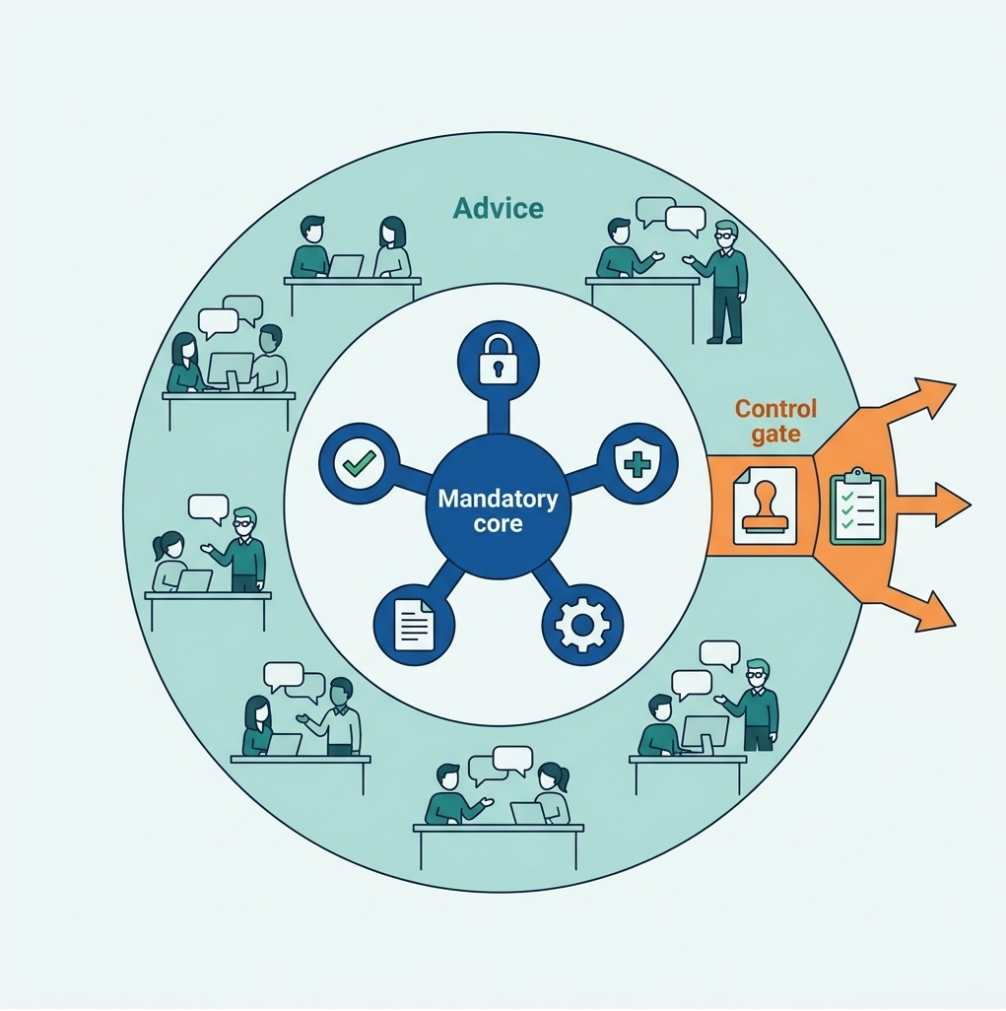


Figure 2: *Federated with a thin mandatory core: the Chief Architect decides a short list of standards, teams decide the rest through advice, and every change to an in-scope system still passes the control gate.*

Decision Rights: Mandatory Standards and Advice

Decision rights state who may take which decision, who must be consulted before it is taken, who can block a specific aspect of it, and who is only informed. An architecture function without them has two possible fates. It can approve everything to stay popular, or it can claim everything and become the queue. Both are common.

Dana's mandate gave her one decision right outright: final decisions on mandatory enterprise-application standards. Her first deliverable was to make that right precise. She defined what counts as a mandatory standard and wrote down whose rights border

hers. The table below is the version she agreed with her peers in November 2026. It uses D for decides, R for recommends, C for consulted (must be heard before the decision), S for sign-off on one specific aspect, and I for informed.

Decision	Dana (Chief Architect)	Leila (CAO and Group Controller)	Ravi (Group CTO)	Brand CTOs	Ingrid (MD, Northmere Pay)	Mei (Head of Internal Controls)
Mandatory enterprise-application standards (canonical financial events, idempotent posting, subledger-to-GL reconciliation, clean-core extension, segregation of duties by design)	**D**	C	C	C	C	S on control impact

Decision	Dana (Chief Architect)	Leila (CAO and Group Controller)	Ravi (Group CTO)	Brand CTOs	Ingrid (MD, Northmere Pay)	Mei (Head of Internal Controls)
Waivers from a mandatory standard, always with an expiry date	**D**	C if accounting is affected	I	R (requester)	C if payments are affected	S if the system is in ICFR scope
Accounting policy, chart-of-accounts design, posting logic meaning	C	**D**	I	I	C	C
Engineering platforms: event backbone, cloud, identity	C	I	**D**	C	C	I
Brand solution design inside the standards	C (advice)	C where it touches posting	I	**D**	C where it touches payments	C where in ICFR scope

Decision	Dana (Chief Architect)	Leila (CAO and Group Controller)	Ravi (Group CTO)	Brand CTOs	Ingrid (MD, Northmere Pay)	Mei (Head of Internal Controls)
Payments platform design: PSP routing, safeguarding, payment methods	C on the payments-to-ledger contract	C	C	I	**D**	C
Whether a change is SOX-relevant, and the evidence it needs	C	C	I	I	I	**D**
Cross-brand capability ownership, such as which payout engine survives	**R**, decided at the Keystone design authority with Leila	**R**	C	C	C	C

Decision	Dana (Chief Architect)	Leila (CAO and Group Controller)	Ravi (Group CTO)	Brand CTOs	Ingrid (MD, Northmere Pay)	Mei (Head of Internal Controls)
Investment-level choices, such as ledger convergence by Central Finance or an accounting hub	**R** with Leila	**R** with Dana	C, co-approves with Tomás	C	C	C

The **brand CTOs** in the table are the technology heads of Stayloft, Wayfare and Tablekin. Their finance-systems leads, Jonas Berg, Hana Kowalski and Arjun Rao, are the people Dana works with day to day. Escalations go to Tomás and Ravi jointly. Northmere's CFO and CTO had agreed to resolve any escalated architecture dispute within ten working days, and to say so publicly. That agreement mattered more than any line in the table.

Three properties make a decision-rights table like this one work.

- **The mandatory list is short and closed.** Five standards, each with a written scope. (In 2027 they became STD-FIN-01 to STD-FIN-05, declared mandatory from 1 April 2027, with the canonical-events and idempotent-posting standards enforced from 1 July 2027 when the schema registry went live; a sixth, STD-FIN-06 for AI agents, followed in Q3 2027. Principles That Decide Things holds the catalog.) Anything not on the list is advisory. Adding a standard is itself a recorded decision with consultation, so the list cannot grow by stealth.
- **Every right borders another right explicitly.** Dana decides *how* posting is made idempotent and reconcilable. Leila decides *what* the posting means. Mei decides *whether* a change needs control evidence. The table prevents the most common finance-architecture dispute, an architect redesigning accounting or a controller redesigning integration, by naming the border.
- **Waivers expire.** A waiver without an end date is a fork with paperwork. Every Northmere waiver records the standard, the reason, the compensating control if the system is in ICFR scope, the owner and the expiry. Expired waivers are reported to the design authority.

Decisions are also routed by **materiality**, so that most never reach a forum.

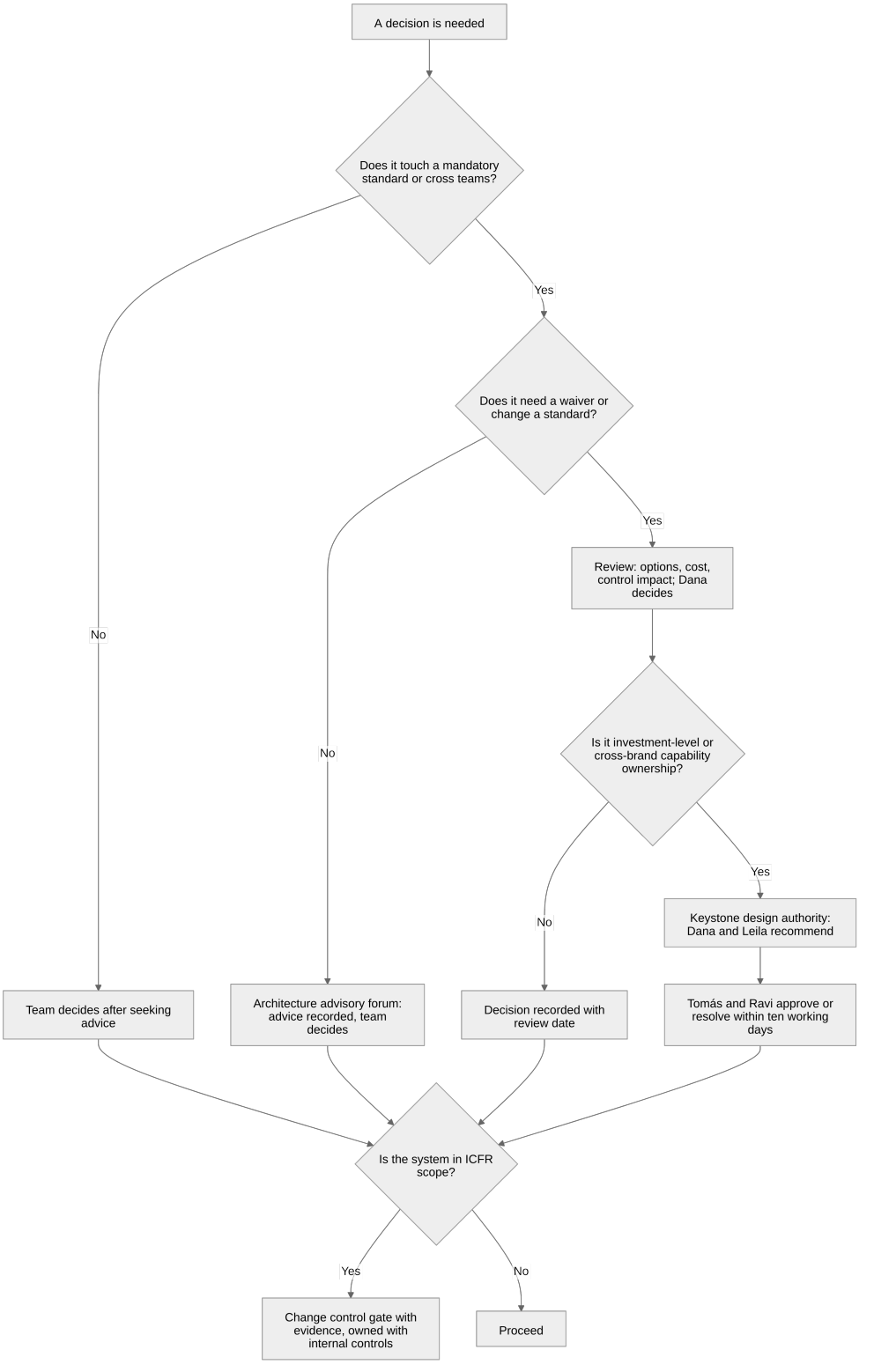

A decision is needed
Does it touch a mandatory standard or cross teams?
Yes
No
Does it need a waiver or change a standard?
Yes
No
Review: options, cost, control impact; Dana decides
Is it investment-level or cross-brand capability ownership?
Yes
No
Keystone design authority: Dana and Leila recommend
Team decides after seeking advice
Architecture advisory forum: advice recorded, team decides
Decision recorded with review date
Tomás and Ravi approve or resolve within ten working days
Is the system in ICFR scope?
Yes
No
Change control gate with evidence, owned with internal controls
Proceed

Figure 3: *Decisions route by materiality, and most never reach a forum. Every path converges on the same control gate for systems in ICFR scope: advice can replace approval for design, never for change evidence.*

The mechanics behind this diagram are owned by Architecture Governance That Speeds Decisions: what a decision record contains, how the review board and the advice forum differ, how waivers are tracked, and how conformance is checked. This chapter's point is narrower. **Decision rights come first, and forums second.** A review board set up before the rights are agreed becomes a debating society, or a court with no jurisdiction.

The Matrix with Brand Finance Engineering

The hardest part of the Chief Architect's job at a group like Northmere is not technical. Dana is accountable for the coherence of an estate whose largest custom component, SFP, is run by about 60 engineers who report to Jonas Berg and, through him, to Stayloft. Wayfare's Oracle Fusion and payout engine belong to Hana Kowalski. Tablekin's Workday Financials and Stripe integration belong to Arjun Rao. This is a **matrix**: brand engineering reports to the brands, and takes standards and architectural direction from Dana.

A matrix works when each side gets something it cannot get alone. Brand finance teams commonly want stable contracts, fast decisions and not rebuilding ledger, tax or reconciliation logic; they fear a central mandate that freezes their roadmap. Architecture needs them to adopt standards, retire duplicates and surface problems early. Dana made her side of the bargain explicit.

What Dana asks of the brands	What Dana owes the brands in return
Publish financial events in the canonical format once it exists, and stop building new bespoke feeds	An adapter kit with schemas, mappings and a test harness, maintained by her integration architects under Lucía Romero
Bring cross-team decisions to the decision service early	A published decision lead time, ten working days for standard requests, and a named architect per request
Accept that the five mandatory standards apply to them	No new mandatory standard without consultation and a reference implementation; no change mid-program without a transition period

What Dana asks of the brands	What Dana owes the brands in return
Retire duplicate capabilities on an agreed schedule	A seat in the decision about which platform survives, and migration support from the central team, not just a deadline
Keep the portfolio data for their systems accurate	Portfolio data that pays for itself in cost allocation and contract renewals

Two things make the matrix fail. The first is **the Chief Architect without a budget**: standards that each brand must fund from its own roadmap will lose to the brand's roadmap every time. Keystone solved part of this for Dana by funding the adapter kit and the first migrations centrally. The second is **the Chief Architect as messenger**, whose authority depends on escalating to the CFO. Each escalation spends trust; Dana's rule was at most one a quarter.

Team design across this matrix, including stream-aligned brand teams, platform teams for the ledger and integration, and enabling architecture, is owned by Who Owns What: Organizing Architecture Across Finance and Technology. Here the point is that the matrix is a contract, and contracts need consideration on both sides.

Riding the Architect Elevator

Gregor Hohpe's image for the architect's position is an elevator. The architect connects the **penthouse**, where business strategy is set, with the **engine room**, where the enabling technologies are built and run, and rides between the floors (The Software Architect Elevator). An architect who stays in the engine room produces designs nobody funds; one who stays in the penthouse produces strategies nobody can implement. The value is in the ride.

In finance, the penthouse speaks of days to close and audit findings, the engine room of summarization levels and settlement file formats. Between them sits a floor technology-side architects underrate: the controller's floor, where accounting policy, close calendars and reconciliations live. Dana answered Tomás's first question by riding all the way down and back up.

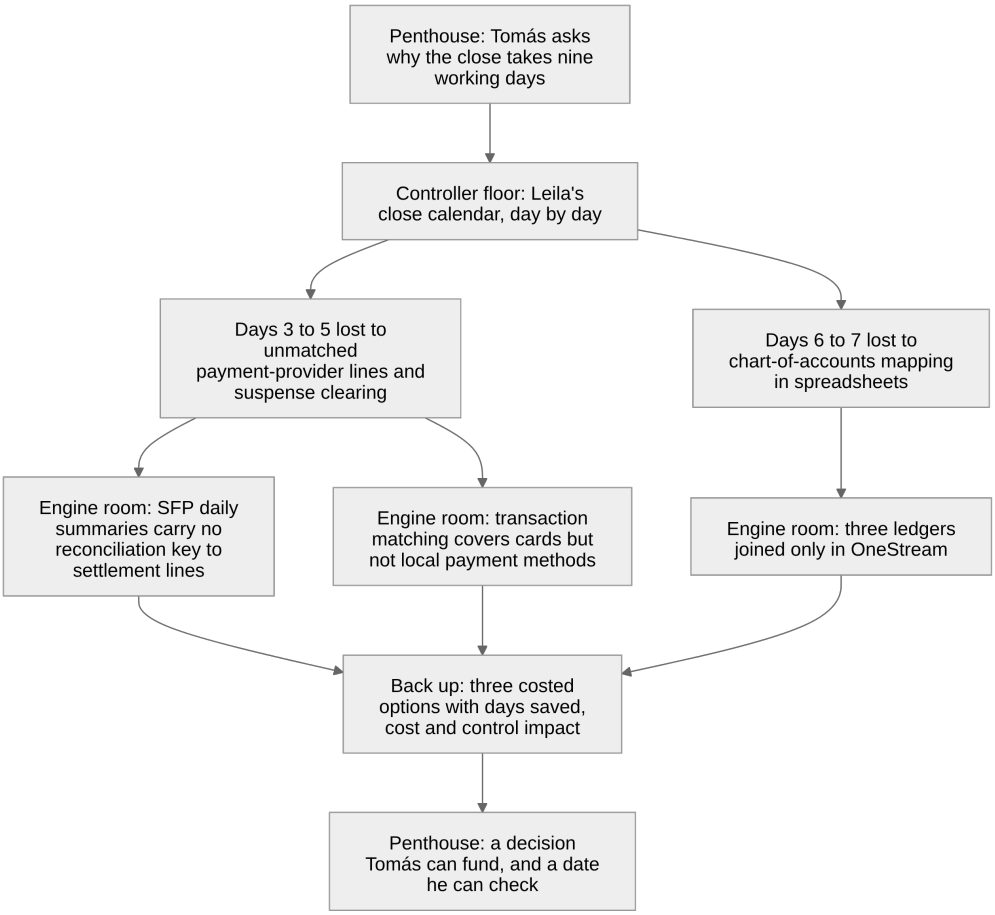


Figure 4: *The elevator ride behind the CFO's first question. The question arrives in the penthouse in days; the causes are found in the engine room as missing keys and missing coverage; the answer goes back up as options priced in days, money and control risk.*

The close walk

Dana did not start with the systems. She asked Leila to let her sit in on the September 2026 quarter close from first working day to last, with each day's critical-path task written down. The result, simplified to the critical path, she called the close walk.

Working day	Critical-path activity	Why it takes that long
1–2	Subledger feeds land in S/4HANA, Oracle Fusion and Workday; accruals booked	SFP's daily summaries are complete only after a final run; late feeds are re-run by hand

Working day	Critical-path activity	Why it takes that long
3–5	Payment-service-provider settlement matched to SFP and to bank; suspense and clearing accounts worked down; intercompany agreed	About 1.9% of settlement lines for Stayloft's local payment methods are unmatched at working day 3; each break is investigated by hand, because summarized SFP postings cannot be traced to settlement lines
6–7	Brand trial balances loaded into OneStream; chart-of-accounts mapping; consolidation and eliminations	Three charts of accounts; mappings maintained in spreadsheets; mapping errors found during load are fixed and reloaded
8–9	Review, top-side adjustments, sign-offs	Adjustments driven by the residual differences of days 3–7

The walk changed the conversation. Nine days was not one problem. It was three, each with a different owner and a different cure. **Days 3 to 5 were a traceability problem**: SFP summarized before posting and kept no key from summary to settlement line, so every break had to be investigated by reconstructing the detail. This is **Lost in Aggregation**, and Reconciled by Design: Traceability and Audit Across Boundaries treats its cure. **Days 6 to 7 were a coherence problem**: three ledgers and spreadsheet mappings, which is **Spreadsheet Glue** on top of an unmade decision about ledger convergence. **Days 8 to 9 were mostly consequences** of the first two, and would shrink as they did.

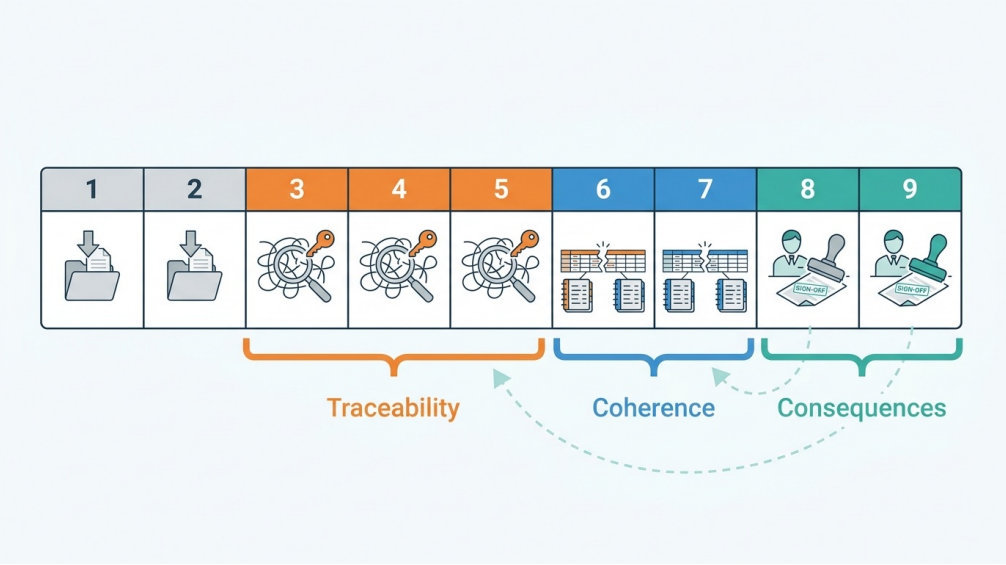


Figure 5: *The close walk split nine days into three problems with different owners: days 3 to 5 lost to traceability, days 6 to 7 to coherence across three ledgers, and days 8 to 9 to the consequences of both.*

Dana did not answer "nine days" with "we need a new ERP", the answer vendors and integrators are commercially inclined to give. Nor did she claim architecture alone would fix the close: the calendar is Leila's, and some of the time is policy and people. Her answer was a contribution, in days, with its assumptions shown.

Dana's Answers to the Three Questions

Dana answered Tomás in writing in mid-November 2026. Each answer was one page, and the three summarized to a table that went into the Keystone board pack.

Question	Short answer	Decision it leads to	What Tomás can check in 2027
Why does the close take nine working days?	Three causes: payments that cannot be traced through summarized postings (about three days), three charts of accounts joined by spreadsheets (about two days), and the review work those two create. The ERP is not the bottleneck	Payments-to-ledger completeness for cards and local payment methods in 2027, with reconciliation keys from SFP summaries to settlement lines; one group chart of accounts planned for 2029	Unmatched settlement lines at working day 3, and the €14 million average suspense balance, reported monthly with targets
Why do we have three ledgers?	Because two acquisitions each brought a working ledger and integrating them at consolidation was the cheapest way to report on day one. That was reasonable in 2023 and 2024. Nobody then decided whether it was the end state, so it became one by default	A recorded decision in Q4 2027 on convergence: SAP Central Finance versus an accounting-hub pattern, with options, costs and control impact; until then, no new ledger anywhere in the group	The decision record itself, dated, with the options and the reasons

Question	Short answer	Decision it leads to	What Tomás can check in 2027
What will Keystone actually change?	In 2027, governance and evidence rather than new platforms: one decision record and review forum, five mandatory standards, payments-to-ledger completeness, a published target state, a portfolio cost baseline, and remediation of the SFP finding. Larger changes follow in later years, and each is tied to a transition state that switches something off	Keystone's year-one scope and the design authority's charter	A quarterly Keystone report on decisions taken and their lead time, systems retired, suspense balances and close days, each against a target

The second answer is the one I would point any new Chief Architect to. "**Three ledgers" was not a mistake. It was a decision deferred until deferral became the decision**. Integrating acquisitions at consolidation level is a common and defensible pattern. It goes wrong when nobody records it as a transition state with a review date. The Chief Architect's contribution is to put a date on the decision, not to decide on day 40 something that needs a year of evidence. Northmere's ledger convergence decision is examined throughout Part III, in Converge or Coexist: Living with More Than One ERP and Systems of Record: Ending the Doppelgänger Record.

Measuring the Architecture Function

A service should be measured as one. The most visible outputs (documents, reviews, diagrams) are the least meaningful, and the meaningful outcomes, such as a shorter close

or fewer duplicate systems, have several causes. A credible scorecard mixes measures the function controls with outcomes it contributes to, and says which is which.

Measure	Definition	Northmere baseline, Q4 2026	2027 target	How it gets gamed	Counterweight
Decision lead time	Working days from when a decision is first raised to when it is recorded, by materiality tier	Median 41 working days across 11 reconstructed decisions	10 days for standard requests; 25 for design-authority decisions	Starting the clock at formal intake, not when the question was first raised; refusing intake	Measure from first raised; report rejected and deferred intakes
Reopened decisions	Share of recorded decisions reopened within 12 months without new facts	3 of 11	Under 10%	Never revisiting decisions that should be revisited	Distinguish reopening on new facts (healthy) from reopening on lost rationale (not)
Reuse of standard patterns	Share of new or changed financial-event feeds built on the canonical events and adapter kit	0% (no canonical events existed)	First two brand feeds	Counting thin wrappers around bespoke feeds as adoption	Count only feeds that pass the conformance harness

Measure	Definition	Northmere baseline, Q4 2026	2027 target	How it gets gamed	Counterweight
Duplicate capabilities retired	Duplicates switched off, with contract ended and a read path archived where retention requires	Baseline list: two payout engines, three billing approaches, three reconciliation tools, two tax engines	Retirement plan agreed for each; OneStream reconciliation module retired	Declaring retirement while the system still runs "for reference"	Retired means no transactions, no license, and an archive with an audit read path
Waivers	Active waivers, and the share closed by their expiry date	No standards, so none	All waivers dated; over 80% closed on time	Zero waivers, which usually means the standards are trivial or not enforced	Report waivers next to conformance results
Close-days contribution	Working days removed from the close that trace to architecture decisions, agreed with the controller	Close at 9 days	Contribution to the group target of 5 days by end of 2028	Claiming the whole improvement	Leila owns the close metric; architecture reports its contribution with assumptions

Three cautions about vanity metrics. **Counting decision records** rewards writing them, not deciding. **Counting reviews** rewards a bottleneck. **Conformance percentages** without the standard's difficulty attached reward easy standards. I have seen architecture functions report 95% conformance to standards that no team would ever have broken. The strongest single measure, in my experience, is decision lead time with

reopened decisions next to it, because the pair is hard to improve without actually deciding faster and better.

Dana deliberately left a portfolio-wide application count off the year-one scorecard: it would have rewarded retiring easy, cheap applications first (Manage the Application Portfolio as an Investment).

Integration with the Rest of the Estate and the Book

Coherence is the job, so the role touches every chapter. Four dependencies matter from the start. Decision rights over which platform owns a capability presuppose a capability map (Map What Finance Does Before What Runs It). Duplication claims and retirement plans need the portfolio as their fact base (Manage the Application Portfolio as an Investment). Internal controls co-owns any change to an in-scope system (Controls by Design, Not by Audit Season). And architects must understand double entry, subledgers and revenue recognition well enough to know where their own decision rights end (Accounting for Architects: Ledgers, Journals and Revenue). Part IV builds the governance mechanics, the standards and the organization on the decision rights set out here.

Failure Modes

Architecture functions fail in recognizable ways. Most of these are failures of decision rights or of the service model, not of technical judgment.

- **The ivory tower**. The function produces a target state and a principles set that brand teams never read, because none of it arrived with a reference implementation or a funded migration. Symptom: the same integration built three times while the target state says it should exist once.
- **Architecture police**. Every change needs a review, reviews take weeks, and teams learn to describe changes so they fall below the threshold. Symptom: nobody measures decision lead time, and teams talk about "getting through architecture".
- **The Permanent Exception by waiver**. Waivers are granted without expiry or compensating controls. Five years later the "mandatory" standard applies to two of five brands, and nobody remembers why.
- **The vendor's architect**. The target state is the vendor's reference architecture with the enterprise's logo on it, and the roadmap tracks the vendor's release schedule. This is **The Borrowed Roadmap**, and the symptom is a target state that contains no system the vendor does not sell (Own Your Roadmap: Vendor Strategy Without Lock-In Panic).

- **Program capture**. The transformation program absorbs all architecture capacity, decisions are optimized for its milestones, and the run estate drifts. Symptom: a design authority that rules only on program scope.
- **Accountability without authority**. The Chief Architect is blamed for incoherence they had no right to prevent, such as a brand's own reconciliation tool. The cure is the decision-rights table, agreed before the next incident.

The Chief Architect's View

The role, as this book uses it, is accountable for four things and deliberately not accountable for several others.

Accountable for:

1. **The coherence of the estate**. Each capability has an owner platform or a recorded reason for two. Systems that describe the same money have a designed way to agree. The target state and its transition states are current, funded and believed.
2. **The quality and speed of architecture decisions**. Including those the Chief Architect does not take: the function answers for its advice and process, not only its rulings.
3. **The mandatory standards**. A short list, each with a reference implementation, conformance checks and an honest waiver register.
4. **The enterprise's options**. Vendor exits that are possible, data and events that are portable, and new brands and markets that can be onboarded without a new ledger.

Not accountable for: accounting policy (the controller), control effectiveness (internal controls and management's ICFR assessment), payments-platform product decisions, brand roadmaps, or program delivery. The Chief Architect contributes to each, as the decision-rights table specifies.

What to decide first. The decision-rights table, because every later argument depends on it; the mandatory standards, the only lever that does not depend on persuasion; and dates for the large deferred decisions, because a decision with a date stops being a default.

What to watch. Decision lead time and reopened decisions; the age of the oldest waiver; whether brand teams bring problems early or late; and whether the CFO's questions are getting more specific. When Tomás stopped asking "why nine days?" and started asking "what happens to working day 4 if we turn on matching for local payment methods in June?", the function had started to work.

Dana's First 90 Days

Dana's plan, as she wrote it in her first week and revised twice, ran from mid-September to mid-December 2026. It ended as the board approved Keystone, so year one could start with rights, standards and a baseline already agreed.

Window	Focus	What Dana did	Outputs	With whom
Days 1–30 (mid-Sept to mid-Oct 2026)	Listen and establish the facts	One-to-ones with Tomás, Leila, Ravi, Ingrid, Mei, Kofi and the three brand finance-systems leads; sat in on the September quarter close end to end; reconstructed eleven recent cross-team decisions from email and minutes; sampled LeanIX entries against reality	The close walk; the decision reconstruction (median 41 working days, three reopened); a list of open and deferred decisions; the duplicate-capability list	Leila's close team; Jonas, Hana and Arjun; her own architects

Window	Focus	What Dana did	Outputs	With whom
Days 31–60 (mid-Oct to mid-Nov)	Answer the questions and propose the rights	Wrote one-page answers to Tomás's three questions; drafted the decision-rights table and negotiated it peer by peer; proposed five mandatory standards; worked with Mei and Jonas on the design side of the SFP access and change-approval remediation; reshaped the Keystone year-one scope around governance and payments-to-ledger completeness	Answers to the three questions; agreed decision-rights table; draft standards; Keystone year-one scope	Tomás, Leila, Ravi, Mei, Ingrid, brand CTOs

Window	Focus	What Dana did	Outputs	With whom
Days 61–90 (mid-Nov to mid-Dec)	Commit and baseline	Chartered the Keystone design authority with Leila; issued the first two decisions under the new rights (the change-approval standard for SFP payout rules, and the scope of payments-to-ledger matching for local payment methods); set baselines for the function's scorecard; planned the Q1 2027 launch of the decision record, the review forum and the capability map	Design authority charter; first two recorded decisions; scorecard baseline; Q1 2027 plan	Leila, Mei, Ingrid, Lucía, the Keystone program office

Three choices in the plan are worth naming. **She sat in on the close before proposing anything**, which earned her credibility on the controller's floor and gave her evidence that slides could not. **She took her first two decisions on problems Finance already felt**, a control finding and a reconciliation gap, not on an architecture topic like integration technology, where the benefit would have been invisible to Tomás. **She**

did not try to fix the three integration technologies, the 2,300 custom objects or the ledger question in 90 days. She gave each of them an owner and a date. The first quarter establishes the right to decide; decisions that bind for years come after.

Questions to Ask

1. Which decisions in our finance estate are expensive to reverse, and who holds the right to take each of them today?
2. What is on our list of mandatory standards, how long is it, and does each standard have a reference implementation a team can copy?
3. How many active waivers do we have, and how many have no expiry date?
4. How long does a cross-team architecture decision take from first raised to recorded, and how many get reopened?
5. Where do the days of our close go, day by day, and which of them are caused by architecture rather than policy or people?
6. Which capabilities do we run in more than one platform, and is each duplication a recorded decision or an accident?
7. For systems in ICFR scope, where does our architecture decision process end and our change-control evidence begin?
8. Which of our large decisions are being deferred, and by what date will each be taken?

From Accountability to a Map

The Chief Architect's accountabilities all depend on one thing Dana did not yet have in December 2026: a shared description of what Northmere's finance function must be able to do, independent of which system does it. Without that description, "which payout engine survives" is a contest between two teams' attachments to their platforms. With it, the question becomes which platform best serves a capability whose requirements everyone has agreed. The next chapter, Map What Finance Does Before What Runs It, builds that description for Northmere and uses it to expose four overlapping capabilities that the landscape view alone hid.

Sources and Evidence

Vendor documentation

- SAP completes acquisition of LeanIX, SAP News Center, 2023. Supports the ownership and date of the EA repository vendor and the independence caution.

Standards and regulation

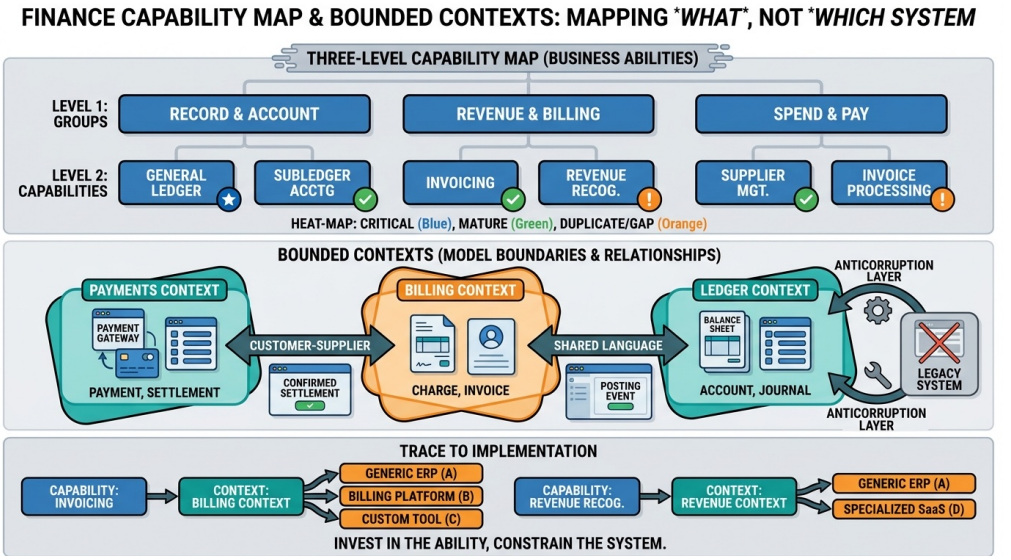
- The TOGAF Standard, The Open Group, 10th Edition, 2022. Reference framework for enterprise architecture methods. The edition's date and its split into Fundamental Content and Series Guides are widely reported; this chapter relies on the framework only as background.

Practitioner and research sources

- Software Architecture Guide, Martin Fowler, martinfowler.com. Ralph Johnson's definition of architecture as shared understanding and "the important stuff"; the IEEE Software column "Who Needs an Architect?" is linked there.
- The Software Architect Elevator, Gregor Hohpe, O'Reilly. The elevator metaphor connecting business strategy with the IT engine room. The publisher and year (2020) are from the author's bibliography and were not shown on the page consulted.
- Architecture: Selling Options, Gregor Hohpe, 2016. Architecture as options whose value rises with uncertainty; basis for the optionality purpose and its caveat.
- Scaling the Practice of Architecture, Conversationally, Andrew Harmel-Law, martinfowler.com, 2021. The advice process, Architecture Advisory Forum, decision records and team-sourced principles behind the advisory operating model.
- Facilitating Software Architecture, Andrew Harmel-Law, O'Reilly, 2024. Book-length treatment of decentralized architecture decision-making.
- Documenting Architecture Decisions, Michael Nygard, Cognitect, 2011. The decision-record format referenced by the decision service; owned in depth by the governance chapter.

Northmere is fictional, and so are all of its figures, people and events. The architectural recommendations in this chapter are the author's judgment.

5. Map What Finance Does Before What Runs It



IN THIS CHAPTER, YOU WILL: Build a three-level finance capability map you can reuse, heat-map it to find overlap and weakness, decompose finance into bounded contexts with explicit relationships, and trace capabilities to applications and teams to find the gaps in ownership.

WHY FINANCE CARES: The CFO's questions are about abilities, not software: "Can we pay partners on time in a new currency?", "How many ways do we reconcile?", "Who owns tax determination?" A capability map answers them in Finance's language, and shows where money is spent twice for the same ability.

WHY ARCHITECTS SHOULD CARE: Every later decision in this book (portfolio disposition, system of record, convergence, team design) needs a stable frame that does not change when a vendor or a reorganization does. The capability map is that frame, and bounded contexts turn it into boundaries that software and teams can respect.

KEY POINTS:

- **Name what finance does, not who or what does it.** A business capability is a stable noun ("Partner Payout Management"), not a system ("the SFP payout engine"), a department ("Settlement Ops") or a process ("Book-to-Settle"). Get the names right and the map survives every reorganization and every vendor change.
- **Heat-map to decide, not to decorate.** Score each level-2 capability on strategic importance, maturity, duplication and cost. Duplication is only bad when nobody chose it: Northmere's four overlaps (payouts twice, billing three ways, reconciliation in three tools, tax determined in two engines) are a problem because no one decided them.
- **Capabilities say what; bounded contexts say where the model and the language stop.** The capability map frames investment; the context map frames software and team boundaries, with explicit relationships (customer-supplier, conformist, anticorruption layer, published language). You need both, and an owner for each capability that cuts across them.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Capability Creep** — A capability map is the only view in which three reconciliation tools show up as one capability with three implementations.
- **The Permanent Exception** — Capabilities named per brand ("Stayloft Billing", "Wayfare Billing") make every brand fork look like a distinct need; one map across brands exposes the fork.
- **The Borrowed Roadmap** — A map copied from a vendor's module catalog bakes the vendor's packaging into the enterprise's planning frame.

Every finance estate is described, sooner or later, by a list of systems. The list is easy to produce, because the EA tool already holds it, and it answers the questions an infrastructure team asks: what runs where, on what version, at what cost. It does not answer the questions a CFO asks. "How many ways do we pay partners?" is not a question about systems. It is a question about an **ability** the enterprise has, and about how many times the enterprise has paid to acquire it.

A **business capability** is what an organization is able to do, independent of how, where, by whom or with which system it does it. The Business Architecture Guild's BIZBOK Guide defines it as a particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome (BIZBOK v12 glossary, as reproduced by the Government of Canada). A **capability map** arranges those abilities into a hierarchy (here three levels) that covers the domain once, without gaps and without

overlap. **The value of the map is that it does not move when the organization chart, the vendor list or the process design moves**. That stability is what makes it a planning frame.

The second half of this chapter adds a different decomposition with a different job. A **bounded context**, in domain-driven design, is the boundary within which one model and its language apply consistently. Capability maps tell you where to invest; bounded contexts tell you where one model should stop and a translation should begin. **Finance needs both**, because finance is a domain where the same word ("account", "settlement", "revenue") means different things a few systems apart, and the cost of confusing them is a misstated balance.

Setting the Scene at Northmere: The Question the Portfolio Could Not Answer

Northmere, the fictional group this book follows (see Meet Northmere: Five Brands, One Set of Books), approved Program Keystone in December 2026. In January 2027, at the first Keystone steering meeting, Tomás Ferreira, the Group CFO, asked Dana Whitlock, the Chief Architect for Enterprise Applications and Financial Systems, a question she could only half answer that afternoon: "How many of the things we do in finance do we do more than once, and which of those did anyone decide?"

Dana had a duplicate-capability list from her first month, built from interviews: two payout engines, three billing approaches, three reconciliation tools, two tax engines. She could not tell Tomás whether the list was complete, because the fact base could not confirm it.

SAP LeanIX, the EA repository, listed **214 finance-relevant applications**. Each had an owner, a cost center, a lifecycle state and a capability tag. The tags were the problem. They had been entered by application owners over five years, and they came in three dialects:

- **System dialect**: "BlackLine Recs", "Vertex Tax", "SFP Payouts". The tag repeated the application's name, so every application had its own capability.
- **Department dialect**: "Accounts Payable", "Settlement Operations", "Wayfare Finance". The tag named the team that used the tool.
- **Process dialect**: "Record to Report", "Order to Cash", "Month-end close". The tag named an end-to-end flow that ran through dozens of systems.

Filtering by tag gave nonsense. "Reconciliation" returned BlackLine but not the reconciliation module bought with OneStream in 2022, which was tagged "Consolidation". It did not return the intercompany reconciliation that Wayfare ran in Oracle Fusion, which was tagged "Wayfare Finance". Tax determination returned Vertex and missed

Oracle's tax engine, which Wayfare used for some markets and which was tagged as part of "Oracle Fusion ERP".

Dana's commitment in the Keystone plan was a **capability map and a system-of-record matrix, version 1, by the end of Q1 2027**. This chapter follows the capability map. The system-of-record matrix is the subject of Systems of Record: Ending the Doppelgänger Record.

Concepts: Capability, Process, Value Stream, Application, Organization Unit

Five concepts get confused in almost every finance capability workshop I have attended. They describe the same enterprise from five angles, and each answers a different question.

Concept	Question it answers	Named as	Changes when	Northmere example	Typical owner
Business capability	What must the enterprise be able to do?	A noun phrase, stable across years: "Partner Payout Management"	The business model changes (a new revenue model, a license, a regulation)	Partner Payout Management exists because Stayloft holds merchant funds and pays properties	A business capability owner in Finance
Value stream	How does value reach a stakeholder, end to end?	A stakeholder outcome: "Settle a partner"	The stakeholder or the outcome changes	"Settle a merchant-model property": from booking to payout settled	A value-stream sponsor, often a finance leader

Concept	Question it answers	Named as	Changes when	Northmere example	Typical owner
Process	In what order, by whom and with which rules is the work done?	A verb phrase: "Calculate payout net of commission and release to the payment factory"	Policy, tooling or organization changes	The Stayloft payout run: daily cut-off, holds, approval	A process owner (operational)
Application	Which software implements part of a capability?	A product or system name	A sourcing, upgrade or retirement decision	SFP payout engine; Wayfare payout engine	An application owner in technology
Organization unit	Who does the work and is accountable for people and budget?	A team or department name	A reorganization	Stayloft Finance Engineering; Wayfare Finance Systems	A line manager

The BIZBOK Guide defines a **value stream** as an end-to-end collection of activities that creates a result for a customer, who may be the ultimate customer or an internal end user (BIZBOK v12 glossary). The usual business-architecture convention is that capability maps show *what*, value streams show *how value is delivered*, and processes are the detailed *how* inside each value-stream stage. Each value-stream stage is enabled by several capabilities. "Settle a merchant-model property" crosses Payment Settlement Reconciliation, Contract Liability Management, Partner Payout Management, Outbound Payment Execution and Subledger Accounting.

Two clarifications save weeks of argument.

Record-to-Report, **Procure-to-Pay and Order-to-Cash are not capabilities**. They are end-to-end process families (value streams, in BIZBOK language) that cut across capabilities. Finance teams use them as organizational labels, and many vendors present

them as top-level menu items. I keep them, but on a different view: the process view in Order to Cash, Procure to Pay, Record to Report. When R2R appears as a box on a capability map, the map has silently mixed two decompositions, and every application ends up "supporting R2R".

A capability has no location in the organization chart. Partner Payout Management is performed by Stayloft Finance Engineering's software, Wayfare's software, Northmere Pay's payment factory and a payout operations team in Dublin. The capability is one. The organizations performing it are four. This is the point of the exercise: once you can see one capability performed four ways, you can ask whether that is a choice.

Common modeling mistakes

These are the mistakes I see most often in finance capability maps, with the Northmere version of each and the correction.

Mistake	Northmere example (from the LeanIX tags)	Why it breaks the map	Correction
Named after a system	"BlackLine Recs", "SFP Payouts"	Every application becomes its own capability, so overlap is invisible by construction	Name the ability: "Account Reconciliation", "Partner Payout Management"
Named after a department	"Accounts Payable", "Settlement Operations"	The map changes with every reorganization and hides capabilities performed by several teams	Name what the team does: "Supplier Invoice Processing", "Partner Payout Management"
Verbs and sequences	"Reconcile PSP settlements daily, then post differences"	A sentence with a sequence is a process; it contains "how" and "when"	Noun phrase: "Payment Settlement Reconciliation"; the daily cadence belongs in the process model

Mistake	Northmere example (from the LeanIX tags)	Why it breaks the map	Correction
"How" mixed into "what"	"Automated three-way matching in BlackLine"	Embeds a solution choice; when the tool changes, the capability appears to disappear	"Payment Settlement Reconciliation"; automation level is a *maturity* attribute, the tool is an *application* mapping
Named per brand	"Stayloft Billing", "Tablekin Billing"	Turns brand variation into separate capabilities, legitimizing forks (The Permanent Exception)	One capability, "Invoicing", with brand and business model as dimensions of the heat map
Copied from a vendor's module catalog	Level 2 equal to an ERP's module list	Bakes in the vendor's packaging and omits what the vendor does not sell (payouts, safeguarding) (The Borrowed Roadmap)	Use vendor reference models as a checklist, never as the skeleton
Uneven granularity	"Finance" next to "FX Rate Upload" at the same level	Makes heat-map comparisons meaningless	Test each level-2 capability: could it be owned, funded and measured on its own?

The rule I use when a name is disputed: **if the name would still be true after replacing every system and reorganizing every team, it is a capability name**. "Indirect Tax Determination" passes. "Vertex Tax" fails. "Group Tax Team Services" fails.

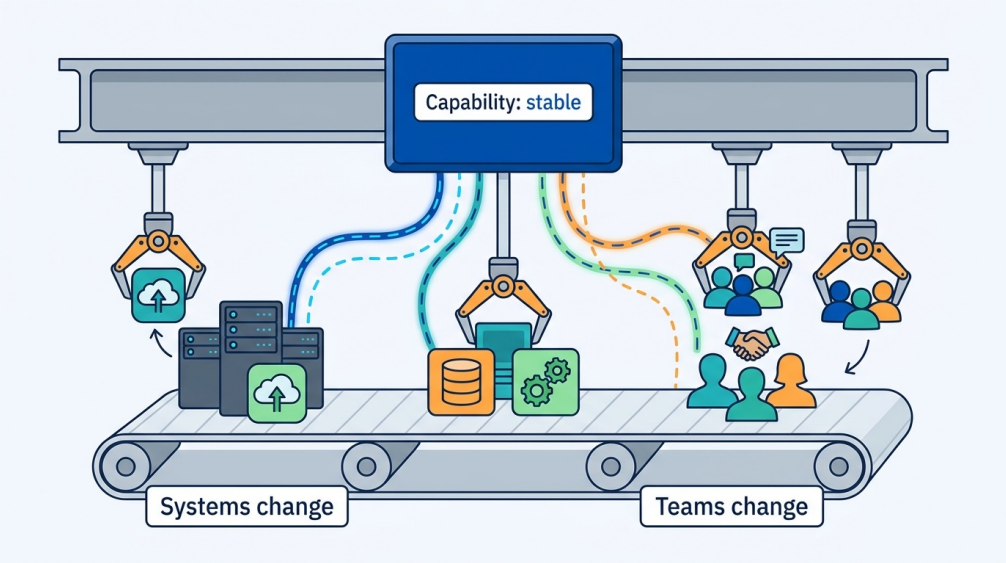


Figure 1: *A capability name passes the rename test when it would stay true after every system is replaced and every team is reorganized; that stability is what makes the map a planning frame.*

Architecture: A Three-Level Finance Capability Map

The map below is the one Dana's team took into the Finance workshops as a strawman, and the one that came out of them as version 1 in March 2027. It has **ten level-1 groups and 45 level-2 capabilities**, with level 3 shown only where Keystone needed it for a decision. It is written for a multi-brand, multi-entity commerce group with marketplace and subscription models. A manufacturer would add Inventory Valuation and Product Costing; a bank would replace most of Money Movement. The structure travels; the content should be challenged.

Three design choices shape it:

- **Level 1 follows Finance's own domains of accountability**, not an ERP's modules. Money Movement and Revenue & Billing are separate groups because, in a marketplace, collecting and paying out money is a different ability from earning and invoicing revenue, and they fail differently.
- **Controls and Finance Data are level-1 groups**, not footnotes. Both are performed across every other group, both have owners (the Head of Internal Controls, the Head of Finance Data), and both are where audit findings and "three revenue numbers" problems come from.
- **Payment acceptance itself is outside the map**. Checkout, PSP routing and fraud scoring belong to the payments product owned by Northmere Pay. The finance

map starts where a payment becomes a financial fact. The boundary is drawn on the map's edge so nobody assumes it is missing by accident.

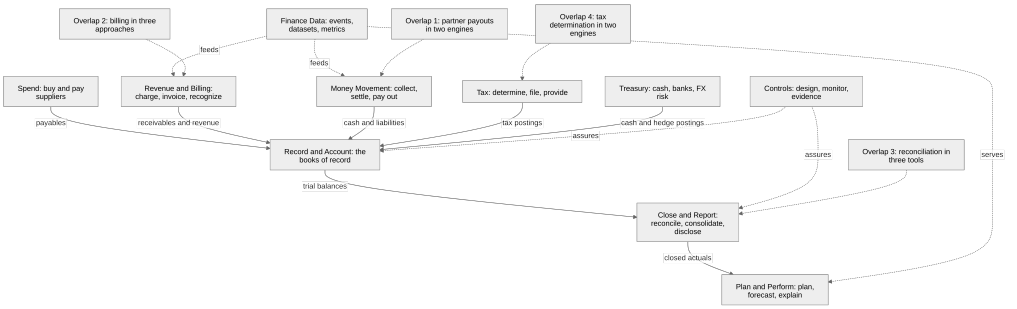


Figure 2: *Northmere's ten level-1 capability groups and the main flows of financial facts between them; the four overlaps Keystone must resolve sit on Money Movement, Revenue and Billing, Close and Report, and Tax.*

The arrows are not integrations. They show which group's outputs another group depends on, which is what a reader needs to understand why a weak capability in Money Movement shows up as a late close.

The map

Definitions are one sentence each, written to be read aloud in a workshop. Level 3 is selective.

Level 1	Level 2 capability	Definition	Selected level 3
1. Record and Account	1.1 General Ledger Management	Maintaining the authoritative books of record per legal entity, ledger and period.	Journal posting and validation; period open and close control; parallel ledger (multi-GAAP) maintenance; manual journal governance

Level 1	Level 2 capability	Definition	Selected level 3
	1.2 Subledger Accounting	Turning high-volume business events into accounting entries at transaction detail, and summarizing them to the ledger with drill-back.	Accounting event capture; posting rule and account derivation; open-item management and clearing; summarization with reconciliation keys
	1.3 Financial Structure Management	Defining and governing the chart of accounts, company codes, cost and profit centers, segments and the mappings from local to group structures.	Account and dimension governance; local-to-group mapping; structure change approval
	1.4 Legal Entity and Intercompany Accounting	Recording the legal-entity structure and accounting for transactions between group entities, including transfer-pricing charges.	Intercompany agreement management; intercompany matching; transfer-pricing charge calculation; intercompany settlement and netting
	1.5 Foreign Currency Accounting	Applying exchange rates to transactions and balances and accounting for revaluation.	Rate management; open-item revaluation; realized and unrealized FX accounting

Level 1	Level 2 capability	Definition	Selected level 3
	1.6 Fixed Asset and Lease Accounting	Capitalizing, depreciating and retiring assets and accounting for leases.	Capitalization (including capitalized software); depreciation; lease accounting
2. Money Movement	2.1 Payment Record Management	Turning PSP and payment-method events and reports into one finance-ready payment record with fees and FX.	Payment event normalization; settlement report ingestion and completeness checks; fee and interchange attribution
	2.2 Payment Settlement Reconciliation	Proving that orders, payments, PSP settlements and bank receipts agree, and working the exceptions.	Three-way matching; exception management and aging; clearing-account certification
	2.3 Refund and Dispute Accounting	Accounting for refunds, chargebacks and their outcomes, including recovery and provisioning.	Dispute lifecycle accounting; partner recovery; loss provisioning
	2.4 Partner Payout Management	Calculating, scheduling and releasing amounts owed to partners net of commission, holds and adjustments.	Payout calculation; payout scheduling and holds; partner bank-detail change control; payout status and remittance

Level 1	Level 2 capability	Definition	Selected level 3
	2.5 Customer Funds Safeguarding	Calculating and evidencing the protection of customer funds held by licensed entities.	Daily relevant-funds calculation; safeguarding reconciliation; regulatory safeguarding returns
	2.6 Outbound Payment Execution	Turning approved payment instructions into bank payments, with approval, screening and confirmation.	Payment batch approval; sanctions-screening hand-off; payment file generation and dispatch; payment status tracking
3. Revenue and Billing	3.1 Charge Calculation and Rating	Determining what is owed for a transaction, a subscription or a unit of usage under the applicable commercial terms.	Commission calculation; usage rating; subscription charge calculation; price and fee schedule management
	3.2 Invoicing	Issuing compliant invoices, credit notes and statements to customers, partners and advertisers.	Invoice generation; credit notes and adjustments; e-invoice issuance; invoice numbering per jurisdiction

Level 1	Level 2 capability	Definition	Selected level 3
	3.3 Receivables and Collections	Managing amounts owed to the enterprise until they are collected or written off.	Cash application; dunning; dispute handling; credit-risk and write-off management
	3.4 Revenue Recognition	Determining when and how much revenue is recognized, and in which presentation, under the applicable standard.	Principal-versus-agent determination; performance-obligation scheduling; revenue release; revenue disaggregation
	3.5 Contract Liability Management	Tracking customer money received before performance and releasing it on delivery, cancellation or payout.	Per-booking liability tracking; liability aging; liability release
4. Close and Report	4.1 Close Orchestration	Planning, executing and certifying the period-end close across entities.	Close calendar and task management; close certification; close gating for consolidation
	4.2 Account Reconciliation	Proving each balance-sheet account balance is supported, and certifying it.	Balance-sheet reconciliation; subledger-to-ledger tie-out; intercompany reconciliation; reconciliation risk rating

Level 1	Level 2 capability	Definition	Selected level 3
	4.3 Consolidation	Combining entity results into group results with translation, eliminations and ownership.	Data collection and mapping; currency translation; intercompany elimination; ownership and minority interests
	4.4 External and Statutory Reporting	Producing the published and filed financial statements and disclosures.	SEC filing preparation; statutory accounts per entity; XBRL tagging; disclosure tie-out
5. Plan and Perform	5.1 Planning and Budgeting	Setting the annual plan and budget by brand, segment and cost center.	Driver-based planning; headcount and cost planning; plan approval
	5.2 Forecasting and Scenario Modeling	Projecting results and testing scenarios during the year.	Rolling forecast; scenario modeling; cash-flow forecast input
	5.3 Management Reporting and Analysis	Explaining performance against plan with governed metrics.	Variance analysis; management packs; metric commentary
	5.4 Cost Allocation and Profitability	Attributing costs and margins to brands, products and markets.	Allocation rules; recharges; unit economics

Level 1	Level 2 capability	Definition	Selected level 3
6. Tax	6.1 Indirect Tax Determination	Determining VAT, GST, sales and transaction taxes on each taxable event, with the rule version recorded.	Tax rule and content management; transaction tax calculation; exemption and registration handling
	6.2 Tax Compliance and Reporting	Preparing and filing tax returns and tax-related platform reports.	VAT and sales tax returns; platform reporting such as DAC7; digital services tax reporting; e-invoicing clearance reporting
	6.3 Income Tax Provision	Calculating current and deferred income tax for interim and annual reporting.	Provision calculation; effective tax rate analysis; deferred tax
	6.4 Global Minimum Tax Management	Collecting data for and calculating top-up tax under the OECD Pillar Two rules.	Jurisdictional effective tax rate calculation; GloBE information return data
	6.5 Transfer Pricing Management	Setting and documenting the pricing of intercompany transactions.	Policy and benchmark management; documentation; year-end true-up
7. Treasury	7.1 Cash Positioning and Forecasting	Knowing and projecting cash by entity, bank and currency.	Daily cash position; liquidity forecast; trapped-cash visibility

Level 1	Level 2 capability	Definition	Selected level 3
	7.2 Bank Relationship and Account Management	Managing bank relationships, accounts, signatories and connectivity.	Bank account lifecycle; signatory management; bank connectivity management
	7.3 FX Risk and Hedge Management	Measuring currency exposure and executing and accounting for hedges.	Exposure capture; hedge execution; hedge accounting documentation
	7.4 Liquidity and Funding Management	Moving and funding cash across the group within legal and regulatory limits.	Cash pooling; intercompany funding; payout funding by currency
8. Spend	8.1 Supplier Management	Onboarding, qualifying and maintaining suppliers and their contracts.	Supplier onboarding and risk checks; supplier bank-detail change control; contract management
	8.2 Requisitioning and Purchasing	Requesting, approving and ordering goods and services.	Guided buying; purchase approval; purchase order management
	8.3 Supplier Invoice Processing	Capturing, matching, approving and scheduling supplier invoices for payment.	Invoice capture and e-invoice receipt; two- and three-way matching; payment proposal
	8.4 Employee Expense Management	Capturing, approving and reimbursing employee expenses.	Expense capture; policy checks; reimbursement

Level 1	Level 2 capability	Definition	Selected level 3
9. Controls	9.1 Control Design and Monitoring	Maintaining the register of financial-reporting controls and monitoring their operation.	Risk and control matrix; automated control monitoring; deficiency management
	9.2 Access and Segregation of Duties Governance	Granting, reviewing and constraining access to financial systems so that incompatible duties are separated.	Role design; access review; segregation-of-duties rules; emergency access logging
	9.3 Audit Evidence Management	Producing and retaining evidence that controls operated and numbers are supported.	Evidence collection; auditor request handling; evidence retention
	9.4 Regulatory Reporting	Producing prudential and conduct returns for licensed entities.	E-money prudential returns; incident reporting; regulator submissions
10. Finance Data	10.1 Financial Event Management	Receiving, validating and distributing financial events from operational systems under a canonical contract, with completeness evidence.	Event contract validation; enrichment with entity and account keys; per-source control totals; replay

Level 1	Level 2 capability	Definition	Selected level 3
	10.2 Finance Data Provisioning	Providing curated, governed finance datasets for analysis, audit sampling and planning.	Ledger-to-warehouse tie-out; dataset access control; lineage capture
	10.3 Finance Metric Governance	Defining each finance metric once, with an owner, logic and version.	Metric catalog; metric certification; metric change control

A note on 10.1. Purists object that event integration is a technical capability and does not belong on a business map. I keep it, deliberately, because in a marketplace finance estate **completeness of financial events is a business ability with a business owner**: if 0.1% of bookings never become financial events, revenue is understated and nobody will notice from the ledger. The definition is written so that it says what must be true (validated, complete, evidenced), not which technology does it.

Readers will notice what is absent. There is no "Integration", "Reporting Platform" or "ERP" capability, because those are implementations. There is no "Shared Services" capability, because that is an organization unit. And there are no brand-named capabilities.

In Practice: Heat-Mapping Northmere

A capability map becomes useful when it carries attributes. Northmere scored every level-2 capability on four dimensions, each with a defined scale so that two people scoring the same capability get the same answer.

Dimension	Scale	Evidence used	Who scores
Strategic importance	*Differentiating* (a source of competitive advantage), *Critical* (failure is material or regulatory; not differentiating), *Commodity* (necessary, standard)	Business model, regulatory exposure, revenue at stake	Capability owner, challenged by the CFO staff
Maturity	1 Ad hoc; 2 Repeatable with heavy manual effort; 3 Defined and mostly automated; 4 Measured, with metrics and controls operating; 5 Optimized and continuously improved	Metrics (match rates, cycle times), control results, audit findings	Capability owner and architects together, checked against metrics
Duplication	Count of distinct implementations of the same capability for comparable scope	LeanIX application-to-capability mapping, verified in interviews	Architects
Cost	Annual run cost in US dollars: licenses, hosting, engineering and operations headcount allocated to the capability	LeanIX cost data, finance cost-center allocations	Architects with Finance business partners

The importance scale borrows deliberately from domain-driven design. Eric Evans's *Core Domain* and *Generic Subdomains* (DDD Reference, 2015) draw the same line:

invest design talent where the business is distinctive, and buy or standardize the rest. In finance, the third category, *Critical*, matters most: a general ledger is not differentiating, but an error in it is a restatement. Critical-but-commodity capabilities should be bought and run to a high standard, not built and not neglected.

The Northmere heat map

The table shows the twelve level-2 capabilities that mattered most to Keystone's first year. Costs are 2027 estimates from LeanIX and cost-center allocations, rounded, and fictional like everything about Northmere.

Capability	Importance	Maturity	Duplication	Annual run cost	Reading
1.1 General Ledger Management	Critical	3	3 (S/4HANA, Fusion, Workday)	~$14.0m	Brand-ledger duplication; the convergence question, owned by Converge or Coexist: Living with More Than One ERP
1.2 Sub-ledger Accounting	Differentiating	2	2 (SFP merchant sub-ledger, FI-CA agency)	~$11.5m	Split by business model, not by choice; heavy summarization, weak drill-back

Capability	Importance	Maturity	Duplication	Annual run cost	Reading
1.3 Financial Structure Management	Critical	1	3 charts, mapped in spreadsheets	~$1.2m	Lowest maturity on the map; precondition for convergence
2.2 Payment Settlement Reconciliation	Critical	2	1 (BlackLine Transaction Matching) plus manual APM work	~$4.8m	€14m average month-end suspense; 1.9% of APM settlement lines unmatched at working day 3
2.4 Partner Payout Management	Differentiating	3 (SFP), 2 (Wayfare)	**2 engines**	~$7.6m	**Overlap 1**
2.6 Outbound Payment Execution	Critical	4	1 (Kyriba payment factory)	~$2.1m	Healthy; the reuse target for every payout path
3.2 Invoicing	Critical	3	**3 approaches** (SFP commission billing, FI-CA invoicing, Stripe Billing)	~$6.3m	**Overlap 2**; e-invoicing mandates multiply the cost of each approach

Capability	Importance	Maturity	Duplication	Annual run cost	Reading
3.4 Revenue Recognition	Critical	2	3 (by brand ledger)	~$3.9m	Most material judgment; evidence scattered across three ledgers
4.1 Close Orchestration	Critical	3	1 (BlackLine Task Management)	~$1.8m	Nine-day close is a symptom of capabilities upstream
4.2 Account Reconciliation	Commodity	4 (BlackLine), 1 (OneStream module), 2 (Fusion intercompany)	**3 tools**	~$3.4m	**Overlap 3**
6.1 Indirect Tax Determination	Critical	4 (Vertex), 2 (Oracle tax engine)	**2 engines**	~$2.6m	**Overlap 4**; ViDA and e-invoicing raise the stakes
10.1 Financial Event Management	Critical	2	3 integration technologies	~$5.2m	Kafka, SAP Integration Suite and MuleSoft; ~70 spreadsheets-in-the-loop

Two cautions about reading a heat map like this.

First, **duplication is not a verdict**. Subledger Accounting appears twice because the agency and merchant models were introduced at different times and have different shapes: FI-CA holds about 850,000 agency partner contract accounts, and SFP holds

merchant-model traveler liabilities. That duplication is a known transition state with a dated plan (the merchant subledger moves to the group core in 2029). General Ledger Management appears three times because Northmere bought two companies with their own ledgers; that is the convergence question, with its own decision in 2027 Q4. **The four overlaps Dana highlighted are different: no decision record created them, no owner arbitrates between the implementations, and no plan retires either.** That is the definition of Capability Creep, and it is why these four go to the steering committee and the other duplications do not.

Second, **the cost column is not a savings estimate.** Removing the OneStream reconciliation module saves its license. Removing the Wayfare payout engine saves almost nothing until Wayfare's ground-transport suppliers are migrated onto the Stayloft payout path, which costs more in its first year than the engine costs to run. Manage the Application Portfolio as an Investment turns these numbers into investment decisions; here they only rank attention.

The four overlaps, one at a time

Overlap 1: Partner payouts in two engines. Of the roughly 9 million partner payouts Northmere makes each month, SFP calculates and schedules those for Stayloft merchant-model properties; Tablekin's venue payouts run through Stripe Connect inside Tablekin's payments-facilitation product, and the workshop recorded them but did not count them as an overlap. Wayfare's own engine pays ground-transport suppliers. Both calculate amounts net of commission, both hold partner bank details, and both release instructions to banks. The payout engine is also where Northmere's FY2026 significant deficiency sits: developers had production access to SFP, and payout-rule changes lacked documented approval. The capability map turns an engineering argument ("our engine is better") into a capability question: **one Partner Payout Management capability, one owner, one control design for bank-detail changes**, with two implementations allowed only while a dated migration runs. Dana's first decision under her new rights, in December 2026, set the change-approval standard for SFP payout rules; the capability view extends that standard to both engines.

Overlap 2: Billing in three approaches. SFP calculates and bills commission for merchant-model properties as part of the payout statement. SAP FI-CA (contract accounting, SAP's high-volume subledger for business-partner accounts) issues commission invoices, dunning and partner balances for agency-model properties. Tablekin bills subscriptions and usage with Stripe Billing, so the Austin venue on the "Pro" plan receives an invoice dated 1 April 2027 for $1,029 before sales tax (the April subscription of $249 plus March usage of 3,120 covers at $0.25). The map separates what is legitimately different from what is duplicated. Charge Calculation and Rating differs genuinely by business model: commission on a stay and usage rating per seated cover are different calculations. **Invoicing, the issuance of a compliant document with jurisdiction-correct numbering and, increasingly, e-invoice clearance, is the same capability**

three times, and every new national e-invoicing mandate has to be implemented in all three. (Northmere Ads' custom ad-billing service sits under the same capability; the workshop scored it as low volume and recorded it, but did not add it to the overlap list.)

Overlap 3: Reconciliation in three tools. BlackLine reconciles about 6,800 balance-sheet accounts a month and runs transaction matching for PSP settlements. The reconciliation module bought with OneStream in 2022 is licensed but used by only a handful of entities. Wayfare reconciles intercompany balances inside Oracle Fusion. The map splits what the LeanIX tag had merged: **Payment Settlement Reconciliation (2.2) and Account Reconciliation (4.2) are different capabilities**, with different volumes (millions of lines against thousands of accounts) and different owners. The overlap is in 4.2. Account Reconciliation is a commodity capability at maturity 4 in BlackLine; there is no business reason to perform it in a second and third tool.

Overlap 4: Tax determination in two engines. Vertex determines indirect tax for S/4HANA and for SFP commission invoicing. Wayfare uses Oracle's tax engine for some markets. Two engines mean two sets of tax content (rules, rates, exemptions), two update processes and two places the auditor and the tax authority must examine. When the EU's ViDA package changes the rules for short-term accommodation platforms, the change must be implemented, tested and evidenced twice. Indirect Tax Determination is critical, not differentiating: the case for one engine is strong, and the exceptions (a market where only one engine has content) are what the decision record should name.

Architecture: From Capabilities to Bounded Contexts

A capability map is the right frame for investment and ownership. It is the wrong frame for software design, because it says nothing about **meaning**. Two capabilities can use the same word for different things; one capability can be implemented across several models. That is what domain-driven design's bounded contexts are for.

Eric Evans's DDD Reference defines a **bounded context** as a description of a boundary (typically a subsystem, or the work of a particular team) within which a particular model is defined and applicable (DDD Reference, 2015). Inside the boundary, every term has one meaning, the **ubiquitous language**. Across boundaries, meanings differ, and the relationship between contexts has to be designed. Martin Fowler's short explanation uses a utility where "meter" meant three different things in three parts of the business (Bounded Context, 2014).

Finance is full of such words. The strongest example is **account**:

Context	What "account" means	Northmere example
General Ledger	A line in the chart of accounts, carrying balances by entity and period	GL account for partner payables in the Northmere Payments B.V. ledger
Subledger (contract accounting)	A contract account: the business-partner-level container for open items	One of ~850,000 agency partner contract accounts in FI-CA
Payments	A merchant account at the acquirer, or a connected account on a platform	Stayloft's Adyen merchant account; a Tablekin venue's Stripe connected account
Treasury	A bank account with signatories and a balance at a bank	One of ~410 accounts at 27 banks in Kyriba
Billing and Invoicing	A customer or billing account with terms and an invoice history	The Austin venue's Stripe Billing customer
Partner Settlement	A partner's payout profile and balance	The Lisbon property's payout profile, with bank details under change control

"Settlement" is almost as treacherous. In Payments it means the acquirer transferring funds to Northmere Payments B.V. In Partner Settlement it means paying the property. In Treasury it can mean settlement of an FX trade. In intercompany accounting it means clearing balances between entities. A single "settlement" table shared by these contexts is a defect waiting to happen.

Northmere's finance bounded contexts

Dana's team proposed ten finance bounded contexts, plus two upstream contexts that finance does not own. Each context has a model, a language and an owning team; each implements one or more capabilities.

Bounded context	Core concepts in its language	Capabilities it implements	Current implementations
Payments	Payment, capture, PSP settlement line, fee, dispute	2.1, 2.2 (matching rules), 2.3	Northmere Pay platform; Adyen and Stripe as external models
Partner Settlement	Partner balance, payout, hold, remittance, payout profile	2.4, part of 3.1 (commission netting)	SFP payout engine; Wayfare payout engine
Billing and Invoicing	Charge, rated usage, invoice, credit note, e-invoice	3.1, 3.2, 3.3	SFP commission billing; FI-CA invoicing; Stripe Billing; ad-billing service
Revenue	Performance obligation, recognition schedule, principal or agent, release	3.4, 3.5	SFP; Fusion; Workday
Subledger	Accounting event, posting rule, open item, summary with reconciliation key	1.2	SFP merchant subledger; FI-CA
General Ledger	Journal entry, GL account, ledger, period, company code	1.1, 1.3, 1.4, 1.5, 1.6	S/4HANA; Fusion; Workday Financials
Consolidation	Entity, scenario, elimination, translation, group account	4.3, 4.4, 5.x shared dimensional model	OneStream; Workiva downstream
Tax	Taxable event, jurisdiction, determination, return	6.1–6.5	Vertex; Oracle tax engine; provision tooling

Bounded context	Core concepts in its language	Capabilities it implements	Current implementations
Treasury	Cash position, bank account, exposure, hedge, payment instruction	2.6, 7.1–7.4	Kyriba
Spend	Supplier, requisition, purchase order, supplier invoice, expense report	8.1–8.4	Coupa; SAP Concur; S/4HANA AP
Booking (upstream)	Booking, stay, cancellation, traveler	none in finance	Brand booking platforms
People (upstream)	Worker, position, cost-center assignment, approver	none in finance	Workday HCM

Account Reconciliation (4.2) and Close Orchestration (4.1) do not get their own context in this first cut. The close is a coordinating activity across the General Ledger and Consolidation contexts, and BlackLine is a **generic subdomain** in Evans's sense: valuable, bought, and configured, not modeled from scratch. Teams that insist on a "Reconciliation context" usually end up building a second ledger.

The context map

A **context map** shows the bounded contexts and the relationships between them. The DDD Reference lists the relationship patterns: Partnership, Shared Kernel, Customer/-Supplier Development, Conformist, Anticorruption Layer, Open-host Service, Published Language, Separate Ways and Big Ball of Mud (DDD Reference, 2015). Five do almost all of the work in a finance estate:

- **Customer-supplier**. The downstream context's needs are planned into the upstream's work. Partner Settlement is the customer of Payments: it cannot pay a property until Payments confirms the funds settled.
- **Conformist**. The downstream adopts the upstream model as-is, because changing it is not possible or not worth it. Every SaaS platform makes its customers conformists to some degree: Tablekin's Billing context conforms to Stripe Billing's object model; Consolidation conforms to OneStream's dimensional model.

- **Anticorruption layer (ACL).** A translation layer that keeps an upstream model from leaking into the downstream model. In finance the most important ACLs sit at the **ERP boundary** and at the **PSP boundary**.
- **Open-host service.** A context offers a defined protocol that any consumer may use. The Tax context offers tax determination as a service to every context that issues invoices or charges.
- **Published language.** A documented, shared language for exchange between contexts. Keystone's canonical financial events (`BookingConfirmed`, `PaymentSettled`, `PartnerPayoutInstructed` and the rest) are Northmere's published language; Canonical Events: A Common Language for Money owns their design.

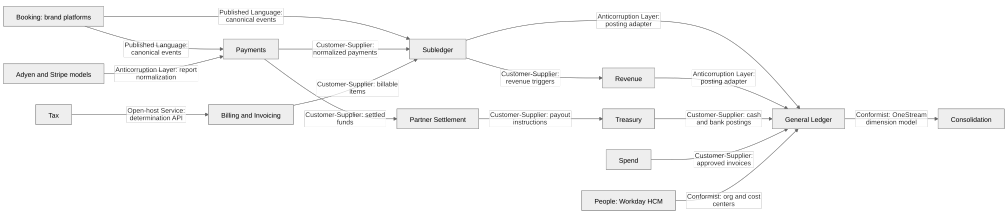


Figure 3: *Northmere's target context map: canonical events as the published language from the brand platforms, anticorruption layers at the PSP and ERP boundaries, and customer-supplier relationships where finance contexts depend on each other's facts.*

Three relationships on this map deserve a sentence of justification each, because teams will argue about them.

Anticorruption layer at the ERP boundary. SFP posts about 38,000 summarized GL lines a day into S/4HANA, summarized from about 3 million subledger lines. The posting adapter translates Subledger facts (accounting events, posting rules, reconciliation keys) into S/4HANA's document model. The rule I apply: **no ERP-specific concept (document types, company-code numbers, posting keys) appears in a brand or subledger model**; it lives only in the adapter. The day Northmere decides between Central Finance and an accounting hub (2027 Q4), the adapter changes and the Subledger context does not. Without the ACL, SFP would quietly become a conformist to S/4HANA, and every S/4 change would become an SFP change.

Conformist where conforming is cheaper than translating. Not every vendor boundary deserves an ACL. Consolidation conforming to OneStream's dimensional model is a reasonable choice, because nobody else consumes that model and the translation would be pure cost. The test I use: **build an ACL where the vendor's model would otherwise spread into contexts you own; conform where the vendor's model stays inside the vendor's context.** Payments needs an ACL because Adyen's and Stripe's report models are different from each other and both would otherwise leak into reconciliation, revenue and payouts.

Published language, not shared database. The brand platforms do not write into the Subledger's tables, and the Subledger does not read the booking database. They exchange canonical events through the integration hub. The temptation to share one "financial transaction" table between Payments, Subledger and Revenue (the *Shared Kernel* pattern, at its worst) is strongest in a custom platform like SFP, where all three started as modules of one codebase. Duplicates and Dependencies: The Hidden Costs of a Grown Estate explains why that coupling is the most expensive kind to undo.

One booking across the contexts

The Lisbon booking makes the boundaries concrete. A traveler from Germany books a three-night stay in a Lisbon apartment on Stayloft on 12 March 2027: €600.00 including VAT, paid by card through Adyen, stay 18–21 April 2027, 15% commission. Each context sees a different object, in its own language:

Context	What it records	Amount	Its language
Booking	A confirmed booking, published as `BookingConfirmed`	€600.00	Booking, stay dates, traveler
Payments	A capture through Adyen, then a settlement line in Adyen batch 1047 on 14 March (bank credit 16 March) and an Adyen fee	€600.00; fee €1.74	Capture, settlement line, fee
Revenue	A performance obligation: Stayloft acts as agent, so revenue is the commission, recognized when the service is delivered	€90.00	Obligation, net presentation, release

Context	What it records	Amount	Its language
Subledger	Accounting events: a liability to the property and to the traveler until the stay, open items, summarized to the GL with a reconciliation key	€600.00 held; €510.00 owed to the property	Open item, posting rule, clearing
Partner Settlement	A payout instructed on 22 April, settled 26 April	€510.00	Partner balance, payout, remittance
Tax	A determination on the commission (simplified here; VAT on commission depends on the property's status and location)	on €90.00	Taxable event, jurisdiction
General Ledger	One contribution to a daily summarized journal line among ~38,000	(summarized)	Journal entry, GL account, period

Five contexts and one booking: the same €600 is a capture, a liability, an open item and part of a summary, and €90 of it is revenue. That is not duplication. It is **five legitimate models of one economic event**, each owned by a team that understands it. The design problem is not to merge them into one model; it is to keep the relationships explicit and the totals reconcilable at every boundary, which is the subject of Reconciled by Design: Traceability and Audit Across Boundaries.

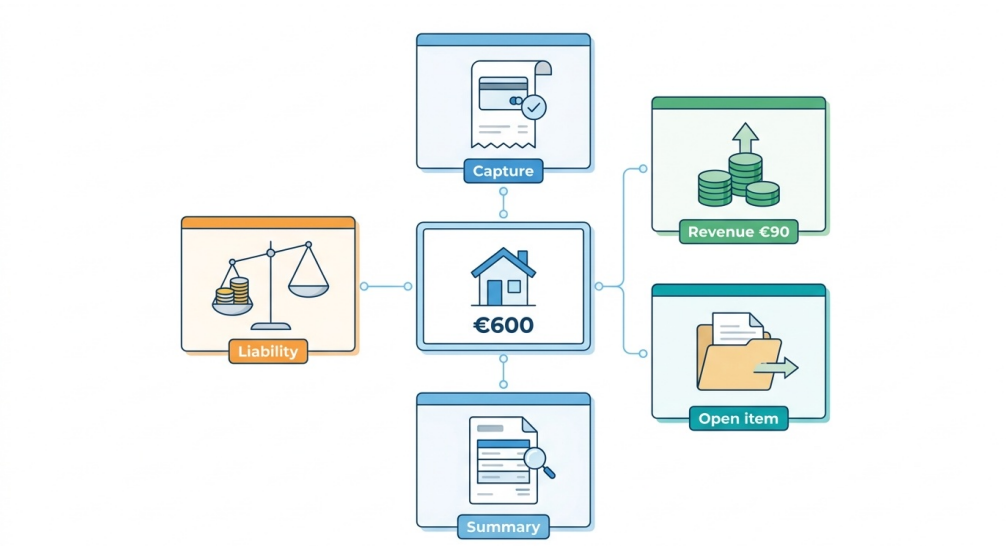


Figure 4: *One Lisbon booking, five legitimate models: the same €600 is a capture, a liability, an open item and part of a summary, and €90 of it is revenue; the design task is to keep the boundaries reconcilable, not to merge the models.*

How Capability Maps and Bounded Contexts Complement Each Other

The two decompositions answer different questions and have different lifespans. Confusing them is the root of two opposite errors: treating the capability map as a service catalog (one microservice per capability) and treating the context map as an investment plan (fund contexts, ignore capabilities that span them).

	Capability map	Context map
Question	What must we be able to do, how well, at what cost?	Where does one model and one language stop, and how do models relate?
Audience	CFO, controllers, portfolio and investment decisions	Architects, engineering teams, integration designers
Unit	Business capability (a noun, an ability)	Bounded context (a model boundary, usually one team's)
Stability	Years; changes with the business model	Months to years; changes with team and system design

	Capability map	Context map
Used for	Heat maps, investment, ownership, portfolio, vendor scope	Service and team boundaries, integration contracts, translation layers
Fails when	Named after systems, departments or processes	Drawn around database tables or vendor modules

They meet in a **many-to-many mapping**, and the shape of the mapping is diagnostic:

- **One capability, several contexts** is normal for broad capabilities. Payment Settlement Reconciliation's matching rules live in Payments, its certifications in the close. It needs a **capability owner** who holds the outcome across both.
- **One context, several capabilities** is normal and healthy when the capabilities share a language. Billing and Invoicing implements Charge Calculation, Invoicing and Receivables.
- **One capability, several implementations in the same context** is the overlap signal. Partner Payout Management in two engines that both belong to Partner Settlement is two implementations of one model, and it should be one.
- **A capability with no context** is usually a manual activity or a spreadsheet. At Northmere, the local-to-group account mapping in Financial Structure Management had no context at all; it lived in spreadsheets loaded into OneStream (Spreadsheet Glue, in the vocabulary of Unreconciled: Fourteen Ways Finance Architecture Goes Wrong).

Conway's law, in Fowler's summary, says organizations design systems that mirror their communication structure, and the inverse Conway maneuver deliberately shapes teams to get the architecture you want (Conway's Law, Fowler, 2022). Fowler also warns that reorganizing alone does not quickly fix a rigid legacy system. Bounded contexts are the unit where this matters: **a context owned by two teams decays into two models; two contexts owned by one overloaded team decay into one**. Who Owns What: Organizing Architecture Across Finance and Technology takes the team design further using Team Topologies.

Capabilities to Applications to Teams: Finding the Ownership Gaps

The final step of version 1 was a three-column trace for every level-2 capability: the capability, the applications that implement it, and the team that owns each application, plus a fourth column for the **capability owner**, a named Finance leader accountable for the outcome regardless of tool. The trace is mechanical. What it reveals is not.

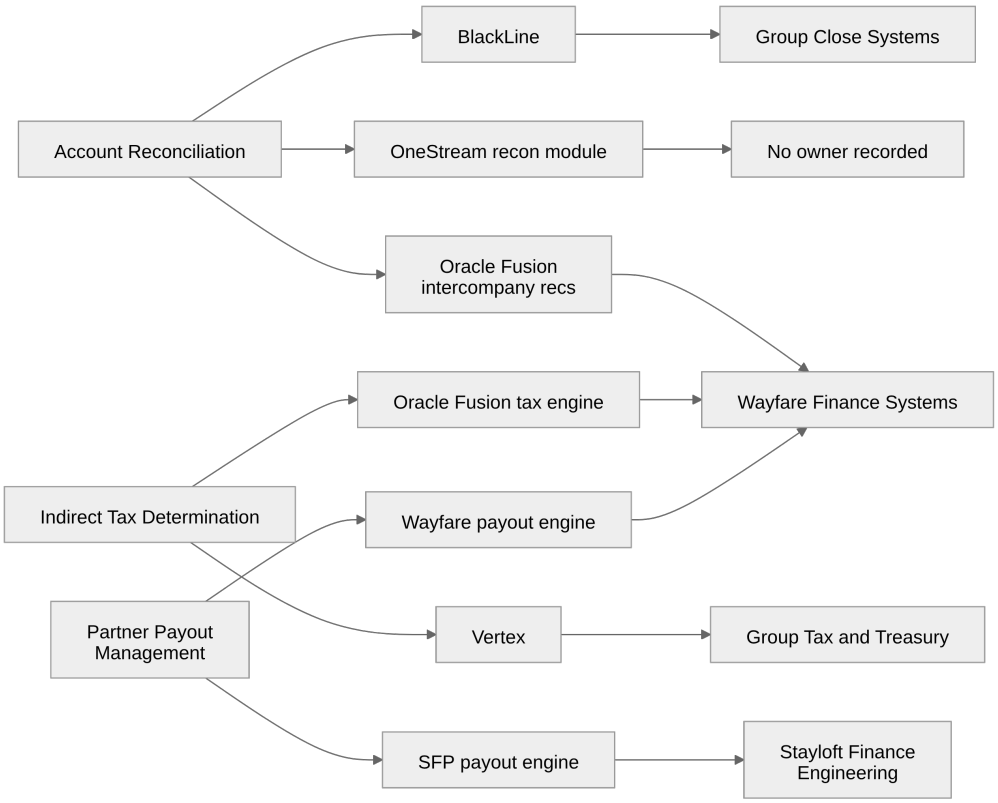


Figure 5: *Three overlapping capabilities traced to applications and owning teams: one application has no owner, and one team owns the minority implementation of all three overlaps.*

Capability	Applications	Application owners	Capability owner (v1 proposal)	Gap revealed
4.2 Account Reconciliation	BlackLine; OneStream reconciliation module; Oracle Fusion (intercompany)	Group close systems team (Leila's organization); none recorded; Hana Kowalski	Leila Haddad, CAO and Group Controller	**Orphan application**: the OneStream module has a license and no owner

Capability	Applications	Application owners	Capability owner (v1 proposal)	Gap revealed
6.1 Indirect Tax Determination	Vertex; Oracle Fusion tax engine	Kofi Asante's team; Hana Kowalski	Kofi Asante, Head of Tax and Treasury	**Owner without reach**: the tax content in Oracle is maintained by Wayfare finance systems staff, outside the Tax function
2.4 Partner Payout Management	SFP payout engine; Wayfare payout engine	Jonas Berg; Hana Kowalski	Proposed: a Head of Partner Settlement in the CAO's organization	**No capability owner**: two engineering owners, no business arbiter; the FY2026 significant deficiency sits here
3.2 Invoicing	SFP commission billing; SAP FI-CA; Stripe Billing; ad-billing service	Jonas Berg; Sven Aalto; Arjun Rao; Northmere Ads engineering	Leila Haddad, delegated to a group billing lead	**Four implementers, no shared standard** for invoice numbering and e-invoicing
1.3 Financial Structure Management	S/4HANA, Fusion and Workday structures; OneStream mappings; spreadsheets	Sven Aalto; Hana Kowalski; Arjun Rao; no one for the spreadsheets	Leila Haddad	**Capability performed in spreadsheets**: 14 mapping workbooks, loaded each close

Capability	Applications	Application owners	Capability owner (v1 proposal)	Gap revealed
10.1 Financial Event Management	Kafka; SAP Integration Suite; MuleSoft	Platform Engineering (Ravi Menon); Sven Aalto; brand teams	Lucía Romero as interim architecture owner, pending a business owner	**Technology owner, no business owner:** completeness of financial events is nobody's measured outcome

Four kinds of gap recur in every estate I have mapped this way, and Northmere has all four:

1. **Orphan application.** An implementation nobody owns. Usually a tool bought with another tool and never adopted. Cheap to fix: retire it or assign it.
2. **Capability without an owner.** Several application owners, none accountable for the business outcome. This is where duplication grows, because each engineering team optimizes its own engine. The fix is a named Finance owner with decision rights over the capability's standards.
3. **Owner without reach.** The capability owner exists but part of the implementation sits in another organization's system, maintained by people who do not report to them. Tax content in a brand ERP is the classic case.
4. **Concentration.** One team owns the minority implementation of several overlapping capabilities. Hana Kowalski's Wayfare finance systems team owns the second payout engine, the second tax engine and the third reconciliation tool. That is not a criticism of the team; it is what an acquired company's estate looks like. It does mean that **Wayfare convergence is one conversation, not three**, and that the people who know Wayfare's edge cases are a scarce resource for every Keystone workstream.

The application-owner and capability-owner distinction matters for controls too. The external auditor tests controls on applications; Finance answers for outcomes on capabilities. The SFP significant deficiency was an application-level control failure (access and change management on the payout engine), but its remediation needed a capability-level decision: whether bank-detail and payout-rule changes should follow one control design across both engines. Controls by Design, Not by Audit Season develops the control side.

Trade-offs: How Much Map, and Which Kind

Capability mapping can be done lightly or heavily, and the choice should follow the decision the map must support.

Approach	What you produce	Effort	Right when	Wrong when
Strawman from a reference model, validated in workshops	Levels 1–2 in two weeks, level 3 selectively	Low to medium	You need a frame for a transformation quickly and Finance will engage	Finance will not challenge it, so the strawman becomes the answer unexamined
Derived from vendor reference content	A map aligned to one vendor's modules	Low	You have chosen a single suite and want fit-to-standard scoping	You run several platforms or custom systems; it hides capabilities the vendor does not sell
Inferred bottom-up from the application portfolio	A map of what the applications do today	Low	You need an inventory for rationalization	You need a target; it reproduces today's overlaps as if they were capabilities

Approach	What you produce	Effort	Right when	Wrong when
Only one of the two maps	Either an investment frame or a boundary design	Low to medium	Capability map only: rationalization without platform redesign; context map only: one engineering organization designing its own platform	Capability map only: re-platforming custom systems, where meaning conflicts go undetected; context map only: investment conversations, because contexts are not Finance's language

Northmere used the first approach for the capability map, and context-mapped only where Keystone was changing software: a strawman built by Dana's architects from reference material and the portfolio, challenged by Finance in workshops. I recommend that combination for most multi-brand groups. The conditions under which it fails: when the strawman is presented as finished (people defer to it), or when the architects running the workshops do not know enough accounting to hear what Finance is telling them.

A note on tools. Northmere maintains the map in SAP LeanIX, which SAP acquired in November 2023 (SAP News, 2023). The map's structure should not depend on any tool: keep it exportable as a plain table, so the repository itself can be replaced, and so a vendor-owned EA tool does not become the place where a vendor-neutral frame quietly acquires a vendor's shape.

Running a Capability Mapping Exercise with Finance

The map's quality is decided in the room. This is how I run it and what Dana's team did in February and March 2027.

Who is in the room. The people who own outcomes and the people who know the edge cases, not the most senior available name per department.

- The **CAO or a delegated controller** with authority to settle naming disputes. At Northmere, Leila Haddad attended the first and last workshops and delegated the rest to two group controllers.
- **Capability-domain leads**: tax and treasury (Kofi Asante's deputies), partner settlement operations, payables, FP&A, payments finance (Ingrid Solberg's finance lead).
- **Internal controls** (Mei Tanaka or a deputy), because controls are a level-1 group and because the heat map's maturity scores must agree with control results.
- **Brand finance systems leads** (Jonas Berg, Hana Kowalski, Arjun Rao), as experts on what exists, not as owners of the names.
- **Head of Finance Data** (Owen Mbeki) for the Finance Data group.
- **A facilitating architect** who knows accounting, plus one note-taker who updates the map live.
- **Not in the room**: vendors and systems integrators (their reference models are inputs, not participants), and anyone who would attend only to defend an application.

How long. For a group of Northmere's size:

Step	Format	Duration	Output
Strawman	Architects alone, from reference material and the portfolio	1–2 weeks	Levels 1–2 with draft definitions
Level 1–2 challenge	Two half-day workshops, one week apart	2 × 4 hours	Agreed level-1 groups, level-2 names and definitions
Level 3 deep dives	One 90-minute session per level-1 group, only where a decision needs it	6–8 sessions over 3 weeks	Selected level 3
Heat-map scoring	Separate sessions per group, with evidence prepared in advance	2–3 weeks	Scores with evidence and owner sign-off

Step	Format	Duration	Output
Application and owner trace	Architects with LeanIX data, verified with owners	2 weeks, in parallel	Capability → application → team table
Readout	Steering committee	1 hour	Version 1, the overlap list, named capability owners

Northmere took seven weeks from strawman to steering readout, at the end of March 2027.

What to avoid.

- **Starting from the org chart or the application list**. Both produce a map of today, and today is what you are trying to see clearly.
- **Scoring in the same session as naming**. People score their own capability high and defend the name to protect the score. Agree names first; score later, with evidence.
- **Self-assessed maturity without metrics**. At Northmere, Payment Settlement Reconciliation was self-scored 3 until the €14 million suspense balance and the 1.9% unmatched rate were put on the screen. It became a 2.
- **Endless naming debates**. Time-box each disputed name to ten minutes, decide, and record the alternative in a glossary. A name can change in version 2; a map that never reaches version 1 cannot.
- **Going deeper than the decision needs**. Level 3 for Partner Payout Management was essential; level 3 for Employee Expense Management was not, and was skipped.
- **Letting a vendor's module list become the skeleton** (The Borrowed Roadmap) or letting each brand add its own variant (The Permanent Exception). Brand and business model are dimensions you score against, not capabilities.
- **Publishing and walking away**. The map must have an owner (Dana's team, with Finance's sign-off on changes) and a review cadence, or it decays into another stale slide.

Integration with the Rest of the Estate

The map is useful only if later decisions consume it. Manage the Application Portfolio as an Investment uses the capability-to-application trace to score functional fit and assign

TIME dispositions (tolerate, invest, migrate, eliminate). Anatomy of a Finance Estate: Layers, Planes and Boundaries lays the capabilities onto a layered reference model, where the volume gradient from billions of events to thousands of GL lines explains why Subledger Accounting and General Ledger Management are separate. Order to Cash, Procure to Pay, Record to Report links R2R, O2C and P2P stages to the capabilities that enable them. Systems of Record: Ending the Doppelgänger Record assigns data authority, with the bounded contexts as natural authority boundaries. Duplicates and Dependencies: The Hidden Costs of a Grown Estate owns the **overlap register** that grows from the four overlaps found here, and Canonical Events: A Common Language for Money owns the published language and anticorruption layers named on the context map. Build, Buy or Compose: Choosing What to Own uses the importance scores, and Architecture Governance That Speeds Decisions makes "which capability does this serve?" a required question in every decision record.

Failure Modes

Failure mode	Symptom	Fix
Map as shelfware	No decision record cites a capability	Make the capability a required field in decision records and portfolio requests
System names in capability clothing	Every capability has exactly one application	Apply the rename test; reject any name a sourcing decision would invalidate
Overlap discovered, never decided	The overlap list is unchanged between versions	Give each overlap a decision owner and a date, recorded as an architecture decision
Capability owners in name only	Application owners change scope without consulting them	Give owners decision rights over capability-level standards and a seat in portfolio reviews
Contexts drawn around tables	A "Transactions context" whose language no user recognizes	Draw contexts around language and responsibility; ask what "account" means there

Failure mode	Symptom	Fix
One microservice per capability	45 services, each calling half the others	Services follow bounded contexts, not capabilities
Heat map as a political instrument	No capability scores below 3	Every score cites a metric, a control result or an audit finding

The Chief Architect's View

The capability map is one of the few architecture artifacts a CFO will read and use. That makes it more valuable than its effort suggests, and more dangerous if it is wrong.

What to decide. Five decisions belong to the Chief Architect and Finance together:

1. **The level-1 structure**. It sets the frame for years. Decide it early with the CAO, and change it rarely.
2. **The naming rules**. Nouns, no systems, no departments, no brands. Publish them and enforce them in the EA repository.
3. **Capability owners for every level-2 capability**. Owners come from Finance; architects propose, Finance decides. Where no owner exists, record that as a gap with a deadline, not a blank cell.
4. **What to do about each overlap**. Not to remove all duplication, but to decide each one: accept with a reason, converge with a date, or retire with a plan.
5. **Where bounded contexts sit and which relationships are mandatory**. ACLs at vendor boundaries that would otherwise spread, published language between brand platforms and finance, no shared databases across contexts. These become standards in Principles That Decide Things.

Who decides. At Northmere, Dana holds final decision rights on mandatory enterprise-application standards; Leila owns the capability definitions in her domain and signs off map changes; the Keystone design authority, chaired by Dana with Leila, decides overlaps. That split is deliberate: architects own the structure and the standards, Finance owns the meaning and the outcomes.

What to watch.

- **Overlap count over time**. The four highlighted overlaps should fall to zero decided-but-unresolved items by the end of Keystone year 3, with any remaining duplication recorded as an accepted choice.
- **Capabilities without owners**. Northmere started with six; the target for the end of 2027 is none.

- **Spreadsheet-implemented capabilities**. Of the ~70 spreadsheets-in-the-loop in the interface inventory, the ones that implement a capability (rather than a report) are architecture debt, tracked in Technical Debt on the Balance Sheet.
- **Map stability**. If level 2 changes more than a few names a year, the map is tracking systems or organizations, not capabilities.

The long-term consideration. Northmere will acquire more brands. The capability map is the template for every acquisition: map the acquired company's applications onto the existing capabilities in the first month, and the overlaps appear immediately, before anyone has argued for keeping their tools. That is the cheapest moment to decide them.

Questions to Ask

Take these into the first capability workshop or the next portfolio review:

1. Would every capability name on our map still be true if we replaced every system and reorganized every team?
2. Which capabilities do we perform more than once, and for each, where is the decision that made it so?
3. Who is the named Finance owner of each level-2 capability, and what decision rights do they actually have over its applications?
4. Which capabilities are performed only in spreadsheets or manual work?
5. Are R2R, O2C and P2P on our capability map? If so, which view are we really drawing?
6. What does "account" mean in each of our finance systems, and where do those meanings meet?
7. At which vendor boundaries have we built an anticorruption layer, and where have we become conformist without deciding to?
8. Do our maturity scores agree with our control results, audit findings and reconciliation metrics?
9. Which team owns the minority implementation of several overlapping capabilities, and what does that imply for sequencing?
10. When we next acquire a company, how long will it take to map its applications onto our capabilities?

The capability map tells Northmere what it must be able to do and where it does the same thing twice. It does not yet tell Dana which of the 214 applications to invest in, tolerate, migrate or eliminate, or what each one costs per unit of capability delivered. That is the job of the next chapter, Manage the Application Portfolio as an Investment, which takes the capability-to-application trace built here and turns it into an investment decision list.

Sources and Evidence

Vendor documentation

- SAP Completes Acquisition of LeanIX — SAP News, 2023; supports the date of SAP's ownership of LeanIX and the independence caution about vendor-owned EA tools.

Standards and regulation

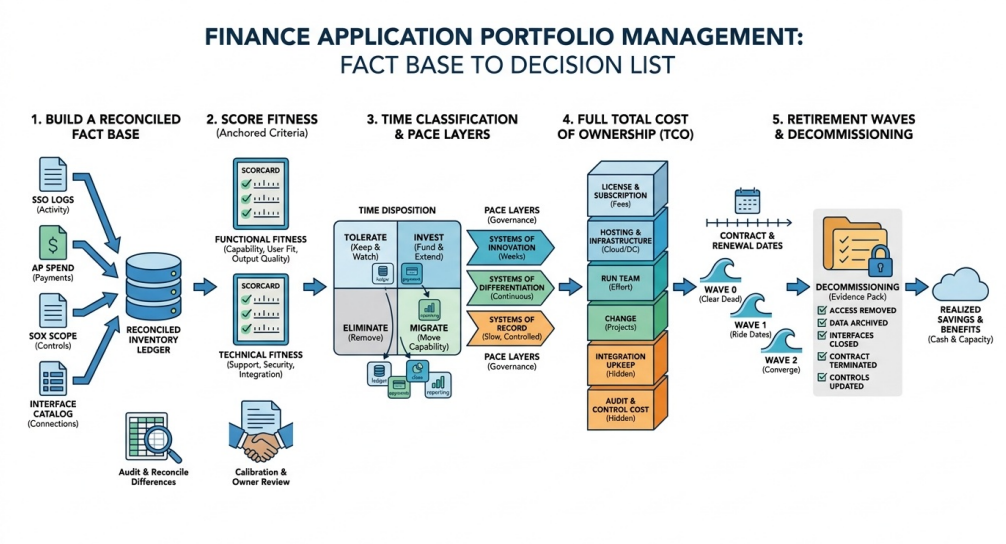
- Business Architecture Key Definitions — Government of Canada GCcollab wiki, reproducing the BIZBOK Guide v12 glossary; supports the definitions of capability and value stream. The current BIZBOK edition is published by the Business Architecture Guild; check its version before quoting.
- ArchiMate 3.2 Specification — The Open Group, 2022; the modeling language whose strategy elements (Capability, Value Stream) support capability and value-stream views.

Practitioner and research sources

- Domain-Driven Design Reference — Eric Evans, 2015; supports the definitions of bounded context and ubiquitous language, the list of context-mapping patterns, and Core Domain and Generic Subdomains.
- Bounded Context — Martin Fowler, 2014; supports the explanation of one term carrying several meanings across contexts.
- Conway's Law — Martin Fowler, 2022; supports the link between team structure and context boundaries and the inverse Conway maneuver.
- Team Topologies, 2nd edition — Matthew Skelton and Manuel Pais, IT Revolution, 2025; background for team ownership of contexts, developed in Who Owns What: Organizing Architecture Across Finance and Technology.

Northmere is fictional; its systems, figures and people are invented for teaching. The capability map, heat-map scales and architectural recommendations are the author's judgment.

6. Manage the Application Portfolio as an Investment



IN THIS CHAPTER, YOU WILL: Learn how to turn an inventory of finance applications into a decision list. The steps are a fact base you can trust, fitness scores with written criteria, the TIME classification, pace layers, a full total cost of ownership, and retirement waves that end in real decommissioning.

WHY FINANCE CARES: The CFO's savings target and the controller's control burden both sit in the portfolio. Every application that should have been retired still costs license fees, run effort, reconciliations and audit testing, and none of those costs appear on the line in the IT budget that carries its name.

WHY ARCHITECTS SHOULD CARE: A portfolio decision is an architecture decision with a price and a date. The application portfolio is where the target state meets the budget and the contract calendar. A Chief Architect who cannot say what each application costs, what it does and when it can go has no credible way to fund a target state.

KEY POINTS:

- **The inventory is a ledger, and it has to reconcile.** Check the EA repository against single sign-on (SSO) logs, accounts-payable spend, the SOX scoping list and the interface catalog. A portfolio that matches none of them is an opinion with a database behind it.
- **Score fitness on written evidence and cost applications in full.** Anchored criteria and calibration sessions keep TIME classifications honest. Total cost of ownership must include integration upkeep and audit and control cost, which together are often a quarter or more of what a finance application really costs.
- **Retire in waves that follow contract dates, and count nothing as retired until it is decommissioned.** Renewal and notice dates set when savings can be taken. Sunset means archived data with a read path for auditors, removed access, closed interfaces, an updated control matrix and a terminated contract, not a lifecycle field changed to "End of life".

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Capability Creep** — The portfolio is the only place where three reconciliation tools and two payout engines appear side by side with their costs, so it is where the overlap gets decided.
- **Spreadsheet Glue** — End-user computing that performs key controls is missing from most application inventories. This chapter registers it, scores it and gives it an owner.
- **The Borrowed Roadmap** — If no one tracks renewal and notice dates, contracts renew automatically, and the vendor's sales cycle decides what the estate keeps.

Application portfolio management (APM) is the practice of managing an organization's applications the way an investor manages holdings. It keeps a fact base on what each application does, who uses it, what it costs and how healthy it is. It decides, on stated criteria, whether to invest in each application, keep it as it is, replace it or remove it, and it follows through on the decision. In a finance estate the practice matters more than elsewhere. Finance applications are expensive to run, sit inside the scope of financial-reporting controls, and hold records that the law requires to be kept long after the application stops being useful.

Most enterprises already own an EA repository full of application records. Few can use it to answer the CFO's plain question: **which systems will we switch off, when, and what will that save?** The repository was filled once for a project and never maintained. The cost fields are empty or show only license fees. The classifications were assigned in workshops without evidence. And the retirements it records are lifecycle fields changed in the tool, not systems that were actually shut down.

This chapter treats the portfolio as an **investment decision instrument**. It shows how to build a fact base that reconciles to independent sources, how to score fitness so that the scores mean something, and how to classify applications with the TIME model and the pace-layer model. It then covers the full cost of a finance application, why contract dates are architecture inputs, and the discipline that turns a decision into a decommissioned system. The capability map from Map What Finance Does Before What Runs It gives the portfolio its spine, and the target state in The Middle Matters: Transition States and the Roadmap gives it its destination.

Setting the Scene at Northmere: 214 Applications and One Question

Northmere, the fictional multi-brand commerce group this book follows (see Meet Northmere: Five Brands, One Set of Books), approved Program Keystone in December 2026. Keystone is a five-year finance transformation with a budget of about $310 million. Its business case promises about **$95 million a year of run-rate savings by 2030** from retired systems, less manual effort and lower payment and bank costs. Year 1 commits to a portfolio return-on-investment (ROI) baseline.

In April 2027 Tomás Ferreira, the Group CFO, asks Dana Whitlock, the Chief Architect for Enterprise Applications and Financial Systems, a direct question. Which applications does the savings case retire, in which year, and how much of the $95 million do they account for? The board's audit committee will ask him the same thing in July.

Dana's team has SAP LeanIX, the EA repository Northmere uses for its application fact base. On paper it holds 203 finance-relevant application records; after the reconciliation described below, the baseline becomes **214 finance-relevant applications**. The finance estate also contains about **1,100 interfaces**, roughly **240** of them flat-file transfers and about **70** of them undocumented spreadsheets that sit inside a process (end-user computing). The duplicates are well known:

- two partner-payout engines, Stayloft Finance Platform's (SFP's) and a Wayfare engine for ground-transport suppliers;
- three billing approaches, SFP commission billing, SAP FI-CA invoicing and Stripe Billing;
- reconciliations performed in BlackLine, in a OneStream module bought in 2022 and rarely used, and in Oracle Fusion for Wayfare intercompany;
- Vertex for indirect tax alongside Oracle's tax engine in some Wayfare markets.

The first audit of the repository is sobering. Only 41% of records carry an annual cost, and most of those show the license line alone. Only 58% have a named business owner. The lifecycle fields say twelve applications are "End of life", and nine of those still have

active users in the SSO logs. Nobody can say from the repository which applications hold the evidence for SOX key controls.

Dana tells Tomás she can give him a decision list in ten weeks and a funded wave plan by the end of Q3. She also tells him what he will not get: a single figure that adds every hoped-for retirement together. The list will carry a confidence level for each saving and a date before which the saving cannot be taken. The rest of this chapter follows how her team does it.

Concepts: What a Portfolio Decision Is Made Of

The application and the fact sheet

An **application**, for portfolio purposes, is a deployed software system that users or other systems depend on and that someone pays to keep running. A vendor product, a module of a vendor platform that is licensed and operated separately, a custom service or a spreadsheet that performs a control can each be an application. The question is whether it carries its own cost, owner and decision. The rule I use: **if it can be retired on its own, it gets its own record**. The OneStream reconciliation module is a separate record, because Northmere can stop using and paying for it without touching consolidation. A posting rule inside S/4HANA is not.

A **fact sheet** is the application's record in the EA repository. Tools such as SAP LeanIX organize their content around fact sheets for applications, IT components, interfaces and business capabilities. SAP completed its acquisition of LeanIX on 8 November 2023. The repository used to rationalize an SAP-heavy estate therefore belongs to the ERP vendor, so I keep its data exportable and treat the tool itself as a dependency with an exit path (Own Your Roadmap: Vendor Strategy Without Lock-In Panic).

Lifecycle states

A **lifecycle state** records where an application is in its life. EA tools ship with phase fields, and SAP LeanIX's application roadmap reports show phases such as planned, phased in, active and phased out, with end of life as a further value. For finance, I recommend making the end of the lifecycle finer than the tool's default, because the costly part of a finance application's life is often the last part:

Figure 1: *A finance application's life does not end when users stop using it. It ends when its records leave retention. The states after "Active" are where retirement value is either taken or lost.*

The two states that matter most are **Retiring** and **Decommissioned**. An application in Retiring still costs money. It still has users with access, it still has interfaces that someone may restart, and it still sits in audit scope. Northmere's rule, adopted in May 2027: **no application moves to Decommissioned without an evidence pack**, and no saving is booked in the Keystone benefits register until it does.

Business value, functional fitness and technical fitness

Portfolio models assess an application on two kinds of quality, under various names.

- **Functional fitness** (often called business fit or functional fit) asks how well the application supports the capabilities and processes it serves, today and against known future demands.
- **Technical fitness** asks how well the application suits the technical environment: support status, security, integration quality, operability, ease of change and conformance to standards.

Some models add **business criticality**, which belongs to the capability rather than the application: the general ledger is critical whichever system runs it.

The TIME model

The **TIME** model sorts applications into four dispositions on the two fitness axes. It is widely attributed to Gartner; I have not found a primary Gartner document, and most published explanations come from EA-tool vendors such as SAP LeanIX. The usual reading:

Disposition	Functional fitness	Technical fitness	What it means
Tolerate	Low or adequate	High	Keep it running and avoid major investment. Watch it and reassess it on a schedule.
Invest	High	High	Fund it, extend it, consolidate onto it.

Disposition	Functional fitness	Technical fitness	What it means
Migrate	High	Low	The need is real but this platform is the wrong home for it. Move the capability elsewhere.
Eliminate	Low	Low	Remove it. Its users go to an existing alternative, or the need goes away.

Variants exist. Some practitioners read the M as "Modernize", which covers re-platforming the same application as well as moving to a different one. The labels matter less than two properties that are easy to lose. First, **TIME is a time-bound statement**. Tolerate is a holding position with a review date, not a permanent home. Second, **TIME is a disposition, not a plan**. "Migrate" says nothing about where to, when, or at what cost. The wave plan answers that.

Pace layering

The **pace-layered application strategy**, popularized by Gartner from around 2011–2012, classifies applications by how fast they change and how distinctive they are (secondary summary). The three layers are:

- **Systems of Record**: established systems that run core transactions and hold critical master data. They change slowly and are often subject to regulation.
- **Systems of Differentiation**: systems that support processes unique to the company or its industry. They are reconfigured often as practices or customers change.
- **Systems of Innovation**: systems built for new needs or opportunities, with short, experimental lives.

Gartner has since moved its narrative toward "composable" ERP, so I present pace layering as a thinking tool, not as current doctrine. It is still the most useful single idea for a finance portfolio, because it tells you **which applications deserve which kind of governance, funding and change cadence**. The secondary sources also record a warning that is easy to forget: the modules of one ERP package can belong to different layers.

Total cost of ownership and technology ROI

Total cost of ownership (TCO) is the full cost of owning an application over a defined period: what it costs to acquire, run, change, connect, control and eventually retire. This chapter owns the term. It is used throughout this book with the six-component model defined below. **Technology ROI** compares the benefits that a technology investment produces with its cost, measured against a baseline taken before the change. Both are needed. TCO tells you what you are paying. ROI tells you whether a change in what you pay, or what you get, was worth making.

Architecture: The Portfolio as a Decision Pipeline

A portfolio practice is a pipeline. Facts come in from independent sources and are reconciled into the fact base. Scoring and classification turn facts into dispositions. Dispositions are sequenced into waves against contract dates and dependencies, and waves end in decommissioning, which feeds evidence back into the fact base and into the benefits register.

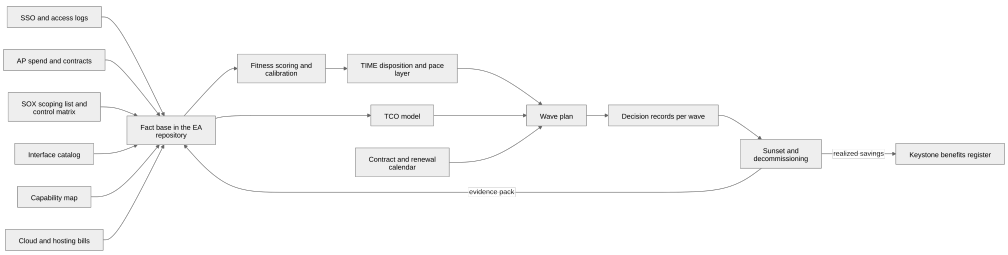


Figure 2: *The portfolio is a pipeline from reconciled facts to evidenced retirements. Every decision traces back to sources outside the EA tool, and every saving traces forward to a decommissioning evidence pack.*

A portfolio manager in Dana's team owns the fact base and the TCO model, with a cost analyst on loan from Tomás's organization. Domain architects own scoring, with process owners confirming functional scores. Procurement owns the contract calendar under architecture review. Dana owns the wave plan, approved through the Keystone design authority, and Keystone program finance owns the benefits register.

Two design choices deserve explanation.

First, the **fact base is fed, not filled**. A repository that depends on application owners updating records goes stale within two quarters. Northmere connects the repository to sources that change by themselves: SSO logs for active users, accounts-payable spend for vendor cost, the cloud bill for hosting, the interface catalog for integration counts. Owners confirm and annotate; they do not type in what a system already knows. The cloud bill becomes a trustworthy hosting figure per application only once it is tagged,

allocated and tied to the ledger, which FinOps Meets the Ledger: Technology Spend as Financial Data designs.

Second, the **decision record is per wave, not per application**. Retiring the Wayfare payout engine and moving 14 interfaces, a supplier master and two key controls is one decision with one set of consequences. It goes through the architecture decision record and review process described in Architecture Governance That Speeds Decisions. The application records link to it.

In Practice: Building a Fact Base You Can Reconcile

Reconcile the inventory before you score it

Dana's portfolio manager starts by treating the inventory the way a controller treats a balance-sheet account: **reconcile it to independent evidence and explain every difference**. The repository is one source. Four others exist outside it:

1. **SSO and access logs.** Every application with real users authenticates somewhere. An application with no logins in 180 days is either a back-end service (and should show up in the interface catalog) or dead.
2. **Accounts-payable spend.** Every vendor application is paid for. A software vendor paid by Northmere with no application in the repository is a missing record; an application with a vendor and no payment is a stale record or a free tier.
3. **The SOX scoping list.** Mei Tanaka's internal controls team keeps a list of systems in scope for internal control over financial reporting (ICFR) because a key control or key report depends on them. Every one of them must be in the portfolio, flagged.
4. **The interface catalog.** Every interface has two ends. An interface whose end is not in the portfolio points to an application nobody registered.

The reconciliation takes four weeks and changes the baseline. It adds 19 applications that the repository never held, among them a commission-adjustment tool that Stayloft's partner-accounting team bought on a credit card and a Python service that splits Northmere Ads invoices by tax jurisdiction. It merges or removes 8 duplicates and dead records. Mei's list contains **90 in-scope systems**, and 11 of them were either missing or mapped to the wrong application. That is the finding that gets the audit committee's attention: the architecture repository and the controls program disagreed about where financial reporting happens. The reconciled baseline is the **214** applications used from here on.

The fields that matter, and how complete they are

A finance fact sheet needs fewer fields than most EA tools offer and more discipline about each. These are the fields Northmere makes mandatory, with completeness before

and after the ten-week push:

Field	Why finance needs it	Source of truth	Complete, April 2027	Complete, June 2027
Business owner and technical owner	Someone accountable for the decision and for the control evidence	Org data from Workday HCM; owner confirms	58%	97%
Annual cost (six components)	TCO and savings cases	AP spend, cost centers, cloud bill, time records	41% (license only)	91%
Active users (90-day)	Usage is the cheapest evidence of value, or of its absence	SSO logs	33%	94%
Contracts: vendor, renewal date, notice period, auto-renewal	Sets when a saving can be taken	Contract repository	47% of vendor apps	100% of vendor apps
Lifecycle state	Distinguishes phasing out from decommissioned	Owner, checked against SSO and interfaces	70% (12 stale)	100%
Capability mapping	Shows overlap; connects the portfolio to the target state	Capability map v1 (Map What Finance Does Before What Runs It)	52%	96%

Field	Why finance needs it	Source of truth	Complete, April 2027	Complete, June 2027
Data classification	Personal data, cardholder data, financial records; drives archive and access design	Data owner; Marco Bellini's classification scheme	39%	88%
SOX relevance	In-scope systems carry control cost and change constraints	Mei's scoping list	61%	100%
Retention obligations	Determines the archive and read path at sunset	Record types held × jurisdictions	0%	64%

The last row is honest about what ten weeks can do. Retention obligations depend on which record types an application holds and in which jurisdictions, and many owners had never been asked. The team recorded "unknown" rather than guessing, and every application proposed for retirement must complete the field before its wave starts.

Registering end-user computing

The 70 spreadsheets in the interface inventory are not in the 214. They are end-user computing (EUC): spreadsheets, desktop databases and scripts built and run by finance users outside IT's change control. They matter to the portfolio for one reason. **Several of them perform or support key controls**, and so carry audit cost and risk that nobody has priced.

Dana's team does not try to register every spreadsheet in the group. It registers an EUC as a lightweight portfolio record when any of three things is true: it performs or evidences a key control, it transforms data that feeds a ledger posting or a reconciliation, or a close task depends on it. That rule brings 31 of the 70 into the portfolio, among them the spreadsheets that hold the mappings from the Wayfare and Tablekin charts of accounts to the group chart in OneStream. Each gets an owner, a control reference

and a disposition. Most are classified **Migrate**, with the destination named: the 14 mapping workbooks were already moving into governed, effective-dated mapping rules in OneStream from May 2027 (Finance Data Architecture: Who Owns the Truth), and reconciliation and accrual workbooks move into BlackLine. The scoring and costing below apply to them as well.

In Practice: Scoring Fitness Without Theater

Anchored criteria

A fitness score is only as good as the criteria behind it. A five-point scale with no definitions produces scores that reflect the confidence of whoever filled them in. Northmere scores each application against written criteria, each with **anchors** that describe what a 1, a 3 and a 5 look like and what evidence is required.

Dimension	Criterion	Weight	Anchor for 1	Anchor for 5	Evidence
Functional	Capability coverage	25%	Covers under half of the mapped capability; the rest is done elsewhere or by hand	Covers the mapped capability for all brands and entities in scope	Capability map, process walk-through
Functional	Manual effort around the application	20%	Recurring manual workarounds, rekeying or spreadsheet steps every period	No routine manual steps outside designed approvals	Close task list, time records

Dimension	Criterion	Weight	Anchor for 1	Anchor for 5	Evidence
Functional	Output quality	20%	Its output is routinely adjusted or reconciled by hand before use	Its output feeds the ledger or reports without adjustment	Manual journal analysis, reconciliation breaks
Functional	Regulatory and policy readiness	15%	Cannot meet a dated mandate (for example an e-invoicing deadline) without major change	Meets known mandates by configuration	Regulatory roadmap
Functional	User fit	20%	Users avoid it or keep parallel records	Users rely on it as the record	SSO data, survey, shadow-tool discovery
Technical	Support and lifecycle	20%	Vendor support ended or ends within 12 months; or no maintaining team	In mainstream support, current release, funded team	Vendor lifecycle dates, release level
Technical	Security and access	20%	Shared or privileged access without monitoring; SoD conflicts open	Role-based access, SoD rules enforced, access reviews clean	Access review results, audit findings

Dimension	Criterion	Weight	Anchor for 1	Anchor for 5	Evidence
Technical	Integration quality	20%	Flat files or manual uploads without completeness checks	Contracted APIs or events, with record counts and control totals	Interface catalog
Technical	Operability and resilience	15%	Frequent incidents; untested recovery	Within its resilience tier's targets; recovery tested	Incident data, recovery test records
Technical	Changeability	15%	Upgrades or changes take quarters; heavy customization	Changes within the team's normal cadence; upgrades routine	Change lead time, upgrade history, custom-object count
Technical	Standards conformance	10%	Violates mandatory standards without a waiver	Conforms; no open waivers	Conformance scans, waiver register

Segregation of duties (SoD) here means the rule that no one person can both initiate and approve, or both record and reconcile, the same transaction. Its controls are the subject of Controls by Design, Not by Audit Season.

Calibration and independence

Two practices keep the scores honest.

- **The owner does not score alone.** The application owner proposes; a domain architect who does not own the application reviews; for functional scores, the process owner confirms. Leila Haddad's record-to-report lead confirms the functional scores for every application that touches the general ledger.

- **Scores are calibrated across domains**. Once a quarter the domain architects sit together and compare applications that scored the same. If the Wayfare payout engine and SFP's payout module both score 4 on integration quality but one runs on flat files with no control totals, one of the scores is wrong. Calibration takes half a day and removes more noise than any weighting scheme.

The scores then place each application on the two TIME axes. The thresholds are explicit: a weighted score of 3.0 or above is "high" on that axis, below 3.0 is "low". Applications within 0.3 of the line on either axis are flagged as **borderline** and reviewed by hand, because a scoring model should not be trusted to make close calls.

What the scores said at Northmere

The first full pass, completed in June 2027, classified the 214 applications as follows:

Disposition	Applications	Share of annual TCO	Examples
Invest	38	41%	SAP S/4HANA group core, SAP FI-CA, BlackLine, OneStream consolidation and planning, Kyriba, Vertex, SFP merchant subledger
Tolerate	97	31%	Workday Financials for Tablekin and Oracle Fusion for Wayfare (both pending the 2027 Q4 convergence decision), Coupa, SAP Concur, most reporting tools

Disposition	Applications	Share of annual TCO	Examples
Migrate	41	19%	Wayfare payout engine, legacy billing variants, the OneStream reconciliation module (its 22 accounts move to BlackLine), reconciliation and accrual workbooks (as registered EUCs)
Eliminate	38	9%	Applications with no active users in 180 days, duplicate reporting instances, a second expense-audit tool, RateKeeper (see below)

Two results were not what anyone expected. The two brand ERPs landed in Tolerate, not Migrate. That is deliberate: their fate depends on the convergence decision due in 2027 Q4 (Central Finance versus an accounting-hub pattern), and scoring them as Migrate before that decision would have prejudged it. Tolerate with a dated review is the honest disposition. And 38 Eliminate candidates account for only 9% of TCO. **The large savings are in the Migrate column**, and Migrate costs money before it saves any.

Pace Layers in a Finance Estate

The TIME classification says what to do with an application. The pace layer says **how to govern it while it lives**. In a finance estate the layers differ less in technology than in three things: how often they change, how much audit scope they carry, and how they are funded.

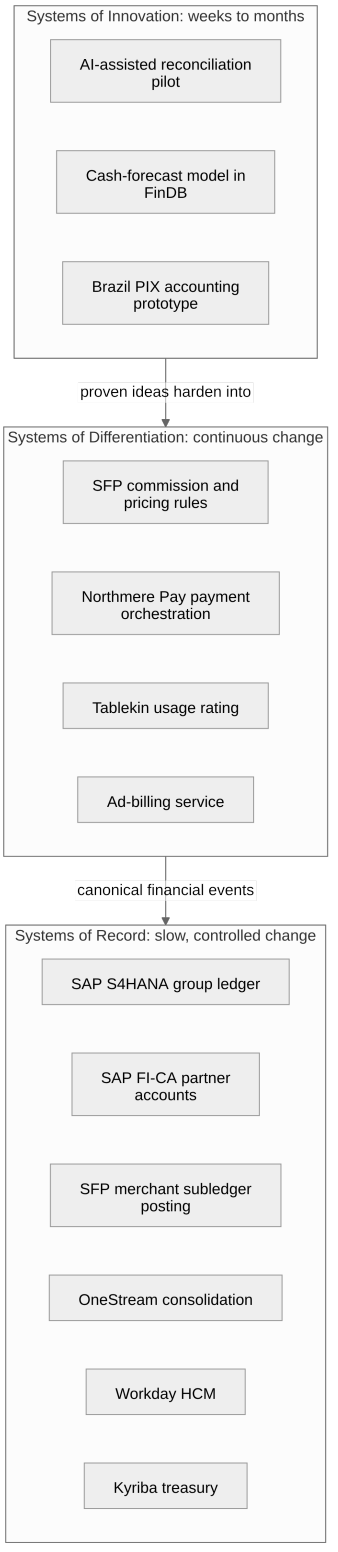

Systems of Innovation: weeks to months
AI-assisted reconciliation pilot
Cash-forecast model in FinDB
Brazil PIX accounting prototype
proven ideas harden into
Systems of Differentiation: continuous change
SFP commission and pricing rules
Northmere Pay payment orchestration
Tablekin usage rating
Ad-billing service
canonical financial events
Systems of Record: slow, controlled change
SAP S4HANA group ledger
SAP FI-CA partner accounts
SFP merchant subledger posting
OneStream consolidation
Workday HCM
Kyriba treasury

Figure 3: *Northmere's finance applications by pace layer. Differentiating systems change weekly and feed the systems of record through a stable event contract, so fast change on top does not become uncontrolled change underneath.*

The layer assignments carry consequences that Dana writes into Northmere's standards:

	Systems of Record	Systems of Differentiation	Systems of Innovation
Typical finance examples	General ledger, subledger posting, consolidation, payroll, treasury	Pricing and commission rules, payment orchestration, usage rating, partner payout logic	Pilots: AI-assisted matching, forecasting models, new-market prototypes
Change cadence	Planned releases, quarterly or slower; close-calendar freezes	Continuous delivery inside contracted interfaces	Weekly; disposable
Governance	Full ITGC change management; review board for structural change; clean-core rules	Advice process for design; ITGC gate where posting logic changes; contract tests on events	Advice and a time box; no posting to ledgers; no production financial data without approval
SOX posture	Usually in scope	In scope where the system computes amounts that reach the ledger	Out of scope by design; any move into scope is a promotion decision
Funding	Multi-year program or run budget; TCO-driven	Product team funding; value-driven	Experiment budget with a kill date

Three lessons from applying the layers to finance.

One application can span layers, and the boundary inside it matters. SFP's commission and pricing rules change every few weeks; that is differentiation. Its merchant subledger and the daily summarized postings to S/4HANA, about 38,000 GL lines a day, are a system of record. The FY2026 significant deficiency on the SFP

payout engine (developer production access, undocumented payout rule changes) is partly a layer error: rules that reach the ledger were governed like differentiating code. The remediation draws the line inside SFP and applies record-layer change control to everything that computes an amount posted to the ledger.

The interface between layers is where the controls live. A differentiating system can change weekly without destabilizing the ledger only if what it sends down is a stable, versioned contract with completeness checks: the canonical financial events (`PaymentSettled`, `PartnerPayoutInstructed`, `CommissionInvoiced` and the others) introduced in Files, Events and APIs: How Finance Systems Talk and owned by Canonical Events: A Common Language for Money. Without that contract, every change on top is a change to the record.

Innovation must not become record by accident. A pilot that starts posting to a ledger has changed layer without a decision. At Northmere it is promoted through a decision record, with an owner, a control assessment and a cost line, before it posts, approves or changes financial data. Otherwise it is **Nobody's Bot** in waiting.

The Full Cost of a Finance Application

Six components

License and hosting are what most portfolios record, because they arrive as invoices. In finance they are often less than half the cost. Northmere's TCO model has six components, each annual:

Component	What it includes	How Northmere estimates it	Often forgotten because
License and subscription	Subscription fees, support and maintenance, true-ups, third-party components	AP spend, contracts	Rarely forgotten; it is the invoice

Component	What it includes	How Northmere estimates it	Often forgotten because
Hosting and infrastructure	Cloud, data center, databases, environments (including non-production), backup, disaster recovery	Cloud bill tags, data-center allocations	Non-production and DR environments are tagged to a platform, not an application
Run team	Operations, support, monitoring, incident handling, user administration	Time records × fully loaded rate	People are budgeted by team, not by application
Change	Enhancements, regulatory changes, upgrades, testing	Time records, project codes	Upgrades are funded as projects, so they do not appear in the run cost
Integration upkeep	Maintaining interfaces, mappings, middleware share, failed-message handling	Interfaces × average upkeep, plus middleware allocation	The cost lands on the integration team, not on either end of the interface
Audit and control	Control operation, evidence preparation, access reviews, internal testing, the attributable share of external audit IT work, remediation	Controls testing hours from the internal controls team; control owners' time	It is spread across finance, internal controls and the auditor's fee, and no one adds it up

Northmere values staff time at a **blended fully loaded rate of $165,000 per full-time equivalent (FTE) year** for finance-systems engineering and operations, a planning rate agreed with Tomás's finance business partner. Time-based figures are cost-based estimates: freeing the time releases capacity, and it becomes cash only if roles are

removed or hiring is avoided. The benefits register keeps cash and capacity savings in separate columns for that reason.

The **audit and control** line is the one I insist on. A SOX-relevant application carries key controls that someone operates every period, access reviews every quarter, change-management evidence for every release, testing by the internal controls team and testing by the external auditor. Retiring the application, or consolidating its controls onto a platform where they already run, removes that cost. A portfolio that leaves it out undervalues every consolidation in the finance estate.

When Dana's team completed the six-component costing for the 214 applications, the total came to about **$172 million a year**. The license and hosting lines, the figures the IT budget had always shown, accounted for about $118 million of it. The remaining **$54 million**, **about 31%**, was run effort, change, integration upkeep and control cost that had never been attributed to an application.

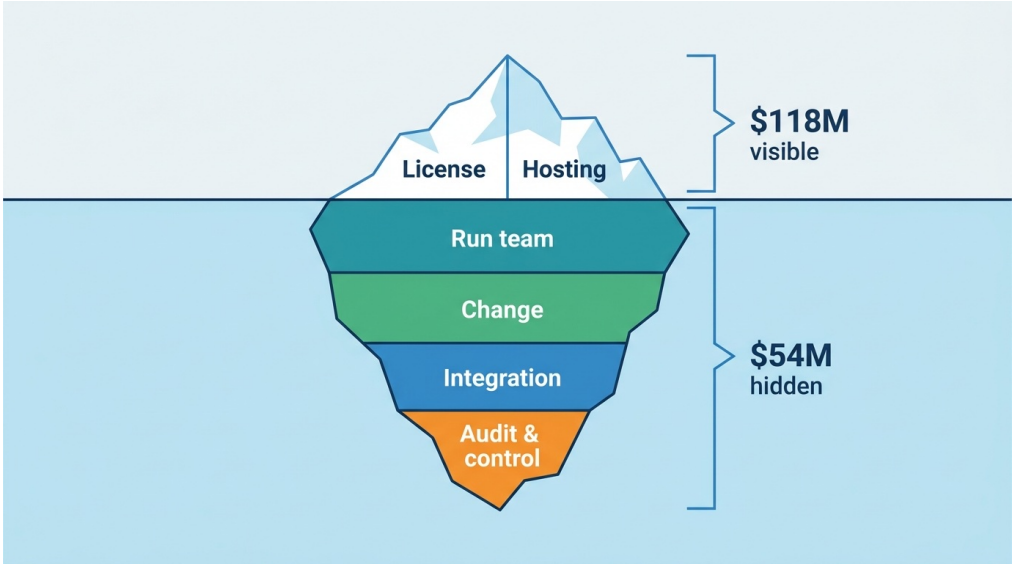


Figure 4: *The IT budget showed about $118 million of license and hosting; the full six-component cost of Northmere's 214 finance applications was about $172 million, with $54 million never attributed to any application.*

A worked TCO comparison

Three retirement candidates show how the model changes decisions. All figures are annual, in thousands of US dollars, for 2027, and carry the confidence labels the benefits register uses.

- **The Wayfare payout engine** pays ground-transport suppliers (transfer companies, rail partners) for Wayfare's merchant bookings. Hana Kowalski's team runs it. Its function overlaps SFP's partner payout capability, and Keystone's year-3 goal is to retire the duplicate.
- **The OneStream reconciliation module** was bought in 2022 with the consolidation platform. OneStream describes account reconciliations and transaction matching among its specialized applications (OneStream FY2025 Form 10-K). At Northmere it is used for 22 balance-sheet accounts of corporate entities, while BlackLine reconciles about 6,800 accounts.
- **RateKeeper** is a home-built indirect-tax rate service that Wayfare wrote in 2016 for rail and transfer bookings in five markets. It was built in the Oracle E-Business Suite (EBS) era and still runs on the colocated infrastructure that hosts the read-only EBS archive. It overlaps Vertex, Northmere's group tax engine.

Component ($k a year)	Wayfare payout engine	OneStream reconciliation module	RateKeeper
License and subscription	60 (two commercial components)	310	0
Hosting and infrastructure	380	0 (inside the SaaS subscription)	140
Run team	990 (6.0 FTE)	66 (0.4 FTE)	248 (1.5 FTE)
Change	660 (4.0 FTE)	17 (0.1 FTE)	83 (0.5 FTE)
Integration upkeep	210 (14 interfaces)	30 (3 interfaces)	60 (6 interfaces)
Audit and control	340 (9 key controls; bank-detail change control; quarterly access reviews)	25 (2 key reconciliations evidenced in it)	120 (manual review of rate changes; tax determination control)
Annual TCO	**2,640**	**448**	**651**
Of which visible in the IT budget as license plus hosting	440 (17%)	310 (69%)	140 (22%)

Component ($k a year)	Wayfare payout engine	OneStream reconciliation module	RateKeeper
One-time exit cost	3,400 (supplier migration, bank-detail re-verification, two parallel payout cycles)	90 (move 22 accounts to BlackLine, re-evidence 2 controls)	520 (configure five markets in Vertex, regression-test historical bookings)
Added cost at the receiving platform	550 (shared payout platform capacity and run effort)	40 (BlackLine user licenses)	110 (Vertex transaction volume)
Net run-rate saving	**2,090**	**408**	**541**
Simple payback on exit cost	about 20 months	about 3 months	about 12 months
Date before which the saving cannot start	Supplier contracts reference the payout schedule; 2029 per Keystone	OneStream renewal, 31 December 2027; notice due 30 September 2027	EBS archive colocation ends 31 December 2028
Confidence	Medium (exit cost depends on the supplier count at migration)	High	Medium

The table changes three conversations.

RateKeeper looks cheap until you count it properly. Its visible cost is $140,000 of hosting. Its full cost is more than four times that, and $120,000 of it is control cost: because RateKeeper's rate tables are maintained by hand, every rate change needs a documented review, and the tax team tests the determination control quarterly. Its retirement is also forced by a date it does not control. When the EBS archive is decommissioned in 2028, its hosting goes with it. The decision is not whether to retire RateKeeper but whether to re-host it for a year or move its markets to Vertex before the colocation ends. Moving costs $520,000 once and saves money from 2029. Re-hosting costs about as much and saves nothing.

The OneStream module is a contract problem, not a technical one. The saving is small but nearly certain, and the payback is three months. All of it depends on one date. The module is co-termed with the OneStream master subscription, which renews on 31 December 2027 with a notice deadline of 30 September 2027. Miss the notice date and

the saving moves out a full term. This is the clearest case for treating **contract dates as architecture inputs**, and it is the first retirement in Northmere's wave plan.

The payout engine's cost is mostly people, and that changes the savings claim. Of its $2.64 million, $1.65 million is run and change effort. Retiring it frees ten FTEs of capacity in Hana's team, but it removes cash cost only if those people move to work that would otherwise need new hires, such as the Brazil launch, or if roles are not refilled. Dana's savings case for Tomás states the $2.09 million net saving as $0.65 million cash and $1.44 million capacity. That honesty costs her a smaller headline number and earns her a figure the audit committee can test.

Technology ROI as Practiced, and Its Limits

Keystone's Year-1 commitment includes a **portfolio ROI baseline**: for each funded decision, the cost and benefit position before the change, so that realized benefits can be measured against it afterward. The practice is simple to state and hard to keep.

At Northmere each decision record that asks for funding carries a value case in three parts: a **baseline** (the six-component TCO of the affected applications and the operational figures the change should move, such as manual reconciliation hours), an **expected benefit** split into cash, capacity, risk reduced and outcomes enabled, each in its own units, and a **measurement plan** with post-implementation reviews six and twelve months after go-live. Risk reduced is stated as the control or finding it addresses, such as the SFP significant deficiency, not as a dollar figure.

ROI works well for consolidation and elimination, where the costs removed are measurable and the baseline is the fact base itself. It works poorly in four situations, and I say so in every Keystone review:

- **Attribution.** If the close shortens from nine working days to seven, how much is due to the reconciliation consolidation, the payments-to-ledger program or a better close calendar? Claimed benefits that add up to more than the observed change are double counting. Northmere assigns each operational improvement to one decision, or splits it explicitly.
- **Avoided cost.** An upgrade not needed, a fine not levied, a material weakness that never appeared: real, and unmeasurable afterward. I state them as reasons and leave them out of the ratio.
- **Different layers earn differently.** A record-layer investment returns lower cost per transaction and fewer audit findings; a differentiation investment returns revenue or speed. One ratio across both favors whatever is easiest to measure.
- **Optionality.** Keeping the 2027 Q4 convergence decision open has value no ROI captures. Claim it only when a dated decision exists to keep open.

The practical result: ROI is a **ranking aid inside a category**, consolidation against consolidation, not one number that ranks the whole portfolio. Across categories the design authority decides on the target state, with ROI as one input.

Contract and License Dates as Architecture Inputs

The OneStream module shows the general point. **The earliest date on which a retirement saves money is set by a contract, not by the architecture.** A portfolio that ignores the contract calendar produces wave plans that cannot be delivered, or savings that arrive a full term late.

The fields that matter, held for every vendor application:

Field	Why it matters for architecture
Term end and renewal date	The date a saving can start
Notice period and auto-renewal	The real decision date is the renewal date minus the notice period
Co-termination and bundling	A module in a bundle can be dropped only when the bundle renews, and sometimes only by renegotiating it
Volume tiers and true-ups	Moving volume onto a platform can trigger a step up in fees; moving it off may not reduce them until the next term
Termination assistance and data return	Determines the exit cost and whether a vendor will help extract records for archive (Own Your Roadmap: Vendor Strategy Without Lock-In Panic)

Dana's team computes a **decision-due date** for each retirement:

> decision-due date = renewal date − notice period − migration lead time − a buffer of one close cycle

For the OneStream module the arithmetic is short: renewal 31 December 2027, notice 90 days, migration lead time 6 weeks for 22 accounts, one close cycle as buffer. The decision was due in mid-July 2027, and it went through the review board on 8 July. For a payout engine the migration lead time is twelve months or more, so the decision is due well over a year before any contract date.

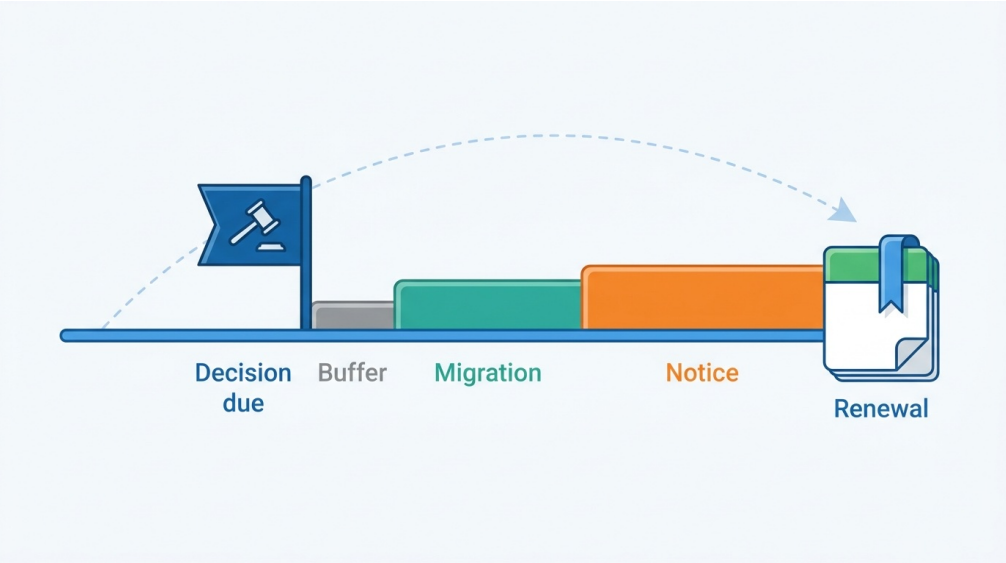


Figure 5: *The decision-due date is the renewal date minus the notice period, the migration lead time and one close cycle; for the OneStream module that put the decision in July for a 31 December renewal.*

Two cautions. First, contract dates are an input, not the architecture. A renewal date is a reason to decide on time; it is not a reason to retire a system that should stay, and the urge to "use the renewal" is a form of **The Borrowed Roadmap** in reverse. Second, the vendors themselves change. OneStream was taken private by Hg, a transaction completed on 1 April 2026. Changes of ownership can change pricing and roadmap priorities at the next renewal, so Northmere reviews strategic-vendor ownership changes as portfolio events.

Rationalization Waves

A **rationalization wave** is a group of retirements and migrations planned, approved and delivered together because they share a trigger, a dependency or a receiving platform. Waves exist because retirements are not independent. Retiring RateKeeper depends on Vertex capacity; retiring the EBS archive depends on RateKeeper leaving the same hosting; retiring the Wayfare payout engine depends on the shared payout capability being ready, which depends on the SFP remediation.

Northmere's wave plan, approved by the Keystone design authority in September 2027:

Wave	Window	Scope	Applications retired	Trigger and dependencies	Exit gate	Run-rate saving (cash / capacity)
0: Clear the dead	Q3 2027 – Q1 2028	Applications with no active users in 180 days; duplicate reporting instances; the OneStream reconciliation module	24	OneStream notice date 30 September 2027; no dependencies beyond archive and access work	Evidence pack per application; control matrix updated for the 2 reconciliations moved to BlackLine	$3.1M / $0.6M

Wave	Window	Scope	Applications retired	Trigger and dependencies	Exit gate	Run-rate saving (cash / capacity)
1: Ride the dates	2028	Wayfare EBS archive; RateKeeper to Vertex; 6 Wayfare satellite tools on the same hosting; 3 superseded tax and reporting tools; 12 control-relevant reconciliation and accrual workbooks moved into BlackLine	11 apps + 12 EUCs	Colocation end 31 December 2028; Vertex configuration for five markets; read path for archived EBS records live before shutdown	Archive accessible to auditors and tax; Vertex determinations reconciled against RateKeeper for one quarter	$4.2M / $2.6M

Wave	Window	Scope	Applications retired	Trigger and dependencies	Exit gate	Run-rate saving (cash / capacity)
2: Converge money movement	2029	Wayfare payout engine to the shared payout capability; billing approaches reduced from three to two; remaining reconciliation outside BlackLine	9	SFP significant deficiency remediated and tested; canonical payout events live; supplier communications	Two parallel payout cycles with zero unexplained differences; supplier bank-detail re-verification complete	$3.8M / $4.6M

Wave	Window	Scope	Applications retired	Trigger and dependencies	Exit gate	Run-rate saving (cash / capacity)
3: One finance core	2029 – 2030	Brand ledgers onto the group finance core; brand-specific close and reporting tools	about 12 (conditional)	Central Finance versus accounting-hub decision (2027 Q4); one group chart of accounts	Parallel close for two periods per brand; consolidation tie-out	to be set by the 2027 Q4 decision (taken in December 2027 for the accounting-hub pattern; see Converge or Co-exist: Living with More Than One ERP)

By the end of wave 2 the plan retires 44 applications and 12 control-relevant EUCs, with about **$11.1 million cash and $7.8 million capacity** of run-rate saving. That is a fraction of the $95 million Keystone target, which also depends on reduced manual effort and lower payment and bank costs, and Dana says so. Tomás now has what he asked for: names, years and amounts, with confidence labels and the dates that constrain them.

Three principles shaped the plan.

- **Start with what is cheap and certain.** Wave 0 saves little, but it proves the sunset discipline on low-risk applications, gives the benefits register its first realized figures and builds trust with the auditor before the payout engine is touched.
- **Let forced dates set the order.** Wave 1 is built around the colocation end date. Work that a date forces is done in the year the date falls, so that it is not done twice.

- **Do not retire into an unready platform.** Wave 2 waits for the SFP remediation. Moving Wayfare's suppliers onto a payout capability with an open significant deficiency would spread the deficiency to a second brand.

The deeper treatment of how to sequence migrations without a big-bang cutover belongs to Modernizing Without Stopping the Close and Changing the Engine in Flight: Modernization Patterns for Finance; the debt carried while waves wait belongs to Technical Debt on the Balance Sheet.

Sunset Discipline: Decommissioning That Counts

Decommissioning criteria

An application is **decommissioned** at Northmere when every item on the following list has evidence attached, checked by architecture and, for SOX-relevant applications, by Mei's internal controls team.

Area	Criterion	Evidence
Usage	No business transactions processed after the cutover date; users moved to the receiving platform	SSO logs; transaction counts at zero for one full close cycle
Data	Records required for retention archived in a readable format with their metadata; archive completeness reconciled against the source (record counts and control totals)	Archive reconciliation report signed by the data owner
Read path	Auditors, tax and legal can search and retrieve archived records without the retired application	Retrieval test with a sample requested by internal audit
Access	User accounts, service accounts, API keys, certificates and privileged access removed; SSO application removed	Access-removal report; identity provider change record

Area	Criterion	Evidence
Interfaces	Every interface to and from the application disabled and removed from the catalog; scheduled jobs deleted; upstream senders confirmed	Interface catalog diff; scheduler export
Controls	Control matrix updated: retired controls closed, migrated controls re-designed and tested at the receiving platform	Control matrix change approved by the internal controls team
Infrastructure	Environments, databases, backups (other than the archive) and DNS entries removed	Cloud resource inventory; hosting termination
Contract	License terminated or reduced; termination confirmed in writing; final invoice checked	Vendor confirmation; AP record
Fact base	Lifecycle set to Decommissioned with the evidence pack linked	EA repository record

The **controls** row is where finance decommissioning differs most from the general IT kind. A key control that ran inside a retired application does not disappear. Either the risk it addressed is gone, or the control must now run somewhere else, designed and tested before the old one stops. Retiring an application mid-year without that work is how a migration silently breaks a key control, which is the outcome the SOX lead fears most. The control side is covered in Controls by Design, Not by Audit Season.

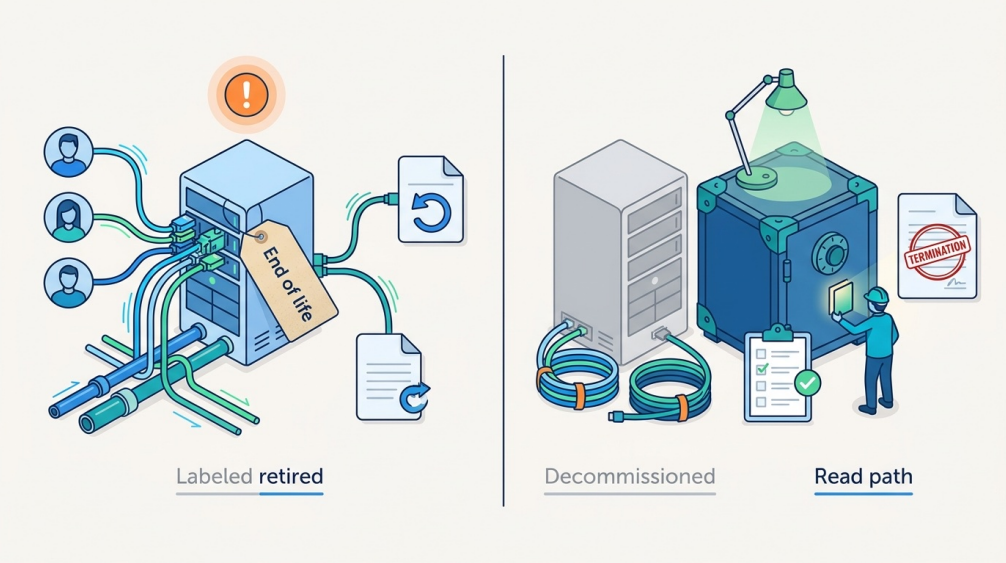


Figure 6: *A lifecycle label is not a retirement: a system counts as decommissioned only when access, interfaces and the contract are gone, its records sit in an archive with a read path, and its controls run somewhere else.*

Retention and the read path

Retiring a finance application rarely means deleting its data. Statutory retention periods vary by country and record type, and the clock usually starts at the end of the year in which the record was created. Some current examples:

- Dutch businesses must keep their records for at least 7 years, and 10 years for records relating to immovable property and for businesses using the EU VAT One Stop Shop Union scheme.
- In Germany, the retention period for accounting vouchers, including invoices, was shortened from 10 to 8 years for records whose period had not expired by 1 January 2025, with a one-year delay for entities supervised by BaFin, Germany's financial supervisor. Books and annual financial statements remain at 10 years.
- Tax audits, transfer-pricing reviews, Pillar Two (the OECD's global minimum tax) and litigation holds can extend the practical period.

The architecture consequence is that **the archive is a system with an owner, a cost and a read path**, and it belongs in the portfolio as its own record, in the "Archive only" state of Figure 1. An archive must keep records retrievable, readable without the original application, complete against the source, and restricted to people with a reason to see them.

Personal data adds a constraint. The GDPR's right to erasure does not apply where processing is necessary for compliance with a legal obligation (Article 17(3)(b)), so records the law requires can be kept for the statutory period. Data that is not required, such as marketing preferences or free-text notes, should not travel into the archive at all. The archive design is the moment to strip it out. Retention design in depth belongs to Finance Data Architecture: Who Owns the Truth.

Wayfare's EBS archive shows the full arc. EBS was kept read-only after Wayfare moved to Oracle Fusion in 2022, and Northmere inherited it with the acquisition in 2024. Vendor support does not force the decision: Oracle has extended Premier Support for EBS 12.2 through at least 2037, according to widely reported announcements, and a read-only archive needs little vendor support anyway. Cost and the colocation contract force it. The 2028 decommissioning extracts the ledger, subledger and invoice records still inside their retention periods into Northmere's archive store, reconciles the extracted balances to the last EBS trial balances, gives tax and audit a search interface and a documented retrieval procedure, and then shuts EBS down.

Trade-offs: How to Run the Portfolio

Each element of the practice involves a choice. These are the main ones, with my verdicts.

Choice	Option A	Option B	When A is right	When B is right	My default for finance
Fact-base model	Owner-maintained records in the EA tool	Records fed from SSO, AP, cloud bills and interface catalogs, confirmed by owners	Small estates with stable ownership	Any estate above about 100 applications, or with frequent reorganizations	B; owner maintenance decays in two quarters

Choice	Option A	Option B	When A is right	When B is right	My default for finance
Scoring depth	Light: a few criteria, owner-scored	Anchored criteria, independent review, quarterly calibration	First pass, to find the obvious eliminations	Any decision that funds a migration	A for a first triage; B before money moves
Cost model	License and hosting	Six-component TCO	Never for finance	Always for finance	B; the forgotten lines are a third of the cost
Definition of retired	Lifecycle field set in the EA tool	Evidence pack checked by architecture and controls	Never	Always	B; nothing else lets savings be audited

The cost of the right-hand column is real. Northmere's portfolio practice takes about four FTEs: a portfolio manager, a cost analyst on loan from finance, and the equivalent of two FTEs of domain-architect time for scoring, calibration and decommissioning checks. That is about $0.7 million a year. Wave 0 alone repays it.

Integration With the Rest of the Estate

The portfolio shares its data with most of the practices in this book. Every application maps to the capabilities it serves (Map What Finance Does Before What Runs It), so duplication is simply a capability with more than one application in the same scope. Waves are approved as decision records, and Tolerate dispositions carry their review dates in the same register (Architecture Governance That Speeds Decisions). The SOX scoping list feeds the fact base, and every decommissioning updates the control matrix (Controls by Design, Not by Audit Season). Technical-fitness scores feed the debt register (Technical Debt on the Balance Sheet), and the contract calendar is shared with vendor tiering (Own Your Roadmap: Vendor Strategy Without Lock-In Panic). One distinction needs care: not every duplicate should go. Two payment service providers are a deliberate redundancy; two payout engines are not (Duplicates and Dependencies:

The Hidden Costs of a Grown Estate). The portfolio marks deliberate redundancy as such so that it is not rationalized away.

The portfolio heat map

The heat map is the view Dana takes to the Keystone steering committee each quarter. It aggregates the 214 applications by capability domain and shows where cost, duplication and poor fitness concentrate. "Heat" combines duplication, low fitness and high TCO; it is a summary for discussion, and every cell can be traced to the application records behind it.

Capability domain	Apps	Overlap	Avg. functional fitness	Avg. technical fitness	SOX-relevant	Annual TCO ($M)	Heat
General ledger and accounting core	9	Three brand ledgers plus EBS archive	3.4	3.1	7	31	Warm: convergence decision pending
Subledger and revenue	14	Merchant subledger in SFP; agency in FI-CA	3.3	2.9	11	18	Warm
Billing and invoicing	17	Three billing approaches	3.0	2.6	12	14	**Hot**
Partner settlement and payouts	12	Two payout engines	3.1	2.4	8	9	**Hot**

Capability domain	Apps	Overlap	Avg. functional fitness	Avg. technical fitness	SOX-relevant	Annual TCO ($M)	Heat
Payments and PSP connectivity	26	Multiple PSPs (deliberate)	3.6	3.2	9	22	Cool: redundancy by design
Close, reconciliation and consolidation	15	Reconciliation in three tools	3.2	3.3	11	12	**Hot**
Planning and performance	13	Brand planning spreadsheets alongside OneStream	2.9	3.0	1	6	Warm
Tax	11	Vertex, Oracle tax engine, RateKeeper	2.8	2.5	6	7	**Hot**
Treasury and banking	10	Bank portals alongside Kyriba	3.5	3.4	5	5	Cool

Capability domain	Apps	Overlap	Avg. functional fitness	Avg. technical fitness	SOX-relevant	Annual TCO ($M)	Heat
Spend (procure-to-pay, expenses)	19	Two expense-audit tools; brand AP tools	3.1	3.2	7	10	Warm
Reporting and disclosure	12	Duplicate reporting instances	3.0	2.9	6	6	Warm
Finance data and analytics	31	Brand data marts alongside FinDB	2.7	2.8	6	17	Warm: ledger-to-warehouse tie-out gap
Integration and middleware	9	Three integration technologies	3.1	2.7	4	9	Warm
Controls, risk and compliance tooling	16	Several control-evidence repositories	2.8	3.0	3	6	Warm
Total	**214**				**90**	**172**	

The hot cells are where Northmere's known duplicates sit: billing, payouts, reconciliation and tax. That is not a surprise, and it should not be one. **The heat map's value is that it puts a price on what everyone already suspected** and shows the cool cells, payments in particular, where duplication is a design choice and should be left alone.

Failure Modes

The inventory as a one-off project. A consultancy fills the EA repository in twelve weeks for a transformation business case. The program starts, ownership changes, two acquisitions close, and within a year the repository describes an estate that no longer exists. The warning sign is a repository with no reconciliation against independent sources and no named portfolio manager. The remedy is the fed fact base and a monthly reconciliation, with breaks investigated like reconciliation breaks.

Scoring theater. Workshops in which owners score their own applications on unanchored scales produce a distribution where everything is a 3 or a 4, every owner's application is strategic, and the TIME grid is a cloud near the center. Decisions get made elsewhere, and the scores are filled in afterward to match. The warning signs are scores without evidence links, no calibration session, and no application anyone is willing to put in Eliminate. The remedy is anchored criteria, independent review and the borderline rule.

Ignoring EUCs. An inventory that stops at applications misses the spreadsheets that do the work between them: the chart-of-accounts mappings, the reconciliation workbooks, the accrual calculations. They carry key controls, they are the most fragile part of the close, and they never appear in a savings case because they cost nothing in the IT budget. This is **Spreadsheet Glue**, and the portfolio is the place to make it visible. The remedy is the three-condition registration rule and a Migrate disposition with a named destination.

Sunset without decommissioning. The replacement goes live, the lifecycle field says "End of life", and the program claims the saving. A year later the old system still has 40 active users, three interfaces still run at month-end, the license renewed automatically, and the auditor finds a key control still evidenced in a system that is officially retired. The warning signs are a growing retirement lag and claimed savings that never appear in cost-center data. The remedy is the evidence pack and the rule that nothing is booked as saved until it is decommissioned.

Retiring into an unready platform, or toward no platform at all. Consolidating onto a platform with open control findings moves the problem rather than removing it, and a Migrate disposition without a named target and date becomes a permanent label. Every wave gate checks the receiving platform's fitness, and every Migrate at Northmere carries a target, a wave and a review date, or reverts to Tolerate.

The Chief Architect's View

The portfolio is where a Chief Architect's authority becomes concrete or turns out to be ceremonial. Five decisions belong to the Chief Architect, or to the forum the Chief Architect chairs:

1. **The portfolio's definitions**. What counts as an application, which EUCs are registered, the lifecycle states, the scoring criteria and their anchors, the TCO components. These are standards, and Dana sets them under her decision rights for mandatory enterprise-application standards.
2. **The dispositions for cross-brand overlaps**. A brand cannot decide alone to keep its own payout engine or tax tool when a shared one exists. These go to the Keystone design authority, which Dana chairs with Leila Haddad.
3. **The wave plan and its gates**. Which retirements go together, in what order, and what must be true before each starts.
4. **The definition of decommissioned**. Jointly with the Head of Internal Controls, because the control matrix is part of it.
5. **What enters the portfolio**. New applications enter through a lightweight intake that requires a capability mapping, an owner, a pace layer and a cost line. Without an intake gate, the portfolio fills up again as fast as waves empty it.

Some decisions do **not** belong to the Chief Architect. The value of a capability is the business's call: Leila owns the functional scores for the ledger, Ingrid Solberg for payments. The savings target is the CFO's. Contract negotiation is procurement's, informed by the decision-due dates architecture supplies.

What to watch, quarterly:

Measure	Why it matters	Northmere reading
Fact-base completeness on mandatory fields	Decisions are only as good as the facts	93% (Q3 2027)
Inventory reconciliation breaks open over 30 days	A stale fact base is the first symptom of decay	6 (Q3 2027)
Tolerate dispositions past their review date	Tolerate turning into permanent	4 (Q3 2027)
Retirement lag (median time from Phasing out to Decommissioned)	Sunset without decommissioning	5 months (wave 0, Q2 2028)
Realized versus claimed savings	Credibility of the whole practice	92% (wave 0, six-month review, Q3 2028)
New applications added versus retired	Whether the estate is shrinking or just churning	7 added, 24 retired (Q3 2027 – Q1 2028)

Measure	Why it matters	Northmere reading
Contract notice dates missed	The Borrowed Roadmap by inattention	2 (wave 0; both contracts renewed for another year, which is most of the 8% shortfall above)

The long-term consideration is that **the portfolio outlives every program**. Keystone will end in 2031; acquisitions, new markets and new regulations will keep adding applications. The practice that lasts is the dull one: the monthly reconciliation, the intake gate, the contract calendar, the evidence pack. A Chief Architect who builds the portfolio for one business case will rebuild it for the next.

Questions to Ask

Take these into a portfolio review, a renewal discussion or a transformation business case:

1. Does our application inventory reconcile to SSO logs, accounts-payable spend, the SOX scoping list and the interface catalog, and who investigates the breaks?
2. Which spreadsheets perform or evidence key controls, and are they in the portfolio with an owner and a disposition?
3. Are our fitness scores tied to written anchors and evidence, and who other than the owner reviewed them?
4. Does our cost figure for each finance application include integration upkeep and audit and control cost, and what share of the total is that?
5. Which Tolerate dispositions have passed their review date, and what happened at the review?
6. For each planned retirement, what is the decision-due date given the renewal date, notice period and migration lead time?
7. Which pace layer is each application in, and does its change governance match? Where does a differentiating system post to a ledger?
8. What does "decommissioned" mean here, who signs it off, and how many "End of life" applications still have users?
9. Where will the records of a retired finance application live, for how long, and how does an auditor retrieve them?
10. Of the savings claimed for last year's retirements, how much appears in cost-center data today?

The portfolio tells Northmere what it owns and what each holding is worth. It does not by itself explain why the estate has the shape it has: why there is a subledger layer

between the booking platforms and the ledger, why reconciliation tools sit beside the ERP rather than inside it, why the volume drops from a billion events to thousands of journal lines. That anatomy, the layered reference model of a financial systems landscape, is the subject of the next chapter, Anatomy of a Finance Estate: Layers, Planes and Boundaries.

Sources and Evidence

Vendor documentation

- Gartner TIME Framework for App Rationalization: Tolerate, Neil Sheppard, SAP LeanIX blog, 2023 — the TIME quadrants on functional and technical fit, and the point that Tolerate is temporary; a vendor explanation of a model widely attributed to Gartner.
- SAP Completes Acquisition of LeanIX, SAP News, 2023 — the date SAP's ownership of the EA repository began.
- OneStream, Inc. Form 10-K for fiscal year 2025, OneStream, 2026 — account reconciliations and transaction matching listed among OneStream's specialized applications.
- OneStream Announces Completion of Acquisition by Hg, OneStream, 2026 — the 1 April 2026 change of ownership cited as a portfolio event.
- Oracle E-Business Suite end of life: what next, ERP Research, 2026 — widely reported extension of EBS 12.2 Premier Support through at least 2037 (secondary source; Oracle's own announcement was not retrievable).

Standards and regulation

- Keeping business records, Netherlands Enterprise Agency (business.gov.nl) — the 7-year Dutch retention period and its 10-year exceptions.
- Viertes Bürokratieentlastungsgesetz, Haufe, 2024 — Germany's reduction of the voucher retention period from 10 to 8 years from 1 January 2025, with the delay for BaFin-supervised entities.
- GDPR Article 17: Right to erasure, Regulation (EU) 2016/679 — the legal-obligation exception that allows statutory records to be retained.

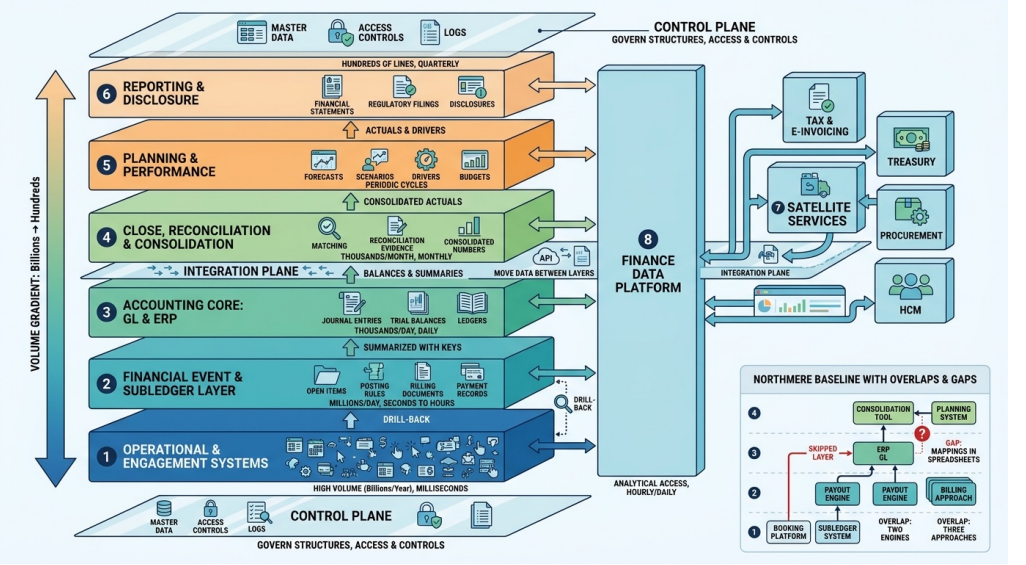
Practitioner and research sources

- Gartner's PACE Layered Application Strategy, CIO Wiki — secondary summary of the three layers and of the point that modules of one ERP package can belong to different layers; Gartner's primary notes were not accessible.

- TIME Framework, GetInSync — a variant of TIME that reads M as "Modernize".
- Scaling the Practice of Architecture, Conversationally, Andrew Harmel-Law, martinfowler.com, 2021 — the advice process referred to in the pace-layer governance table.

Northmere Group and all its figures, systems choices and people are fictional; the architectural recommendations in this chapter are the author's judgment.

7. Anatomy of a Finance Estate: Layers, Planes and Boundaries



IN THIS CHAPTER, YOU WILL: Learn the book's reference model of a finance estate: eight layers and two cross-cutting planes. For each one you will see why it exists separately, what data it masters, what volume and latency it carries, which vendors typically fill it and who usually owns it. Then you will map Northmere's landscape onto the model and find its overlaps and gaps.

WHY FINANCE CARES: The controller's hardest questions are about where a number comes from and who vouches for it at each step. A shared model of the layers turns "which system is right?" into "which layer is responsible for this, and how did the number cross into the next one?" That question has an answer.

WHY ARCHITECTS SHOULD CARE: Most expensive finance-architecture mistakes are placement mistakes: product logic built into the ERP, detail pushed into the general ledger, a warehouse quietly turned into a ledger, an

operational system posting straight to the books. A reference model with stable layer names makes placement an explicit, reviewable decision before it turns into a migration.

KEY POINTS:

- **The volume gradient explains why finance estates have layers.** Northmere's roughly 1.1 billion events a year become about 3 million subledger lines a day, about 38,000 general-ledger lines a day, about 6,800 reconciled accounts a month and a few hundred reported lines. Each step summarizes, and each layer exists to hold detail at one grain and prove it to the layer above.
- **Layers are separated by what they master, not by product boundaries.** Operational systems, financial events and subledgers, the accounting core, close and consolidation, planning, reporting, satellites and the data platform each own different data and answer to different owners; the integration and control planes run across all of them. Vendors span layers. The layers do not move.
- **Most failures are layer failures.** Layer leakage puts logic or detail in the wrong layer; a skipped layer lets operational systems post directly to the ledger; a duplicate layer gives two systems the same job. The model is a thinking tool for spotting these, not a deployment rule that every flow must pass through eight boxes.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Lost in Aggregation** — The volume gradient forces summarization at every layer boundary; the chapter shows where the keys must be kept so detail survives the step up.
- **Capability Creep** — Mapping each system to a layer exposes where several platforms do the same job: two payout engines, three billing approaches and reconciliations in three tools at Northmere.
- **The Swiss-Army ERP** — The model shows which layers an ERP is built for and which it should not be asked to carry at marketplace volume.

Ask five people in a finance organization to draw "the finance systems" and you get five pictures. The controller draws the close. The payments lead draws money moving between providers, banks and partners. The data lead draws pipelines into a warehouse. The enterprise resource planning (ERP) lead draws the ERP's modules, with everything else as an interface. A brand's engineering director draws the product with "finance" as one box at the edge. Each picture is accurate from where its author stands; none can settle a disagreement between the others.

A **reference model** is a shared, vendor-neutral decomposition of a domain into parts whose purpose and boundaries are agreed in advance. This chapter defines the reference model the rest of the book uses: **eight layers and two cross-cutting planes**. When When the ERP Isn't Enough: Custom Finance Platforms says a brand platform sits in the financial event and subledger layer, or BlackLine: Matching, Certifying and Closing places close automation in the close, reconciliation and consolidation layer, they mean exactly the layers defined here.

The model rests on one physical fact about finance data: **it gets smaller as it moves up**. Billions of operational events become millions of subledger lines, thousands of ledger summaries and a few hundred reported lines. Every layer boundary is a summarization, and every summarization is where traceability is preserved or lost. That **volume gradient** explains why there are layers, where detail must live, and what goes wrong when a layer is skipped, duplicated or used for the wrong job.

Setting the Scene at Northmere: Four Teams, One Job

Northmere, the fictional multi-brand commerce group this book follows (see Meet Northmere: Five Brands, One Set of Books), launched its architecture review board in the first quarter of 2027 under Program Keystone. One of the first agenda items, in February, came from Northmere Pay. Ingrid Solberg's team proposed a "payments ledger" service: ingest Adyen and Stripe settlement reports, normalize them, keep a balance per payment service provider (PSP) and currency, and post the results to SAP S/4HANA. The target was the payments-to-ledger gap. About 1.9% of settlement lines for Stayloft's alternative payment methods (APMs: local wallets, bank transfers and other non-card methods) were still unmatched on working day 3 of each close, and month-end balances in PSP clearing and suspense accounts averaged €14 million in 2026.

Within ten minutes, three other teams said they already did this, or should. Jonas Berg said the Stayloft Finance Platform (SFP) already held the merchant subledger the settlements clear against. Sven Aalto said SAP FI-CA, the contract-accounting subledger introduced in 2025, was built for exactly this kind of open-item clearing. Owen Mbeki pointed out that FinDB, the Snowflake finance data platform, already held every settlement line and ran half of Finance's matching analysis. Leila Haddad, the Chief Accounting Officer, asked the question nobody could answer: **which of these is the book, and which are copies?**

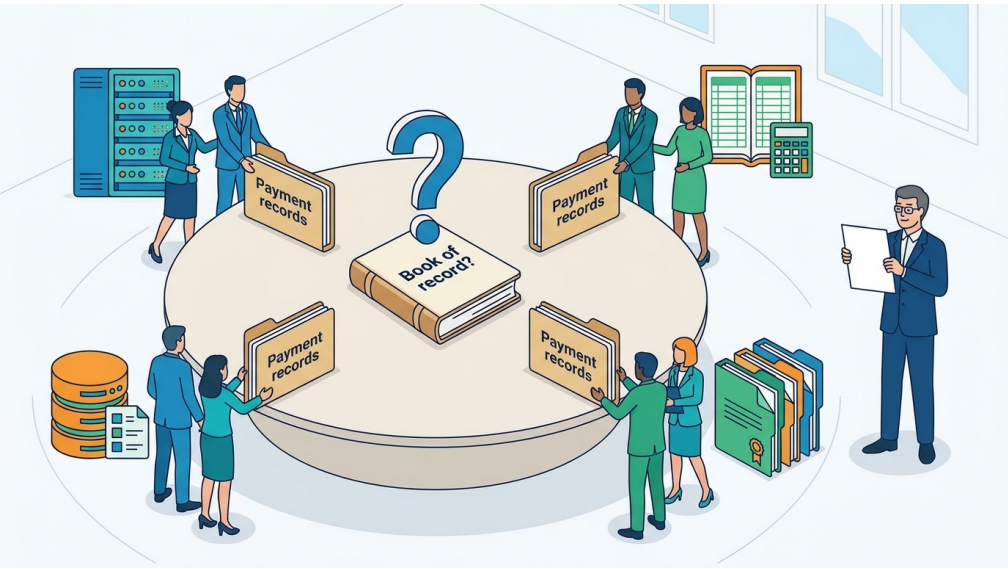


Figure 1: *Four teams claimed the same payment records; until the board shared one picture of the layers, nobody could say which system was the book and which were copies.*

Dana Whitlock, the Chief Architect, took no position that day. The four teams were not disagreeing about payments; they were disagreeing about **what kind of system each of them was**, and each drew the estate around itself. Until the board shared one picture of the layers, agreed before any proposal arrived, every placement decision would be argued from scratch.

By March the board had adopted the model in this chapter. Every decision record now states which layer a proposal belongs to, what data it would master, and which existing system already claims that layer. The payments proposal came back as a layer-2 question: which layer-2 component should own payment records and PSP clearing, and how would it prove completeness to the ledger? Whether the general ledger or the warehouse should do the job was no longer on the table. The answer belongs to Payments-to-Ledger: Draining the Suspense Swamp; this chapter builds the frame that made the question answerable.

What a Reference Model Is, and What It Is Not

Four artifacts are often confused. Each answers a different question.

Artifact	Question it answers	Unit	Owner in this book
Capability map	What must finance be able to do?	Business capability, independent of systems	Map What Finance Does Before What Runs It
Process model	In what order does work happen, and who hands off to whom?	Process step and hand-off	Order to Cash, Procure to Pay, Record to Report
Reference model	What classes of system does a finance estate need, and how do they relate?	Layer or plane	This chapter
Application landscape	Which systems does this enterprise actually run?	Application and interface	Manage the Application Portfolio as an Investment

The reference model sits between the capability map and the landscape. It is more concrete than a capability map, because it assigns responsibilities to classes of system with their own data, volume and controls, and more abstract than a landscape, because it names no products. That makes it the tool for **placement**: which layer does a proposed system belong to, and which system already holds that layer?

I separate layers with four tests. A candidate deserves its own layer only if it differs from its neighbors on at least two:

1. **What it masters**. The general ledger (GL) masters balances by account and period; a subledger masters open items; consolidation masters group numbers.
2. **Grain and volume**. Per event, per open item, per account per period, per reported line: grains that differ by orders of magnitude.
3. **Rate of change and latency**. Some layers change logic weekly and respond in milliseconds; others change structure yearly and work monthly.
4. **Owner and control profile**. Brand engineering, the controller, financial planning and analysis (FP&A), tax, the data team, each with a different position in the scope of internal control over financial reporting (ICFR).

"Reporting" in general fails these tests: management dashboards, consolidated statements and regulatory returns have different masters, grains and owners. "Integration" fails them too, because it runs between every layer. That is why the model has planes as well as layers.

The Volume Gradient

The numbers come first, because they explain the shape. These are Northmere's 2026 figures, rounded.

Level	What a record is	Northmere volume	Grain
Operational events	A booking, modification, cancellation, payment, refund, payout or billed ad click	**~1.1 billion** a year (about 3 million a day), plus ~420 million payment transactions a year through Northmere Pay	Per business event
Subledger lines	An accounting-relevant line with treatment applied: a liability per booking, a payable per property, a fee per settlement line	**~3 million** a day in SFP alone	Per open item or accounting event
General-ledger lines	A summarized journal line by date, entity, account and dimensions	**~38,000** a day from SFP into S/4HANA (about 1.1 million a month); ~2,600 a day from Wayfare's booking platform into Oracle Fusion	Per summarization key
Reconciled accounts	A balance-sheet account per entity, substantiated and certified	**~6,800** a month, in BlackLine	Per account per entity per period

Level	What a record is	Northmere volume	Grain
Reported lines	A line in the consolidated statements, segment notes and disclosures	A few hundred	Per line per quarter

The ratios matter more than the totals: roughly **80 to 1** from SFP to the ledger, more than 100 to 1 from a month of ledger lines to reconciled accounts, another factor of ten to the statements. From event to reported line, the data shrinks by six to seven orders of magnitude.

Figure 2: *The volume gradient. Each step up summarizes by one or more orders of magnitude; each dotted arrow is the path an auditor or controller needs back down. A boundary that drops the key on the dotted path is where Lost in Aggregation begins.*

Three consequences follow, and the rest of the chapter depends on them.

Detail must live at the lowest layer that has to answer questions about it. The general ledger's job is balances by entity, account and period, audited as the book of record. "Which bookings make up this partner payable?" and "which settlement lines are unmatched?" are subledger questions, asked at subledger grain. The rule I use: **every layer keeps what it must prove, and points down for the rest.**

Every upward step needs a key down and a control across. A summarized ledger line is acceptable only if it carries a **summarization key** (the date, entity, account combination and batch that let the subledger list every line behind it) and only if somebody proves each period that the subledger total equals the ledger's control account. That proof, **subledger-to-GL reconciliation**, is defined in Accounting for Architects: Ledgers, Journals and Revenue.

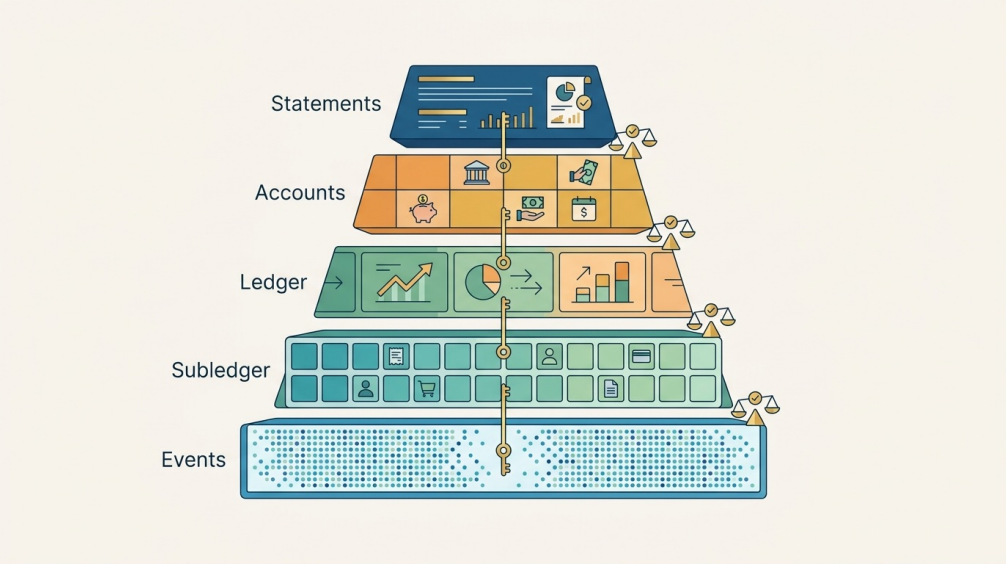


Figure 3: *Every layer keeps what it must prove and points down for the rest: each summary carries a key back to the detail below and a reconciliation across the boundary.*

Latency and change cadence fall as you go up, from milliseconds and weekly rule changes at the bottom to quarterly statements at the top. A layer boundary is also a cadence boundary, which is why a fast-changing brand platform must meet a slow-changing ledger through a stable contract (Files, Events and APIs: How Finance Systems Talk), never a shared database.

Where detail lives	Strength	Cost	When it is right
In the general ledger (every transaction a journal line)	One place to look; nothing to reconcile below it	Volume; operational dimensions bloat the chart; product changes touch the book of record	Low volume, standard processes such as supplier invoices
In a subledger, summarized with keys	Each layer works at its own grain; product logic stays out of the ledger	A subledger to own, control and reconcile	High-volume, product-specific flows: bookings, payments, usage, payouts

Where detail lives	Strength	Cost	When it is right
In the data platform only	Cheap; analysts already work there	An uncontrolled subledger: no open items, no period locks, outside ICFR scope	Never for amounts that reach the ledger

The Reference Model: Eight Layers and Two Planes

The numbering follows the gradient: business events at the bottom, reported numbers at the top, satellites and data beside the stack.

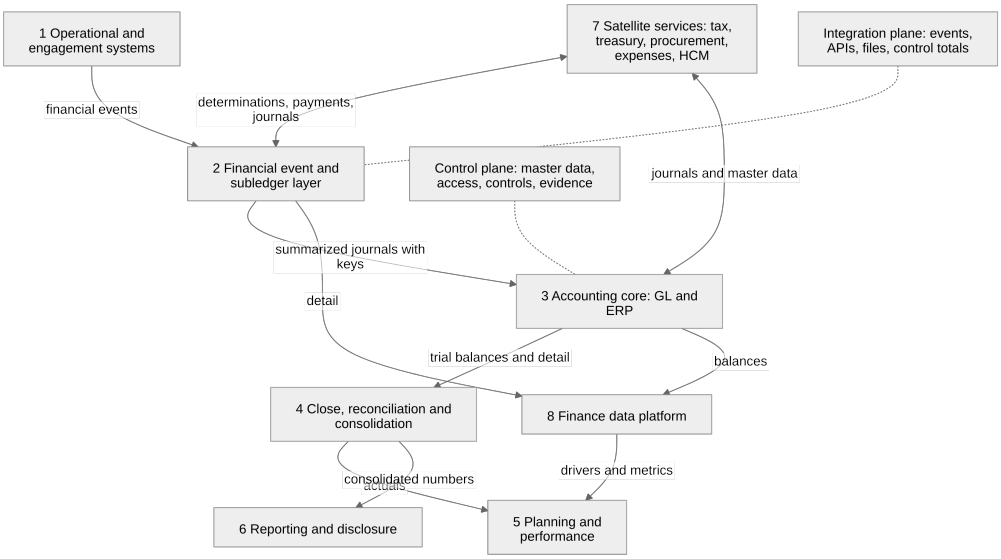


Figure 4: *The finance reference model. Six layers form the stack from business event to published number; satellite services attach at several layers; the data platform reads from all of them and is the book for none. The integration plane carries every arrow; the control plane governs every box.*

Layer	Purpose	Masters	Grain and volume	Latency	Typical owner
1 Operational and engagement systems	Run the business: sell, book, deliver, charge	Business events	Per event; highest volume	Milliseconds	Brand and product engineering
2 Financial event and subledger layer	Turn events into financial facts and open items	Billing documents, payment and settlement records, partner balances, payouts, customer funds, revenue schedules	Per open item; millions a day	Seconds to hours; daily summaries	Finance engineering and payments; controller as process owner
3 Accounting core: GL and ERP	Keep the book of record per legal entity	Journal entries and balances; ERP-native payables, receivables, assets	Per summary line; thousands a day	Daily; period locks	Controller; ERP platform team
4 Close, reconciliation and consolidation	Prove the books and combine them	Reconciliation evidence, close tasks, consolidated actuals, eliminations, top-side adjustments	Per account per entity per period	Monthly, moving toward daily	Group controller; record-to-report operations

Layer	Purpose	Masters	Grain and volume	Latency	Typical owner
5 Planning and performance	Plan, forecast, explain variance	Plans, forecasts, scenarios, drivers	Per dimension member per period	Monthly cycles	FP&A
6 Reporting and disclosure	Publish to regulators, investors, authorities	Filed statements, disclosures, statutory accounts, regulatory returns	Hundreds of lines	Quarterly; daily for some returns	Chief Accounting Officer; regulatory reporting
7 Satellite services	Specialized domains with fast-changing rules or external networks	Tax determinations, cash positions, suppliers, expenses, workers	Varies	Synchronous to monthly	Tax, treasury, procurement, people
8 Finance data platform	Analytical, audit and planning access across layers	Analytical datasets and metric definitions, no books	Every grain, copied	Hourly to daily	Finance data team
Integration plane	Move data between layers completely and once	Delivery records, control totals	Every flow	Every cadence	Integration architecture; platform engineering

Layer	Purpose	Masters	Grain and volume	Latency	Typical owner
Control plane	Govern structures and people every layer depends on	Chart of accounts and master data, access and segregation of duties, controls and evidence	Reference data and logs	Change-controlled	Controller; internal controls; security

Layer 1: Operational and engagement systems

Booking engines, checkouts, subscription products, ad servers, partner portals and the payments platform's routing. They create the events finance cares about but are not finance systems: they are measured on conversion and availability, and their logic changes with the product. A booking engine can say a booking exists; it should not decide that €90 of it is revenue on 22 April. In digital businesses this layer is mostly built. PSPs such as Adyen and Stripe are external operational systems here, authorizing and capturing for the checkout; their settlement output feeds layer 2.

The one obligation finance should impose is to **emit complete, well-formed financial events**, the canonical events of Canonical Events: A Common Language for Money such as `BookingConfirmed`, `PaymentCaptured` and `StayCompleted`. In return, layer 1 is spared from knowing accounting.

Layer 2: Financial event and subledger layer

This is the layer this chapter owns, and the one most enterprise diagrams leave out.

Purpose. The **financial event and subledger layer** turns business events into financial facts at transaction grain, applies the accounting treatment, keeps the resulting open items until they clear, and summarizes them to the ledger with keys. A **subledger**, in the sense of Accounting for Architects: Ledgers, Journals and Revenue, is a detailed record for one area whose total must equal a control account in the general ledger. Typical components:

- financial event intake and **posting rules** (the accounting engine);

- billing and invoicing: commission, subscription, usage, advertiser invoices;
- payment record normalization and settlement, one finance-ready record per payment, fee and settlement line whichever provider sent it;
- partner balances, payout calculation, holds and instructions;
- customer funds and contract liabilities per booking;
- revenue recognition schedules under ASC 606 (US GAAP) or IFRS 15;
- refund and dispute accounting;
- summarization to the ledger.

Why it is separate. Volume: a marketplace cannot post a billion events a year into its ledger, and its chart of accounts would have to carry booking, partner and payment-method attributes it was never designed for. Logic: commission rules, payout timing and refund policies are commercial decisions that change with the product; in the ERP, every pricing change becomes an ERP change (When the ERP Isn't Enough: Custom Finance Platforms sets the boundary rules). Layer 2 lets the product move while the book of record stays still.

Typical vendors. The most mixed sourcing in the estate:

- **ERP subledgers and accounting hubs**. SAP's contract accounting (FI-CA) is a high-volume subledger for business-partner receivables and payables, part of the SAP Billing and Revenue Innovation Management (BRIM) family that SAP's training describes as subscription billing plus contract accounting and invoicing. Oracle's Accounting Hub applies the Subledger Accounting rules engine to transactions from non-Oracle sources and transfers detailed journals to the general ledger. Workday's Accounting Center turns high volumes of external business events into rules-based entries and posts summarized journals with drill-down to source.
- **Billing and revenue specialists**. Zuora positions its billing product as an accounts receivable subledger; Stripe Revenue Recognition is "built … on top of a double-entry accounting ledger" for Stripe-originated revenue.
- **Custom builds**. In marketplaces, most of the layer: commission engines, payout engines, customer-funds ledgers, payment normalization.

These are not interchangeable. An accounting hub applies rules and keeps detail for drill-back; it is not by itself an operational subledger with partner balances, clearing and payout lifecycles, which FI-CA comes closer to. Never Update, Only Append: Designing Custom Ledgers compares them.

Data, volume, latency. Millions of open items a day, processed per event or in intraday batches and summarized daily, on the providers' rhythm: Adyen's settlement details report lists every credit and debit in a payable batch with fees; Stripe records each movement of funds as a balance transaction with gross, fee, net and the date funds become available.

Owner. Usually split, and that is the layer's central governance problem. Engineering teams build the components; the controller owns the accounting they produce. Every component that computes an amount that reaches the ledger is in ICFR scope, whoever wrote it. Northmere's FY2026 significant deficiency on the SFP payout engine, where payout rules were changed with developer production access and no documented approval, is that lesson learned the hard way (Controls by Design, Not by Audit Season).

Layer 3: Accounting core (GL and ERP)

Purpose. The **accounting core** keeps the **book of record**: the legally authoritative accounting record of each legal entity, as balanced journal entries and balances by account, dimension, ledger and period. Period control, parallel ledgers for several accounting standards, revaluation, intercompany accounting and the trial balance live here. So do the ERP's native subledgers for back-office processes (supplier payables, customer receivables, fixed assets), because their volumes are modest and their processes standard.

Why it is separate. Its structures (charts of accounts, ledgers, entities) outlive any product, and every published number must reconcile to it. A layer that changes with the product cannot also be the layer the auditor relies on.

Typical vendors. SAP S/4HANA, whose Universal Journal holds general ledger, controlling, asset and material-ledger line items in one table; Oracle Fusion Cloud ERP, where a ledger combines a chart of accounts, calendar, currency and accounting method and balances sit in Essbase-based cubes; Workday Financial Management, whose datasheet describes a ledger that keeps operational attributes alongside accounting and generates accounting as transactions are approved; NetSuite and mid-market ERPs in smaller entities. Three philosophies for holding accounting and its context; none is the reference model. SAP S/4HANA: One Journal, Clean Core, Oracle Fusion: An Accounting Hub for High-Volume Events and Workday: The Org Chart Behind the Ledger cover each.

Owner. The controller as process owner (Leila at Northmere); an ERP platform team to run it (Sven Aalto for S/4HANA, Hana Kowalski for Wayfare's Fusion, Arjun Rao for Tablekin's Workday Financials).

Layer 3 is not the ERP *product*. SAP's suite around S/4HANA also offers FI-CA (layer 2), group reporting (layer 4), treasury (layer 7) and Business Data Cloud (layer 8). **The layer is the responsibility**; **the ERP is one possible home for several responsibilities**, and keeping the two apart lets an architect ask, rather than assume, whether it is the right home for each.

Layer 4: Close, reconciliation and consolidation

Purpose. Prove the books and combine them. **Account reconciliation** substantiates and certifies each balance-sheet account. **Transaction matching** pairs high-volume

records across sources: PSP settlement to bank, settlement to subledger, intercompany to intercompany. **Close orchestration** runs the task calendar across entities. **Consolidation** loads each entity's trial balance, maps local accounts to the group chart, translates currencies, eliminates intercompany balances and produces group numbers.

Why it is separate. Its unit of work is the account per entity per period, and its users are record-to-report teams and group controllers. With more than one ledger it must sit above all of them (OneStream: Consolidating Actuals, Planning the Future), and reconciliation evidence must be independent enough of the systems it checks to be credible.

Typical vendors. BlackLine, whose FY2025 10-K groups its products into close and consolidation, intercompany, and invoice-to-cash suites; Oracle's Account Reconciliation, with Transaction Matching and Oracle's Financial Consolidation and Close; SAP's Advanced Financial Closing and S/4HANA Finance for group reporting; and corporate performance management (CPM) platforms such as OneStream, which its FY2025 10-K describes as one platform for close and consolidation, planning and reporting, and which Hg took private in April 2026; and CCH Tagetik. Vendors cross the line both ways: BlackLine sells consolidation, OneStream sells reconciliations and matching.

Data, **volume**, **latency**. Thousands of accounts per period, plus very high-volume payment matching, which in practice is layer-2-scale work done for layer-4 purposes. The cycle is monthly and quarterly; the aspiration is continuous, which BlackLine: Matching, Certifying and Closing treats as an architecture rather than a tool. At Northmere, BlackLine reconciles about 6,800 accounts a month under Leila's record-to-report organization.

Layer 5: Planning and performance

Budgeting, forecasting, scenarios, driver-based plans, allocations and variance analysis. The layer masters something no other layer has: **intentions**. Plans and forecasts are not facts and must never be confused with them. It is dimensionally richer than the ledger (room nights, take rate, merchant mix) and owned by FP&A on its own cadence. Typical vendors are OneStream, Oracle EPM Planning, Workday Adaptive Planning, Anaplan and SAP Analytics Cloud. Whether planning shares a platform with consolidation is a real choice: one dimensional model for actuals and plan removes a mapping; separate platforms let each be chosen on its merits. Actuals come from layer 4; operational drivers increasingly come from layer 8.

Layer 6: Reporting and disclosure

The numbers the enterprise publishes or files: SEC filings (10-K, 10-Q) in inline XBRL (eXtensible Business Reporting Language, the machine-readable format the SEC requires), statutory accounts per entity, returns for licensed entities such as daily e-money safeguarding, and tax filings drawn from ledger data. The output is legally

consequential and frozen once filed; the data is material non-public information until release; the controls are tie-out, tagging review and sign-off. Workiva is the common disclosure platform, linking consolidation output to narrative, tagging and filing (ERP Research overview); CPM reporting modules and, for licensed entities, often built tools fill the rest. Management dashboards are a related job but belong to layers 5 and 8, because nothing there is filed.

Layer 7: Satellite services

Purpose. Specialized domains that attach to the stack at several points: indirect tax determination and e-invoicing; direct tax provision and Pillar Two; treasury (cash positioning, bank connectivity, hedging, the payment factory); procurement and payables capture; employee expenses; and human capital management (HCM) as the source of workers, organizations and approvers.

Why they sit beside the stack. Each owns a narrow domain whose rules change fast or depend on external networks, and each attaches at more than one layer. A tax engine is called by layer-2 billing and layer-3 payables and feeds layer-6 returns; a treasury system takes payout instructions from layer 2, posts cash and hedge entries to layer 3 and supplies bank balances to layer-6 safeguarding returns. A vertical layer would hide those attachment points, which is where the integration and reconciliation design lives (Around the Core: Where Specialist Platforms Earn Their Place owns the patterns).

Satellite	Example vendors
Indirect tax and e-invoicing	Vertex O Series, Avalara, Thomson Reuters ONESOURCE, Sovos, SAP Document and Reporting Compliance
Treasury and bank connectivity	Kyriba, FIS Treasury and Risk Manager – Quantum Edition, ION Treasury, SAP Treasury and Risk Management
Procurement and payables capture	Coupa (private since Thoma Bravo closed its acquisition in February 2023), SAP Ariba, Oracle Procurement, Workday Strategic Sourcing
Expenses	SAP Concur, Navan
HCM	Workday HCM, SAP SuccessFactors, Oracle Fusion HCM

Each satellite is owned by the function it serves (Kofi Asante for tax and treasury at Northmere). That is also why satellites drift: each owner optimizes its domain, and

nobody owns the attachment points unless architecture does.

Layer 8: Finance data platform

Purpose. Analytical access across every layer: transaction-grain subledger and payment detail, ledger balances, close status, tax aggregates and operational drivers, for analysis, audit sampling, planning drivers and governed metrics.

Why it is separate. It reads every layer at every grain, serves analysts and increasingly AI agents, and is the only place a billion events and the ledger can be joined cheaply. Its defining rule is that **it is the book for nothing**: it masters analytical datasets and metric definitions, not financial facts, and every financial number it publishes must **tie out** to the ledger it came from (One Number, Many Views: Finance Data from Operations to Analytics owns that design).

Typical vendors. Snowflake, Databricks and BigQuery; and the ERP vendors' finance data offerings: SAP Business Data Cloud, Oracle Fusion Data Intelligence (formerly Fusion Analytics Warehouse) and Workday Prism Analytics, whose capabilities are being packaged into Workday Data Cloud with finance data targeted for 2027 as of October 2026. Owner: a finance data team (Owen Mbeki at Northmere), with metric owners in Finance.

The integration plane

The integration plane moves financial data between layers **completely**, **exactly once in effect**, **in order where order matters**, **and with evidence**: the event backbone, APIs, managed file transfer for bank and PSP files, the integration platform for vendor connectors, and the delivery records and **control totals** (counts and amounts per source per period) that prove a flow was complete. Typical technology: Kafka-class event platforms, vendor integration suites (SAP Integration Suite, Oracle Integration, Workday's integration tools, MuleSoft) and managed file transfer. Files, Events and APIs: How Finance Systems Talk owns the styles and their finance-specific requirements.

It is a plane because it masters no financial data and touches every arrow in Figure 4. Drawn as a layer, it invites mapping logic, account derivations and silent corrections that belong in layer 2 or the control plane. Ownership is usually split between platform engineering (the runtime; Ravi Menon's team runs Kafka at Northmere) and integration architecture (the contracts; Lucía Romero leads the canonical financial events).

The control plane

The control plane governs what every layer depends on:

- **financial master and reference data**: chart of accounts, legal entities, cost and profit centers, local-to-group mappings, FX rates, calendars;

- **identity, access and segregation of duties (SoD)**: who can post, approve and change what;
- **the control register, monitoring and audit evidence**;
- **retention and archive**.

No layer can own it alone. A chart-of-accounts change must reach posting rules, ledger, consolidation mappings, plan dimensions and metrics on the same effective date; an SoD rule must hold across the payout engine and the ERP that pays the payout. Without the plane, each layer keeps its own copy: **The Doppelgänger Record** for master data, **Spreadsheet Glue** for mappings. Typical tools: SAP Master Data Governance or Oracle Enterprise Data Management, SAP GRC and identity platforms, controls repositories, write-once archives. The controller owns financial structures, internal controls (Mei Tanaka) owns the control register, security (Marco Bellini) owns privileged access. Finance Data Architecture: Who Owns the Truth and Controls by Design, Not by Audit Season go deeper.

Where Vendors Sit

Vendors sell suites, so a product rarely fits one layer. "Primary" is the layer a product was designed for; "extends into" is where the vendor also sells. The table is a map of where overlaps will appear, not a recommendation.

Vendor or product	Primary layer	Extends into
SAP (S/4HANA and its suite)	3	2 (FI-CA, BRIM), 4 (group reporting, closing), 7 (treasury, tax compliance), 8 (Business Data Cloud)
Oracle (Fusion Cloud ERP and its suite)	3	2 (Accounting Hub, Revenue Management), 7 (procurement, tax), 8 (Fusion Data Intelligence); 4 and 5 through the separate Fusion Cloud EPM
Workday (Financial Management and its suite)	3	2 (Accounting Center), 5 (Adaptive Planning), 7 (HCM, procurement, expenses), 8 (Prism, Data Cloud)

Vendor or product	Primary layer	Extends into
BlackLine	4 (close, reconciliation)	4 (consolidation); layer-2 work in invoice-to-cash
OneStream	4 (consolidation)	5, 6; reconciliations and matching within 4
Workiva	6	ESG and sustainability reporting, still layer 6
Adyen, Stripe	1 (external payment acceptance)	2 (settlement data; Stripe Billing and Revenue Recognition)
Zuora, Chargebee	2 (billing, revenue)	Payment gateway connections
Vertex, Avalara, Sovos	7 (tax)	6 (returns, e-invoice reporting)
Kyriba	7 (treasury)	2 (payment factory for payouts)
Coupa, SAP Ariba	7 (procurement)	Payables capture feeding layer 3
Snowflake, Databricks	8	Vendors build layer-2 and layer-4 features on them

Buying a suite does not decide which layer a responsibility lives in; it only adds candidates. Every long "extends into" cell is a future overlap, decided either in a decision record or by whichever team switches the module on first. Duplicates and Dependencies: The Hidden Costs of a Grown Estate owns the overlap register; Own Your Roadmap: Vendor Strategy Without Lock-In Panic owns the dependency questions.

In Practice: Northmere Mapped onto the Model

Dana's team gave each of the 214 finance-relevant applications in LeanIX a mandatory primary layer. The core of the result fits on one diagram.

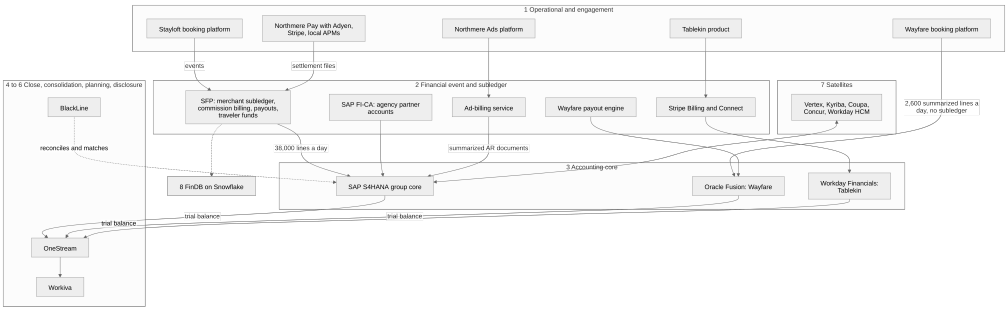


Figure 5: *Northmere's 2027 baseline on the reference model. Stayloft's flow uses every layer; Wayfare's booking platform posts summaries straight into its ledger; the three ledgers meet only at consolidation. Layers 4 to 6 are drawn together for readability, since OneStream spans consolidation, planning and reporting.*

Overlap means several systems claim one responsibility in a layer; **gap**, a responsibility no system holds, or only a spreadsheet; **leak**, a responsibility in the wrong layer.

Layer	Northmere systems	Findings
1 Operational	Booking platforms, Tablekin, Ads, Northmere Pay with Adyen ($_{68\%\ \text{of payments}}$), S_{tripe} $_{(22\%}$), local APMs (~10%)	Clean while it emits events and keeps no accounting
2 Event and subledger	SFP; SAP FI-CA (~850,000 agency partner accounts); Stripe Billing and Connect; ad-billing service; Wayfare payout engine	**Overlap**: two payout engines. **Overlap**: three billing approaches (SFP, FI-CA, Stripe Billing). **Gap**: no shared financial-event intake across brands. **Gap**: no layer 2 for Wayfare's booking flow. **Question**: ad billing posts summarized documents to S/4HANA receivables, splitting invoice detail from open items. Which is the subledger?

Layer	Northmere systems	Findings
3 Accounting core	S/4HANA (Stayloft, Northmere Pay, Ads, corporate); Oracle Fusion (Wayfare, 12 entities); Workday Financials (Tablekin)	**Overlap, by design for now**: three ledgers, three charts of accounts. **Leak**: ~2,300 custom S/4HANA objects, some carrying product logic. **Overlap**: Fusion's USD reporting currency duplicates OneStream's translation
4 Close and consolidation	BlackLine (~6,800 accounts a month; PSP matching); OneStream; Fusion intercompany	**Overlap**: reconciliation in three tools (BlackLine, OneStream's module, Fusion). **Gap**: brand-to-group mappings in spreadsheets
5 Planning	OneStream; Workday Adaptive Planning at Tablekin	Mostly clean; plan and actuals share one dimensional model. **Overlap, minor**: Tablekin plans in Adaptive Planning and submits a top-down plan to OneStream
6 Reporting	Workiva; Northmere Pay regulatory returns	Clean; safeguarding depends on layer-2 traveler funds and layer-7 bank balances
7 Satellites	Vertex, Kyriba (~410 bank accounts at 27 banks), Coupa, SAP Concur, Workday HCM	**Overlap**: Vertex and Oracle's tax engine both determine tax for Wayfare in some markets
8 Data platform	FinDB on Snowflake with dbt	**Gap**: no ledger-to-warehouse tie-out. **Leak risk**: matching analysis already runs here

Layer	Northmere systems	Findings
Integration plane	SAP Integration Suite, Kafka, MuleSoft; ~1,100 interfaces, ~240 flat-file transfers	**Overlap**: three integration technologies. **Gap**: ~70 undocumented spreadsheets in the loop
Control plane	S/4HANA master data, BlackLine evidence, the Sarbanes-Oxley (SOX) program	**Gap**: no single governed source for the group chart and its mappings. **Gap**: change control on SFP payout rules

Three findings changed how Northmere talked about its estate.

Wayfare skips a layer. Wayfare's booking platform handles about 150 million financial events a year, summarizes them itself and posts about 2,600 lines a day into Fusion GL through file-based data import (FBDI), with no subledger in between. The treatment of the €84.00 Paris–Lyon rail ticket (a €6.72 margin and a €77.28 payable to the operator) is decided in booking-platform code and reaches the ledger already summarized; listing the tickets behind a ledger line takes an engineering ticket. The design authority had treated the Central Finance-versus-accounting-hub decision, due in the fourth quarter of 2027, as a ledger question. On the model, part of it is a **layer-2 question for Wayfare**: something must apply posting rules and keep detail between the booking platform and whichever ledger wins.

The group exists only from layer 4 up. Wayfare and Tablekin join as trial balances in OneStream, mapped through spreadsheets; layers 2 and 3 are federated by brand. That is a legitimate transitional state (Converge or Coexist: Living with More Than One ERP compares the options), and it explains why drill-back from a group number into a Wayfare transaction crosses two brand systems and a spreadsheet.

The payments proposal was a duplicate. Northmere Pay's payments ledger would have been a third system holding payment open items, beside SFP and FI-CA, with FinDB as a fourth, informal copy. The board's decision is told in Payments-to-Ledger: Draining the Suspense Swamp; its framing came from this map: **one layer-2 owner for payment records and PSP clearing, one layer-4 tool to certify the clearing accounts, and FinDB as a consumer of both.**

Alternative Decompositions

The layered model is one useful cut of the estate. Others answer different questions well.

- **Process-centric** views (Order to Cash, Procure to Pay, Record to Report) show hand-offs and control points, but processes cross layers: order-to-cash at Northmere touches layers 1, 2, 3, 4 and 7, and a process view alone does not show that three processes share one subledger, or that two have each built their own.
- **Vendor-suite-centric** views follow the dominant ERP's modules and erase what the vendor does not sell: payouts, customer funds and safeguarding become "interfaces". As a skeleton, that is a mild form of **The Borrowed Roadmap**.
- **Capability-centric** and **domain-centric** views (Map What Finance Does Before What Runs It) are stable across reorganizations and right for investment, team boundaries and contracts, but they draw capabilities and contexts as peers and hide the physics: at Northmere, Subledger Accounting carries about eighty times the line volume of General Ledger Management and must summarize into it.
- **Postmodern or composable ERP**, Gartner's prediction from 2013 or 2014 (sources disagree) that the monolithic ERP would break into loosely coupled applications (TechTarget summary), is a sourcing philosophy, not a decomposition. In my judgment it needs a layered model more than a suite does, because **every seam between composed components is a reconciliation point**; without a model of which piece masters what, a composable estate becomes **Point-to-Point Spaghetti** with better marketing (Build, Buy or Compose: Choosing What to Own).

Decomposition	Best for	Blind spot	Use it when
Layered reference model	Placement of responsibilities and detail	Process sequence; team boundaries	Reviewing a new system or module; finding overlaps, leaks, skipped layers
Process-centric	Hand-offs, controls, cycle time	Components shared across processes	Redesigning the close or order-to-cash
Vendor-suite-centric	Licenses, configuration, implementation scope	What the vendor does not sell	Planning an ERP release
Capability-centric	Investment, duplication, target state	Volume and data flow	Portfolio decisions and roadmaps
Domain-centric	Team ownership, integration contracts	Upward summarization	Designing services and interfaces

Decomposition	Best for	Blind spot	Use it when
Composable ERP	Sourcing strategy	Structure	Build, buy or compose for one layer

I use three together: the capability map says what the estate must do, the reference model says where each responsibility belongs, and the process model says how work flows across both. A Northmere decision record now names a capability, a layer and the processes affected.

A thinking tool, not a deployment rule

The layers are **logical responsibilities, not physical tiers**. Misreading this does harm in four ways.

1. **One product can host several layers.** S/4HANA running FI-CA, the ledger and group reporting hosts layers 2, 3 and 4. That is fine if each responsibility is governed and reconciled as if separate: FI-CA-to-ledger is still a reconciliation inside one database.
2. **One layer can span several products.** Northmere's layer 2 is SFP, FI-CA, Stripe Billing and the ad-billing service. The goal is **one owner per responsibility**, not one product per layer.
3. **Not every flow needs every layer.** Coupa invoices go into S/4HANA payables, a layer-2 responsibility hosted inside layer 3, and that is right at modest volume. The test is not "did the flow pass through layer 2?" but "**is the accounting treatment applied somewhere designed and controlled for it, with detail retrievable below the ledger?**"
4. **The model does not prescribe an org chart.** It requires a named owner for every layer responsibility and a contract and reconciliation at every boundary between owners.

The model describes a multi-entity commercial group. In banks and insurers, core banking and policy administration are subledgers of extraordinary depth; the model still applies, but layer 2 becomes most of the estate.

Integration With the Rest of the Estate

Systems of Record: Ending the Doppelgänger Record decides authority per data object, which often moves between layers: the booking in layer 1, the receivable in layer 2, the balance in layer 3. Files, Events and APIs: How Finance Systems Talk designs the integration plane; Controls by Design, Not by Audit Season and Reconciled by Design:

Traceability and Audit Across Boundaries design the control plane and the reconciliation at each boundary. Never Update, Only Append: Designing Custom Ledgers goes deep into layer 2, Part II places each vendor on the model, and The Life of a Booking: From €600 Payment to $97.38 of Revenue follows one booking through every layer from event to filing.

Failure Modes

Most failures in a finance estate are layer failures.

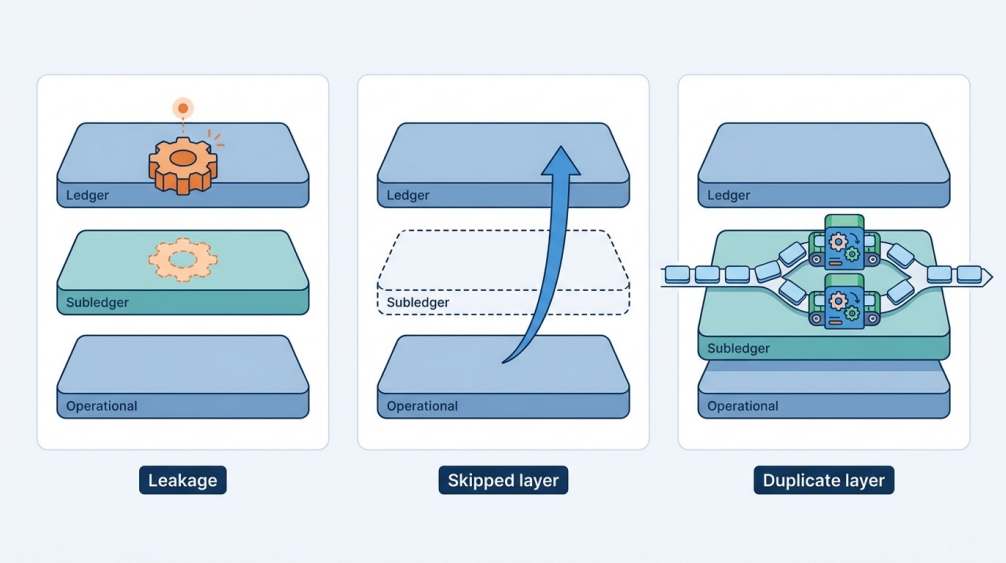


Figure 6: *Three families of layer failure: a responsibility in the wrong layer, an operational system posting straight to the ledger, and two systems doing one layer's job.*

Layer leakage: responsibility in the wrong layer

- **Product logic in the general ledger.** Commission or revenue-timing rules written as ERP custom code: every product change becomes an ERP change, every upgrade a product regression test. Northmere's ~2,300 custom S/4HANA objects and seven-month upgrades are partly this (**Customization Concrete**).
- **Operational dimensions in the chart of accounts.** Channel or payment method become account segments; the chart multiplies and the ledger carries layer-2 detail.
- **Ledger detail in the warehouse.** The data platform holds the only transaction-level view and starts driving matching, accruals and revenue splits, posted as manual journals: an uncontrolled subledger, **The Shadow Ledger**. FinDB is one step away.

- **Accounting in operational systems**. A brand platform computes its own "net revenue", and its numbers drift from the ledger.
- **Top-side adjustments that never reach the ledgers**, so group numbers and entity books drift apart (OneStream: Consolidating Actuals, Planning the Future).

Skipped layers: operational systems posting to the ledger

When layer 1 posts directly to layer 3, three things are lost at once: a place that holds the accounting treatment as configuration under change control; detail below the ledger that Finance can query; and a subledger total to reconcile with the control account. The booking platform's summary *is* the evidence. Wayfare is the Northmere case, the pattern is **Lost in Aggregation**, and the symptom is a finance team filing engineering tickets to answer audit requests.

Skipping is not always wrong; low-volume back-office flows rightly use the ERP's own subledgers. It is wrong when **volume or product-specific logic** is high enough that the treatment needs its own home, usually in a business that grew past its original design.

Duplicate layers: two systems, one job

- **Two subledgers for the same open items**, reconciled to each other by spreadsheet every month.
- **Two reconciliation systems of record for the same accounts**, so the auditor asks which certification counts (**Capability Creep**; Northmere has three).
- **Two payout engines**, **two tax engines**, **three billing approaches**: each rational when acquired, together doubling controls and audit scope.
- **Two masters of the chart of accounts and its mappings**: **The Doppelgänger Record**.

Two quieter failures complete the list. A **missing control plane** leaves master data, mappings and access governed inside each layer, so mappings live in spreadsheets (**Spreadsheet Glue**) and controls arrive when the auditor asks (**Audit Archaeology**). An **integration plane that becomes a hidden layer** accumulates mapping tables, account derivations and fix-up transformations where neither Finance nor system owners can see them.

The Chief Architect's View

For a Chief Architect the reference model is a **governance instrument** first and a diagram second. The decisions I would make with it, roughly in order:

Make it the shared vocabulary. At Northmere, "primary layer" became a mandatory field on every LeanIX fact sheet and on every decision record that adds or changes a system. Once the names are stable, "is this a layer-2 or a layer-8 responsibility?" takes thirty seconds to ask and one meeting to answer.

Set a few placement rules as standards, each with waivers that expire:

1. **One book of record per legal entity.** The layer-3 ledger is authoritative for balances; other layers may hold copies, never alternatives.
2. **Detail in layer 2, summaries in layer 3, a key on every summary.** High-volume, product-specific flows need a controlled layer-2 home before they post to a ledger.
3. **A contract and a reconciliation at every layer boundary**: control totals on the integration plane, and an equality proof (subledger to GL, ledger to consolidation, ledger to warehouse) at the boundary.
4. **One owner per responsibility within a layer.** A second system that wants to master the same open items, accounts or reconciliations needs a decision record.
5. **Layer 8 is the book for nothing.** Nothing posts to a ledger from the data platform without a controlled layer-2 step.

Decide who decides. Placement is the Chief Architect's call under the mandatory standards; accounting treatment within a layer is the controller's; control design is shared with internal controls; overlaps that cross budgets go to the design authority, under the decision rights agreed in November 2026 (The Chief Architect's Job: Making Many Systems Tell One Story).

Watch four measures, which Technical Debt on the Balance Sheet turns into fitness functions: operational systems posting directly to a ledger without a waiver; the share of high-volume ledger lines with a working drill-back key; responsibilities with more than one system of record in a layer; and whether the ledger-to-warehouse tie-out passes. Northmere's 2027 baseline: one skipped layer (Wayfare), drill-back keys on SFP lines but not on ad-billing or Wayfare lines, at least five duplicate responsibilities, no tie-out.

Change the model rarely. Layer names should outlive vendors, reorganizations and Keystone itself; what changes, through decisions, is where systems sit. I review the model yearly and change it only when a responsibility appears that no layer explains.

Questions to Ask

1. Which layer does this system belong to, and which existing system already holds that responsibility for this business?
2. What data will it master, and which system masters the same data today?
3. For every high-volume flow, where is the accounting treatment applied, and can Finance change it under change control without a product release?

4. Does any operational system post directly to a general ledger? At what volume, and how is the detail behind each line retrieved?
5. For every summarized ledger line, what is the summarization key, and how long does it take to list the transactions behind it?
6. Which reconciliation proves each layer boundary (subledger to ledger, ledger to consolidation, ledger to warehouse), who certifies it, and in which tool?
7. Where are local-to-group account mappings maintained, and who approves a change?
8. Which vendor modules overlap with a responsibility another system already holds, and who decided the owner? Is anything in the data platform posted back to the ledger?

From Layers to Ledgers

The reference model explains the shape of a finance estate: why there are layers, where detail must live and how the layers fail. It leans on accounting ideas it has not fully explained: what a journal entry must contain, why a subledger's open items must equal a control account, why cash received is not revenue, and why a marketplace that collects €600 may earn only €90 of it. Those ideas decide what each layer must record and preserve, and they are the subject of the next chapter, Accounting for Architects: Ledgers, Journals and Revenue.

Sources and Evidence

Vendor documentation

- How Oracle Accounting Hub Cloud Works, Oracle, 26C documentation, 2026 — Accounting Hub applies the Subledger Accounting rules engine to non-Oracle transactions and transfers detailed journals to GL.
- Overview of Oracle General Ledger Balances Cubes, Oracle, 26C documentation, 2026 — Fusion GL balances held in Essbase-based cubes.
- Transaction Matching concepts, Oracle Account Reconciliation documentation — high-volume matching in Oracle's reconciliation product.
- Oracle Cloud EPM Financial Consolidation and Close, Oracle — Oracle's consolidation platform, separate from Fusion ERP.
- Oracle Fusion Analytics Warehouse is now Oracle Fusion Data Intelligence, Oracle blog — the rename of Oracle's finance analytics offering.
- SAP Billing and Revenue Innovation Management, SAP Learning — BRIM as subscription billing plus contract accounting and invoicing (seen via search; widely reported).

- Exploring the Architecture of SAP Business Data Cloud, SAP Learning, 2025 — the components of Business Data Cloud.
- Workday Financial Management datasheet, Workday, 2024 — ledger with operational attributes; accounting generated as transactions are approved.
- Workday Accounting Center, Workday — high-volume external events turned into rules-based entries, summarized posting, drill-down.
- From Prism to Data Cloud, Workday, 2026 — packaging of Prism into Data Cloud and finance-data availability targets.
- BlackLine, Inc. Form 10-K for fiscal year 2025, BlackLine, 2026 — product suites across close, consolidation, intercompany and invoice-to-cash.
- OneStream, Inc. Form 10-K for fiscal year 2025, OneStream, 2026 — one platform for close, consolidation, planning and reporting, with reconciliation and matching applications.
- OneStream Announces Completion of Acquisition by Hg, OneStream, 2026 — private ownership from 1 April 2026.
- Zuora products, Zuora — billing positioned as an accounts receivable subledger.
- Stripe Revenue Recognition methodology, Stripe — revenue recognition built on a double-entry ledger.
- The Balance Transaction object, Stripe API reference — gross, fee, net and availability date per movement of funds.
- Settlement details report, Adyen documentation — every credit and debit in a payable batch, with fees.
- Vertex Indirect Tax O Series, Vertex — indirect tax determination deployable on-premise, in the cloud or at the edge.
- Kyriba, Kyriba — SaaS treasury and bank connectivity.
- Thoma Bravo Completes Acquisition of Coupa Software, Thoma Bravo, 2023 — Coupa's ownership and closing date.

Standards and regulation

- ASC 606 and IFRS 15, revenue from contracts with customers — the basis of layer-2 revenue scheduling; sources in Accounting for Architects: Ledgers, Journals and Revenue.
- Sarbanes-Oxley Act section 404 and ICFR — the control profile of each layer; sources in Controls by Design, Not by Audit Season.

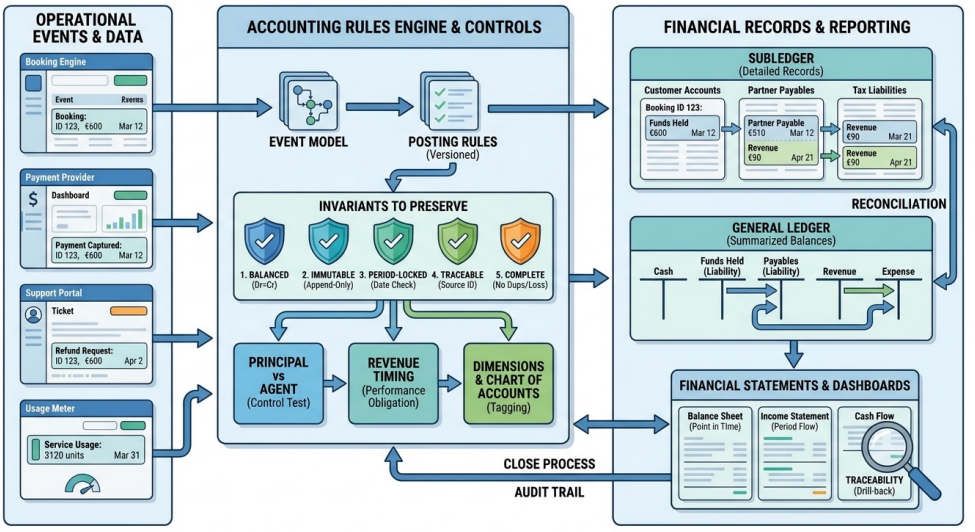
Practitioner and research sources

- What Is SAP's Universal Journal?, SAP PRESS blog — one line-item table for GL, controlling, assets and material ledger (secondary; SAP Help was not fetchable).

- Workiva overview, ERP Research — SEC filing with inline XBRL and linked data from ERPs (secondary).
- Evolution of postmodern ERP to composable ERP explained, TechTarget — Gartner's postmodern and composable ERP concepts.

Northmere Group and all its figures, system choices and people are fictional; the reference model, the placement rules and the architectural recommendations in this chapter are the author's judgment.

8. Accounting for Architects: Ledgers, Journals and Revenue



IN THIS CHAPTER, YOU WILL: Learn the accounting that finance systems are built to carry out (double entry, journal entries, charts of accounts, subledgers, accrual and the period close, revenue recognition, principal versus agent, currency and intercompany) by following one booking, one subscription invoice, one refund and one chargeback through the books, entry by entry.

WHY FINANCE CARES: Every number the CFO signs is a sum of journal entries produced by systems that accountants did not build. When engineers do not understand what an entry means, Finance spends the close finding out: manual journals, unexplained clearing balances and revenue in the wrong period.

WHY ARCHITECTS SHOULD CARE: Accounting rules are design inputs, not reporting details. Principal versus agent decides which events you must capture. The period close decides your cut-off and lock design. The chart

of accounts decides which detail lives in the ledger and which stays in the subledger.

KEY POINTS:

- **Every published number is a sum of balanced, immutable journal entries.** Accountants decide which entries to make; architects own the invariants that make the sums trustworthy: balanced, immutable, period-locked, traceable and complete.
- **Cash received is not revenue, and collecting the cash does not make you the principal.** In the Lisbon booking, Northmere holds €600 for five weeks, earns €90 of it and owes €510 to the property. The principal-versus-agent judgment decides which liabilities, events and accounts the systems must carry.
- **Accounting choices become system requirements.** Revenue timing sets which operational events must reach finance. The close sets cut-off and lock design. Dimensions decide what lives in the general ledger and what stays in the subledger. Currency rules decide what every line must store.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Lost in Aggregation** — The chapter shows exactly what a summarized ledger line must keep so that a balance can be traced back to the bookings behind it.
- **The Shadow Ledger** — The worked examples show why a brand platform that applies its own accounting treatment is keeping books, and must be reconciled and controlled as books.
- **The Suspense Swamp** — Clearing accounts are designed to return to zero; the chapter shows the entries that empty them and what it means when they do not.

Most finance-system failures I have seen were not technology failures. They were **accounting misunderstandings built into software**. A payment service that called captured cash "revenue". A refund flow that corrected a posted entry by editing it. A ledger that summarized a day's bookings so well that nobody could tell which booking a balance belonged to. Each was a reasonable engineering decision made by someone who did not know what the numbers would be used for.

This chapter gives architects the accounting they need to avoid those mistakes. It is not an accounting course; it does not teach you to prepare financial statements. It teaches you what the systems you design must **record, preserve and prove**, and why. Every concept comes with the design decision it drives, and every rule is shown in a worked journal entry with real numbers.

Other chapters build on this one. When Never Update, Only Append: Designing Custom Ledgers discusses posting rules, when Payments-to-Ledger: Draining the Suspense Swamp designs clearing accounts, when Controls by Design, Not by Audit Season discusses audit trails, and when The Life of a Booking: From €600 Payment to $97.38 of Revenue follows a booking into the published statements, they assume the vocabulary and the entries set out here. The chapter ends with the **accounting invariants your systems must preserve**, the five properties that later chapters turn into designs, standards and tests.

Setting the Scene at Northmere: The Event That Could Not Be Booked

Northmere, the fictional group this book follows, is a multi-brand digital commerce group listed in New York. It reports in US dollars under US GAAP, and its accommodation brand Stayloft takes about 64% of its gross transaction value through the **merchant model**: Stayloft collects the traveler's payment at booking, holds it until after the stay and pays the property net of commission.

It is February 2027. Lucía Romero, Dana Whitlock's lead integration architect, is reviewing the first draft of the Keystone canonical financial events with Jonas Berg's Stayloft Finance Engineering team. The draft has one event for a merchant booking: `BookingPaid`, carrying the booking id, the amount and the currency. It is clean, small and easy to publish from the booking engine.

Leila Haddad, Chief Accounting Officer, reads the draft and asks three questions. When the traveler pays €600 in March for an April stay, how much of it is Stayloft's revenue in March? When the property is paid in April, where does the €510 come from in the books? And if the traveler cancels on 2 April, which entry undoes what? The draft cannot answer any of them. A single "paid" event tells the ledger that money arrived. It does not say that the money belongs mostly to someone else, that the part that does belong to Stayloft is not yet earned, or when it will be.

Dana's conclusion, recorded in the first Keystone design-authority minutes, is the premise of this chapter: **the event model is downstream of the accounting model**. Before her architects design an event, a subledger or a summarization job, they need to know which accounts the business touches, when each amount moves between them, and what has to stay true at every step. The rest of this chapter is the briefing she asked Leila's team and her own architects to write together.

Double Entry: The Invariant Behind Every Number

Double-entry bookkeeping records every economic event as a **journal entry** with at least two lines, where the total of the debits equals the total of the credits. It rests on the

accounting equation:

Assets = Liabilities + Equity, where equity accumulates **income minus expenses** over time (retained earnings).

Every event changes at least two terms of the equation and leaves it in balance. A traveler's €600 arriving at Northmere's bank raises an asset (cash) and, because Northmere owes it onward, a liability by the same amount. Equity does not move: nothing has been earned.

Debit and **credit** are a sign convention, not a judgment. They mean left and right, and which one increases an account depends on the account's class:

Account class	Normal balance	Increased by	Appears in	Lisbon booking example
Asset	Debit	Debit	Balance sheet	Cash at bank, receivable from Adyen
Liability	Credit	Credit	Balance sheet	Traveler funds held, payable to the property
Equity	Credit	Credit	Balance sheet	Retained earnings (where profit ends up)
Revenue	Credit	Credit	Income statement	Commission revenue
Expense	Debit	Debit	Income statement	Payment processing fees

For an engineer, the useful translation is to store each line as a signed amount, debit positive and credit negative (or the reverse, chosen once and documented), so that the invariant becomes:

`SUM(amount) = 0` for every journal entry, per currency, per ledger.

That invariant is the reason a ledger is trustworthy in a way an application database is not. It is checkable for every entry, by machine, at the moment of posting. A ledger that only checks it afterwards, in a nightly job, has already accepted an entry it should have rejected. Two further engineering rules follow directly: never represent money in binary floating point, and decide where rounding happens and which line absorbs the

rounding difference, because an entry that is out of balance by one cent is out of balance. Never Update, Only Append: Designing Custom Ledgers covers money representation and rounding policy in depth.

What a balanced trial balance does not prove

A **trial balance** lists every account's balance at a date; total debits equal total credits. It is the starting point for the financial statements and for the auditor. A balanced trial balance proves that the double-entry invariant held. It does **not** prove that the books are right. A booking posted twice, revenue posted to the wrong period, a refund posted to the wrong account, a payment never posted at all: every one of these leaves the trial balance perfectly balanced. Balance is a necessary condition, cheap to enforce. Completeness and accuracy are the expensive part, and they are what reconciliations, controls and traceability exist to prove.

Anatomy of a Journal Entry

A journal entry is not just "debit this, credit that". In an enterprise ledger it carries the context that lets a controller, an auditor or a regulator understand it years later, and every field exists because something downstream depends on it. The table below lists the fields I expect any system that produces journal entries to populate, and why each matters to an architect.

Field	What it is	Why the architect cares
Entry id	Unique, immutable identifier	Idempotency and drill-back key. Some jurisdictions require gapless sequential numbering for invoices; few require it for every ledger entry.
Ledger or book	Which accounting book: group US GAAP, local statutory, tax	One event can post to several ledgers with different rules (parallel ledgers).
Legal entity	The company whose books these are	Statutory reporting, tax and intercompany all key off it; it cannot be inferred later.

Field	What it is	Why the architect cares
Posting date (accounting date)	The date that decides which period the entry belongs to	The basis of cut-off and period locks; it is a business decision, not a server timestamp.
Document or event date	When the business event happened	Kept alongside the posting date so late events can be explained.
Entered at, entered by	System timestamp and user or service identity	Audit trail; for automated postings, the identity of the service and the rule version.
Source system and source reference	Where the entry came from and which record it represents	Traceability back to the booking, invoice or settlement line.
Lines	Account, dimensions, debit or credit amount	The economic content.
Currency data per line	Transaction currency and amount, rate, rate type, functional-currency amount	FX traceability; revaluation needs the original rate.
Entry type	Standard, accrual, reversal, adjustment, elimination	Drives reversal logic and reporting; a reversal links to the entry it reverses.
Approval status	Workflow and approver for manual entries	Manual journals are a key control area; the evidence must be in the record.

One rule sits above all the fields: **a posted entry is never edited or deleted**. Errors are corrected by new entries, typically a reversal that mirrors the original and a corrected re-post. This is an audit expectation everywhere and a legal requirement in some jurisdictions; Germany's principles for electronic bookkeeping (GoBD) are the example practitioners usually cite. For an engineer this means a journal table is append-only, and an `UPDATE` statement on a posted line is a defect, whatever the reason.

The Chart of Accounts and Its Dimensions

The **chart of accounts** (CoA) is the list of accounts an entity posts to. A modern chart separates two things:

- The **natural account** says *what* the amount is: cash at bank, trade receivables, traveler funds held, commission revenue, payment processing fees.
- **Dimensions** (also called segments) say *where, for whom and why*: legal entity, brand, business model, market, channel, cost center, product, intercompany counterparty.

The combination of a natural account and its dimension values is the **accounting string** or account combination. The design rule that practitioners converge on is to keep natural accounts few and stable (hundreds, not tens of thousands) and push analytical variety into dimensions. The classic anti-pattern is account explosion, where "Commission revenue – Stayloft – Portugal – mobile app – merchant" becomes its own account, and every new market or channel means a chart change, a mapping change and a consolidation change.

The question architects face is not only "natural account or dimension?" It is a four-way placement decision for every piece of detail, and it is one of the most consequential decisions in a finance design:

Where the detail lives	What belongs there	Northmere example	Cost of getting it wrong
Natural account	The economic nature of the balance, at the level the financial statements and auditors need	`Traveler funds held`, `Partner payables`, `Commission revenue`	Too many accounts: chart sprawl and mapping spreadsheets. Too few: balances that mix different obligations.

Where the detail lives	What belongs there	Northmere example	Cost of getting it wrong
Dimension on the GL line	Anything needed to *report, consolidate or eliminate* from the ledger: entity, brand, business model, market, intercompany counterparty	`brand = Stayloft`, `business_model = merchant`, `ic_partner = NMP-NL`	Missing at posting time means retrofitting history, which is expensive and sometimes impossible.
Subledger	Per-counterparty and per-document detail needed to *operate and reconcile*: each booking, each property's open items	The €510 owed to the Lisbon property, linked to booking id	Putting it in the GL makes the GL slow and unreadable; omitting it means nobody can explain a balance.
Operational system only	Detail with no accounting meaning: room type, guest names, search channel	Apartment amenities, traveler's preferences	Copying it into finance creates privacy exposure and noise.

The test I use for the second row: if the CFO's reporting, the segment note, a statutory filing or an elimination needs to cut a balance by an attribute, that attribute must exist as a dimension *at posting time*. US segment reporting (ASC 280) and new disclosure rules such as the US expense disaggregation standard (ASU 2024-03, effective for US public companies for annual periods beginning after 15 December 2026, as of 2026) are examples of "disclosure changes" that are really data-model changes. Northmere's three charts of accounts, one in each ERP and mapped only in OneStream spreadsheets, are the cost of not deciding this once; Finance Data Architecture: Who Owns the Truth owns the group chart of accounts program.

Operational Records, Subledgers and the General Ledger

Three layers hold financial truth in an enterprise, and much of finance architecture is about the boundaries between them.

- **Operational records** are the business events themselves: bookings, modifications, payments, refunds, usage, payouts. They are high volume, per event, and owned by the brand and payments platforms.
- A **subledger** holds accounting-relevant detail for one area, with the accounting treatment already applied: every booking's funds held, every property's payable, every customer's open invoices. Accounts receivable, accounts payable, fixed assets and Stayloft's merchant subledger in the Stayloft Finance Platform (SFP) are subledgers.
- The **general ledger** (GL) holds balances by account and dimension, summarized, and is the source of the financial statements.

Each subledger has a **control account** in the GL (for example `Partner payables`) whose balance must equal the sum of the open items in the subledger. Proving that equality every period, **subledger-to-GL reconciliation**, is one of the most important key controls in any finance estate.

Figure 1: *Detail narrows as it moves toward the statements: millions of operational lines become tens of thousands of GL lines. The design works only if every summarized line can be traced back to its detail and the subledger's open items can be proven equal to the GL control account.*

At Northmere the volume gradient is concrete: SFP turns about 3 million subledger lines a day into about 38,000 GL lines posted to SAP S/4HANA. That ratio is not a problem; it is the reason the subledger exists. The problem appears when the summarization drops the information needed to go back. A summarized line must carry a **summarization key** (the date, entity, account combination and batch it summarizes) that the subledger can use to list every line behind it. Without that key, Northmere has the failure pattern **Lost in Aggregation**: a balance nobody can explain. Anatomy of a Finance Estate: Layers, Planes and Boundaries places the subledger layer in the wider reference model, and Never Update, Only Append: Designing Custom Ledgers designs the summarization.

A point that matters for brand platforms: a system that decides how an operational event is treated in accounting (what is a liability, what is revenue, when it moves) is **keeping books**, whatever it is called. SFP is a subledger in this sense, and so is any brand service

that computes "net revenue" for its own dashboards. If its numbers are not reconciled to the GL, it is **The Shadow Ledger**.

Accrual Accounting and the Period

Financial statements under US GAAP and IFRS (apart from the cash-flow statement) are prepared on the **accrual basis**: revenue is recorded when earned and expenses when incurred, regardless of when cash moves. Operational systems, by contrast, are mostly event- or cash-driven. The finance stack's job is to turn one into the other, and the instruments it uses are a small set:

Instrument	What it does	Northmere example
Accrued expense	Records an expense incurred but not yet invoiced	Cloud hosting used in March, invoiced in April
Accrued revenue / unbilled receivable	Records revenue earned but not yet billed	Tablekin usage in March, invoiced 1 April
Prepaid expense	Defers an expense paid in advance	An annual software license paid in January
Deferred revenue / contract liability	Defers revenue billed or collected before the service is delivered	Tablekin's April subscription, invoiced 1 April
Reversing entry	Automatically reverses an accrual on day one of the next period, so the real invoice posts normally	The March hosting accrual, reversed 1 April
Adjusting entry	Corrects or trues up balances at period end	A refund reserve based on cancellation history

The period close, and what it asks of systems

A **period** is the unit of financial reporting, a month for Northmere's management accounts, a quarter for its SEC filings. The **period close** is the routine that makes a period's books complete, accurate and final: cut off the subledgers, post accruals and deferrals, run revaluations, match intercompany, reconcile balance-sheet accounts, consolidate, review and finally **lock** the period so nothing more can post to it. Northmere's close takes nine working days; Keystone's target is five by the end of 2028. Order to Cash,

Procure to Pay, Record to Report follows the close as a process; this chapter is concerned with what it demands of the systems underneath.

Three design consequences follow.

1. **Every event needs two dates.** The **event date** says when something happened; the **posting date** says which period it belongs to in the books. Usually they fall in the same period. When an event arrives after its period is locked (a settlement file for 31 March delivered on 3 April after the March cut-off), it is posted in the current open period with its original event date preserved, so the late arrival can be explained and measured. The systems must never silently post into a locked period, and must never discard the event date.
2. **Cut-off is a design, not a moment.** For each feed, finance and engineering agree what "all of March" means: by which event date, in which time zone, received by when, proven complete by which control total. A cut-off that depends on a batch job happening to run after midnight in Amsterdam is not a cut-off.
3. **Locks are enforced by the ledger, not by convention.** Period status (open, closing, closed) is master data that every posting path checks, including APIs used by brand platforms and scripts used by finance operations. Never Update, Only Append: Designing Custom Ledgers treats this as bi-temporality: the difference between when something happened and when the books recorded it.

Revenue Recognition: Five Steps, Translated Into System Requirements

Revenue is the most scrutinized number in the statements, and its rules are where engineering intuition fails most often. IFRS 15 and its US counterpart, ASC Topic 606, were issued jointly in May 2014 and are substantially converged; both organize revenue recognition into five steps. Northmere reports under US GAAP, so its policies cite ASC 606; its European subsidiaries' statutory books apply IFRS 15 or local GAAP. The table translates each step into what systems must hold.

Step	What the standard asks	What the systems must hold	Northmere example
1. Identify the contract	An enforceable agreement with a customer	The contract and its terms as data: who the customer is, cancellation rights, effective dates	The booking and its cancellation policy; the Tablekin subscription
2. Identify the performance obligations	Each distinct promise to the customer	A catalog of promises per product and business model	Stayloft's promise to arrange the stay; Tablekin's platform access and its per-cover service
3. Determine the transaction price	The consideration expected, including variable amounts such as refunds and usage, constrained to what is unlikely to reverse	Price, discounts, expected refunds and the history needed to estimate them	€90 commission, subject to cancellation until the stay; $0.25 per seated cover
4. Allocate the price	Split across obligations by relative standalone selling price	Standalone selling price tables as governed, versioned reference data	Not material in the Lisbon booking; matters in bundles
5. Recognize revenue when (or as) each obligation is satisfied	At a point in time or over time	Recognition triggers (events) and schedules, with the rule version applied	Commission at stay completion; subscription ratably over the month

Three consequences matter more than the steps themselves.

- **Billing is not revenue.** An invoice is a legal and tax document; revenue follows

performance. An invoice for a service not yet delivered creates a contract liability, not revenue, and usage delivered but not yet invoiced is revenue without an invoice. A billing system is not a revenue subledger, and the two must reconcile.

- **Revenue timing decides which operational events finance needs**. If Stayloft recognizes commission at stay completion, a `StayCompleted` event (or a reliable derivation of it) is a financial event, not an operational detail. If the policy recognized at check-in, check-in would be the trigger.
- **Policy changes need re-runs**. When Leila's team changes a recognition policy or an estimate, the systems must be able to recompute the effect across every open contract. That is only possible if open bookings are held as data with their history, not as already-aggregated balances.

Principal or Agent: The Judgment That Decides What You Capture

When a third party is involved in providing the service, an entity must decide whether it is the **principal**, which controls the service before it is transferred to the customer and reports revenue **gross** (the full price, with the third party's share as a cost), or an **agent**, which arranges for the other party to provide it and reports revenue **net** (its commission or margin). Control is the test. Indicators such as primary responsibility for fulfillment, inventory risk and discretion over price support the control assessment; they are not a checklist that overrides it. The assessment is made per specified good or service, so one platform can be principal for some products and agent for others.

For a marketplace like Stayloft the contrast between the two business models looks like this:

	Agency model	**Merchant model**
Who collects the traveler's money	The property, usually at the stay	Stayloft (through Northmere Pay), at booking
Who provides the room night	The property	The property
Stayloft's likely conclusion	Agent	Still commonly agent, because Stayloft does not control the room night before it is transferred

	Agency model	Merchant model
Stayloft's revenue	Commission, invoiced to the property	Commission, retained from the traveler's payment
What else Stayloft's books carry	A receivable from the property	Funds held for travelers, payables to properties, PSP clearing, refunds, chargebacks, safeguarded cash

The teaching point in the right-hand column is the one engineers most often miss: **collecting the cash does not make you the principal**. Being the merchant of record, holding funds, and having tax obligations are separate questions from the revenue standard's control assessment. Under the EU's VAT in the Digital Age package (ViDA), short-term accommodation platforms will become deemed suppliers for VAT from 1 July 2028 (member states may delay to 1 January 2030) where the underlying provider does not charge VAT. That makes the platform liable for the VAT; it does not make it the principal for revenue. The same transaction can be "agent" for revenue, "deemed supplier" for VAT, "marketplace facilitator" for US sales tax and "reporting platform" under the EU's DAC7 platform-reporting rules. Systems therefore need the tax treatment and the accounting treatment as **separate attributes** of a transaction, determined by separate services.

> **What this chapter simplifies.** The principal-versus-agent conclusion is a judgment made by the accounting team, documented in policy, agreed with the auditor, and specific to contract terms. This chapter treats Stayloft as agent in both models because Northmere's (fictional) accounting policy says so, not because the merchant model always leads there; real companies disclose different conclusions for different product lines. The *timing* of an agent's revenue (at booking, at check-in or at stay completion) is also a judgment that depends on what the agent promised and on cancellation terms; Northmere's policy recognizes Stayloft commission at stay completion, while Wayfare, which is merchant of record for rail tickets but also an agent, recognizes its €6.72 margin on the €84.00 Paris–Lyon ticket at ticket issue, because its promise (arranging and issuing the ticket) is complete then. Architects implement these policies; they do not make them.

Why the architect needs this

The principal-versus-agent conclusion determines the **shape of the data**, not only a line in the income statement:

If the conclusion is…	Revenue line	Events finance must capture	Accounts the systems must carry
Agent, agency model	Commission	Stay completed, commission invoiced, invoice paid	Receivables from properties, commission revenue, bad-debt allowance
Agent, merchant model	Commission (net)	Payment captured, settled, fees, stay completed, refunds, chargebacks, payout instructed and settled	Funds held for travelers, PSP clearing, partner payables, safeguarding bank accounts, payouts in transit, commission revenue
Principal	Gross price	All of the above, plus the cost of the room night and any inventory commitments	Gross revenue, cost of revenue, supplier payables, inventory or prepayments

A change of conclusion, for example a new product where Stayloft pre-buys room allotments and bears unsold-inventory risk, is therefore not a reporting change. It changes which events must be captured, what the subledger stores and how the GL is structured. That is why I ask, in every design review of a new business model, "has accounting policy concluded on principal versus agent yet?" before I ask anything about the event model.

Three Kinds of Money That Are Not Revenue

The merchant model fills the balance sheet with money that is not Northmere's income, and the systems must keep three kinds of it apart.

- A **contract liability** (commonly called **deferred revenue**) is Northmere's obligation to deliver a service it has already been paid for, or has billed. It becomes Northmere's revenue when the service is delivered. Tablekin's April subscription, invoiced on 1 April, is a contract liability until April passes.
- **Funds held for customers** (Northmere's account is `Traveler funds held`; public filings in the travel industry often call this balance **deferred merchant bookings**) is cash collected from travelers for stays not yet taken. It is mostly *not* Northmere's revenue at all: most of it will be paid to properties or refunded, and only the commission share will ever be earned. Treating all of it as deferred revenue overstates future revenue by several times.
- A **partner payable** is the amount owed to a property once the stay has happened and the obligation is fixed.

Because Northmere Payments B.V. is a licensed e-money institution, travelers' funds are also subject to **safeguarding**: the E-Money Directive (2009/110/EC), Article 7, requires e-money institutions to safeguard funds received, typically by holding them in segregated accounts protected from other creditors. Safeguarding turns a balance-sheet distinction into a regulatory control: Northmere must show, daily, that the cash in its safeguarding accounts covers what it owes travelers and properties. That is only possible if the chart of accounts separates safeguarded cash from Northmere's own operating cash, and the subledger can say at any moment how much is held, for which bookings, in which entity and currency. Payments for Finance Architects: Eleven Reasons the Numbers Disagree covers the regulation; Global by Design: Entities, Currencies, Jurisdictions and Brands covers licensed entities.

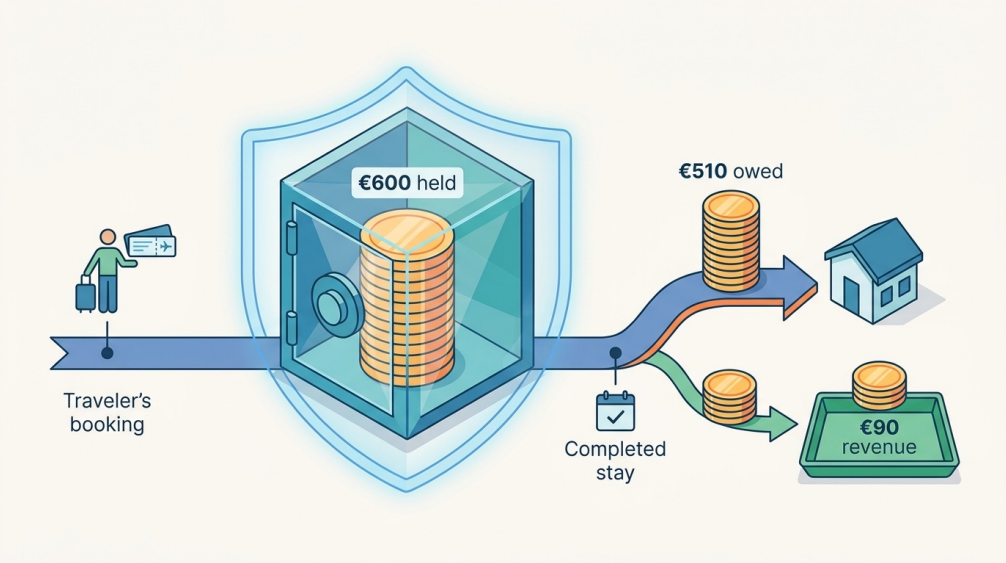


Figure 2: *Of the €600 a traveler pays, €510 is owed to the property and only €90 ever becomes Stayloft's revenue; until the stay, all of it is safeguarded funds held, not deferred revenue.*

The Lisbon Booking, Entry by Entry

The canonical worked example of this book is a merchant-model booking. A traveler from Germany books a three-night stay in a Lisbon apartment on Stayloft: **€600.00** including VAT, paid by card through Adyen on **12 March 2027**, for a stay on **18–21 April 2027**. The commission rate is **15%**, so Stayloft's commission is **€90.00** and the property is owed **€510.00**. Adyen books the capture into its settlement batch on Sunday **14 March** and the batch payout reaches Northmere's bank on Tuesday **16 March** (illustrative), after deducting a **€1.74** processing fee (illustrative). The payout is instructed on **22 April** and settles on **26 April**.

Simplifications stated up front

- **One entity view.** The entries below show the group's books as if one euro-functional entity performed every step. In Northmere's structure the cash sits in Northmere Payments B.V. and the commission in Stayloft B.V.; the intercompany section later in this chapter splits the entries across the two.
- **VAT.** The €600 includes the property's VAT on the accommodation, which is the property's obligation in the agency relationship and passes through Stayloft's books inside the partner payable. For the commission, I assume a business-to-business supply to a VAT-registered Portuguese property that accounts for the VAT

itself under the reverse-charge mechanism, so no output VAT appears in Stayloft's entries. Where a host is not VAT-registered, the platform may have to charge VAT on its commission, and ViDA's deemed-supplier rules change the picture from mid-2028. Kofi Asante's tax team and Vertex decide those treatments; the entries here are for teaching.

- **Booking-level entries**. In production, Adyen settles thousands of payments in one batch and SFP posts daily summaries. The entries show one booking so that each movement is visible.
- **Policy**. Revenue at stay completion and agent treatment are Northmere's (fictional) accounting policy, as stated above.

The accounts

Account	Class	Meaning
PSP clearing – Adyen	Asset	Money Adyen owes Northmere for captured payments, not yet settled to the bank. Should return to zero as settlements arrive.
Bank – safeguarding	Asset	Segregated account holding travelers' funds.
Bank – operating	Asset	Northmere's own cash.
Traveler funds held	Liability	Cash collected for stays not yet taken; refundable on cancellation.
Partner payables	Liability	Owed to properties for completed stays.
Payouts in transit	Liability	Payouts instructed but not yet confirmed settled by the bank.
Commission revenue	Revenue	Stayloft's net revenue as agent.
Payment processing fees	Expense	PSP fees.

The entries

#	Date	Trigger event	Account	Debit	Credit	Note
1	12 Mar	`PaymentCaptured`	PSP clearing – Adyen	600.00		Adyen now owes Northmere the traveler's money
			Traveler funds held		600.00	Nothing earned; fully refundable under the cancellation policy
2	14 Mar	`PSPFeeCharged`	Payment processing fees	1.74		Fee deducted in the settlement report
			PSP clearing – Adyen		1.74	
3	16 Mar	`PaymentSettled`	Bank – safeguarding	598.26		Net amount settled in Adyen batch 1047 on 14 Mar; simplified straight to the bank, which the batch payout credits on 16 Mar
			PSP clearing – Adyen		598.26	Clearing for this booking is now zero

#	Date	Trigger event	Account	Debit	Credit	Note
4	16 Mar	Safeguarding top-up	Bank – safeguarding	1.74		Northmere's policy safeguards the full €600 held, so own funds cover the fee
			Bank – operating		1.74	Done in aggregate daily, not per booking
5	21 Apr	`StayCompleted`	Traveler funds held	600.00		The obligation to the traveler is fulfilled
			Partner payables		510.00	Now owed to the property
			Commission revenue		90.00	Earned: revenue recognized, net, as agent
6	22 Apr	Own-funds sweep	Bank – operating	90.00		The commission is no longer travelers' money
			Bank – safeguarding		90.00	Moved after the daily safeguarding reconciliation
7	22 Apr	`PartnerPayoutInstructed`	Partner payables	510.00		Payout file sent to the bank
			Payouts in transit		510.00	

#	Date	Trigger event	Account	Debit	Credit	Note
8	26 Apr	`PartnerPayoutSettled`	Payouts in transit	510.00		Bank confirms the transfer
			Bank – safeguarding		510.00	

The booking confirmation itself creates no entry. `BookingConfirmed` carries the data every later entry needs (entity, business model, amounts, stay dates, cancellation terms, property), but no money has moved and nothing is owed until the payment is captured. Entries 4 and 6 are not driven by booking events at all; they come from treasury's daily safeguarding process. They are shown because they are what the bank statements will contain, and the reconciliation must explain them.

The balances, period by period

Account	After 16 Mar	At 31 Mar close	After 21 Apr	After 26 Apr
PSP clearing – Adyen	0.00	0.00	0.00	0.00
Bank – safeguarding	600.00 Dr	600.00 Dr	600.00 Dr	0.00
Bank – operating	1.74 Cr	1.74 Cr	1.74 Cr	88.26 Dr
Traveler funds held	600.00 Cr	600.00 Cr	0.00	0.00
Partner payables	0.00	0.00	510.00 Cr	0.00
Payouts in transit	0.00	0.00	0.00	0.00
Commission revenue (cumulative)	0.00	0.00	90.00 Cr	90.00 Cr
Payment processing fees (cumulative)	1.74 Dr	1.74 Dr	1.74 Dr	1.74 Dr

"After 26 Apr" shows the bank balances after the 22 April sweep and the 26 April payout. "Cr" on the operating bank account means Northmere's own cash went down by that amount.

Read the "At 31 Mar close" column as Leila would. The booking has put €600 of cash in a safeguarded account and an equal liability on the balance sheet; March's income statement shows a €1.74 expense and no revenue. A dashboard that called the €600 "March revenue" would overstate March by €600 for this booking, and by €510 even if it netted out the property's share. When the stay completes in April, revenue of €90 appears, the liability changes character from "owed to the traveler" to "owed to the property", and by 26 April every balance-sheet account the booking touched is back to zero except Northmere's own cash, up €88.26, which equals revenue minus the fee.

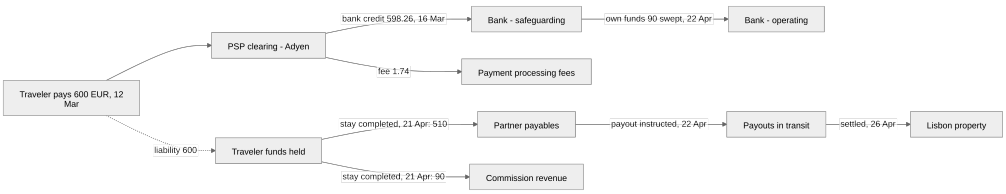


Figure 3: *One merchant booking moves through five balance-sheet accounts before it settles. Only €90 ever becomes revenue; every other account it touches must return to zero, which is what reconciliation checks.*

What the design must therefore do

The entries tell the architect what the systems need, more precisely than any requirements workshop:

- **Six booking and payment events** drive six of the entries (treasury's daily safeguarding process drives the other two), and each must carry the booking id as correlation key so that the eight entries can be tied together and to the bank.
- **Two clearing accounts** (PSP clearing and payouts in transit) exist to be emptied. Each must be emptied by matching specific items, not by netting totals. When they do not return to zero, the items left behind are the €14 million of unreconciled PSP clearing balances Northmere carried at month-end in 2026. That is **The Suspense Swamp**, and Payments-to-Ledger: Draining the Suspense Swamp designs the cure.
- **Revenue depends on an event from the stay**, which originates in the booking platform, not in payments. If `StayCompleted` is late or missing, revenue is late or missing, and the funds-held liability is overstated.
- **The subledger must hold the liability per booking**. At 31 March Northmere must be able to say which bookings make up its traveler funds held, by entity and currency, for the balance sheet, for safeguarding and for the auditor.

The Same Stay Under the Agency Model

Under the agency model the traveler pays the Lisbon property directly, at the stay. Stayloft never touches the €600. It earns the same €90 commission and invoices the property monthly through SAP FI-CA, Northmere's contract-accounting subledger for agency partner accounts. Assume the commission is recognized at stay completion, invoiced on 30 April and collected by direct debit on 15 May (dates illustrative).

#	Date	Trigger event	Account	Debit	Credit	Note
1	21 Apr	`StayCompleted`	Unbilled receivables	90.00		Commission earned, not yet invoiced
			Commission revenue		90.00	Same revenue, same timing as the merchant model
2	30 Apr	`CommissionInvoiced`	Trade receivables – partners	90.00		Monthly commission invoice to the property
			Unbilled receivables		90.00	
3	15 May	Direct debit collected	Bank – operating	90.00		
			Trade receivables – partners		90.00	

The income statement is identical: €90 of commission revenue in April. Everything else differs:

	Merchant model	Agency model
Journal entries for one booking	8 (plus refunds and chargebacks when they occur)	3
Peak balance sheet exposure	€600 of travelers' funds held	€90 receivable
Main financial risk	Refunds, chargebacks, fraud, payout errors, safeguarding breaches	Credit risk: the property does not pay the invoice
Systems involved	Booking platform, Northmere Pay, Adyen, SFP, treasury, banks, S/4HANA	Booking platform, FI-CA, S/4HANA
Reconciliations	Booking to PSP, PSP to bank, payouts to bank, safeguarding daily, subledger to GL	Invoices to collections, subledger to GL
Regulatory exposure	E-money license, safeguarding, payment regulation	Invoicing and VAT rules

This is why the rise in Stayloft's merchant share from 51% of GTV in 2023 to 64% in 2026 is an architecture event, not only a commercial one. Revenue barely changed shape; the volume of entries, reconciliations, liabilities and regulated cash multiplied. The agency invoice also shows why "commission revenue at stay" and "commission invoice at month end" are different events: the revenue belongs in April whether or not the invoice run happens on 30 April or slips to 2 May.

When the Money Goes Back: A Refund and the €420 Chargeback

Reversals are where weak designs break, because they arrive later, in different systems and often in a different period.

A cancellation before the stay

Suppose the Lisbon traveler had canceled on 2 April, within the free-cancellation window, and received a full refund through Adyen.

#	Date	Trigger event	Account	Debit	Credit	Note
1	2 Apr	`BookingCancelled`, `RefundIssued`	Traveller funds held	600.00		The obligation to the traveler is discharged by repayment
			PSP clearing – Adyen		600.00	Adyen will deduct the refund from a later settlement batch

No revenue is reversed, because none was ever recognized: a fully refunded cancellation is never revenue. The €1.74 processing fee is commonly not returned when a payment is refunded (it depends on the PSP contract), so the canceled booking costs Northmere €1.74 and earns nothing. The PSP clearing account now carries a *credit* item, to be matched against the deduction in a later settlement report.

A partial refund is more demanding. If the traveler had shortened the stay to two nights and received €200 back, the funds held would fall to €400, and the eventual commission (€60) and payout (€340) would be recalculated from the modified booking. That recalculation depends on `BookingModified` reaching finance before `StayCompleted`; if it arrives late, the subledger recognizes revenue on the old amount and someone corrects it during the close. Late modifications are among the commonest causes of manual journals in marketplace finance.

The €420 chargeback

A **chargeback** is a reversal of a card payment initiated by the cardholder's bank after a dispute. The book's second canonical example is a separate €420 merchant booking: booked 2 April 2027, stay 1–4 May 2027, commission €63.00, payout €357.00 settled on 7 May. On **13 June 2027**, 40 days after check-out, the traveler's bank raises a chargeback ("apartment not as described"). Adyen debits €420 from Northmere's next settlement and charges a dispute fee (€15.00, illustrative). By then, April's and May's books are closed, revenue has been recognized and the property has been paid.

#	Date	Trigger event	Account	Debit	Credit	Note
1	13 Jun	`ChargebackReceived`	Chargebacks under dispute	420.00		An asset: Northmere may win the dispute
			PSP clearing – Adyen		420.00	Deducted from the next settlement
2	13 Jun	`PSPFeeCharged`	Chargeback fees	15.00		Charged win or lose (illustrative)
			PSP clearing – Adyen		15.00	

Northmere Pay submits evidence (**representment**). Two outcomes are possible; in the book's example the issuer rules for the cardholder on 26 July, so 3b is what happens (Payments for Finance Architects: Eleven Reasons the Numbers Disagree):

#	Date	Outcome	Account	Debit	Credit	Note
3a	26 Jul	`ChargebackResolved`, won	PSP clearing – Adyen	420.00		Funds returned in a later settlement
			Chargebacks under dispute		420.00	
3b	26 Jul	`ChargebackResolved`, lost, recovered from the property	Tenant receivables	357.00		Offset against the property's next payout, as its contract allows
			Refunds and chargebacks (contra-revenue)	63.00		The commission is given up

#	Date	Outcome	Account	Debit	Credit	Note
			Chargebacks under dispute		420.00	

The outcome in 3b is a policy choice, not an accounting law. Whether a lost chargeback is recovered from the property depends on the partner contract and on the reason; fraud chargebacks on a stay that genuinely took place are commonly absorbed and expensed as losses; whether Northmere's lost €63 is a reduction of revenue or an expense is an accounting policy that Leila's team sets. What is not a choice is **where** the correction lands: in June and July, the periods in which the chargeback was received and resolved. May's revenue of €63 is not reopened. At the June close, the €420 under dispute is assessed for recoverability, and the refund-and-chargeback reserve, an estimate of future reversals based on history, is trued up. That estimate is itself a key control and needs governed data by cohort, which the subledger must keep.

For architects the chargeback carries three requirements. The booking id must survive for months across payments, subledger and partner systems, so that a chargeback can find its booking and its property. The partner subledger must be able to hold a *receivable* from a partner, not only payables, and net it against future payouts. And every reversal must post in the current open period, linked to the original entries, never by editing them.

Tablekin: One Invoice, Two Revenue Treatments

Tablekin, Northmere's restaurant-software brand, sells subscriptions plus usage. The book's canonical example is a venue in Austin on the **Pro plan**: **$249 a month** plus **$0.25 per seated cover**. In March 2027 the venue seated 3,120 covers, so usage is **$780**, and the invoice total is **$1,029** before sales tax. Stripe Billing produces the invoice; Workday Financial Management is the ledger.

Northmere's configuration, a common pattern for subscriptions with metered usage, bills the fixed fee **in advance** and the usage **in arrears**: Stripe aggregates metered usage at the end of the billing period. The invoice dated **1 April 2027** therefore contains two charges that belong to two different months: the **April subscription** ($249) and the **March usage** ($780). The March subscription was billed on 1 March.

Under ASC 606, platform access is a stand-ready obligation satisfied **over time**, so the subscription is recognized ratably across the month it covers. The per-cover fee is variable consideration that relates to a distinct period of service; Northmere's policy allocates it to the month in which the covers were seated (an alternative practical expedient lets an entity recognize revenue in the amount it has the right to invoice, when

that amount corresponds directly to the value delivered). Either way, March usage is March revenue.

#	Date	Trigger event	Account	Debit	Credit	Note
1	31 Mar	Month end, March subscription	Contract liability	249.00		March subscription invoiced 1 March, now delivered
			Subscription revenue		249.00	
2	31 Mar	`UsageRated` (3,120 covers, summarized)	Unbilled receivables	780.00		Usage delivered in March, invoice not yet issued
			Usage revenue		780.00	
3	1 Apr	`SubscriptionInvoiced`	Trade receivables	1,029.00		The invoice, excluding sales tax
			Unbilled receivables		780.00	March usage is now billed
			Contract liability		249.00	April's subscription: billed, not yet earned
4	1 Apr	Card charge succeeds	Stripe clearing	1,029.00		Stripe's payout and fees are a payments-to-ledger matter
			Trade receivables		1,029.00	

#	Date	Trigger event	Account	Debit	Credit	Note
5	30 Apr	Month end, April subscrip-tion	Contract liability	249.00		
			Subscription revenue		249.00	

The coincidence in this example is instructive. March revenue for this venue is $249 + $780 = **$1,029**, and the 1 April invoice is also **$1,029**, but the two numbers are made of different things. A design that used the invoice as the revenue record would get March right only by luck, and would be wrong in every month in which usage changes. The billing system's invoice lines and the ledger's revenue schedule are two records that must reconcile, which is the point of the misconception "billing equals revenue".

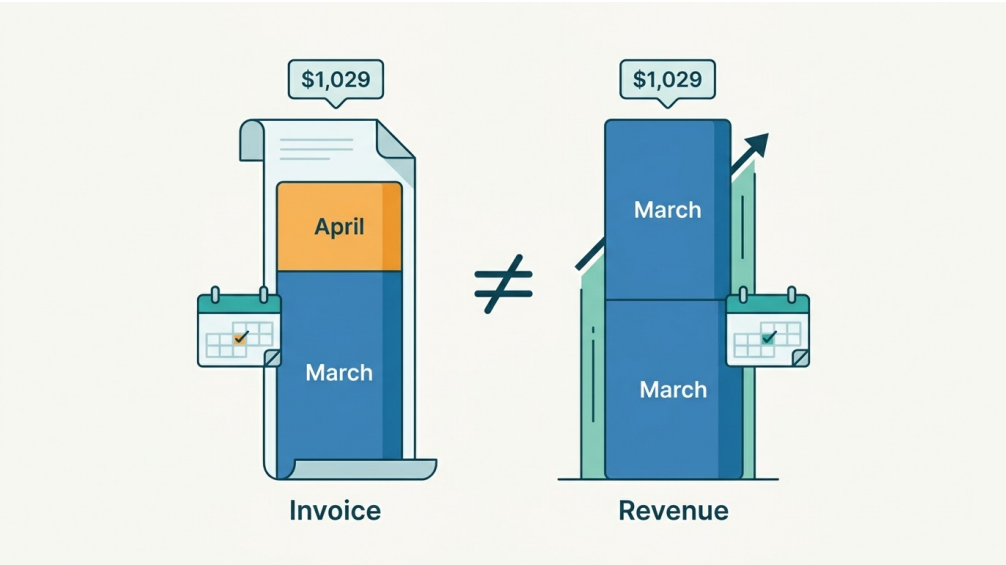


Figure 4: *The 1 April invoice and March revenue are both $1,029 but made of different things: the invoice carries April's $249 subscription and March's $780 usage, while March revenue is March's subscription plus March's usage.*

Three further points. Sales tax charged on the invoice is owed to the taxing authorities and posts to a sales-tax payable, not to revenue; US GAAP allows an entity to exclude such taxes from the transaction price, and Tablekin does. Whether recognition is daily or monthly is a policy choice with system consequences (a daily schedule is more precise for mid-month upgrades and cancellations and much more voluminous). And where the revenue schedule lives (in a billing tool's revenue module, in Workday or in a custom

service) is a system-of-record decision that Systems of Record: Ending the Doppelgänger Record addresses; what matters here is that exactly one system owns it.

Currency: Revaluation Is Not Translation

Northmere customers pay in 46 currencies and partners are settled in 31. Three currency concepts govern how the systems record this, and the standards, IAS 21 under IFRS and ASC 830 under US GAAP, distinguish them carefully:

- **Transaction currency** is the currency of an individual transaction.
- **Functional currency** is the currency of the primary economic environment in which an entity operates. It is determined by facts (prices, costs, financing), not chosen. Stayloft B.V.'s is the euro.
- **Presentation currency** is the currency the statements are presented in; Northmere's group presentation currency is the US dollar.

Three processes use them, and systems that conflate them produce errors that are hard to find:

1. **Initial recognition**: a foreign-currency transaction is recorded in the functional currency at the rate on the transaction date.
2. **Revaluation (remeasurement)**: at each reporting date, **monetary items** (cash, receivables, payables) in a foreign currency are retranslated at the closing rate, and the difference goes to **profit or loss**. Non-monetary items, such as deferred revenue in most analyses, stay at their historical rate.
3. **Translation**: when consolidating a foreign operation, its statements are converted into the presentation currency (assets and liabilities at the closing rate, income and expenses at transaction-date or average rates), and the balancing difference goes to **other comprehensive income** as the **cumulative translation adjustment** (CTA), outside profit or loss.

A revaluation worked through

A New York hotel is contracted to Stayloft B.V. (euro functional currency) and settled in US dollars. A stay completes on 30 March 2027 and the property is owed **USD 1,700.00**, paid on 6 April. Rates are illustrative and quoted as US dollars per euro; amounts round to the cent.

#	Date	Rate (USD per EUR)	Account	Debit (EUR)	Credit (EUR)	Note
1	30 Mar	1.0800	Partner payables (USD 1,700)		1,574.07	Recognized at the stay-completion rate (the funds-held side of the entry is omitted)
2	31 Mar	1.0900	Partner payables	14.44		Remeasured at the closing rate: €1,559.63
			Unrealized FX gain (P&L)		14.44	The dollar weakened, so the euro cost of the debt fell
3	6 Apr	1.0850	Partner payables	1,559.63		Carrying amount cleared
			Realized FX loss (P&L)	7.19		
			Bank – USD account (USD 1,700)		1,566.82	Euro value of the dollars paid at the payment-date rate

Over the life of the payable, Northmere gained €7.25 (€1,574.07 − €1,566.82): €14.44 in March and −€7.19 in April. Two simplifications: some ledgers reverse the month-end revaluation on day one of the next period instead of carrying it forward (the total is the

same; the split between realized and unrealized differs), and the US dollar bank account is itself a monetary item that is revalued, which partly offsets the payable's movement. Commission revenue recognized from the stay is *not* revalued later: it stays at the rate on the day it was recognized.

The design requirements are specific. Every line in a foreign currency must store the transaction amount, the rate applied, the rate type (spot, closing, average) and the functional-currency amount. Rates come from one governed rate table with a source and a timestamp, not from whatever service each system happens to call. And revaluation must know which accounts are monetary, which is a property of the account in the chart, set by accounting.

Translation, briefly

When OneStream consolidates Stayloft B.V. into the US-dollar group, the Lisbon booking's €90 of April commission is translated at April's average rate (1.0820, illustrative): **$97.38**. Stayloft B.V.'s euro balance sheet is translated at the 30 April closing rate, and because income, balance sheet and equity use different rates, a difference appears that is posted to CTA. Translation does not change any euro amount in Stayloft B.V.'s own books, and it does not go through profit or loss. Global by Design: Entities, Currencies, Jurisdictions and Brands owns the multi-currency model, rate types and hyperinflationary cases.

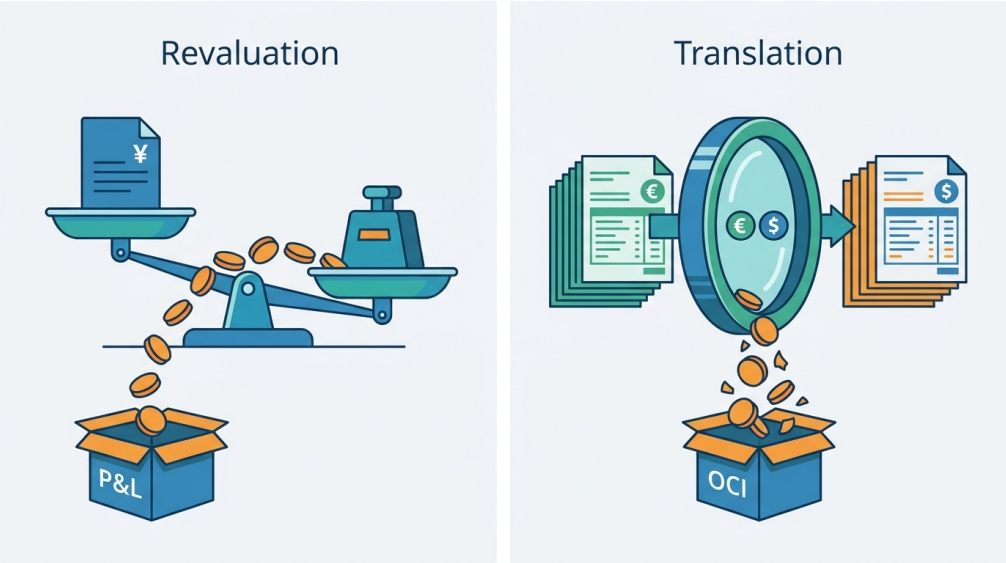


Figure 5: *Revaluation remeasures an entity's foreign-currency monetary items and runs through profit or loss; translation converts a whole entity's statements into the presentation currency and parks the difference in other comprehensive income.*

Intercompany: The Booking Across Two Legal Entities

Groups trade between their own legal entities, and each side records its half. In consolidation, intercompany balances, income and expenses are **eliminated** so that the group presents only transactions with outside parties. Intercompany is one of the most persistent close bottlenecks, because the two halves are often posted by different systems, at different times, at different rates.

In Northmere's (fictional) structure the Lisbon booking involves two entities. Northmere Payments B.V., the e-money institution, receives the Adyen settlement and holds travelers' funds in safeguarding accounts. Stayloft B.V. is the contracting entity with the traveler and the property, and earns the commission. Adyen's fee is incurred by Northmere Payments B.V. and recharged to Stayloft B.V. The real allocation of legal obligations between such entities is a legal, regulatory and tax design; this split is illustrative. At stay completion the entries become:

Entity	Account	Debit	Credit	Note
Northmere Payments B.V.	Traveler funds held	600.00		
Northmere Payments B.V.	Partner payables		510.00	Paid out from the safeguarding account
Northmere Payments B.V.	Intercompany payable – Stayloft B.V.		90.00	The commission belongs to Stayloft B.V.
Stayloft B.V.	Intercompany receivable – Northmere Payments B.V.	90.00		
Stayloft B.V.	Commission revenue		90.00	
Consolidation	Intercompany payable – Stayloft B.V.	90.00		Elimination
Consolidation	Intercompany receivable – Northmere Payments B.V.		90.00	Group shows only revenue, payable and cash

The fee recharge follows the same pattern: Northmere Payments B.V. records recharge

income of €1.74 and Stayloft B.V. a fee expense of €1.74; consolidation eliminates the pair, leaving the group with the original €1.74 fee to Adyen, an outside party. After elimination the group's figures equal the single-entity tables earlier in the chapter, which is why those tables can be read as the consolidated view.

The architectural lessons are the ones Northmere's intercompany backlog teaches every month. Both halves must carry the **intercompany counterparty** as a dimension from the moment of posting, since elimination depends on it. Both halves should be generated from the same event, with the same amount, rate and posting date, by one posting rule, rather than by two systems that hope to agree. And intercompany settlement (the actual movement of the €90 between the entities) is a treasury process with its own entries and reconciliation. Global by Design: Entities, Currencies, Jurisdictions and Brands owns the legal-entity model and transfer pricing.

Accounting Invariants Your Systems Must Preserve

Everything above reduces to five properties that every system producing or holding financial records must preserve. They are the contract between Finance and Engineering: Finance decides what the entries are; Engineering guarantees these properties hold for every entry, at every volume, through every failure. Later chapters turn each into designs, standards and controls; this section defines them.

Invariant	What it means	How a system enforces it	What the auditor sees	Deepened in
1. Balanced	Every entry's debits equal its credits, per currency and per ledger; every subledger's open items equal its GL control account	Reject unbalanced entries at the posting boundary; generate both sides from one rule; reconcile control accounts each period	Posting-engine validation as an automated control; subledger-to-GL reconciliations	Never Update, Only Append: Designing Custom Ledgers

Invariant	What it means	How a system enforces it	What the auditor sees	Deepened in
2. Immutable	Posted entries are never changed or deleted; corrections are new, linked entries	Append-only storage; reversals generated by the system with a link to the original; no update path for posted lines, including for administrators	A complete history of corrections; change-management and privileged-access controls over the store	Controls by Design, Not by Audit Season
3. Period-locked	Nothing posts into a closed period; every entry carries an event date and a posting date	Period status checked on every posting path; late events posted to the open period with the original event date	Period-status change log with approvals; late-event reports	Never Update, Only Append: Designing Custom Ledgers, Order to Cash, Procure to Pay, Record to Report
4. Traceable	Every GL balance can be traced to the events and documents behind it, and every event forward to its GL line	Correlation ids (booking id) on every event and line; summarization keys on GL lines; drill-back from GL to subledger	Walkthroughs from statement line to booking and back	Reconciled by Design: Traceability and Audit Across Boundaries

Invariant	What it means	How a system enforces it	What the auditor sees	Deepened in
5. Complete	Every financial event produces exactly one accounting effect: none lost, none duplicated	Idempotent consumers keyed on event id; control totals per batch and period; clearing accounts that must return to zero	Completeness reconciliations (bookings to PSP to bank); aging of clearing items	Files, Events and APIs: How Finance Systems Talk, Payments-to-Ledger: Draining the Suspense Swamp

I add a sixth to my own reviews, because the other five depend on it: **posting rules are deterministic and versioned**. The same event, processed under the same rule version, always produces the same entry, and every entry records which rule version produced it. That is what makes a policy change replayable, a historical entry explainable and an automated posting auditable. It is the subject of Never Update, Only Append: Designing Custom Ledgers.

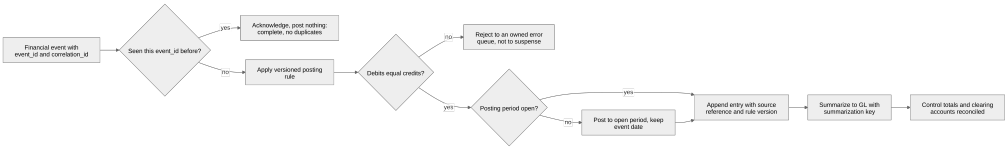


Figure 6: *Each invariant becomes a check on the posting path: idempotency for completeness, a balance check before acceptance, a period check that redirects rather than rejects late events, append-only storage with source references, and reconciliation of what was summarized.*

Two notes on the invariants. "Immutable" is a control concept, not a technology mandate: an append-only table with reversal entries and controlled access satisfies auditors; nothing requires a blockchain. And the invariants apply to every system that keeps books in the sense used above, including brand platforms, not only to the ERP. A brand subledger that updates rows in place is not exempt because its summaries post correctly to S/4HANA.

Where the Accounting Treatment Should Be Applied

Every finance estate must decide where operational events become journal entries, and the decision shapes ownership, controls and change cost for a decade. Northmere today has all three answers at once: SFP applies Stayloft's treatment, Oracle Fusion and Workday apply Wayfare's and Tablekin's, and S/4HANA receives summaries.

Option	How it works	Strengths	Weaknesses	Right when
In each brand platform	The booking or billing system computes entries and posts them	Fast to build; close to product knowledge	Accounting policy implemented N times; brand engineers own controls they do not see; Shadow Ledger risk	A single brand, early stage, with a plan to centralize
In a central subledger or accounting engine	Brands publish canonical financial events; one rules engine creates entries and summaries	One implementation of policy; consistent invariants; one place for auditors to test	Needs a canonical event model and governance; becomes a critical, high-volume system	Several brands or business models with shared policies (Northmere's Keystone direction)
Directly in the ERP general ledger	Detailed transactions post as ERP documents	Strong standard controls; no extra system	Volume and latency limits; detail clutters the GL; product change waits on ERP change	Low-volume, standard flows such as procurement and expenses

A related decision is the **granularity of what reaches the GL**: line-level (every booking), summarized with keys and drill-back, or summarized without keys. The first does not scale at Northmere's billion-event volume; the third is Lost in Aggregation. Summarized with keys is the default I recommend, and the design work is in choosing

the summarization dimensions (entity, account, business model, currency, day) so that the GL can answer every reporting question and the subledger every "which bookings?" question. When the ERP Isn't Enough: Custom Finance Platforms discusses when a brand-specific platform is justified, and Never Update, Only Append: Designing Custom Ledgers covers the accounting engine.

Integration With the Rest of the Estate

Each entry in the Lisbon table maps to a Keystone canonical event (`PaymentCaptured`, `PSPFeeCharged`, `PaymentSettled`, `StayCompleted`, `PartnerPayoutInstructed`, `PartnerPayoutSettled`, `RefundIssued`, `ChargebackReceived`, `ChargebackResolved`), whose envelope carries entity, business model, correlation id and amount in minor units; Files, Events and APIs: How Finance Systems Talk introduces that model. PSP clearing, fees and chargebacks are where Northmere Pay, Adyen and Stripe meet the ledger (Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Payments-to-Ledger: Draining the Suspense Swamp). Accounts, dimensions, rate tables and period calendars are master data with owners (Finance Data Architecture: Who Owns the Truth). Posting validations, period locks and reconciliations are the controls the auditor tests (Controls by Design, Not by Audit Season). BlackLine reconciles the balance-sheet accounts these entries create and OneStream translates and eliminates them (BlackLine: Matching, Certifying and Closing, OneStream: Consolidating Actuals, Planning the Future). The Life of a Booking: From €600 Payment to $97.38 of Revenue follows the Lisbon booking all the way into the 10-K.

Failure Modes

These are the accounting-shaped failures I would look for first in any finance estate, each with its mechanism:

- **Revenue at capture**. A payments or booking service emits "revenue" when cash arrives. Dashboards and sometimes ledgers then count the full €600, or even €90 before the stay, as March income. The fix is in the event model: payments publish payment facts; only the recognition rules create revenue.
- **Funds held treated as deferred revenue**. All of the travelers' cash is booked to a deferred-revenue account and "released" at stay. Revenue is overstated by the partners' share unless something nets it later, and safeguarding cannot be evidenced from the ledger.
- **Edits instead of reversals**. A correction updates a posted line. The trial balance still balances; the audit trail is gone, and so is the auditor's reliance on the system.

- **Event date used as posting date, with no lock**. Late settlement files post into March after March has closed, so reported figures change after they are published.
- **Clearing accounts used as suspense**. Unmatched items are parked in PSP clearing and netted at month end. The account never returns to zero, and its balance grows without explanation. Northmere's €14 million month-end average, with about €2.1 million older than 90 days, is what this looks like at scale.
- **Missing dimensions at posting time**. No intercompany counterparty, no business model, no market. Eliminations, segment notes and merchant-versus-agency analysis then depend on spreadsheets, the failure pattern **Spreadsheet Glue**.
- **Rates looked up, not stored**. Each system fetches its own FX rate at posting time and keeps only the converted amount. Revaluation and audit cannot reconstruct what was done, and the two halves of an intercompany pair disagree by a rounding difference that nobody can trace.
- **Reversals unlinked from their originals**. A chargeback lands as a free-standing entry. Nobody can tell which booking, property or period it relates to, so recovery from the partner and the reserve calculation both fail.

The Chief Architect's View

The most important governance point in this chapter is a boundary of decision rights. **Accounting policy belongs to the Chief Accounting Officer; the systems that implement it belong to architecture and engineering; the posting rules that connect the two are owned jointly**. At Northmere, Leila decides that Stayloft is an agent and that commission is earned at stay completion. Dana's architecture function decides how that policy is implemented once, for every brand, with the invariants preserved. Neither can do the other's job, and every serious failure I have seen in this area started with one side assuming the other had done it.

The decisions I would put on a Chief Architect's agenda:

1. **One accounting event catalog**. Every financial event type, the posting rule it triggers, the accounting policy that rule implements, and the owner of each. At Northmere this extends the Keystone canonical event list with a column that Leila's team signs off.
2. **The invariants as a mandatory standard**. Balanced, immutable, period-locked, traceable and complete, plus versioned posting rules, applied to every system that keeps books, including brand platforms. Exceptions are explicit, time-limited and recorded, which is how Architecture Governance That Speeds Decisions treats any standard.
3. **Where treatment is applied**. A target-state decision on the trade-off above, with a dated plan for each brand. At Northmere this is bound up with the 2027 Q4 decision between Central Finance and an accounting hub, which SAP S/4HANA:

One Journal, Clean Core and Converge or Coexist: Living with More Than One ERP discuss.

4. **The dimension set**. Which dimensions every GL line carries at posting time, decided with the controller, tax and FP&A, and governed as master data.
5. **Policy-change readiness**. A test that any recognition policy can be re-run across open contracts, rehearsed before it is needed.

The metrics I would watch, because each reveals an invariant weakening before an auditor does: the number and value of manual journal entries per close and their causes; aging of items in clearing accounts; volume of events arriving after cut-off; unexplained subledger-to-GL differences; and the share of GL lines that can be drilled back to source automatically.

Long-term, the question is not whether Northmere's systems can produce the right entries today, but whether they can keep doing so as business models change. The 2023–2026 shift toward the merchant model, ViDA's deemed-supplier rules in 2028 and a Brazil launch with local payment methods will each change which events matter and which accounts they touch. An estate where policy is implemented once, as versioned rules over canonical events, absorbs those changes as configuration. An estate where policy is scattered across brand code absorbs them as projects.

Questions to Ask

Take these into the next design review of any system that touches money:

1. Which accounting policy does this system implement, who owns that policy, and where is it documented?
2. Has accounting concluded on principal versus agent, and on revenue timing, for every product this system handles?
3. Which events does the system publish, and does each one carry entity, business model, correlation id, event date and amount in minor units with currency?
4. Where are posting rules defined, are they versioned, and does every entry record the rule version that produced it?
5. Can any posted entry be changed or deleted, by anyone, through any path?
6. What happens to an event that arrives after its period is closed?
7. Which clearing accounts does this design create, what empties each of them, and who owns the items that do not clear?
8. Can you take one GL line and list every booking behind it, and take one booking and find every GL line it touched?
9. Which dimensions are on every GL line at posting time, and which reporting or elimination needs are not covered by them?

10. For foreign-currency lines, where do rates come from, and is the rate stored on the line?

The next chapter, Order to Cash, Procure to Pay, Record to Report, takes these entries out of the ledger and into the processes that produce them: Record-to-Report, Order-to-Cash and its marketplace variant, Procure-to-Pay and the close, and the hand-offs between systems where the invariants are most often lost.

Sources and Evidence

Standards and regulation

- IFRS 15 Revenue from Contracts with Customers, IFRS Foundation, 2014 (amended 2016) — the five-step model and joint issuance with the FASB in May 2014.
- FASB Accounting Standards Codification Topic 606, *Revenue from Contracts with Customers* (access-controlled at asc.fasb.org) — the US GAAP counterpart applied by Northmere; principal-versus-agent guidance in ASC 606-10-55-36 to 55-40 and the sales-tax presentation election, cited through the research brief.
- IAS 21 The Effects of Changes in Foreign Exchange Rates, IFRS Foundation — functional versus presentation currency, revaluation and translation; ASC 830 is the US counterpart.
- Directive 2009/110/EC (E-Money Directive), European Union, 2009 — Article 7 safeguarding requirements for e-money institutions.
- VAT in the Digital Age (ViDA), European Commission, 2025 — deemed-supplier rules for short-term accommodation platforms from 1 July 2028, with possible delay to 2030.
- Deloitte DART: FASB clarifies effective date of ASU 2024-03, Deloitte, 2025 — effective dates of the US expense disaggregation standard (secondary source).

Vendor documentation

- Record usage for billing, Stripe — metered usage is aggregated at the end of the billing period, supporting billing usage in arrears.

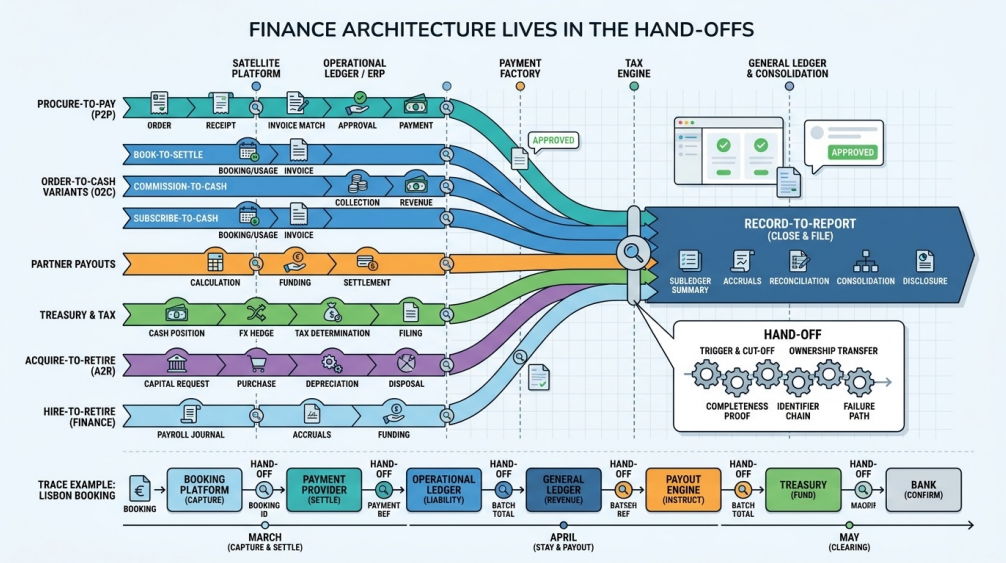
Practitioner and research sources

- Research brief *Accounting, Controls and Regulation for Finance-Systems Architects* (this book's research base, October 2026) — journal-entry anatomy, chart-of-accounts design, accrual and close mechanics, principal-versus-agent illustrations, misconceptions and cautions; foundational mechanics there are textbook material rather than newly verified claims.

- Productscape digest, tutorial concepts "Following the money" (this book's research base, generalized) — the merchant-booking walkthrough, deferred merchant bookings, subledger-versus-GL and reconciliation concepts.

Northmere Group, its brands, entities, systems, people, rates, fees and accounting policies are fictional; the journal entries are teaching illustrations, not accounting advice, and the architectural recommendations are the author's judgment.

9. Order to Cash, Procure to Pay, Record to Report



IN THIS CHAPTER, YOU WILL: Walk through the end-to-end finance processes an enterprise estate serves (Record-to-Report, Procure-to-Pay, Order-to-Cash and its marketplace variants, partner payouts, treasury, tax, Plan-to-Perform, Acquire-to-Retire and the finance side of Hire-to-Retire) and learn to read each one as a chain of hand-offs, with the controls, failure points and metrics that belong to each hand-off.

WHY FINANCE CARES: The controller's close, the treasurer's cash position and the tax director's filing all depend on documents that crossed five or six systems before they arrived. When a number is wrong, it was almost always lost, duplicated, mis-timed or mis-identified at one of those crossings.

WHY ARCHITECTS SHOULD CARE: Systems are bought and built one at a time, but finance runs end to end. The hand-offs between systems carry the cut-offs, identifiers and reconciliations that auditors test and that the close waits on, and nobody owns them unless the architecture assigns them.

KEY POINTS:

- **A finance process is a chain of hand-offs, and the architecture lives at the links.** Each hand-off changes at least one of system, owner, identifier or timing; each must declare its trigger and cut-off, the identifier it carries forward, who owns the item on each side, the control total that proves completeness, and where failures go.
- **The marketplace variants of Order-to-Cash are different processes, not configurations of one.** Book-to-Settle (merchant model), Commission-to-Cash (agency model) and Subscribe/Usage-to-Cash (SaaS) differ in who holds the money, when revenue is earned and which party is paid, so they need different subledgers, controls and metrics.
- **Every process ends in Record-to-Report, so the close inherits every upstream weakness.** Days to close is a lagging measure of hand-off quality; the leading measures are first-pass match rates, touchless rates and aging at each crossing, owned by a named process owner rather than by whichever system happens to hold the item.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Suspense Swamp** — Unmatched items collect wherever a hand-off has no identifier, no control total or no owner for its exceptions.
- **The Doppelgänger Record** — Supplier, partner and bank details are created at more than one point in the process, and the fraud and the mismatches enter there.
- **Spreadsheet Glue** — Accruals, cut-off lists and mapping tables that bridge two systems at month-end live outside any system and any control.

Finance teams describe their work in processes: **Record-to-Report** (R2R), **Procure-to-Pay** (P2P), **Order-to-Cash** (O2C). Architects describe the estate in systems and capabilities. The two views meet at the **hand-off**, where a financial document leaves one system, owner or identifier and continues under another. A supplier invoice approved in a procurement suite, posted in the ERP, paid through a treasury platform and confirmed by a bank statement crosses four systems and three owners, and can be lost, duplicated, mis-dated or misdirected at each crossing.

This chapter reads each end-to-end process the way an architect should: purpose, steps, systems, hand-offs, **key controls** (the checks auditors rely on to prevent or detect a material misstatement), typical failure points and a few metrics. The argument is simple: **the architecture of a finance estate is mostly the architecture of its hand-offs.** Platforms do their internal work well; the money and the audit findings are lost between them.

Two documents run through the chapter: a supplier invoice that enters through Coupa and leaves through a bank, and the Lisbon booking, a €600 merchant-model stay that touches seven processes before it becomes €90 of revenue. Following both is the fastest way I know to find where an estate's design is weak.

Setting the Scene: Two Documents Dana Asked to Follow

Northmere, the fictional multi-brand digital commerce group this book follows, closes its books in nine working days in early 2027; Program Keystone, its finance transformation, targets five by the end of 2028. In February 2027 Dana Whitlock, the Chief Architect for Enterprise Applications and Financial Systems, asked Leila Haddad, the Chief Accounting Officer: "Where do the nine days go?" Leila answered with a close calendar. Dana asked instead for two documents to be followed from origin to the consolidated statements, with every change of system, owner and identifier recorded on one sheet.

The first was a March 2027 invoice from Kestrel Media GmbH, a German agency running a campaign for Stayloft B.V., Stayloft's Dutch contracting entity: €40,000 for March services, on a purchase order raised in Coupa. The second was the Lisbon booking: a German traveler's €600 three-night stay in a Lisbon apartment, paid by card through Adyen on 12 March 2027, stayed 18–21 April, with €90 commission and a €510 payout to the host.

The sheet for the invoice had 11 hand-offs across five systems. The sheet for the booking had 19 hand-offs across eight systems and three month-ends. Six of the 30 hand-offs had no agreed owner for their exceptions, four changed the identifier without keeping the old one, and three depended on a spreadsheet maintained by one person. None of the nine close days was spent in the systems themselves. Almost all were spent proving that the hand-offs had worked.

Processes, Capabilities and Hand-offs

Process versus capability

A **business capability** is what the organization must be able to do, stable over time and independent of how it is done: *account for partner liabilities*, *determine indirect tax*, *execute payments*. Map What Finance Does Before What Runs It owns the capability map and explains why architects should map capabilities before systems. A **process** is how work flows across capabilities, in sequence, to produce an outcome a stakeholder cares about: a supplier paid correctly, a period closed, a partner paid on time. The capability map tells you what must exist; the process map tells you the order in which those capabilities are exercised and where responsibility passes from one to the next.

The two views fail differently. A capability map with no process view hides hand-offs: "execute payments" is one box, but P2P, payouts, payroll and tax each invoke it with different cut-offs and approvals. A process map with no capability view hides duplication: Northmere's "pay partner" step is implemented by two payout engines, which nobody notices until both are drawn against the same capability.

Naming conventions

End-to-end processes are named as *start-to-finish* pairs. The names are conventions, not standards; reference models such as the APQC Process Classification Framework offer one taxonomy, and ERP vendors publish their own. I use the following, matching how Northmere's finance organization is structured:

Process	Starts with	Ends with	Northmere process owner (2027)
Record-to-Report (R2R)	A posting to any ledger	Filed and published financial statements	Leila Haddad, CAO
Procure-to-Pay (P2P)	A purchase need	A cleared supplier payable	Group Head of Procurement, with Leila for the payables ledger
Order-to-Cash (O2C), and its variants	A customer commitment	Cash applied and revenue recognized	Brand finance directors, with Ingrid Solberg for the payment leg
Partner payouts	A liability to a partner	A confirmed, reconciled payout	Ingrid Solberg (Northmere Pay)
Treasury	A bank balance or exposure	A funded, hedged, reconciled cash position	Kofi Asante
Tax: determine–report–pay	A taxable transaction	A filed and paid return	Kofi Asante

Process	Starts with	Ends with	Northmere process owner (2027)
Plan-to-Perform	A strategy and a set of drivers	A plan, a forecast and a variance explained	Group FP&A director
Acquire-to-Retire (A2R)	A capital need	A disposed or fully depreciated asset	Leila Haddad
Hire-to-Retire (H2R), finance touchpoints	A position approved	A final payroll and closed obligations	Chief People Officer, with Leila for payroll accounting

In this book "P2P" means Procure-to-Pay; Plan-to-Perform is always written out. The last column carries the table's most important design decision: a **process owner** is the executive accountable for the outcome end to end, across systems and organizational lines. Without one, each system owner optimizes their own segment and the hand-offs become nobody's job.

Anatomy of a hand-off

A **hand-off** is any point where a financial item changes system, owner, identifier or time basis. I ask the same five questions of every one:

Property	The question	Example of getting it wrong
Trigger and cut-off	What event releases the item, and which period does it belong to when it arrives late?	A March service receipted on 4 April is missing from the March accrual.
Ownership transfer	Who owns the item, and its exceptions, before and after the crossing?	A rejected payment file sits between Accounts Payable and Treasury, each assuming the other will act.

Property	The question	Example of getting it wrong
Identifier	Which key is carried forward, and is the upstream key preserved?	A PSP settlement line carries the payment reference but not the booking id, so matching falls back to amount and date.
Completeness proof	What control total, count or reconciliation proves nothing was lost or duplicated?	SFP's daily summary posts to S/4 without a record count; a re-sent file doubles a day's revenue.
Failure path	Where do rejected or unmatched items go, who sees them, and how do they age?	Unmatched items land in a suspense account with no owner and no aging report.

These five properties are the unit of design in this chapter. Files, Events and APIs: How Finance Systems Talk covers the mechanics behind them (idempotency, control totals, the transactional outbox), and Reconciled by Design: Traceability and Audit Across Boundaries turns them into a reconciliation catalog. Here the job is to find them.

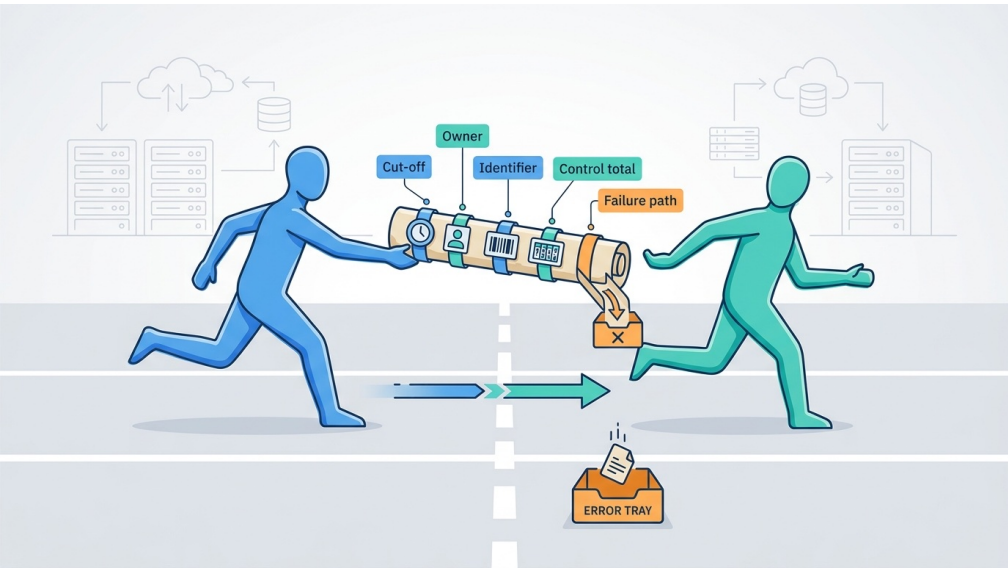


Figure 1: *A hand-off is designed when its cut-off, ownership transfer, identifier, completeness proof and failure path are all agreed; platforms work well internally, and items are lost between them.*

The Process Map of a Finance Estate

Every process eventually feeds Record-to-Report. That is the structural fact that makes the close the place where every upstream weakness surfaces.

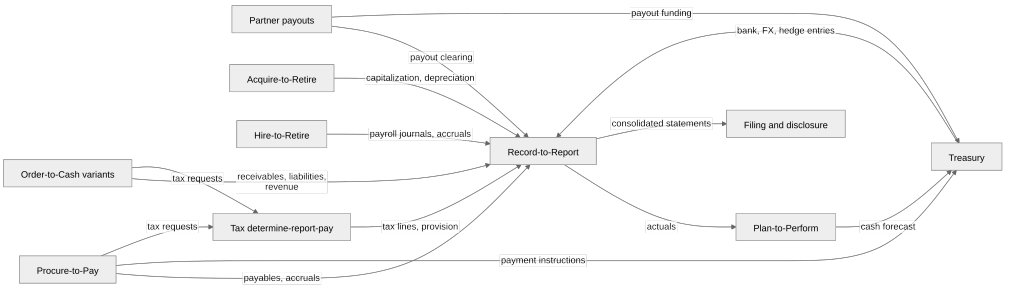


Figure 2: *Every finance process converges on Record-to-Report, and treasury and tax are shared services that the operational processes call; a weak hand-off anywhere on the left shows up as a late or wrong number on the right.*

Two points follow. Treasury and tax behave as **shared services** called from inside other processes, so their hand-offs are numerous and repetitive and are the first candidates for standard interfaces. And Plan-to-Perform consumes R2R's output and feeds treasury's forecast, so a late close degrades the cash forecast, a cost the controller rarely sees.

In the sections that follow, metrics are **Northmere illustrations**: fictional baselines and targets that show what to measure, not industry benchmarks.

Record-to-Report: Where Every Process Lands

Purpose. R2R turns postings from every ledger and subledger into closed, reconciled, consolidated and published financial statements. It is the process the auditor walks through first and the one the CFO is measured on.

Steps at Northmere.

1. **Subledger cut-off and summarization**. SFP, the Stayloft Finance Platform, posts daily summarized journals to S/4HANA (about 38,000 GL lines a day from about 3 million subledger lines). FI-CA, SAP's contract-accounting subledger, holds agency partner accounts. Wayfare closes in Oracle Fusion; Tablekin closes in Workday Financials.
2. **Accruals and adjustments**. Uninvoiced services, unbilled usage, bonuses, PSP fees not yet settled. The digest's process research records a sentence I hear in every close review: manual journals are the largest error source.

3. **Reconciliation**. About 6,800 balance-sheet accounts are reconciled monthly in BlackLine, risk-rated, with subledger-to-GL proofs for SFP and FI-CA and transaction matching for PSP settlements.
4. **Intercompany matching and FX revaluation**. Each entity books its side; the two sides must agree in amount, currency and period before elimination.
5. **Trial balance load and consolidation**. OneStream loads trial balances from S/4, Fusion and Workday, maps three charts of accounts to the group chart, translates to US dollars, eliminates intercompany and produces group results.
6. **Review, disclosure and filing**. Flux review by brand, then Workiva for the 10-Q or 10-K.

Hand-offs that matter. Subledger-to-GL (summary with drill-back key), ERP-to-consolidation (trial balance plus mapping) and consolidation-to-disclosure (linked figures). The second is Northmere's weakest: mappings live in spreadsheets, so a new Fusion account can reach consolidation unmapped. Finance Data Architecture: Who Owns the Truth takes up the group chart of accounts program.

Aspect	At Northmere
Key controls	Period locks per ledger; journal approval with preparer–approver segregation; subledger-to-GL reconciliation per subledger; risk-rated reconciliations with reviewer sign-off; intercompany match before elimination; consolidation only after entity certifications.
Typical failure points	Summaries posted twice or not at all (no record count); late changes pushed to the next period without a flag; top-side adjustments that never flow back to entity ledgers; stale mapping spreadsheets.
Metrics (illustrative)	**Days to close**: 9 (2027), target 5 by end-2028. **Reconciliations auto-certified** (closed by rule): 22%, target 50%. **Post-close adjustments**: target zero material.

Days to close is a lagging indicator: **it measures how long it takes to prove that every upstream hand-off worked**. The levers sit in the other processes; if settlements matched first time and accruals came from systems rather than spreadsheets, the close would shrink without anyone working faster. Controls by Design, Not by Audit Season covers the controls; BlackLine: Matching, Certifying and Closing and OneStream: Consolidating Actuals, Planning the Future the platforms.

Procure-to-Pay: The Supplier Invoice Across Four Systems

Purpose. P2P turns a purchase need into a cleared supplier payable: buy from an approved supplier, under contract, at the agreed price, pay exactly once, to the right account, on time. Its three outcomes are **spend on contract**, **touchless invoices** and **no duplicate or fraudulent payments**.

Steps. Requisition; supplier onboarding (tax ID, bank details, sanctions checks); purchase order; receipt; invoice capture; two-way (invoice against PO) or three-way (against PO and receipt) match; exception approval; posting; payment proposal and approval; payment execution; bank confirmation and clearing.

Systems at Northmere. Coupa handles requisitions, POs, receipts and supplier invoices for indirect spend. S/4HANA holds the payable, the GL posting and the payment run. Vertex determines tax. Kyriba is the **payment factory**, the central hub that formats, screens and routes payment files to the banks, and returns statements to S/4 for clearing. Employee expenses run in parallel through SAP Concur into S/4.

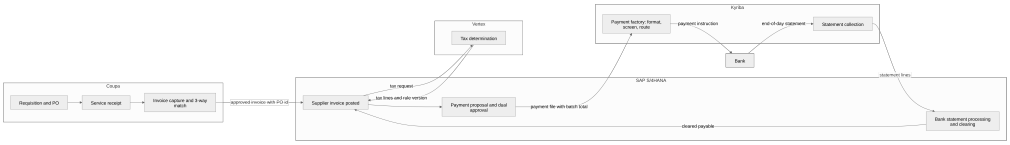


Figure 3: *Procure-to-Pay at Northmere crosses four systems and a bank; the five arrows between subgraphs are the hand-offs that need identifiers, control totals and an exception owner.*

Tracing the Kestrel invoice

Kestrel Media GmbH is onboarded in Coupa in January 2027: VAT ID validated, bank account verified by call-back to a known contact, sanctions screening clear. Coupa is the system of record for the supplier master, which replicates to S/4 as a business partner. On 3 February a first-quarter campaign PO is approved: €120,000 in three monthly service lines of €40,000, coded to Stayloft's marketing cost center. Kestrel issues structured e-invoices to all customers, because German domestic issuing obligations begin in 2027.

Date (2027)	Step	System	Hand-off and what must hold
31 Mar	March invoice, €40,000 net, as a structured e-invoice	Kestrel → Coupa	The structured file is the legal original and is stored as received.
31 Mar	No service receipt yet	Coupa	**Cut-off**. No goods-received/invoice-received (GR/IR) entry exists in S/4, so March cost is missing unless accrued.
WD1 April	€40,000 accrued for March from open PO lines past their service date	Coupa extract → S/4	The extract is **information produced by the entity** (IPE): a control depends on it, so its completeness must be provable. In 2027 it is a spreadsheet.
4 Apr	Service confirmed by the marketing manager	Coupa	Receipt date and service period must both be kept.
6 Apr	Three-way match within tolerance; invoice approved	Coupa	An **application control**: no payment for services not ordered or received.
6 Apr	Posted in S/4 with April posting date; the March accrual reversed on 1 April	Coupa → S/4	Coupa invoice id and PO id carried into the S/4 document; duplicate check in both systems.

Date (2027)	Step	System	Hand-off and what must hold
6 Apr	Reverse charge: Dutch input and output VAT of €8,400 each (21%)	S/4 → Vertex → S/4	Determination, inputs and rule version stored; S/4 tax lines reconcile to Vertex before the return.
4 May	Payment proposal, two approvers	S/4	Neither approver can maintain supplier bank details.
5 May	Payment file (ISO 20022 pain.001) with count and batch total	S/4 → Kyriba → bank	Kyriba checks the total, screens and routes; a rejected file belongs to Treasury Operations, not AP.
7 May	End-of-day statement (camt.053); payable cleared (value date 6 May)	Bank → Kyriba → S/4	Matching key is S/4's payment reference; if the bank truncates it, clearing falls back to amount and date.
May close	Bank and GR/IR accounts reconciled	S/4 → BlackLine	GR/IR aging, where receipts and invoices that never met accumulate, is the P2P health check.

Three things in that table are architecture, not operations. The **cut-off**: a March service receipted in April can only be accrued from PO data, so the PO must carry a service period. The **identifier chain**: PO id, invoice id, S/4 document, payment reference and statement reference must each carry the previous one, or the bank line cannot be traced to the PO. The **ownership transfer at the payment file**: once Kyriba has it, an error is Treasury's, and the status must flow back so AP does not pay twice. The reverse charge nets to zero but must still appear in the VAT return, so the tax hand-off is mandatory even when no cash tax is paid; Accounting for Architects: Ledgers, Journals and Revenue covers the accruals and reversals.

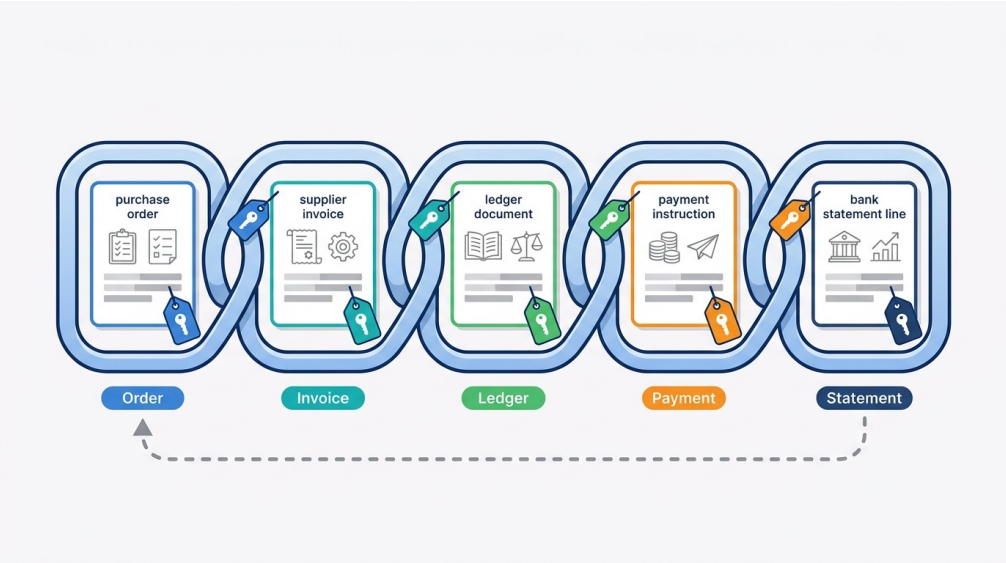


Figure 4: *Each document on the Kestrel invoice's path must carry the previous one's identifier; if any link drops its key, the bank line can no longer be traced back to the purchase order.*

Aspect	At Northmere
Key controls	One front door for supplier creation and bank-detail changes (Coupa onboarding), with independent verification; three-way match with tolerances; duplicate-invoice check in Coupa and S/4; two payment approvers without supplier-master rights; count and total checked between S/4 and Kyriba.
Typical failure points	Changed supplier bank details, **a classic fraud vector** when updated from a convincing email outside the workflow; services never receipted; duplicate suppliers across brands; unreadable e-invoice formats; bank rejections nobody owns.
Metrics (illustrative)	**Touchless invoice rate**: 58%, target 80%. **First-pass match rate**: 81%. **On-time supplier payment**: 93% within terms.

Order-to-Cash and Its Marketplace Variants

Classic O2C (customer setup, order, fulfillment, invoice, receivable, collection, cash application, revenue) fits Northmere Ads, whose advertiser receivables sit in S/4. It does not fit most of Northmere, because in a marketplace "who holds the money", "when is revenue earned" and "who gets paid" have different answers per business model. **I treat the three marketplace variants as different processes** with different subledgers, controls and metrics; treating them as one O2C with options produces one design that is wrong for two of them.

Variant	Northmere brand and model	Who holds the customer's money	Revenue	Who is paid onward	Subledger
Book-to-Settle	Stayloft merchant; Wayfare rail and transfers	Northmere, from booking until payout	Net commission or margin (agent), when Northmere's own promise is met	The partner, net of commission	SFP (Stayloft); Oracle Fusion (Wayfare)
Commission-to-Cash	Stayloft agency; Wayfare flights	The partner (property, airline)	Commission, when earned	Nobody: Northmere collects from the partner	FI-CA (Stayloft); Oracle Fusion (Wayfare)
Subscribe/Usage-to-Cash	Tablekin	Stripe, then Tablekin	Subscription ratably; usage as consumed	Venues, for payments facilitated through Stripe Connect	Stripe Billing; Workday Financials

The principal-versus-agent judgment and revenue timing belong to Accounting for Architects: Ledgers, Journals and Revenue.

Book-to-Settle: the merchant model

Purpose. Collect the traveler's payment, hold it as a liability until the partner delivers, recognize the commission when earned, pay the partner the rest, and prove at every month-end that the money held matches the bookings outstanding. It is Northmere's dominant flow: 64% of Stayloft's 2026 gross transaction value.

Steps. Booking published as a financial event; payment captured; PSP settlement received net of fees; liability recorded in the subledger (**deferred merchant bookings**, cash held for trips not yet taken); changes and cancellations adjust it; stay completed and revenue recognized; partner payable created; payout instructed and settled; three-way reconciliation of booking, PSP and bank; summary to the GL.

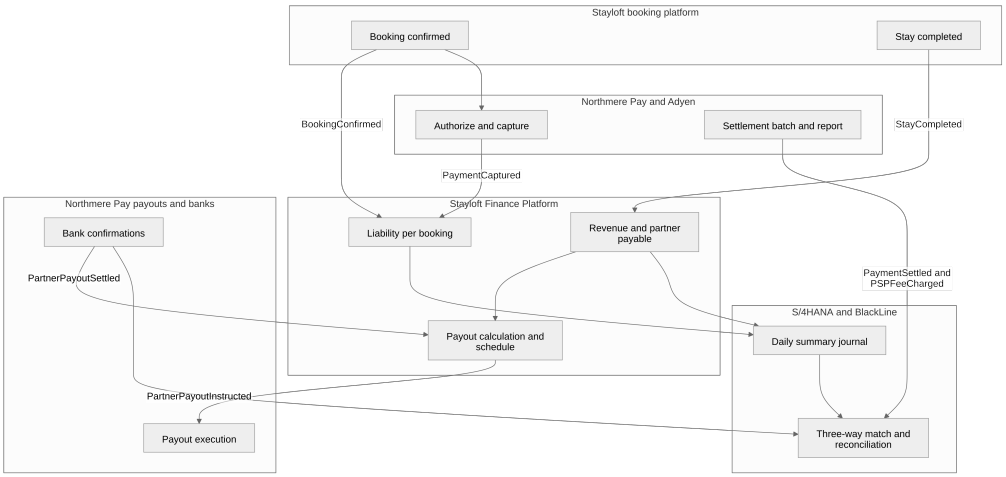


Figure 5: *Book-to-Settle as swimlanes: the booking, the money and the accounting run in different systems on different clocks, and BlackLine's three-way match is where they are proven to agree.*

Tracing the Lisbon booking. On 12 March the booking platform emits `BookingConfirmed` with the booking id as correlation id; Northmere Pay captures €600 through Adyen and emits `PaymentCaptured` with the same correlation id plus Adyen's payment reference; SFP records a €600 liability. On 14 March Adyen books the capture into settlement batch 1047, and on 16 March the batch payout credits Northmere Payments B.V., the group's licensed e-money institution, with €598.26 for this booking (€600 less the €1.74 fee), one line among thousands. Adyen documents settlement reconciliation at transaction level ("each payment, refund, and chargeback included in a single payout batch and the costs for each transaction") and at batch level, matched to the bank statement. Northmere needs both: one ties the €598.26 to the booking, the other ties the batch to the bank.

At the **March close** the €600 is a liability, not revenue, the €1.74 is an expense, and the €600 is traveler funds that Northmere Payments B.V. must safeguard. On 21 April the

stay completes; under the policy this book uses, SFP recognizes €90 of commission and turns €510 into a payable to the host (the commission's VAT is simplified away here; the tax section explains why it cannot be in practice). The payout is instructed on 22 April and settles on 26 April. At the **April close** the liability is gone, the revenue sits in April, and the payout must match the bank.

Aspect	At Northmere
Key controls	Booking-event completeness by brand and day; settlement-report completeness per PSP and batch; three-way match with tolerance rules; daily SFP-to-GL proof; daily safeguarding reconciliation; payout release only after the safeguarding check.
Typical failure points	Modified bookings that lose the link to their payment; each PSP and alternative payment method (APM) reporting fees, FX and timing differently; settlement straddling month-end; refunds reaching payments before SFP; chargebacks after the partner is paid.
Metrics (illustrative)	**First-pass settlement match at working day 3**: 98.1% by count for Stayloft APMs (2026), target 99.7%. **Suspense**: €14 million average month-end, €2.1 million older than 90 days. **Deferred merchant bookings roll-forward explained**: 100%.

The €14 million is Northmere's clearest case of The Suspense Swamp, and its root cause is a hand-off property: in an April 2027 analysis of about 40,000 unmatched APM lines, 48% lacked the booking reference. Payments-to-Ledger: Draining the Suspense Swamp fixes it; Payments for Finance Architects: Eleven Reasons the Numbers Disagree explains why settlement is hard. Wayfare runs the same process in a different estate: the €84 Paris–Lyon rail ticket is paid through Stripe, with Wayfare as merchant of record but, under Northmere's policy, agent for the rail operator. Oracle Fusion recognizes the €6.72 margin at ticket issue and holds the €77.28 fare as a payable to the operator, who is paid monthly by Wayfare's own payout engine. **Same process, second implementation.**

Commission-to-Cash: the agency model

Purpose. Invoice the commission earned on stays the partner collected for, with the right tax, collect it, and keep partner receivables clean.

Steps. Accrue commission on stayed bookings (partners mark no-shows late); determine VAT from the partner's location and status; issue the monthly invoice with booking-level

detail in the partner extranet; collect, dun, and **net against merchant payouts** where a partner has both; clear receivables; feed the bad-debt reserve from aging.

Systems. SFP's commission billing calculates; FI-CA (since 2025, about 850,000 partner contract accounts) holds the open item, dunning and clearing; Vertex determines VAT; S/4 receives summaries. Netting is a hand-off between two variants, an FI-CA receivable against an SFP payable for the same property, and **netting across two subledgers requires one partner identity**: a master-data requirement, not a payments feature.

Aspect	At Northmere
Key controls	Commission base reconciled to stayed bookings; invoice numbering per entity and jurisdiction; logged tax determination; write-off thresholds; FI-CA-to-GL reconciliation.
Typical failure points	Stay-data disputes; invalid VAT IDs; payments without invoice reference, so cash sits unapplied; small balances costlier to chase than to write off.
Metrics (illustrative)	**Days sales outstanding (DSO)**: 41 days, target 32. **Unapplied cash** older than 30 days. **Disputes** per thousand invoices.

Subscribe/Usage-to-Cash: the SaaS model

Purpose. Bill a subscription plus metered usage, collect it, and recognize revenue on the right schedule. Tablekin's Austin venue on the Pro plan pays $249 a month plus $0.25 per seated cover; in March 2027 it seats 3,120 covers, so usage is $780 and the invoice $1,029 before sales tax.

Steps. Usage events rated (each seated cover is one event from Tablekin's product); invoice finalized on the first of the month, with the subscription billed in advance and usage in arrears; sales tax added; payment collected and paid out by Stripe; revenue recognized (subscription over its month, usage as consumed); daily summarized journals posted to Workday Financials.

The cut-off hand-off. The $1,029 invoice is dated 1 April and mixes two periods: the **April** subscription ($249, a contract liability until April is served) and **March** usage ($780). At the March close Workday Financials must therefore already hold the $780 as accrued revenue, estimated from usage Stripe has rated but not yet invoiced, and must defer the $249 when the invoice posts. An integration that books invoices as revenue on their date gets both months wrong. On the cash side, Stripe's payout reconciliation report matches "the payouts you receive in your bank account with the batches of payments and other transactions that they relate to", and its itemized view carries

`invoice_id` and `subscription_id`: the identifiers that trace a bank credit back to the venue's invoice. Vendor change reaches hand-offs too: Stripe now recommends Metronome over its Meters API for new usage-based billing integrations.

Aspect	At Northmere
Key controls	Usage completeness (seated covers against rated usage); Stripe AR reconciled to the Workday AR control account; revenue schedule review; payout-to-bank match.
Typical failure points	Usage events lost or duplicated; unexplained mid-period proration; revenue posted only at finalization; fees and disputes netted inside payouts.
Metrics (illustrative)	**Usage completeness**: rated ÷ seated covers, 100.0% within a day. **Billing-to-revenue variance** at close.

Partner Payouts

Payouts sit inside Book-to-Settle but deserve their own view. They are Northmere's largest outbound flow by count, about 9 million a month across Stayloft and Tablekin. They depend on **the most fraud-sensitive master data in the estate**, partner bank details. And they produced the FY2026 significant deficiency: developers had production access to the SFP payout engine, and rule changes lacked documented approval.

Steps. Calculate the net payout, usually tied to check-in or check-out; apply holds (fraud, disputes, missing tax data); fund the run from the right entity and currency, with the safeguarding check passed before client funds are released; release in batches by currency, method and bank cut-off; confirm settlement; re-queue returns; clear partner payables.

Systems. SFP calculates and schedules Stayloft payouts and hands payout instructions to Northmere Pay, which executes them; Kyriba funds the accounts they are paid from. Tablekin venue payouts run inside Stripe Connect; Wayfare's ground-transport suppliers are paid by Wayfare's own engine. Three routes for one capability.

The Lisbon payout. On 22 April SFP emits `PartnerPayoutInstructed` for €510, carrying the booking id and its own payout id, in a euro batch whose count and total Northmere Pay checks before execution. On 26 April the bank confirms; SFP emits `PartnerPayoutSettled` and clears the payable. If the host's bank returns the payment, it must come back to SFP as a payout status, not as an unexplained credit on

a Northmere bank account.

Aspect	At Northmere
Key controls	Bank-detail changes independently verified, second-approved and logged; payout rule changes under change management (the FY2026 remediation); safeguarding check before release; SFP-to-Northmere Pay batch totals; returns routed to a named owner.
Typical failure points	Bank details changed in a support tool without the second approval; adjustments after payout (the €420 chargeback arrived 40 days after the stay); returns aging without owners; missed country cut-offs.
Metrics (illustrative)	**On-time partner payout**: 98.6% (2027), target 99.5%. **Returned payouts** and their age. **Payouts with no post-payment adjustment**.

Treasury: Cash, FX and Bank

Purpose. Know where the group's cash is, every day; fund every outbound flow from the right account in the right currency; manage FX exposure within policy; keep bank connectivity secure; and, for Northmere's e-money institutions, keep safeguarded funds separate and reconciled.

Steps. Collect prior-day statements (camt.053 or MT940) and intraday reports (camt.052 or bank APIs) from about 410 accounts at 27 banks; separate safeguarded from operating cash; position cash by entity and currency; forecast from payouts, receipts, AP runs and payroll, with deferred merchant bookings as the leading indicator of payouts; capture FX exposures (partners are paid in 31 currencies) and hedge within policy; post hedge and revaluation entries to the GL; execute every other process's payments through the payment factory.

Systems. Kyriba is the treasury management system and payment factory; Kyriba describes its platform as connecting to banks out of the box through open APIs. S/4 receives statement postings and hedge entries; SFP supplies the payout forecast and the traveler-funds liability; the daily safeguarding calculation compares the subledger's relevant funds with the safeguarding account balances.

Bank formats are often stated wrongly: SWIFT's MT/ISO 20022 coexistence ended in November 2025 for **cross-border interbank payment instructions**, not for corporate statements, and MT940 remains in wide use while banks migrate to camt.053. Treasury's

hand-offs must handle both for years.

Aspect	At Northmere
Key controls	Central approval of bank accounts and signatories; dual approval of manual payments; file integrity between source systems and Kyriba; daily bank and safeguarding reconciliations; hedge designation documented at inception.
Typical failure points	Unconnected banks leaving manual lines in the position; late statements; exposures assembled in spreadsheets; safeguarded and operating cash commingled; returns without the originating reference.
Metrics (illustrative)	**Cash visibility**: share of group cash reported automatically by 09:00 Amsterdam, 94%, target 99%. **13-week forecast accuracy** by entity. **Safeguarding breaks** open at day end: zero.

Tax: Determine, Report, Pay

Purpose. Determine the right tax on each transaction when it happens, record it with evidence, report and pay it on time, and supply data for the income-tax provision and the global minimum tax.

Steps.

1. **Determine** at the transaction: VAT on Stayloft's commission and on merchant sales, reverse charge on Kestrel's invoice, US sales tax on Tablekin's invoices. The determination must record jurisdiction, rate, the rule version and the inputs used.
2. **Record** the tax lines in the ledger with jurisdiction and entity.
3. **Reconcile** the tax engine's committed transactions to the GL tax accounts before filing.
4. **Report**: VAT returns per entity and country; platform reporting under DAC7, which requires EU platform operators to report seller information annually by 31 January; digital services taxes; e-invoices and, increasingly, **clearance**, where the authority or an accredited platform validates an invoice before it is legally issued.
5. **Pay** through the payment factory, from the right entity and account.
6. **Provide** jurisdiction-level data to the income-tax provision and to Pillar Two, the OECD's 15% global minimum tax, which needs consolidation-grade, entity-level data.

Systems. Vertex determines indirect tax for S/4 and for SFP's commission invoicing; Wayfare uses Oracle's tax engine in nine markets. In May 2027 the design authority made Vertex the group standard, called only through a Northmere-owned tax service adapter, which turns tax determination into one standard hand-off instead of one per calling system (Around the Core: Where Specialist Platforms Earn Their Place). OneStream and the Snowflake finance data platform supply provision and Pillar Two data; Kyriba pays.

The Lisbon booking's tax. The €90 commission is a service to a Portuguese host. If the host is a VAT-registered business, the general B2B rule places the supply where the customer is established and the host self-assesses under reverse charge; if the host is a private individual, the answer changes. **The same booking can produce different tax outcomes depending on master data that lives in another process**, which is why each determination is logged with its inputs. The host reappears in January 2028 in Northmere's DAC7 report, with €510 paid and €90 in fees. DAC7 and VAT data must come from the same partner master and transaction record, or they will disagree in front of a tax authority.

The horizon. The EU's VAT in the Digital Age package, adopted in March 2025, makes e-invoicing the default for intra-EU cross-border B2B supplies from 1 July 2030 and lets member states mandate domestic e-invoicing without derogation; as of 2026 Belgium, Poland, France and Germany are already phasing in mandates. Under clearance, **invoice issuance moves into the critical path of Commission-to-Cash and P2P**: an outage at an accredited platform stops invoicing. That is an architecture decision, not only a tax one; Global by Design: Entities, Currencies, Jurisdictions and Brands goes further.

Aspect	At Northmere
Key controls	Engine rule updates under change control; determination log retained for the statute period; engine-to-GL reconciliation before filing; return approval; archive of the legally valid e-invoice.
Typical failure points	Missing VAT IDs and location evidence; entity–registration mismatches; fallback rates never recalculated after an outage; DAC7 data from a different master than payouts; disputes that outlast the systems that produced the data.
Metrics (illustrative)	**Returns filed on time**: 100%. **Unexplained engine-to-GL difference** at filing: zero above threshold. **Determination exceptions** per million transactions.

Plan-to-Perform

Purpose. Set targets, forecast against them and explain variances: the process where finance meets the brands' operating data.

Steps and systems. Load closed actuals into OneStream; build driver-based plans (for Stayloft: room nights, average daily rate, take rate, merchant mix, marketing) from drivers in Snowflake's FinDB and headcount from Workday HCM; reforecast monthly with FX and demand scenarios; review performance; feed treasury's cash forecast.

The Lisbon booking in the plan. It is one of millions in the drivers: 3 room nights at €200, a 15% take rate, merchant model. The plan needs the **definitions** behind the drivers to match the ledger. If FinDB counts room nights at booking and the ledger recognizes revenue at stay, the take rate computed from the two drifts whenever the booking curve shifts, and the review becomes a debate about whose number is right. One Number, Many Views: Finance Data from Operations to Analytics resolves it with governed metrics.

Aspect	At Northmere
Key controls	Planning actuals tied out to consolidated results; governed driver definitions; locked plan versions. Mostly outside internal control over financial reporting (ICFR), but numbers derived from it (impairment tests, guidance) are not.
Typical failure points	Shadow reforecast spreadsheets; brand-specific driver definitions; actuals restated after load.
Metrics (illustrative)	**Revenue forecast at month −3** within ±3%. **Close-to-forecast cycle**: 8 working days, target 4.

Acquire-to-Retire

Purpose. Account for long-lived assets from acquisition to disposal: capitalize the right costs, depreciate correctly, track leases, test for impairment, retire correctly.

Steps and systems. Capital request; purchase through Coupa with an asset flag; costs on an asset under construction in S/4 asset accounting; capitalization when in service; monthly depreciation; impairment review; disposal. Two categories dominate at Northmere: **leases** (offices, data center capacity) in a lease subledger, and **capitalized internal-use software**, which under US GAAP (ASC 350-40) requires engineering time on qualifying development to be captured and capitalized.

Hand-offs that matter. P2P-to-asset (an invoice expensed that should have been capitalized, or the reverse) and engineering-time-to-capitalization, which draws on brand engineering tools never designed as financial inputs. If a key control relies on a time-tracking export, that export becomes IPE and its pipeline enters audit scope.

Aspect	At Northmere
Key controls	Capitalization rules on asset-flagged POs; register-to-GL reconciliation; depreciation run completeness; lease completeness; capitalized software reviewed by project stage.
Typical failure points	Assets under construction never capitalized; unrecorded disposals; leases embedded in service contracts; capitalization from a spreadsheet of engineers' time.
Metrics (illustrative)	**Assets under construction older than 12 months**. **Register-to-GL difference**: zero.

Hire-to-Retire: The Finance Touchpoints

Hire-to-Retire belongs to HR, but finance touches it at five points:

1. **Cost center.** Workday HCM (group-wide since 2019, about 24,000 workers) assigns each worker to a cost center, and every payroll journal inherits it.
2. **Payroll accounting.** Workday Payroll (US, UK, Canada) and third-party providers elsewhere each post payroll journals to S/4 that must reconcile to the payroll register.
3. **Funding and statutory payments** through Kyriba, on dates that cannot slip.
4. **Accruals** for bonuses, leave and equity compensation at each close.
5. **Approvers and access.** Workday's manager hierarchy drives approval chains in Coupa and Concur, and its leaver events drive access removal. **A leaver who keeps an approval right in Coupa is an access-control failure that starts in HR data.**

Aspect	At Northmere
Key controls	Payroll journal reconciled to the payroll register by entity; net pay matched to the bank; approval hierarchy synchronized daily from Workday; leaver access removed within a defined window.

Aspect	At Northmere
Typical failure points	Cost-center changes not reflected in open approvals; third-party payroll journals mapped by hand; provider reports used as evidence without IPE testing; leavers keeping approval rights in satellites.
Metrics (illustrative)	**Payroll journals posted without manual mapping**, by country. **Leaver access removed within 24 hours**.

The Hand-off Matrix

The matrix below, the most useful artifact from Dana's exercise, puts each process against Northmere's systems by role and names its weakest hand-off.

Process	Originating system	Subledger or operational ledger	General ledger	Money movement	Satellites	Close and reporting	Weakest hand-off in 2027
Record-to-Report	All ledgers	SFP, FI-CA	S/4, Fusion, Workday Fin	—	Vertex (tax accounts)	BlackLine, OneStream, Workiva	Three charts of accounts mapped in spreadsheets
Procure-to-Pay	Coupa, Concur	S/4 AP	S/4	Kyriba, banks	Vertex	BlackLine (GR/IR, bank)	Coupa open-PO accrual extract

Process	Originating system	Subledger or operational ledger	General ledger	Money movement	Satellites	Close and reporting	Weakest hand-off in 2027
Book-to-Settle	Stayloft and Wayfare platforms	SFP; Fusion (Wayfare)	S/4; Fusion	Northmere Pay, Adyen, Stripe, APMs	Vertex	BlackLine Transaction Matching	APM settlement lines without booking id
Commission-to-Cash	Stayloft platform	SFP billing, FI-CA	S/4	Direct debit, bank	Vertex, e-invoicing	BlackLine	Incoming payments without invoice reference
Subscription-to-Cash	Tidalkin product	Stripe Billing	Workday Fin	Stripe	Sales tax	OneStream (via Workday TB)	Unbilled usage at month-end
Partner payouts	SFP; Wayfare engine; Stripe Connect	SFP, Fusion	S/4, Fusion	Northmere Pay, banks, Stripe; funded by Kyriba	—	BlackLine	Returned payouts without payout id
Treasury	Banks	Kyriba	S/4	Kyriba	Safeguarding calculation	BlackLine (bank recs)	Banks without automated statements

Process	Originating system	Subledger or operational ledger	General ledger	Money movement	Satellites	Close and reporting	Weakest hand-off in 2027
Tax	All transacting systems	Vertex, Oracle tax	S/4, Fusion, Workday Fin	Kyriba	Snowflake (DAC7, DST)	OneStream (provision, Pillar Two)	Two engines for one capability
Plan-to-Perform	OneStream, FinDB	—	Actuals from OneStream	—	Workday HCM	OneStream	Driver definitions differing by brand
Acquire-to-Retire	Coupa, engineering tools	S/4 asset accounting, lease subledger	S/4	via P2P	—	BlackLine	Engineering time to capitalization
Hire-to-Retire (finance)	Workday HCM	Workday Payroll, third-party payroll	S/4	Kyriba	—	BlackLine	Third-party payroll mapping

Read down a column and the platform's blast radius appears: Kyriba is in the money path of five processes, so its outage stops supplier, payroll and tax payments at once and leaves payouts unfunded; Vertex sits in five. Read across a row and the process's dependency chain appears: Book-to-Settle needs eight systems to agree before one booking is proven.

Both Documents, Across Every Process

Date (2027)	Kestrel invoice (€40,000 net)	Lisbon booking (€600)	Process
12 Mar	—	Captured; liability €600 in SFP; safeguarded	Book-to-Settle; Treasury
14–16 Mar	—	Adyen batch 1047 booked 14 Mar (this booking €598.26 net of the €1.74 fee); the batch payout of €1,380.52 credited to the bank 16 Mar	Book-to-Settle; R2R
31 Mar	Invoice issued; service not receipted	—	P2P
March close	€40,000 accrued from open PO	€600 in deferred merchant bookings; matched three ways	R2R
6 Apr	Matched, posted, reverse charge €8,400 / €8,400	—	P2P; Tax
21 Apr	—	Stay completed; €90 revenue; €510 payable	Book-to-Settle; R2R
22–26 Apr	—	€510 payout instructed and settled	Partner payouts; Treasury
April close	Accrual reversed against invoice	Liability cleared; revenue in April	R2R
5–7 May	Paid via Kyriba; cleared from camt.053	—	P2P; Treasury
Q1, Q2 VAT returns	Reverse charge reported in NL	Commission VAT reported per host status	Tax
Jan 2028	—	Host reported in DAC7	Tax

Every row in the right-hand column is a different process owner. **No single system sees either document whole**, and only a deliberately designed chain of identifiers lets anyone reconstruct it. The Life of a Booking: From €600 Payment to $97.38 of Revenue traces the booking in full depth, hop by hop, with the invariants that must hold at each one.

Trade-offs: Where Process Logic Lives and Who Owns It

Two design questions recur in every process: where the logic that moves an item across a hand-off should live, and how the process is owned.

Option for hand-off logic	When it is right	What it costs	Northmere example
Inside the ERP's native workflow	The process runs mostly in one platform and the vendor's standard fits	Couples the process to the ERP's release cycle; tempting to customize	Payment proposal and approval in S/4
Inside the satellite	The satellite owns the domain and its network (suppliers, banks, tax authorities)	Satellite's data model becomes a dependency; logic duplicates across satellites	Three-way match in Coupa; bank formats in Kyriba
In the integration layer	Mapping and routing between platforms with no business judgment	Business rules creep into mappings, invisible to process owners	Today's MuleSoft flows for Wayfare and Tablekin
In a custom orchestration service	Volume, business-model novelty or cross-brand logic no vendor supports	You own controls, audit evidence and run cost	SFP's payout scheduler

The rule I use: **put judgment in the system that owns the domain, put translation in the integration layer, and never put judgment in the integration layer.** A mapping

that decides whether a cost is capital or expense is a business rule wearing integration clothes, and it will not be found by an auditor or a process owner until it fails.

On ownership, the choice is between system-aligned ownership (clear technical accountability, orphaned hand-offs), process-aligned ownership (one owner for end-to-end metrics and exceptions, but it needs authority over brand teams) and capability-aligned ownership (prevents duplicates, but nobody owns the outcome). I recommend the second paired with the third: **the process owner signs the hand-off contract**; **the capability owner implements it**. Where a process owner cannot be given authority over brand engineering, the hand-off contract should at least be an architecture standard the review board enforces.

Integration With the Rest of the Estate

This chapter owns the process view; other chapters own the layers it crosses. Anatomy of a Finance Estate: Layers, Planes and Boundaries places each system of the matrix in the reference model. Accounting for Architects: Ledgers, Journals and Revenue owns the journal entries behind each step. Finance Data Architecture: Who Owns the Truth owns the master data every hand-off depends on. Controls by Design, Not by Audit Season turns the key controls here into a control framework. Files, Events and APIs: How Finance Systems Talk supplies the idempotency and control-total mechanics, Canonical Events: A Common Language for Money the canonical events the Lisbon trace used, and Systems of Record: Ending the Doppelgänger Record formalizes how authority over an item moves from booking platform to subledger to GL.

Failure Modes

The orphaned exception queue. Rejected items land in a queue neither side owns. In one week of 2026 a bank format change caused 312 payout instructions to be rejected downstream of SFP; SFP still showed them as instructed, and partners called before finance noticed. The fix is a failure path with an owner and a status that flows back to the originator.

Identifier amnesia. A system mints its own key and drops the upstream one: APM settlement lines without booking ids, bank-truncated payment references, Coupa invoice ids lost in S/4. Every hand-off contract should require the upstream key to be preserved, and tests should prove it.

Cut-off by convention. Periods assigned by when the file arrived rather than by the event's date, so late items slide into the next period unflagged and the controller spends a day explaining the flux. Carry event date, posting date and period separately.

The accrual spreadsheet. Accruals built from extracts and finished by hand are IPE, key to the close, and owned by one person: Spreadsheet Glue in its purest form, and usually the first thing to automate because the source data already exists.

Two front doors for master data. Supplier bank details editable in Coupa *and* in S/4; partner bank details editable in SFP's console *and* a support tool. Every second door is a fraud path and a reconciliation break. The fix is one front door per master object with replication outward, which Finance Data Architecture: Who Owns the Truth develops.

Duplicate process implementations. Two payout engines, three billing approaches and two tax engines run "the same" process differently, so metrics are not comparable and controls are tested twice. The process view exposes them; the capability view decides which survives.

The Chief Architect's View

Decide that processes have owners, and that hand-offs are architecture. At Northmere, Dana proposed and Tomás Ferreira, the CFO, approved that every process in the table above has a named owner, and that every hand-off between systems has a contract (the five properties) approved by the process owner and registered in the architecture repository. That turns the hand-off from tribal knowledge into a governed artifact the review board can check. A process owner without influence on brand roadmaps will write good contracts nobody implements; the review board described in Architecture Governance That Speeds Decisions is what gives the contracts teeth.

Standardize the hand-offs that repeat. Payment instructions to the payment factory, tax determination requests, summary postings to the GL and bank statement returns each recur across several processes. One contract for each, with one identifier convention and one control-total rule, removes more risk than any single platform project. Keystone's canonical events are the O2C half of this; payment-factory and tax-request standards are the other half.

Measure the leading indicators, not only the close. The close metric tells the CFO how long proof took. First-pass match rates, touchless rates, unapplied cash, suspense aging and returned-payment aging tell the architect which hand-off to fix next. I would put five of them on the Keystone dashboard next to days to close.

Treat regulation as a process-change trigger. Clearance e-invoicing puts tax authorities inside P2P and Commission-to-Cash; DAC7 and ViDA put partner master data inside tax; safeguarding puts the subledger inside treasury. Each changes a hand-off. The architecture function should review every regulatory change against the hand-off matrix, not leave it to the tax or legal team to discover the systems impact.

Questions to Ask

- Who is the named owner of each end-to-end process, and do they have authority over the hand-offs, or only over their own system?
- For the three hand-offs that feed the most close days, what are the trigger, cut-off, identifier, control total and failure path?
- Can we trace one supplier invoice and one customer transaction from origin to the consolidated statements today, and how long does it take?
- Which hand-offs depend on spreadsheets or manual extracts that a key control relies on?
- Where can supplier and partner bank details be changed, and how many front doors exist for each?
- Which processes are implemented more than once across brands, and which implementation is the target?
- How do late-arriving events (refunds, chargebacks, late receipts, restated usage) find their period, and are they visible when they do?
- Which upcoming regulatory changes alter a hand-off, and who is redesigning it?

Processes describe how items flow; they cannot work unless the items flowing through them are described consistently in every system they touch. The Kestrel invoice depended on one supplier record and one chart of accounts; the Lisbon booking depended on one partner identity across SFP, Kyriba, Vertex and the DAC7 report. The next chapter, Finance Data Architecture: Who Owns the Truth, takes up the master data, systems of record and governance that every hand-off in this chapter quietly assumes.

Sources and Evidence

Vendor documentation

- Settlement reconciliation, Adyen documentation, accessed 2026 — transaction-level and batch-level settlement reconciliation used in the Book-to-Settle trace.
- Payout reconciliation report, Stripe documentation, accessed 2026 — matching payouts to the transactions they settle; `invoice_id` and `subscription_id` columns used in the Subscribe/Usage-to-Cash trace.
- Usage-based billing, Stripe documentation, accessed 2026 — Metronome as Stripe's primary usage-based billing platform; Meters API remains supported.
- Kyriba platform overview, Kyriba, accessed 2026 — bank connectivity and open APIs, as the vendor describes them.
- Vertex Indirect Tax O Series, Vertex — indirect tax determination for sales and purchase transactions.

Standards and regulation

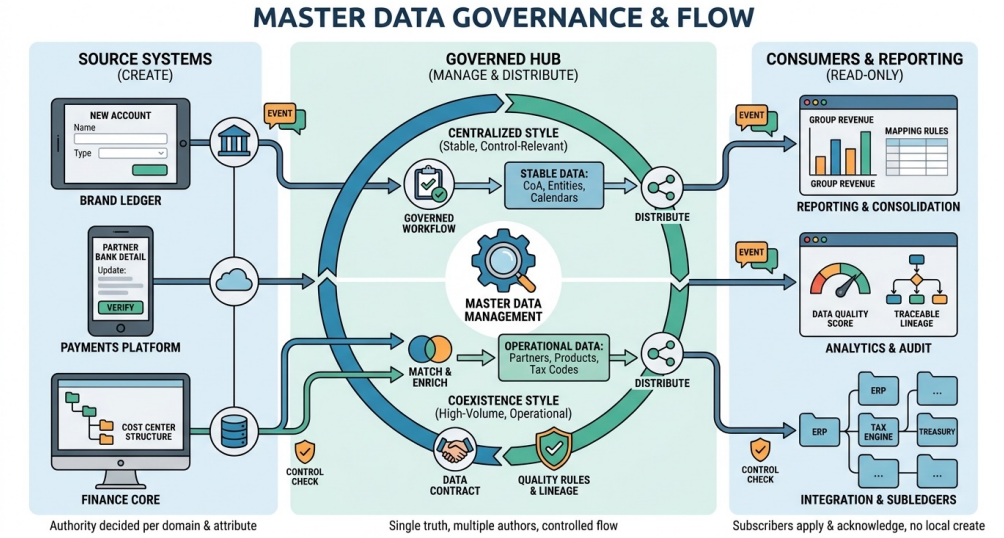
- Adoption of the VAT in the Digital Age package, European Commission, 2025 — ViDA adoption and the 1 July 2030 e-invoicing default for intra-EU B2B.
- Peppol: about, OpenPeppol — the four-corner model behind network e-invoicing referred to in the tax section.
- Deadline for first DAC7 reporting, PwC Belgium, 2024 — DAC7 scope and the 31 January annual reporting deadline (secondary source).
- Germany e-invoicing timeline, Avalara — German receive (2025) and issue (2027/2028) obligations (vendor source; verify against the German Federal Ministry of Finance before relying on dates).
- IFRS 15 Revenue from Contracts with Customers, IFRS Foundation — revenue timing behind Book-to-Settle and Subscribe/Usage-to-Cash.
- IFRS 16 Leases, IFRS Foundation — lease accounting in Acquire-to-Retire.
- ISO 20022 and Swift MT/MX coexistence FAQ, PCBB — end of MT/MX coexistence for cross-border interbank payment instructions in November 2025 (secondary source).

Practitioner and research sources

- Book research digest (this book's `_research/productscape-digest.md`), 2026 — the twelve end-to-end finance streams, their steps and pain points, generalized from a public-source domain model of a multi-brand travel group.
- Book research brief on accounting, controls and regulation (`_research/brief-accounting-controls-regulation.md`), 2026 — accruals, revenue models, ITGCs, application controls, IPE, DAC7, safeguarding.
- Book research brief on satellite platforms (`_research/brief-satellites-ai.md`), 2026 — tax determination patterns, treasury patterns, payment factory, AP intake channels.
- APQC Process Classification Framework, APQC — a widely used cross-industry process taxonomy; named here as an example of process reference models (not re-verified in this session).

Northmere Group, its people, systems, figures and metrics are fictional; the architectural recommendations are the author's judgment.

10. Finance Data Architecture: Who Owns the Truth



IN THIS CHAPTER, YOU WILL: Learn which financial master data domains an enterprise finance estate depends on, how to choose a master data management style for each, and how to design and govern a group chart of accounts and a partner master that several platforms share.

WHY FINANCE CARES: Every reported number is a transaction multiplied by its master data: the account it hit, the entity, brand and market it was tagged with, the partner it was owed to. When master data is wrong or inconsistent, the close spends its days reclassifying instead of closing, and the auditor tests mappings instead of balances.

WHY ARCHITECTS SHOULD CARE: Master data is the coupling nobody draws on the integration diagram. Every platform needs it, every platform ships its own create screen, and the authority decision, once missed, turns into mapping tables, reconciliations and duplicate records that outlive every system replacement.

KEY POINTS:

- **Authority is decided per domain and per attribute, not per system.** A partner's name may be authored in the brand platform, its VAT number checked by a steward and its bank account only in the payments platform. One record, several authorities, each written down and enforced through integration.
- **Choose the master data management style by domain.** The chart of accounts, legal entities and fiscal calendars belong in a centralized hub with a governed workflow; a partner master shared by a brand platform and an ERP subledger needs coexistence; registry and consolidation are steps on the way, not end states.
- **A mapping table is master data and a control.** The rules that translate brand charts of accounts into the group chart decide which line of the financial statements a balance lands on. They need an owner, effective dating, version history, completeness checks and change approval, exactly like the chart itself.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Doppelgänger Record** — partners, entities and accounts are created in several systems with no agreed authority, and every copy drifts.
- **Spreadsheet Glue** — the mapping from three charts of accounts to the group lives in workbooks that decide where revenue is reported.
- **The Permanent Exception** — each acquired brand keeps its own account structure and dimensions, so group comparability depends on translation forever.

Master data is the comparatively stable data that transactions refer to: the legal entity that books a sale, the account and cost center a cost lands on, the partner who is owed money, the currency and the rate it was converted at. A finance estate processes about a billion events a year, and each one is only as correct as the dozen pieces of master data it points to. A booking with the wrong market code is still a balanced journal; it is just reported in the wrong segment.

Most finance data problems that reach a Chief Architect are master data problems in disguise. "The close takes nine days" turns out to include two days of reclassifying postings that carried the wrong dimensions. "We cannot onboard partners faster" turns out to be five systems that each need the partner created by hand. "Consolidation is a black box" turns out to be **a mapping maintained in spreadsheets** that nobody can reproduce for last year's numbers.

This chapter covers the master data domains, the four management styles, ownership and stewardship, chart of accounts governance, data quality, lineage, data contracts and

retention. It stays light on one topic: the full matrix of **which system is authoritative for which object at which lifecycle stage** belongs to Systems of Record: Ending the Doppelgänger Record. Here the question is how master data is designed, owned and kept correct.

Setting the Scene at Northmere: The Usage Revenue That Went Missing

Northmere Group, the fictional multi-brand digital commerce group this book follows, closes the first quarter of 2027 in April. On working day 7, Leila Haddad, the Chief Accounting Officer and Group Controller, sees Tablekin's revenue down sharply against forecast at group level while Tablekin's own Workday Financials reports look normal. The analytical review finds the cause within the day. Tablekin had created a new ledger account for usage fees in January to separate per-cover usage from subscription revenue. The group mapping workbook, one of 14 workbooks that translate brand accounts into the group structure in OneStream, had no row for it. A catch-all rule mapped every unrecognized Tablekin revenue account into "Other revenue", so **$41 million of usage revenue** sat in the wrong caption at group level for the whole quarter until the review caught it.

Nothing in any system was broken. Workday posted correctly. OneStream applied the rules it was given. The workbook did what workbooks do. The failure was a missing decision: nobody owned the rule that a new account in a brand ledger must have a group mapping before it can be posted to.

Dana Whitlock, Northmere's Chief Architect for Enterprise Applications and Financial Systems, is asked by Tomás Ferreira, the Group CFO, a question that sounds simple: "How many charts of accounts do we have?" The honest answer is four. There is the SAP S/4HANA chart used by Stayloft, Northmere Pay, Northmere Ads and the corporate entities, with about **3,900 general ledger accounts**, many of them country or product variants of the same account. There is Wayfare's Oracle Fusion chart, a segmented string with about **2,100 natural account values**. There is Tablekin's Workday Financials structure, about **640 ledger accounts** plus worktags. And there is the fourth, invisible one: about **6,100 mapping rows** across those 14 workbooks, maintained by three people in group reporting, which is the only place where the other three become one group.

Dana's follow-up finding is worse for operations than for reporting. Stayloft has about **850,000 partners** (hotels, apartment operators and individual hosts). The same partner exists in the Stayloft Finance Platform (SFP), as a business partner with a contract account in SAP FI-CA (contract accounting, the high-volume subledger for Stayloft's agency partners), in the payout records of Northmere Pay (including, where payouts run through Adyen's balance platform, as a legal entity with a verified payout bank account there), and in Vertex as the party whose VAT registration determines the tax on

Stayloft's commission invoices. Each system has its own identifier and its own create path. Program Keystone, Northmere's finance transformation, commits to one group chart of accounts by 2029. This chapter is the design behind that commitment, and behind the partner master that has to come with it.

Four Kinds of Finance Data

Finance data comes in four kinds that differ in volume, change rate and governance. Mixing them up produces the classic mistakes: transactional data managed with master data workflows (too slow), or master data managed like transactional data (no approval, no history).

Kind	What it is	Examples at Northmere	How it is governed
Reference data	Code lists and values defined outside the enterprise or fixed by policy, used to classify other data	ISO currency and country codes, FX rates, VAT rate tables, fiscal calendar, payment method codes, card scheme reason codes	One distribution point per list; changes are mostly mechanical, but some (fiscal calendar, rate types) need controller approval
Master data	The enterprise's own long-lived business entities that transactions reference	Legal entities, chart of accounts and dimensions, cost and profit centers, partners, suppliers, customers, bank accounts, products and fee types, tax codes	Owned domains, stewards, workflow, four-eyes approval for sensitive attributes, full change history

Kind	What it is	Examples at Northmere	How it is governed
Transactional data	Records of business events and their accounting	Bookings, payments, refunds, payouts, invoices, journal entries, subledger postings	Controlled by application and interface controls; corrected by reversal, never by edit
Derived data	Data computed from the other three	Balances, revenue schedules, reconciliation results, consolidated statements, metrics, deferred merchant bookings	Reproducible from inputs and rule versions; lineage to its sources

Two boundaries cause most arguments. The first is **reference versus master**: FX rates look like reference data, but the choice of rate source, rate type and cut-off time is a policy decision with direct P&L effect, so the *rate policy* is mastered and governed while the daily rates are distributed mechanically. The second is **master versus configuration**: a mapping rule, a posting rule or an account determination entry looks like system configuration but behaves like master data. It changes during the year, it decides where amounts land, and it needs history. I treat any table that changes which account or dimension a transaction lands on as master data, whatever the vendor calls it.

The Financial Master Data Domains

Ten domains carry nearly all of a finance estate's master data. The table names each, its main attributes, who should own it and why it matters. "Golden source" here means the place where the authoritative version of a record or attribute is created and maintained; Systems of Record: Ending the Doppelgänger Record distinguishes that carefully from the system of record for a transaction at each lifecycle stage.

Domain	Key attributes	Natural golden source	Why it is hard
Legal entity	Code, legal name, registration, country, functional currency, tax registrations, licenses, ownership percentage, consolidation method	Legal entity register owned by group reporting, with group legal	Few records, enormous fan-out; a change reaches every platform. Global by Design: Entities, Currencies, Jurisdictions and Brands owns the entity model
Chart of accounts and dimensions	Natural account, account type, normal balance, reporting hierarchy, dimension values and their hierarchies, validation rules	Group finance core	Must be stable for years and still absorb acquisitions, new business models and new disclosures
Cost and profit centers	Code, owner, hierarchy, validity, linked entity and brand	Finance core for structure; the human capital management (HCM) system for the people assigned and the approvers	Two authorities meet: Finance owns the structure, HR owns who sits where
Organizational hierarchy	Supervisory organizations, positions, managers, approval roles	Workday HCM	Reorganizations happen on HR's calendar, not the close calendar

Domain	Key attributes	Natural golden source	Why it is hard
Business partners	Identity, legal form, addresses, tax identifiers, know-your-customer (KYC) status, roles (customer, supplier, marketplace partner, venue), contract accounts	Depends on role; see the partner master below	Millions of records, many created by self-service onboarding, several systems that each need a copy
Bank accounts	House bank accounts (Northmere's ~410 accounts at 27 banks) and partner and supplier payout accounts	Treasury system for house banks; payments platform for payout accounts	The most fraud-sensitive attributes in the estate; every change must be verified and dual-approved
Products, services and fee types	Product line, fee type (commission, service fee, subscription, usage, ad click), revenue treatment, tax category	Product catalog per brand, mapped to a group product hierarchy	Product teams create fee types faster than Finance classifies them
Tax codes and tax categories	Tax codes in ledgers, product tax categories, partner tax status, exemption evidence	Tax engine for determination content; ledger for tax codes	Tax content changes by jurisdiction; ledger tax codes must stay aligned
FX rates	Rate type (spot, closing, average, budget), source, cut-off time, currency pairs	Treasury policy; a single market rate feed	Two systems using different rates for the same day create intercompany and reconciliation breaks

Domain	Key attributes	Natural golden source	Why it is hard
Fiscal calendars	Fiscal year, periods, adjustment periods, period status per entity and ledger	Group finance core	Period status is master data with control significance: who can open a closed period

Three observations matter more than the table. First, **fan-out decides governance effort, not record count**: Northmere has 71 legal entities and 850,000 partners, but a wrong entity attribute reaches every platform, while a wrong partner address reaches one invoice. Second, **domains often have split authority**, with structure in one system and assignment in another; cost centers are the standard example: where HCM and the ERP come from different vendors, a common pattern is for the ERP to own the cost-center structure and feed it to HCM, while HCM owns which worker sits in which cost center. Third, some of the most consequential master data is not called master data by any vendor: the mapping tables, account determination rules and summarization keys that decide where an amount lands.

Management Styles: Registry, Consolidation, Coexistence, Centralized

Master data management (MDM) is the set of processes and technology that keeps master data consistent across systems. Four implementation styles are commonly cited, usually attributed to Gartner (no Gartner primary source is cited here; vendor explanations such as Informatica's describe them consistently). They differ in one thing: **where a record is authored and whether the hub writes back**.

Style	Where records are authored	What the hub does	Writes back to sources?	Finance fit	Northmere use
Registry	In each source system	Indexes and cross-references source records; holds a global identifier and match keys	No	Good first step to measure duplication; weak as a control, because nothing stops a bad record being created	2027: partner cross-reference across SFP, FI-CA, Northmere Pay and Vertex, to count duplicates and gaps
Consolidation	In each source system	Builds a golden record downstream, for analytics and reporting	No	Good for reporting views (customer analytics, spend analysis); does nothing for the transactions themselves	FinDB's conformed partner and supplier views for analytics and audit sampling

Style	Where records are authored	What the hub does	Writes back to sources?	Finance fit	Northmere use
Coexistence	In sources *and* in the hub	Matches, merges, publishes the golden record and synchronizes back	Yes	Fits domains where operational systems must keep creating records (self-service onboarding), but authority per attribute must be enforced	2028 target for the partner master between the brand platform, FI-CA and payouts
Centralized (transactional)	Only in the hub, through a governed workflow	Is the system of record; distributes to consumers, which cannot create	Distributes (consumers are read-only)	Fits low-volume, high-fan-out, high-control domains	Group CoA, dimensions, legal entities, fiscal calendar, rate policy, cost-center structure

The rule I use is simple. **Centralize where the record is rare, shared and control-relevant; coexist where the record is born in an operational process that cannot wait for a workflow**. Nobody should create a natural account in a brand ledger, so the chart of accounts is centralized. A host signing up on Stayloft at midnight cannot wait for a steward, so the partner is born in the brand platform, and the hub's job is to match, enrich, govern sensitive attributes and distribute.

Registry and consolidation are useful but transitional. A registry tells you how bad the

duplication is; it does not stop the next duplicate. A consolidated golden record in the data platform makes reports agree; it does not make the payout file agree with the tax engine. I fund them as measurement and as a migration step, and I do not let a program declare victory on either.

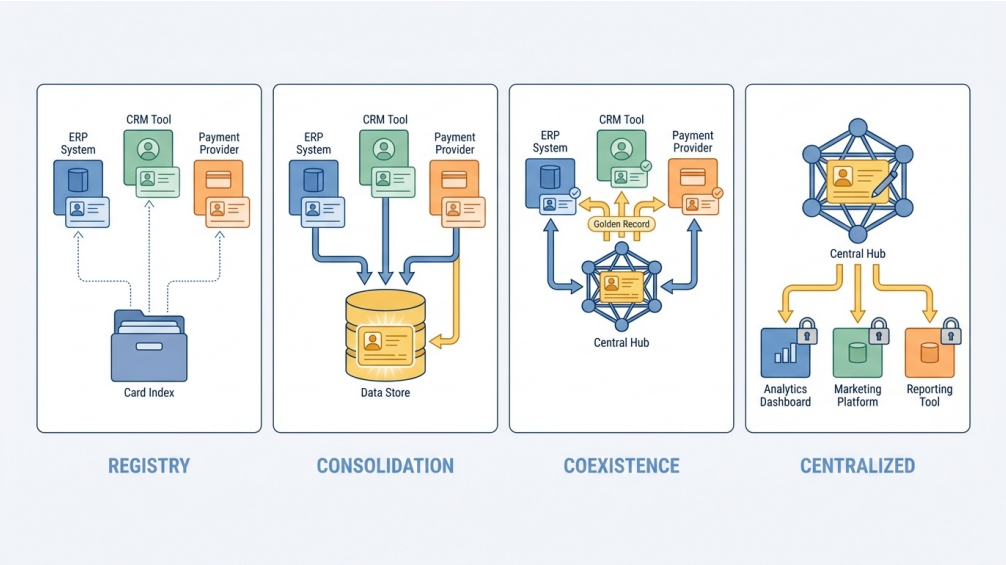


Figure 1: *The four MDM styles differ in where records are authored and whether the hub writes back: centralize what is rare, shared and control-relevant; coexist where records are born in operational processes.*

In SAP-centric estates, the centralized style is commonly implemented with SAP Master Data Governance (MDG), SAP's governance application for change requests on master data such as financial objects and business partners (SAP S/4HANA: One Journal, Clean Core covers its fit; Northmere does not run MDG in 2027); the coexistence and consolidation styles are commonly built with a specialist MDM platform or a custom partner hub. Which product implements the style is a secondary decision. **The style is the architecture**; **the tool is a sourcing choice**, and Build, Buy or Compose: Choosing What to Own covers how to make it.

The Architecture of Financial Master Data

The target shape is a small number of authoring points, a governed distribution path, and consumers that subscribe rather than create.

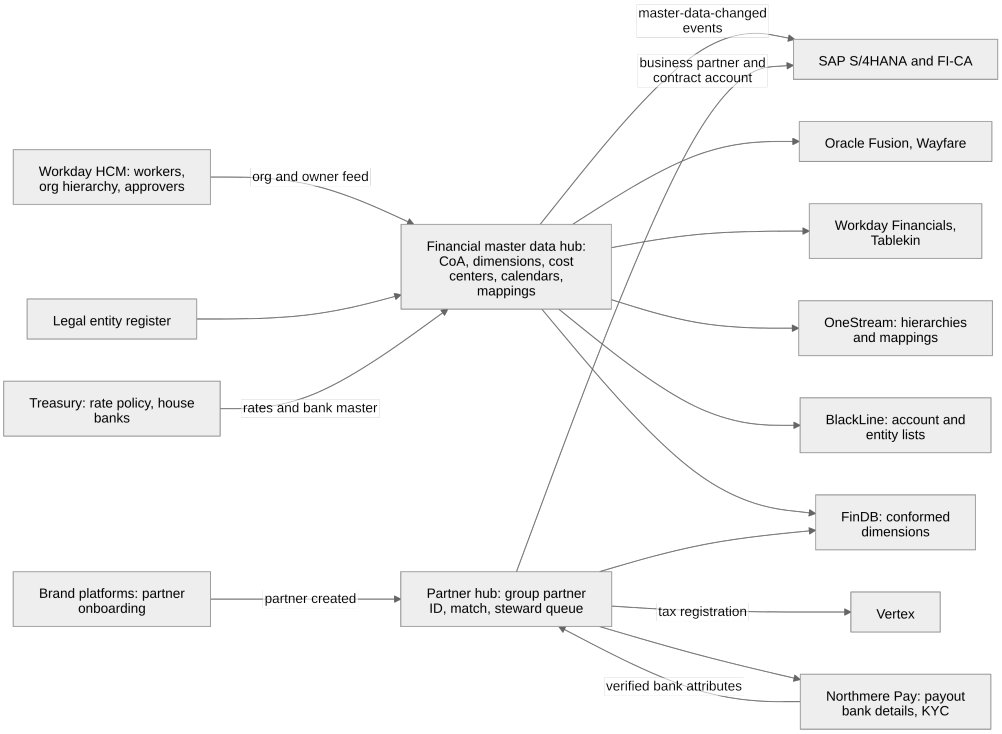


Figure 2: *Financial master data has few authoring points and many subscribers. Centralized domains are authored in one governed hub; the partner master coexists with the brand platforms that create partners, with bank attributes authored only in the payments platform in the target state.*

Five design decisions sit behind the picture.

Subscribers cannot create. The enforcement mechanism for authority is not a policy document; it is the absence of a create path. In S/4HANA, Oracle Fusion and Workday Financials, the roles that can create natural accounts, dimension values, entities and cost centers are removed from everyone except the integration user that loads from the hub, and a detective control compares each consumer's lists with the hub daily. Where a vendor platform needs local master data that the hub does not hold (an Oracle balancing segment value, a Workday worktag), the hub publishes the value and the local system holds it as a subscribed copy.

Changes are events with effective dates. Every change is published as a versioned `master-data-changed` event carrying the object, the attributes changed, the old and new values, the effective-from date and the approval reference. Consumers apply the change for the right period. Effective dating matters because finance reports past periods: a cost center moving from one brand to another in July must not move its January costs.

Identifiers are owned and immutable. The hub assigns the group identifier (group

account, group entity code, group partner ID) and never reuses one. Local identifiers are kept as cross-references. Renaming a record never changes its identifier.

Distribution is complete and checked. A change is not "done" when the hub approves it; it is done when every consumer has applied it. The hub records acknowledgments and raises an exception when a consumer has not applied a change within its agreed window. This is the check that would have caught the renamed Asian entity that reached four systems in a month and the fifth only when treasury noticed it.

Analytics subscribe to the same masters. FinDB, Northmere's Snowflake-based finance data platform, uses the hub's dimensions as its conformed dimensions. If the warehouse builds its own brand or market hierarchy, management reporting and statutory reporting will disagree by construction, and Owen Mbeki's team will spend its time explaining the difference.

Designing the Group Chart of Accounts

A **chart of accounts** (CoA) is the list of accounts a ledger posts to. Modern charts separate **what** happened, the **natural account** (cash, partner payables, commission revenue, salaries), from **where**, **who and why**, the **dimensions** (entity, brand, market, cost center). The combination of account and dimension values on a posting is often called the accounting string. Accounting for Architects: Ledgers, Journals and Revenue covers the accounting; this section covers the design choices.

The design guidance most practitioners accept is short: keep the natural account list small and stable, push analytical variety into dimensions, define the intercompany counterparty as a dimension from day one, map accounts to reporting hierarchies rather than duplicating accounts, and make sure the dimensions that segment reporting needs exist **at posting time**, because they are very expensive to retrofit. New disclosure rules make the last point sharper. US public companies adopt the expense disaggregation requirements of ASU 2024-03 for annual periods beginning after 15 December 2026 (as of 2026), and a disclosure that needs a category the ledger does not carry becomes a data project.

Northmere's group CoA, version 2

The Keystone design team, led by Leila's group reporting function with Dana's data architects, proposes a group chart of about **1,150 natural accounts** with seven dimensions. The SAP chart's 3,900 accounts collapse mainly because country, brand and product variants of the same account move into dimensions.

Element	Values (2027 design)	Authority	Required on	Example
Natural account	~1,150, six digits; 1 assets, 2 liabilities, 3 equity, 4 revenue, 5 cost of revenue, 6 operating expense, 7 other income and expense, 8 income tax	CoA council	Every posting	410100 Commission revenue
Entity	71 legal entities	Legal entity register	Every posting	Stayloft B.V.
Brand	6: Stayloft, Wayfare, Tablekin, Northmere Ads, Northmere Pay, Corporate	CoA council	All P&L accounts; balance-sheet accounts where brand-specific	STL
Business model	7: agency, merchant, subscription, usage, payments facilitation, advertising, other	CoA council with accounting policy	Revenue and cost of revenue	MER
Market	ISO 3166 country codes; ~110 active	CoA council; definition set by accounting policy (country where the service is delivered)	Revenue, cost of revenue, partner balances	PT

Element	Values (2027 design)	Authority	Required on	Example
Product	~40 product lines in a three-level hierarchy	CoA council, proposed by brand finance	Revenue and cost of revenue	ACC-APT (accommo-dation, apartments)
Intercompany partner	71 entity codes, plus "none"	Legal entity register	Required on intercom-pany accounts; forbidden elsewhere	—
Cost center	~4,800, hierarchy to brand and function	Finance core for structure; Workday HCM for owners	Operating expense; optional elsewhere	—

Under this design, the €90.00 commission on the Lisbon booking (the €600.00 merchant-model stay that recurs through this book) posts to account 410100 with entity Stayloft B.V. (the contracting entity), brand STL, business model MER, market PT and product ACC-APT. In the current SAP chart it posts to one of eleven country-specific commission accounts, and the business model is inferred from which account was used. The new string can answer "merchant commission in Portugal for apartments" without anyone knowing an account number.

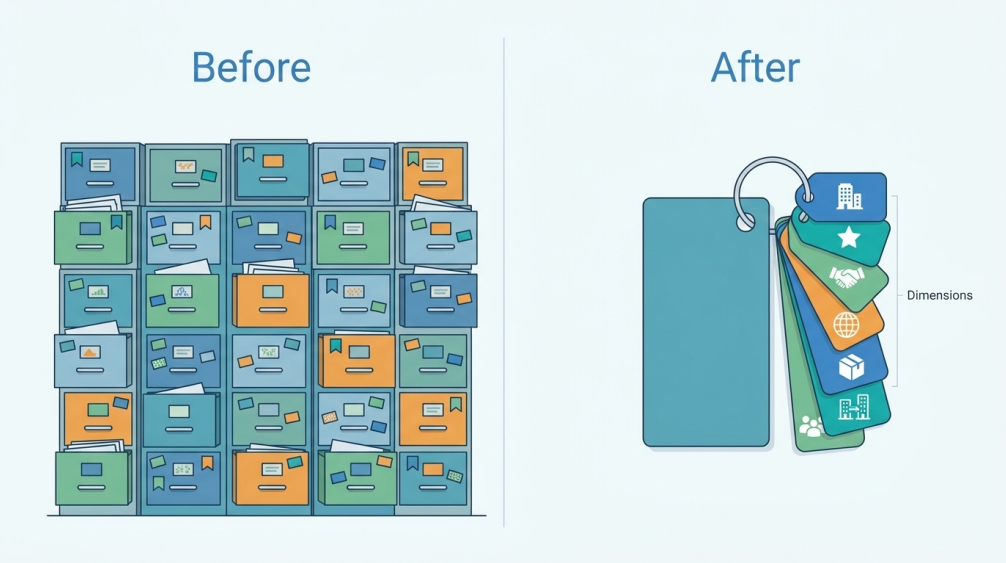


Figure 3: *Group CoA v2 collapses the SAP chart's 3,900 accounts into a group chart of about 1,150 natural accounts by moving country, brand and product variants into seven dimensions on every posting.*

Three deliberate exceptions keep the design honest.

1. **Clearing accounts stay granular.** Payment service provider (PSP) clearing accounts remain separate natural accounts per provider and currency (Adyen EUR, Stripe EUR and so on), even though a "PSP" dimension would be tidier. The reconciliation unit drives this: BlackLine reconciles accounts, and a clearing account that mixes providers cannot be matched to one provider's settlement report. Payments-to-Ledger: Draining the Suspense Swamp designs the clearing structure.
2. **Some splits must happen in the natural account.** Partner payables (money owed to properties and venues, often safeguarded funds) and trade payables (money owed to suppliers) are different liabilities with different controls and different regulatory meaning. They are separate natural accounts, not one account with a dimension, because their reconciliations, owners and disclosures differ.
3. **Dimensions are required by account type, not everywhere.** A required market on a cash account adds friction and invites junk values ("XX"). The validation rules say which dimensions are required, optional or forbidden for each account range, and the forbidden list matters as much as the required one.

The design deliberately has no customer or booking dimension (that detail stays in subledgers, linked by identifiers; see Never Update, Only Append: Designing Custom Ledgers), and it does not serve local statutory charts directly: those are reached through mapped reporting hierarchies or, in Oracle Fusion, through secondary ledgers, which Oracle documents may differ from the primary ledger in chart of accounts.

Platform realities

The same design lands differently in each ledger, and the differences are architectural, not cosmetic.

- **Oracle Fusion** defines the chart as segments of an accounting key (Wayfare's has six: company, cost center, natural account, product line, intercompany and a reserved future segment), and Oracle states that the chart of accounts is one of the elements combined to create a primary ledger. Oracle's standard GL balances cubes have one dimension per chart segment and, in Oracle's words, "dimensions are predefined and new ones can't be added" (Oracle GL balances cube dimensions). For Wayfare, adopting the group design inside Fusion therefore means a new chart and a new ledger, not an edit; that is one reason the group CoA arrives with ledger convergence in 2029, not before.
- **Workday Financials** uses its Foundation Data Model: transactions carry **worktags** (company, cost center, revenue category, spend category and others), and account posting rules derive the ledger account from them (Workday Financial Management datasheet). Tablekin's mapping must therefore read ledger account *and* worktags. A common implementation failure is a dimensional model designed as a copy of the legacy account list, which keeps the account explosion and loses the benefit of worktags.
- **SAP S/4HANA** records postings in one line-item journal, the Universal Journal, with account assignments such as cost center, profit center and segment on each line (SAP S/4HANA: One Journal, Clean Core covers the details and their limits). Northmere's SAP redesign is the cheapest of the three to start, because SAP is already the group core, and the most expensive to finish, because SFP, FI-CA account determination, Northmere Ads and about 2,300 custom objects all reference today's account numbers.

In Practice: The Mapping Table as Master Data

Until 2029, three ledgers keep their own charts and the group is assembled in OneStream. The decision Dana and Leila take in May 2027 is not to wait. **The mapping comes out of the workbooks in 2027**, into governed transformation rules in OneStream, authored through the same change workflow as the chart itself, with every rule effective-dated and versioned. The reason is simple: in consolidation on an enterprise performance management (EPM) platform, the mapping tables *are* the translation between local and group charts, so they deserve the ownership, change control and testing given to the charts themselves.

A worked mapping table

The table below shows a sample of Northmere's mapping rules from source ledger to group CoA v2. The codes are illustrative.

#	Source ledger	Source key	Source meaning	Group account	Dimensions set by the rule	Rule type
1	SAP S/4HANA	4100117	Commission revenue, Stayloft agency, Portugal	410100 Commission revenue	brand STL, model AGY, market PT	Many-to-one: country moves from account to dimension
2	SAP S/4HANA	4100217	Commission revenue, Stayloft merchant, Portugal	410100 Commission revenue	brand STL, model MER, market PT	Many-to-one
3	Oracle Fusion	3105-0000-40210-RAIL-0000-000	Wayfare France company, service fee revenue, rail product line	410300 Service fee revenue	entity from company segment 3105 via entity cross-reference; brand WAY; product RAIL; model MER	Segment-based

#	Source ledger	Source key	Source meaning	Group account	Dimensions set by the rule	Rule type
4	Oracle Fusion	3105-0000-13900-0000-1001-000	Intercompany receivable, intercompany segment 1001	131000 Intercompany receivable	IC partner = group code for local company 1001	Depends on the entity cross-reference, itself master data
5	Workday	Ledger account 4010 + revenue category "Usage – covers"	Tablekin usage revenue	420200 Usage revenue	brand TBK, model USG, product TBK-RES, market from company worktag	Account plus worktag
6	Workday	Ledger account 4000 + revenue category "Subscription – Pro"	Tablekin subscription revenue	420100 Subscription revenue	brand TBK, model SUB, product TBK-RES	Account plus worktag
7	Oracle Fusion	3105-0000-21500-0000-0000-000	Supplier payables, mixing ground-transport partners and indirect suppliers	**Cannot be mapped at balance level**	—	Split required in source: new Oracle accounts 21510 and 21520
8	Workday	Any 4xxx not listed	Catch-all	490000 Other revenue	—	**Forbidden** after May 2027

Rows 1 and 2 show the main effect of the redesign: dozens of SAP accounts collapse into one, and the dimension carries the difference. Rows 3 and 4 show a dependency people forget: a segment-based rule is only as good as the **entity cross-reference** that translates a local company value into a group entity code, so that cross-reference is part of the legal entity domain, governed with it. Row 7 is the expensive lesson: **a mapping cannot create information the source did not record**. If one Oracle account mixes two liabilities, no rule at trial-balance level can separate them; the fix is a source change, and until it is made, group reporting depends on a manual reclassification that is itself a control. Row 8 is the April incident. A catch-all rule turns "unmapped" into "wrong", silently. After May 2027 the rule set must cover 100% of accounts with a balance, and an unmapped account stops the OneStream load for that entity and raises an exception to the steward, rather than defaulting.

Rules that make mappings safe

Five rules turn the mapping from Spreadsheet Glue into controlled configuration.

1. **No account without a mapping**. A brand ledger cannot activate a new account until the group mapping exists. The check runs in the change workflow (preventive) and again at every load (detective).
2. **Effective dating, no overlaps**. Each rule has valid-from and valid-to periods; two rules may not cover the same source key in the same period. Reporting a prior period uses the rules valid then.
3. **Versioned and reproducible**. Every consolidation run records the mapping version it used. Restating 2027 in 2029 must reproduce 2027's numbers with 2027's rules, then show the effect of the new ones.
4. **Coverage and balance checks**. After mapping, the group trial balance per entity equals the local trial balance per entity, translated; any difference blocks the load.
5. **Owned rows**. Every rule has a named steward in group reporting and a named counterpart in the brand finance team, and the brand cannot change its chart without the group steward's approval.

The mapping table is also the migration specification. When Wayfare converges onto the group core, every one of these rows becomes either a posting rule in the new ledger or a data migration rule. A governed, tested mapping in 2027 is two years of free preparation for 2029.

Governing Change to the Chart of Accounts

Chart of accounts changes are rare in count and large in effect. A new natural account can change financial statement lines, reconciliation populations, tax reporting,

consolidation, planning models and SOX scoping. That is why the governance belongs to Finance, with Architecture as a required reviewer rather than an owner.

Northmere's **Finance Data Council**, set up under Keystone in Q2 2027, is chaired by Leila as data owner for the chart of accounts and the legal entity register. Members are the brand finance controllers, Kofi Asante for tax and treasury, Mei Tanaka for internal controls, Owen Mbeki for finance data, Sven Aalto for the SAP platform and a data architect from Dana's team. It meets monthly, and its standing agenda is the change log, the data quality scorecard and open exceptions.

Changes are classified by impact.

Class	Examples	Approval	Lead time
Standard	New cost center under an existing node; new market value; new product line under an existing family	Domain steward, four-eyes	Days; any day of the month
Significant	New natural account; change to an account's reporting-line mapping; new mapping rule for a brand account; new required dimension on an account range	Steward, group reporting lead, internal controls review where the account is in a SOX-relevant caption	Two weeks; effective from a period start
Structural	New dimension; change to a dimension's definition (what "market" means); new entity; change to the account numbering scheme	Finance Data Council, with the design authority for platform impact	One quarter; effective from a quarter or year start

The flow for a significant change looks like this.

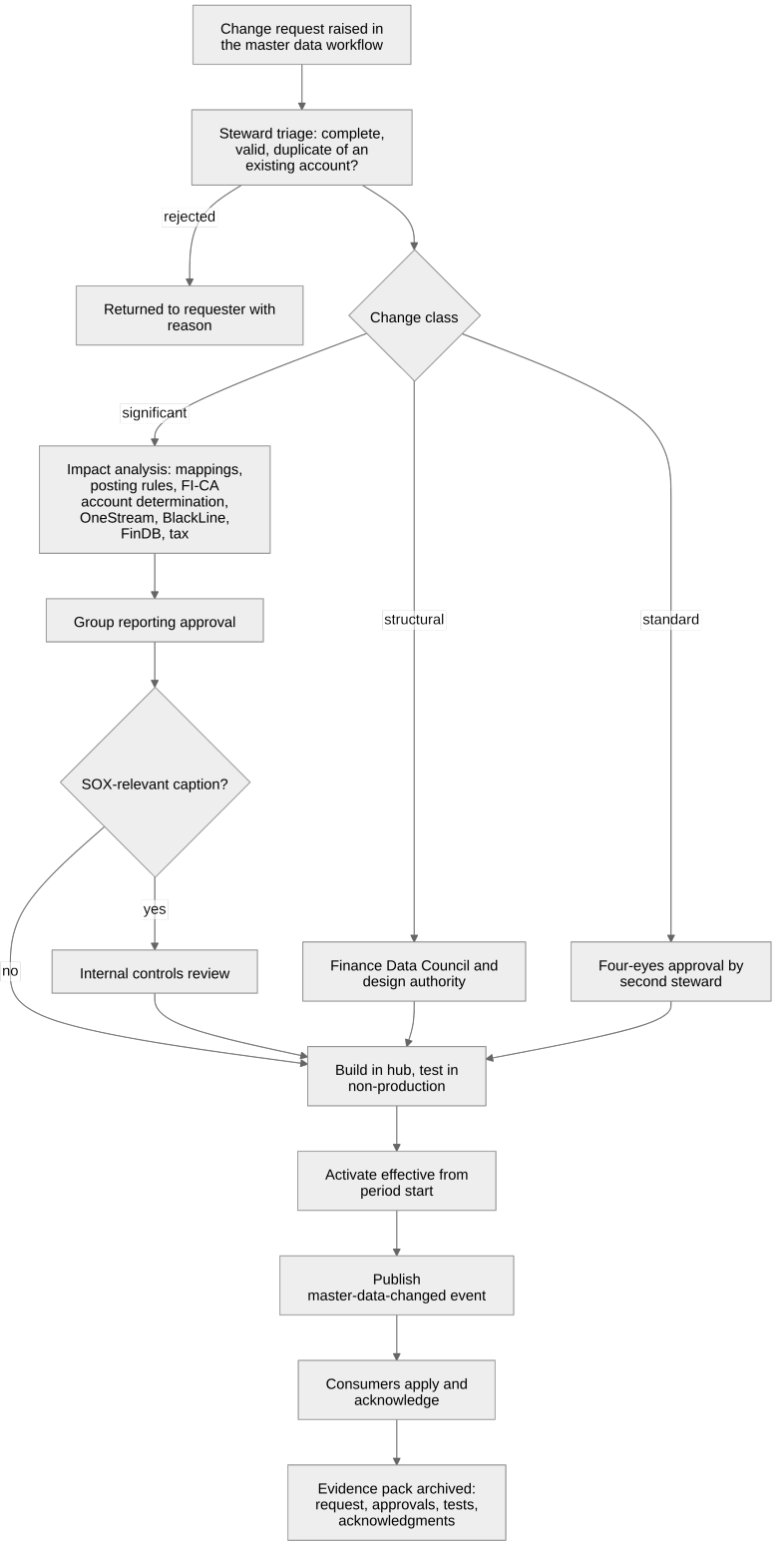

Change request raised in the master data workflow
Steward triage: complete, valid, duplicate of an existing account?
rejected
Returned to requester with reason
Change class
significant
Impact analysis: mappings, posting rules, FI-CA account determination, OneStream, BlackLine, FinDB, tax
Group reporting approval
structural
standard
SOX-relevant caption?
yes
no
Internal controls review
Finance Data Council and design authority
Four-eyes approval by second steward
Build in hub, test in non-production
Activate effective from period start
Publish master-data-changed event
Consumers apply and acknowledge
Evidence pack archived: request, approvals, tests, acknowledgments

Figure 4: *A chart of accounts change is a controlled change with an evidence trail: classified by impact, analyzed for downstream effects, approved by the owner, activated from a period start, and complete only when every consumer has acknowledged it.*

The link to SOX is direct. Under the US Sarbanes-Oxley Act, management must maintain effective internal control over financial reporting, and the external auditor tests it. A change to an account's reporting-line mapping changes which financial statement caption a balance lands in, which is exactly what the April usage revenue incident did. Mapping and chart changes are therefore in scope of the same change-management discipline auditors expect for financial applications: authorized, tested, approved before production, with evidence that the control operated for every change in the period, not just a policy. Controls by Design, Not by Audit Season covers IT general controls, information produced by the entity and evidence by design; the design point here is that **the master data workflow produces that evidence as a by-product**. Nobody assembles it before the audit.

The Partner Master: Coexistence in Practice

The chart of accounts is the clean case for centralization. Partners are the hard case.

A Stayloft partner is born when a property operator or host signs up in Stayloft's partner extranet. From that moment the partner appears in at least five places, each with its own identifier and each with a legitimate need:

System	Why it holds the partner	Identifier	Attributes it needs
SFP	Commission billing, merchant subledger, payouts instruction	SFP partner ID	Contract terms, commission rate, business model, billing entity
SAP FI-CA	Agency-model commission invoices, dunning, partner balances (~850,000 contract accounts)	Business partner number and contract account	Legal name, address, tax identifiers, billing and dunning settings

System	Why it holds the partner	Identifier	Attributes it needs
Northmere Pay payout records	Merchant-model payouts	Payee ID	Verified bank account, beneficiary name, KYC status, sanctions screening
Adyen balance platform (where payouts run through it)	Payout execution	Adyen legal entity, account holder and transfer instrument	KYC data, verified payout bank account
Vertex	Tax determination on commission invoices	Customer or taxpayer reference	Country, VAT registration, tax status

Owen's team runs a registry-style match in Q2 2027 across SFP, FI-CA and Northmere Pay, matching on tax identifier, normalized legal name and address, and bank account. The illustrative results: about **31,000** FI-CA business partners with no live SFP partner; about **12,400** SFP partners linked to two FI-CA business partners, usually after a change of legal owner; and about **4%** of partners whose VAT number differs between SFP and the tax determination calls. Each of these is a quiet cost: duplicate dunning, commission invoices with the wrong VAT treatment, and payouts that a fraud analyst cannot tie to the invoiced partner.

The answer is coexistence with **authority set per attribute**.

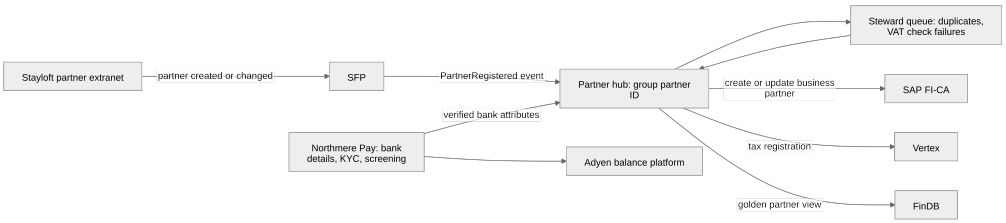


Figure 5: *The partner master as coexistence: partners are born in the brand platform, matched and governed in a hub that assigns the group partner ID, and distributed to FI-CA and Vertex; in the target state, bank attributes are authored only in the payments platform and flow to the hub, never the other way.*

The attribute-level authority is what makes this more than a synchronization project:

- **Identity and contact data** (name, address, contacts) are authored in the extranet through SFP; the hub matches and distributes.

- **Tax identifiers** are entered in the extranet but are not trusted until validated (for EU VAT numbers, against the EU's VAT Information Exchange System, VIES) and, on failure, reviewed by a steward. Until validated, the partner can trade but commission invoices apply the treatment for an unregistered party, as the tax team's policy prescribes.
- **Bank accounts and KYC status** end up authored only in Northmere Pay, with verification and dual approval, because bank detail changes are where payout fraud happens. That is the target state, not the 2027 baseline: in 2027 partner bank details are still authored in SFP and Northmere Pay holds a copy for execution, a daily SFP-versus-Northmere Pay comparison blocks any payout whose details differ, and authority moves to Northmere Pay by 30 June 2028 (Systems of Record: Ending the Doppelgänger Record records the transitional row). From then on no other system can change them, and SFP displays them read-only.
- **FI-CA business partners** are created only by the hub, carrying the group partner ID as an external reference. The manual create transaction is removed from user roles.
- **The group partner ID** is assigned by the hub and never reused; merges keep both old identifiers as aliases so that historical invoices remain traceable.

Northmere's sequence is registry in 2027 (to measure and to stop new duplicates at creation through a match check), coexistence in 2028, and in 2029, as the SFP merchant subledger moves to the group core, the partner hub becomes the only path into FI-CA for both business models. Tablekin's venues, held as connected accounts in Stripe Connect, and Wayfare's ground-transport suppliers in Oracle Fusion join the hub as additional sources when their payout engines are reviewed under Keystone; until then they are cross-referenced, not governed.

Ownership and Stewardship

Data ownership is accountability for a domain's definition, quality and access. **Stewardship** is the day-to-day work of keeping it correct. The vocabulary below follows common data-management practice (the DAMA body of knowledge uses similar roles); the names matter less than having each role filled by a named person.

Role	Accountable for	Northmere example
Data owner	Definitions, policies, approval of significant changes, acceptance of quality levels	Leila for the CoA, dimensions and entity register; Kofi for FX rate policy and house banks; Ingrid Solberg for partner bank and KYC attributes
Data steward	Daily request handling, triage, duplicate resolution, quality exceptions	Group reporting stewards for the CoA and mappings; a partner data team in Stayloft finance operations
Technical steward (custodian)	Platform, workflow, distribution, monitoring	Sven's SAP team for the hub; Northmere Pay engineering for payout data
Producer	Creating records correctly at the source	Stayloft extranet; HR operations in Workday HCM
Consumer	Using subscribed data, not creating local copies, raising defects	Every brand ledger, OneStream, BlackLine, FinDB
Architecture	Authority decisions, integration patterns, enforcement design	Dana's data architects; authority recorded in the SoR matrix

Two ownership rules prevent most disputes. **Owners are business people, not platform teams**: Sven runs the hub, but he does not decide what a natural account means. And **an attribute has exactly one owner**, even when the record has several: the partner record has three owners (Stayloft for identity, tax for tax status, Northmere Pay for bank details), and none of them owns the whole.

Data mesh, in Zhamak Dehghani's formulation of domain ownership, data as a product, a self-serve platform and federated computational governance, fits finance with one limit. Northmere Pay should own and publish settlement data as a product, with a contract. The chart of accounts is different: it is the shared language that makes domains comparable, and federating it is The Permanent Exception by another name. I federate data products and centralize the vocabulary.

Data Quality Rules

Data quality is managed as rules with owners, thresholds and an action, not as a score. The useful split is between **preventive** rules, which stop bad data at creation, and **detective** rules, which find it afterwards. Prefer preventive rules wherever creation is governed; rely on detective ones where creation is self-service.

Rule	Quality dimension	Type	Where it runs	Action on failure
Postings to intercompany accounts carry an IC partner that is a valid entity and is not the posting entity	Validity	Preventive	Ledger validation rule	Posting rejected
Brand is valid for the entity (a Tablekin entity cannot post brand STL)	Consistency	Preventive	Ledger derivation and validation	Posting rejected
Every account with a balance in a brand ledger has an effective group mapping	Completeness	Preventive and detective	Change workflow; OneStream load	Load stopped for the entity; steward alert

Rule	Quality dimension	Type	Where it runs	Action on failure
Closing and average rates exist for all 46 transaction currencies by 06:00 CET on the first working day	Timeliness, completeness	Detective	Rate distribution job	Treasury alert; revaluation held
EU VAT number validates against VIES before the partner is treated as registered	Accuracy	Preventive with fallback	Partner hub	Steward queue; unregistered tax treatment until resolved
No two active partners share tax identifier and bank account	Uniqueness	Detective	Partner hub match job	Steward merge review
Partner bank account change: IBAN checksum, name match, second approver, payout hold of 48 hours on first payout	Accuracy, security	Preventive	Northmere Pay	Change not activated

Rule	Quality dimension	Type	Where it runs	Action on failure
Every cost center has an active owner in Workday HCM	Consistency	Detective	Daily HCM-to-hub comparison	Owner's manager notified; cost center blocked for new purchase orders after 30 days
Subscribed master lists in each consumer match the hub	Consistency	Detective	Daily comparison	Exception to technical steward

Each rule is published with its owner, threshold and trend in the council's monthly scorecard. The scorecard's purpose is not a green dashboard. It is to make visible the rules that keep failing, because a rule that fails every month is a process problem upstream, usually a create path that should not exist.

Lineage and Data Contracts

Data lineage is the recorded path of data from its source to a reported number, including the transformations applied. For finance, lineage has two strands, and most tools capture only the first.

The **data strand** follows records: a group revenue figure comes from OneStream, from a Tablekin trial balance, from Workday journals, from Stripe Billing invoices and usage events. The open OpenLineage specification models this as datasets, jobs and runs, and FinDB's dbt pipelines can emit it.

The **rule strand** follows the master data and rule versions that shaped the number: which mapping version, which account hierarchy, which FX rate set, which posting rule. A reported number is reproducible only if both strands are recorded. When the auditor asks why Tablekin's Q1 2027 usage revenue moved between captions between the original close and the corrected one, the answer is a mapping version change with its approval, and it should take minutes to produce.

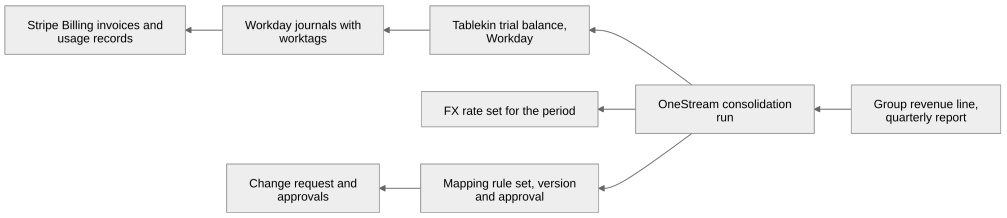


Figure 6: *Finance lineage has a data strand and a rule strand. A reported line is reproducible only when both the source records and the versions of the mapping and rate sets that shaped it are recorded.*

A **data contract** is an agreed, versioned specification between a data producer and its consumers: schema, semantics, quality rules, service levels and owners. The Open Data Contract Standard defines a YAML format with sections for fundamentals, schema, data quality, service levels, roles and team. For finance, a contract has three jobs that a schema alone does not do: it states **completeness** obligations (control totals), **semantics** that accountants rely on (which amounts are gross, which currency, which date governs the period), and the **master data references** that must be valid.

An abbreviated contract for SFP's daily summarized posting to S/4HANA, in the shape of an ODCS contract, looks like this:

```yaml
kind: DataContract
id: sfp-daily-gl-summary
version: 3.1.0
status: active
owner: Stayloft Finance Engineering (Jonas Berg)
consumers: [SAP S/4HANA GL, BlackLine, FinDB]
schema:
  - name: gl_summary_line
    properties:
      - {name: posting_date, type: date, description: "Accounting date; governs the period"}
      - {name: entity, type: string, reference: "group entity code, hub list"}
      - {name: group_account, type: string, reference: "group CoA v2, effective on posting_date"}
      - {name: brand, type: string, reference: "brand dimension"}
      - {name: business_model, type: string, reference: "business model dimension"}
      - {name: market, type: string, reference: "ISO 3166 alpha-2, active market list"}
      - {name: amount_minor, type: integer, description: "Signed, minor units"}
      - {name: currency, type: string, reference: "ISO 4217"}
      - {name: summary_key, type: string, description: "Links to constituent subledger lines"}
quality:
  - "Sum of debits equals sum of credits per entity and currency"
  - "Line count and amount totals equal the batch control record"
```

```
    - "Every reference value is valid in the hub on posting_date"
sla:
  delivery: "Daily batch by 04:00 CET; late batch alerts the GL steward"
  retention: "Detail behind each summary_key retained 10 years"
```

The contract gives Sven's team grounds to reject a batch that references a retired account, and gives Jonas's team notice of a chart change through a new contract version. Files, Events and APIs: How Finance Systems Talk covers control totals and delivery, Canonical Events: A Common Language for Money owns the canonical event contracts, and One Number, Many Views: Finance Data from Operations to Analytics covers how contracted data reaches the warehouse and the governed metrics.

Retention and Privacy

Finance data must be kept for statutory periods and, at the same time, personal data must be minimized and erased when no longer needed. Both are true; the architecture reconciles them.

Retention periods vary by country and record type, and the clock usually starts at the end of the year in which the record was created. Germany is a current example: since 1 January 2025, accounting vouchers such as invoices must be kept for eight years, while books and annual financial statements remain at ten (reported by Haufe). Master data has a derived retention: **a master record must be kept as long as any record that references it**, so a partner that last traded in 2027 must remain resolvable until the last invoice referencing it expires, and the chart of accounts and its mapping versions must outlive every period reported with them.

The GDPR's Article 17(3) disapplies the right to erasure to the extent processing is necessary for compliance with a legal obligation or for legal claims. That covers what the law requires, for as long as it requires it, and no more. Stayloft's partner master makes this concrete, because many hosts are individuals, so the partner master holds personal data. Northmere's design:

- **Ledgers and subledgers hold the group partner ID, not identity.** Name, address and contact data live in the partner master, with its own lifecycle.
- **On an erasure request or at the end of the relationship**, attributes not needed for statutory records (contact preferences, phone numbers, extranet profile) are erased; attributes needed to support retained invoices and payouts (legal name, tax identifier, address on the invoice) are restricted: removed from operational use, visible only to a small finance role, and deleted when the last referencing record's retention expires.
- **Invoices themselves are retained as issued**, in the immutable finance document archive, because the invoice is the statutory record.

- **The data platform pseudonymizes** partner and traveler identifiers for analytics and keeps the re-identification key under access control.

The design decision is to make retention a property of the data asset, recorded in the data catalog with its legal basis, rather than a property of each system. Controls by Design, Not by Audit Season covers the security and residency side of the same problem.

Trade-offs: Where to Put the Authority

The central choice for each domain is where authority sits. The table compares the options for the two Northmere domains that matter most.

Option	Strengths	Weaknesses	When it is right
CoA authored in the group ERP's governance workflow	One workflow, close to the main ledger; native validation; account determination in the same platform	Brand ledgers on other platforms subscribe through integration; ties the master to one vendor	One group finance core is the target (Northmere from 2029)
CoA authored in the consolidation platform	Finance-owned; already holds the group view and mappings	Consolidation is downstream of posting; it cannot stop a wrong posting, only reclassify it	Federated estates with no convergence plan; as an interim home for mappings
CoA authored in a separate MDM platform	Vendor-neutral; strong workflow and multi-domain matching	Another platform to run and integrate; may duplicate ERP governance	Estates with several long-lived ERPs and other domains on the same MDM platform

Option	Strengths	Weaknesses	When it is right
Partner master in the brand platform only	Fast onboarding; one team	Other systems create their own copies; no group identity; The Doppelgänger Record	Single-brand, single-ledger businesses
Partner master centralized in a hub before creation	Strong control, no duplicates	Onboarding waits for workflow; brand resists; often bypassed	Low-volume B2B supplier onboarding (indirect procurement)
Partner master in coexistence with attribute authority	Fast onboarding plus governed sensitive attributes	More integration and matching logic; needs stewards	High-volume, self-service partner onboarding feeding several finance systems

Northmere's choices follow the rule stated earlier: **CoA in the group finance core's workflow** (implementation product selection due Q3 2027), mappings governed in OneStream until convergence, and **partner master in coexistence** with attribute-level authority. The cost is real: the partner hub is a new platform with a matching engine, a steward team of about eight people and integration to five systems. The alternative is paying the duplicates' cost forever, in invoices, payouts and audit sampling.

Integration with the Rest of the Estate

Master data touches every other chapter, and the authority decisions here constrain them. Systems of Record: Ending the Doppelgänger Record builds the full authority matrix, including the lifecycle hand-offs this chapter avoids. Global by Design: Entities, Currencies, Jurisdictions and Brands owns the legal entity, intercompany and multi-currency design that the entity and rate domains serve. Files, Events and APIs: How Finance Systems Talk carries the `master-data-changed` events and enforces contracts at the boundary. OneStream: Consolidating Actuals, Planning the Future covers consolidation mapping in depth, and SAP S/4HANA: One Journal, Clean Core, Oracle Fusion: An Accounting Hub for High-Volume Events and Workday: The Org Chart Behind the Ledger cover each platform's chart and dimension model. Converge or Coexist: Living with More Than One ERP uses the mapping table as the migration specification for convergence, and The Next Brand Should Be Boring: Onboarding as Configuration turns new entities, brands and markets into master data configuration.

Failure Modes

- **Workflow nobody waits for**. A governance workflow that takes three weeks for a new cost center teaches people to reuse an old one. Standard changes must take days; if they do not, the workflow is the defect.
- **Local create paths left open**. Authority on paper, create rights in practice. The test is a role review that shows who can create each object in each system.
- **Dimension sprawl**. Seven dimensions easily become eleven, each added for one report. Every dimension has a cost at every posting; add one only for a recurring, owned reporting need.
- **Retrofitting dimensions**. A disclosure or segment change that needs a dimension the ledger never captured forces manual allocation or restatement. Design dimensions with the controller and the SEC reporting team, not only with the ERP implementer.
- **Bank details in two places**. Any second place where a payout bank account can be changed defeats the approval on the first.

The Chief Architect's View

The Chief Architect's job here is not to run MDM. It is to make five decisions explicit and to make sure they are enforced in the design of every platform the estate buys or builds.

1. **The domain list and the authority per domain and attribute**. Recorded in the system-of-record (SoR) matrix, decided with the data owners, reviewed when a platform is added.
2. **The management style per domain**. Centralized for the vocabulary (CoA, dimensions, entities, calendars, rate policy), coexistence for high-volume operational masters, registry and consolidation as steps.
3. **The rule that consumers do not create**. Every platform design review checks for create paths on mastered objects. This is the single most effective enforcement point the architect controls.
4. **The group chart of accounts as a long-lived design**. Its structure should survive a decade of acquisitions. I would rather spend six months on the dimension definitions than six years on mapping workarounds.
5. **Governance owned by Finance, designed by Architecture**. The Finance Data Council decides what accounts mean. Architecture decides how that decision reaches every system and how the evidence is produced.

What to watch: the number of systems where each master data object can be created (the target is one per attribute), the count and age of mapping exceptions, the share of partners with a group partner ID, the data quality rules that fail repeatedly, and the time from approved change to acknowledgment by every consumer. Those five numbers tell me whether the governance is working long before the close or the auditor does.

Questions to Ask

1. For each master data domain, which system is the golden source, and can that be demonstrated by showing that no other system has a create path?
2. Where do the mappings between local and group charts of accounts live, who can change them, and can last year's consolidated numbers be reproduced with last year's mapping version?
3. What happens today when a brand ledger posts to an account that has no group mapping?
4. How many dimensions does the chart require, and which reporting obligation does each one serve?
5. Which partner, supplier or customer records exist in more than one system, and how many duplicates does a match find?
6. Where can a payout bank account be changed, and does every such place enforce the same verification and approval?
7. Who owns the chart of accounts, who stewards it day to day, and how long does a standard change take?
8. Which master data and mapping changes are in SOX scope, and is the evidence produced by the workflow or assembled before the audit?
9. How long must each master record be retained, given the records that reference it, and how is personal data in it erased or restricted?

Master data decides where amounts land; controls decide whether anyone can trust that they landed correctly. The change workflow, the role design that removes local create paths and the evidence pack at the end of Figure 4 are all control design. Controls by Design, Not by Audit Season takes that up directly: how SOX, IT general controls, segregation of duties and audit evidence shape the architecture of every finance platform.

Sources and Evidence

Vendor documentation

- Ledgers, Legal Entities, Balancing Segments, and Business Units, Oracle, 2026: primary ledgers combine a chart of accounts, calendar, currency and accounting method; secondary ledgers may differ in chart of accounts.

- Dimensions for Standard Balance Cubes, Oracle, 2026: one cube dimension per chart segment; dimensions are predefined and cannot be added.
- Workday Financial Management datasheet, Workday, 2024: dimensions captured at the point of transaction (Foundation Data Model and worktags).
- MDM Implementation Styles, Informatica: description of the registry, consolidation, coexistence and centralized styles (vendor explanation; the styles are commonly attributed to Gartner).

Standards and regulation

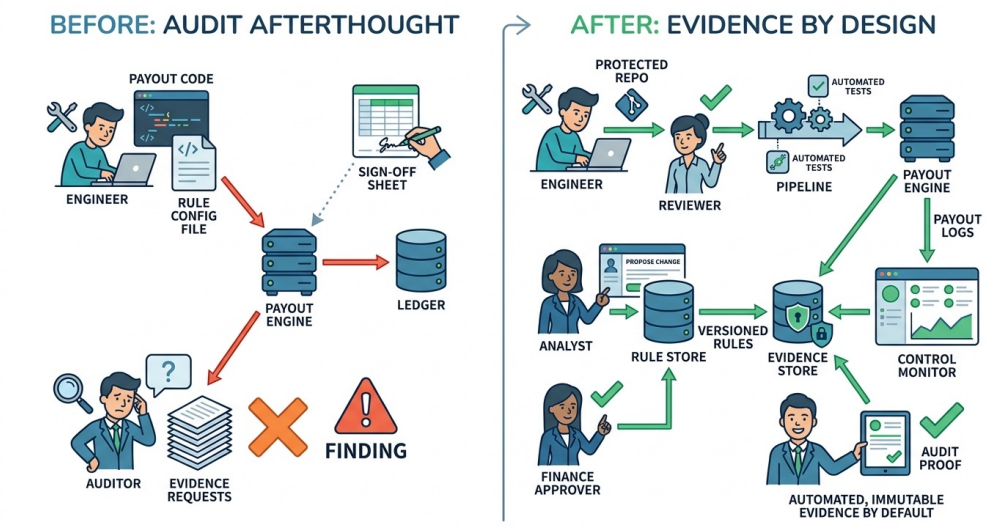
- GDPR Article 17, European Union, 2016: the right to erasure and its exemptions for legal obligations and legal claims.
- Viertes Bürokratieentlastungsgesetz, Haufe, 2024: reported reduction of the German retention period for accounting vouchers to eight years from 2025.
- FASB effective date of ASU 2024-03, Deloitte DART, 2025: effective dates for expense disaggregation disclosures (secondary source).
- Open Data Contract Standard, Bitol project: the YAML data contract structure used for the contract example.
- OpenLineage documentation, LF AI & Data Foundation: the dataset, job and run lineage model.

Practitioner and research sources

- Data Mesh Principles and Logical Architecture, Zhamak Dehghani, 2020: the four data mesh principles.
- DAMA-DMBOK: Data Management Body of Knowledge, DAMA International, 2nd edition, 2017: the common vocabulary for data ownership, stewardship and data quality dimensions.
- Patterns for Accounting, Martin Fowler, 2006 (draft): accounts and entries as the model behind charts of accounts and posting.

Northmere is fictional; architectural recommendations are the author's judgment.

11. Controls by Design, Not by Audit Season



IN THIS CHAPTER, YOU WILL: Learn the control vocabulary auditors use and see how each term becomes a design decision, then follow Northmere's remediation of an audit finding on a custom payout engine as an architecture change rather than a paperwork exercise.

WHY FINANCE CARES: The CEO and CFO personally certify the controls behind every quarterly and annual report. A deficiency on a system that moves money is a conversation with the audit committee, and a material weakness is a disclosure to the market.

WHY ARCHITECTS SHOULD CARE: Every key control lands on a system: on its access model, its deployment pipeline, its configuration store, its logs and its reports. Designed in, these controls cost little and run on every change; retrofitted after a finding, they cost a quarter of a team's roadmap and still produce weaker evidence.

KEY POINTS:

- **Auditors rely on automated controls only as far as they trust the general IT controls beneath them.** Access, change management, operations and implementation controls are the foundation; when they fail, every automated control and every system-generated report above them loses its value as evidence at once.
- **Remediate findings with architecture, not with sign-off sheets.** Northmere's payout-engine finding is closed by a deployment pipeline that cannot ship an unreviewed change, by payout rules held as approved, versioned data instead of code, and by production access that exists only for minutes and leaves a record.
- **Evidence by design means the system that makes the change also produces the proof.** A complete population of changes, approvals and access events, emitted automatically into an evidence store the control owner can query, is what turns quarter-end sampling into continuous monitoring and makes a SaaS vendor's SOC 1 report usable.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Audit Archaeology** — the payout engine was built for speed of change, and its controls were bolted on only after the auditor asked who had approved a rule change.
- **Nobody's Bot** — payout rules changed by engineers in production moved real money with no owner, no approval and no record a reviewer could test.
- **Spreadsheet Glue** — key reports and reconciliations that run on unvalidated extracts and spreadsheets cannot be relied on as evidence, however carefully people work.

Finance architecture has a client that most software does not: the **external auditor**, who must form an opinion on the financial statements and, for a large US-listed company, on the controls over them. When those controls are automated, the auditor tests the system that runs them and the controls over that system. **Controls are architectural requirements**, with the same standing as throughput, latency or availability.

Most engineering organizations learn this the expensive way. A platform is built for speed of change, and one year the auditor asks for every change made to a payment calculation in the period, with its approver. The list does not exist, because nobody designed the system to produce it, and the **remediation** (correcting the deficiency and proving the correction works) competes with the roadmap for a year.

This chapter defines the control vocabulary precisely, because auditors and engineers use the same words differently, and works through one remediation: Northmere's **significant deficiency** on the Stayloft payout engine. The argument is simple: **the cheapest control evidence is a by-product of normal operation**, and that happens only when the control is part of the design.

Setting the Scene: A Significant Deficiency on the Payout Engine

Northmere, the fictional group this book follows, is listed in New York and files with the Securities and Exchange Commission (SEC). It therefore falls under the Sarbanes-Oxley Act: management must report each year on the effectiveness of its internal control over financial reporting, and, as a large accelerated filer, its external auditor must attest to that report.

In February 2027, as the FY2026 audit closes, the auditor reports a **significant deficiency** to the audit committee. It concerns the payout engine inside the Stayloft Finance Platform (SFP), the custom system that holds the merchant-model subledger and instructs Stayloft's share of the roughly 9 million partner payouts Northmere makes each month. The auditor found two things:

1. **Developers had production access.** Fourteen SFP engineers held standing write access to the production payout database and could deploy to production without a second person. Access had been granted during an incident in 2024 and never withdrawn.
2. **Payout rule changes lacked documented approval.** Payout rules are the configuration that decides how much a partner is paid and when: commission rates and overrides per contract tier, the release delay after check-out, holds and reserves. Of the 23 payout rule changes made in 2026, nine had no record of who requested or approved them. They were edits to configuration files, merged by the engineer who wrote them.

The auditor re-performed a sample of payouts and found **no misstatement**. That is why the finding is a significant deficiency and not a material weakness. But the auditor could not rely on SFP's automated payout calculation, because the controls that would make it reliable (who can change it, and how) had not operated. The auditor had to do more substantive testing, and next year's audit will do the same unless the controls are fixed and have operated long enough to test.

Mei Tanaka, Head of Internal Controls, brings the finding to Dana Whitlock, the Chief Architect, and Jonas Berg, who runs Stayloft Finance Engineering and its 60 engineers. Jonas's first proposal is a sign-off spreadsheet: every deployment gets a row, every row gets a manager's initials. Mei's answer frames the rest of this chapter: *"That gives me a control I have to test by hand forty times a week, and the first time someone forgets a row, I have a repeat finding. I want a system that cannot deploy an unapproved change, and that tells me so."*

Program Keystone, Northmere's finance transformation, lists the remediation as a Year 1 deliverable. Dana's question is how to fix it so that it stays fixed, scales to the other custom finance platforms, and does not slow SFP's deployments to a crawl.

Internal Control over Financial Reporting in One Page

Internal control over financial reporting (ICFR) is the process, designed by management and overseen by the board, that provides reasonable assurance that financial reporting is reliable and the financial statements are prepared in accordance with the applicable accounting framework. It is narrower than "all controls": it covers only what could cause a misstatement in the financial statements.

Three pieces of law and standard frame it for a US registrant:

- **Sarbanes-Oxley Act section 302** requires the CEO and CFO to certify each quarterly and annual report, including that they have disclosed significant deficiencies, material weaknesses and fraud to the auditor and the audit committee.
- **Section 404(a)** requires management's annual assessment of ICFR effectiveness against a recognized framework; **section 404(b)** requires the external auditor's attestation for accelerated and large accelerated filers. Non-accelerated filers are exempt from 404(b) but not 404(a). The SEC's 2007 interpretive guidance sets a top-down, risk-based approach to management's evaluation.
- **PCAOB AS 2201**, the Public Company Accounting Oversight Board standard for an audit of ICFR integrated with the financial statement audit, tells the auditor to work top-down: from the financial statements to significant accounts, to the **relevant assertions** (existence or occurrence, completeness, valuation or allocation, rights and obligations, presentation and disclosure), to the controls that address each assertion.

AS 2201 distinguishes two levels of failure. A **material weakness** means there is a reasonable possibility that a material misstatement will not be prevented or detected on a timely basis; it must be disclosed and leads to an adverse opinion on ICFR. A **significant deficiency** is less severe but merits the attention of those charged with governance. The distinction is a judgment about severity, not a property of the control: the same access weakness can be a material weakness in one company and a significant deficiency in another, depending on the accounts it touches and the compensating controls around it. (A time-sensitive note: an SEC proposal of May 2026 would let registrants elect semiannual reporting; as of October 2026 it is still a proposal, and it would change 302 certification frequency, not section 404.)

The framework behind the assessment

Management assesses ICFR against a framework, in practice the **COSO 2013 Internal Control — Integrated Framework**: five components (control environment, risk assessment, control activities, information and communication, monitoring) and seventeen

principles, each of which must be present and functioning (Deloitte summary). Three principles are where system design becomes audit evidence. **Principle 9**, identifying significant change, is why a re-platforming project needs Internal Controls in its steering group. **Principle 11**, general controls over technology, is the foundation of everything in the next section. **Principle 13**, relevant, quality information, is the root of every question about whether a report can be trusted.

Outside the United States: Provision 29 and the EU

The UK Corporate Governance Code 2024 applies to financial years beginning on or after 1 January 2025, and its **Provision 29** applies to financial years beginning on or after **1 January 2026**. It asks the board to declare the effectiveness of its **material controls** (financial, operational, reporting and compliance) as at the balance-sheet date, and to describe any material control that did not operate effectively and the remediation. For December year-ends, the first declarations appear in annual reports published in early 2027. Provision 29 is **not "UK SOX"**: it is broader in scope (all material controls, not only financial reporting), lighter in assurance (the Financial Reporting Council states that the external auditor's scope does not change and that boards decide whether to seek external assurance), applies on a comply-or-explain basis, and leaves the definition of "material" to each board. Northmere, listed only in New York, is not subject to it; a UK-listed group would be, and should not assume its SOX-style evidence answers the board's wider question.

There is **no EU-wide equivalent of SOX** requiring a management assessment of ICFR with auditor attestation. EU requirements arrive through the corporate governance statement in the Accounting Directive, the audit legislation and sectoral rules such as the Digital Operational Resilience Act (DORA) for financial entities, which reaches Northmere Payments B.V. as an e-money institution (see Recover Before the Close: Resilience and Security for Finance). An architect in an EU group should not assume SOX vocabulary maps one-to-one onto local obligations.

A Control Taxonomy for Architects

A **control** is a specific activity that reduces the risk that a financial statement assertion is wrong. Auditors classify controls on several axes at once, and each axis changes what the system must do.

Axis	Values	What it means for the system
Timing	**Preventive** (stops the error) vs **detective** (finds it after the fact)	Preventive controls live in the transaction path and need to be fast and available; detective controls live in reconciliation, monitoring and review, and need complete data
Nature	**Manual**; **automated** (performed entirely by a system); **IT-dependent manual** (a person acts on system-produced information)	Automated controls need ITGCs; IT-dependent manual controls need ITGCs *and* a validated report
Level	**Entity-level** (tone, governance, risk assessment, the close review by the CFO); **process-level** (within a business process, such as payout release); **application** controls; **general IT** controls	Entity-level controls rarely touch systems directly; application and ITGC controls are almost entirely system design
Significance	**Key** controls (relied on to address a risk) vs non-key	Only key controls are tested; designing the system so a few strong automated key controls replace many weak manual ones is the largest lever on audit cost

Application controls are automated controls inside an application: balanced-entry enforcement, posting-period locks, three-way match, duplicate invoice detection, automated calculations such as revenue schedules, and interface completeness checks with record counts and hash totals. An automated control is typically tested once by walking a transaction through it (a "test of one") and then relied on for the whole period, *provided the general IT controls over the application were effective throughout.*

General IT controls (ITGCs) are the controls over the environment in which applications run. Audit firms group them in four domains:

ITGC domain	What the auditor looks for	Typical system design response
Logical access	Approved provisioning; timely removal of leavers; periodic user access reviews; privileged and administrator access restricted and monitored; generic and shared accounts controlled; service accounts owned	Identity provider as the only entry point; joiner-mover-leaver driven from the HR system; just-in-time privileged access; named owners for every service account
Change management	Changes authorized, tested and approved before production; separation between developers and production deployment, or compensating controls; an emergency change process	Protected branches, mandatory review, pipeline-only deployment, change records linked to commits, emergency path with retrospective approval
IT operations	Job scheduling and monitoring; interface failure handling; backup and recovery; incident management	Monitored schedules, interface error queues with owners, restore tests, incident records linked to financial impact
Program development (implementation)	Controls over new systems and data migrations, especially the completeness and accuracy of migrated balances	Migration reconciliations as go-live gates (see Changing the Engine in Flight: Modernization Patterns for Finance)

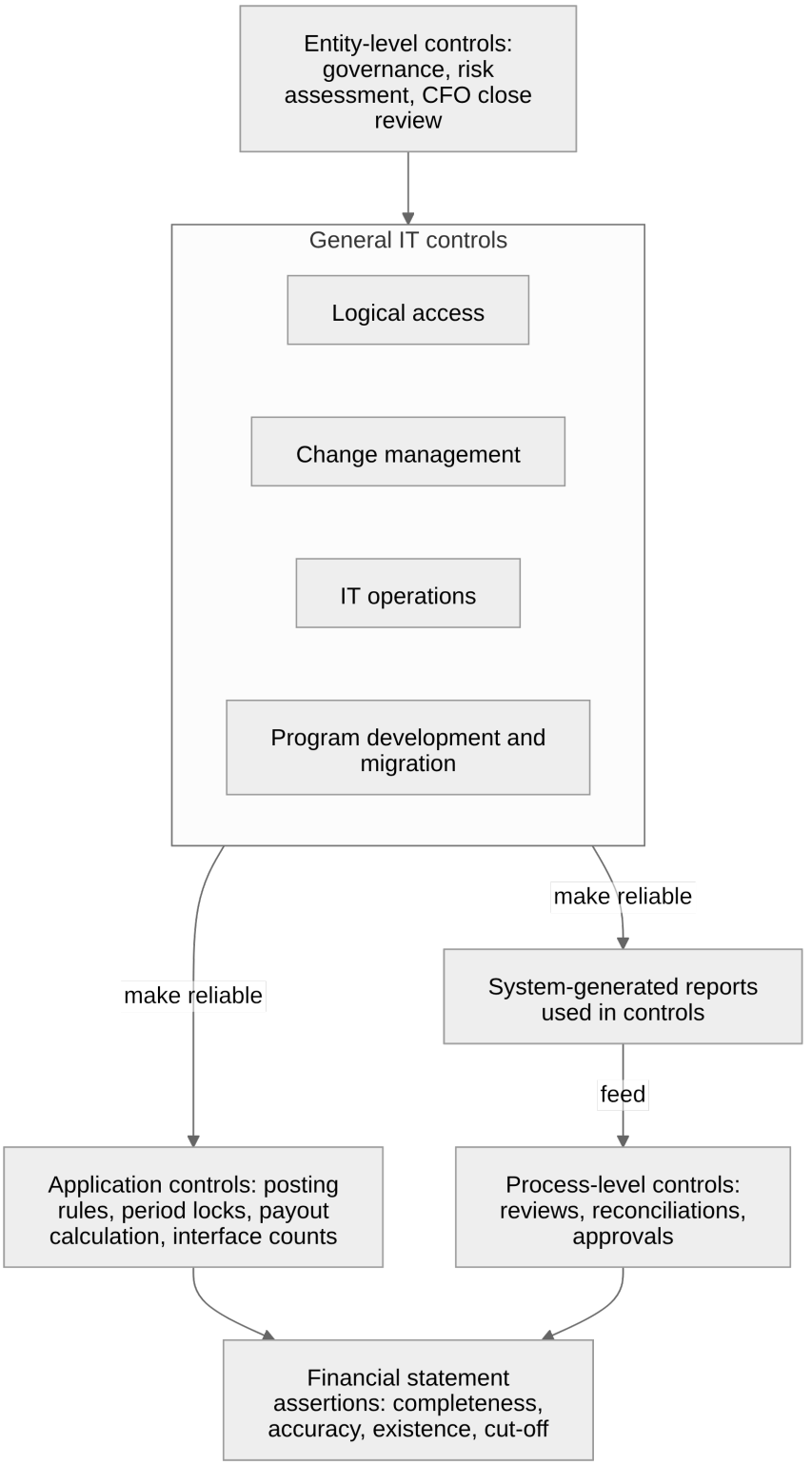

Entity-level controls: governance, risk assessment, CFO close review
General IT controls
Logical access
Change management
IT operations
Program development and migration
make reliable
make reliable
System-generated reports used in controls
feed
Application controls: posting rules, period locks, payout calculation, interface counts
Process-level controls: reviews, reconciliations, approvals
Financial statement assertions: completeness, accuracy, existence, cut-off

Figure 1: *The reliance chain. Automated controls and system-generated reports are trusted only because the general IT controls beneath them operated all year; a failed access or change control removes the foundation from everything that sits on it.*

This chain explains the SFP finding. The payout calculation is an **application control** the auditor could test once with a sample payout. But because developers could change the code and rules without approval, and write directly to the payout tables, nobody could show that the calculation tested in March was the one that ran in November. **The ITGC failure did not make the payouts wrong; it made them unprovable.**

Key Reports and Information Produced by the Entity

Information produced by the entity (IPE) is any report, query, extract or spreadsheet the company produces and uses either in the performance of a control or as audit evidence. A **key report** is IPE that a key control depends on. The auditor must be satisfied that each one is **complete and accurate**: that its source is the right one, its parameters and filters are correct, and the ITGCs over the system that produces it are effective. IPE and management review controls have been a recurring theme in PCAOB inspection findings for about a decade, which is why auditors now test them in detail.

Three Northmere examples show how IPE drags systems into audit scope:

- **The payout release report.** Each day, Stayloft's partner-settlement operations team reviews payouts above a threshold before release. The review is an IT-dependent manual control, and the SFP report listing those payouts is IPE. If the report's threshold filter could be edited by anyone, the review proves nothing.
- **Transaction Matching inputs.** BlackLine matches payment service provider (PSP) settlement lines against the SFP settlement extract (see BlackLine: Matching, Certifying and Closing). The extract is IPE: if it silently drops a file, the reconciliation reports a clean match on an incomplete population.
- **The chargeback reserve.** The chargeback reserve is estimated from a FinDB query in Snowflake. Once a key control uses that query, the dbt models behind it, their change process and the Snowflake roles that can alter them are in ITGC scope.

The consequence: **a data platform becomes an in-scope financial system the day a key control depends on it.** Owen Mbeki's FinDB team did not plan to operate under change-management controls; the chargeback reserve made it so. The response is to decide deliberately which reports are key, keep them few, and produce them from versioned, reviewed queries with control totals that tie to the ledger. A report built ad hoc and pasted into a review is the most common IPE failure, and it is **Spreadsheet Glue** under another name.

Segregation of Duties and Sensitive Access

Segregation of duties (SoD) means no single person can initiate, approve, record and reconcile the same transaction, or change the system that does so. The classic conflicts are creating a supplier and approving payments to it, or posting a manual journal and approving it. **Sensitive access** is access that is risky on its own, without a conflicting second right: changing bank details, releasing payments, opening or closing periods, administering users, deploying code, reading secrets.

SoD is implemented through role design, a **ruleset** of conflicting permissions checked when access is requested and periodically afterwards, and **mitigating controls** (usually a detective review) where a conflict cannot be removed, as in a small team in a minor entity. In SAP estates this is commonly run through a governance, risk and compliance tool with **firefighter access**: a time-boxed emergency identity whose every action is logged and reviewed.

The hard part in a multi-platform estate is **cross-system SoD**. Each vendor platform has its own role model and its own conflict rules, and each is internally clean. The conflict appears only when one person's rights are viewed across systems: a payments engineer who can change partner bank details in SFP and also approve the payment run in S/4HANA, or a Tablekin finance user who can change Stripe Connect payout settings and also reconcile venue payouts in Workday.

Conflict	Systems involved at Northmere	Risk	Design response
Change partner bank details / release payouts	SFP partner master; SFP payout release; Kyriba payment factory	Payout diverted to a fraudulent account	Bank detail changes through a maker-checker workflow with call-back verification; release rights in a different role; cross-system ruleset

Conflict	Systems involved at Northmere	Risk	Design response
Change payout rules / deploy SFP code	SFP rule store; SFP deployment pipeline	A rule and the code that interprets it changed by the same person, unreviewed	Rules approved by Finance in the rule workflow; code approved by a second engineer; neither path can approve itself
Post manual journal / approve it	S/4HANA; Oracle Fusion; Workday; BlackLine Journal Entry	Unsupported entries reach the ledger	Workflow approval with self-approval blocked in every ledger; threshold-based review
Create supplier / approve supplier invoice or payment	Coupa; S/4HANA; Kyriba	Fictitious supplier	Supplier master owned by a master-data team; approval roles in procurement and treasury
Change Stripe Connect payout configuration / reconcile venue payouts	Stripe Dashboard and API keys; Workday Financials	Payout errors hidden by the person who should find them	Stripe roles and restricted API keys mapped to named owners; reconciliation in a separate role

Two practical rules follow. First, **the identity provider must know every finance-relevant role in every system**, including the custom ones; a cross-system conflict cannot be detected in a system that only sees itself. Second, **engineers are users too**. Production database access, deployment rights, pipeline administration and secret-store access are sensitive access and belong in the same ruleset as journal approval. SFP's finding was, at root, an SoD conflict between writing code and deploying it, and between writing rules and approving them, that no ruleset covered because nobody had classified engineering rights as financial access.

Remediation as Architecture: Fixing the Payout Engine

Dana rejects the sign-off spreadsheet: it would satisfy the letter of the finding while creating a manual control tested by sampling, which fails the first time someone is busy and slows deployment enough that engineers route around it. She also makes the remediation the reference design for every custom finance platform, including Wayfare's payout engine, which Hana Kowalski's team runs with the same weaknesses and no finding yet.

The design has five parts.

1. A change pipeline where approval is the evidence

All SFP code reaches production through one deployment pipeline, running under a **service identity**; no human credential can deploy. The pipeline enforces, before it deploys:

- a merged pull request on a protected branch, approved by an engineer other than the author;
- passing automated tests, including a **payout regression suite** that replays a fixed set of historical bookings through the payout calculation and compares results to the penny;
- a link from the change to a work item, so the reason for the change is recorded;
- for changes classified as **SOX-relevant** (anything touching payout calculation, posting rules, the rule store schema or the SFP-to-S/4 interface), a second approval from a named code owner in the payout domain.

Continuous deployment is compatible with these controls, provided there is evidence that they operated **for every change in the period** and that the pipeline's own configuration is under change control. SFP keeps deploying about forty times a week; each deployment now writes a record that proves it was reviewed.

The decisive detail is where the **population** comes from. An auditor samples from the complete list of production changes; a list taken from the ticketing system contains only the changes someone remembered to ticket. SFP's pipeline emits the population from the deployment log itself (every deployment, its commit range, approvers and test results), and a weekly check reconciles that log to the production version history, so a change that arrived by any other route surfaces within days, not at year-end.

2. Segregation of duties in deployment and no standing production access

The fourteen engineers lose standing write access to production. Production data is reached through three routes only:

- **Read access** through a reporting replica, with personal data masked.
- **Operational changes** (re-sending a failed payout instruction, releasing a hold) through audited operational tools that call SFP's own APIs, so the same validations and logs apply as for automated processing.
- **Break-glass access** for incidents, requested through the privileged access tool, approved by an on-call lead other than the requester, limited to four hours, session-recorded, and reviewed the next working day by someone outside the team.

The rule Dana writes into the Keystone standard on segregation of duties by design (STD-FIN-05) is short: *no human writes to a production financial data store except through an application path that validates, logs and can be reviewed.* Jonas's concern is incident response at 3 a.m.; the answer is that break-glass access takes two minutes to approve and is rarely needed once the operational tools exist. Between July and December 2027, break-glass is used eleven times, each reviewed within a day.

3. Payout rules as approved, versioned data

The most consequential change is to move payout rules out of code and configuration files into a **rule store**: configuration held as data, with a schema, validation, effective dates and an approval workflow. A rule change is proposed by a partner-settlement operations analyst (the maker), approved by a finance approver (the checker) who cannot be the maker, and becomes active only at its effective date. Engineers change the rule *engine* through the code pipeline; Finance changes the *rules* through the rule workflow. Neither can do the other's job, and each path has its own evidence.

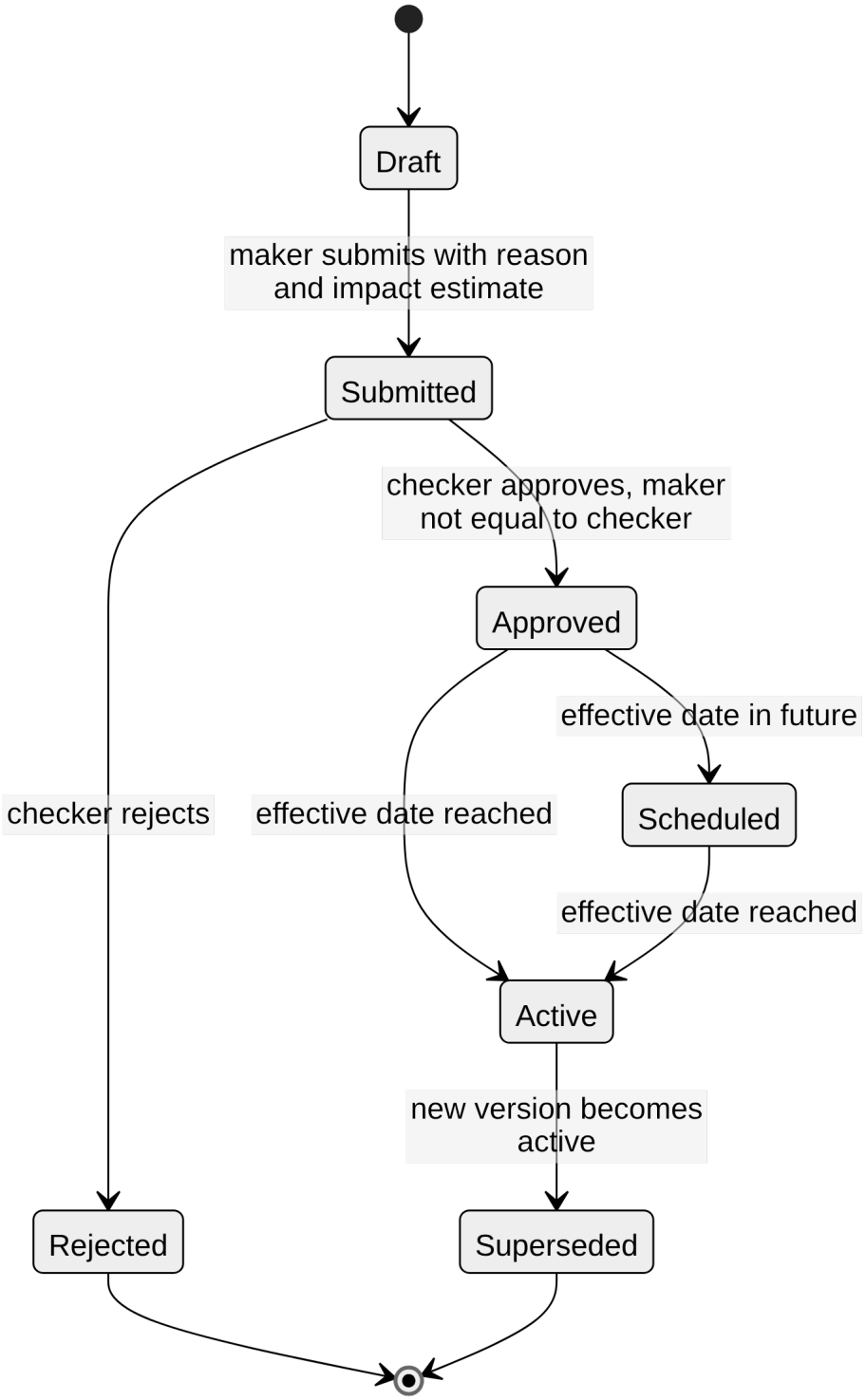


Figure 2: *The lifecycle of an SFP payout rule. Every transition is recorded with who, when*

and why; a rule cannot reach Active without passing an approval by someone other than its author, and a superseded version is kept, never overwritten.

The rule store also gives the payout calculation a property the auditor values: every payout records the **rule version** it was calculated under. A reviewer can take any payout, retrieve the exact rules that applied on that date, and recompute it. That is traceability from a cash movement back to an approved decision, and it is the same pattern Never Update, Only Append: Designing Custom Ledgers applies to posting rules.

A worked example shows why this matters. Take the Lisbon booking that this book follows: €600.00 paid by a traveler on 12 March 2027 for a stay of 18–21 April, Stayloft commission 15%, payout of €510.00 instructed on 22 April. Suppose a partner-success manager wants a promotional commission override of 12% for Lisbon apartments for April stays. Under the old process, still in force that spring, an engineer edits a configuration file and merges it. Under the new one, live from July, the analyst submits the override with a reason, a date range and an impact estimate, and a finance approver signs it off.

Account	Debit	Credit	Note
Traveler funds held (liability to partner and traveler)	€600.00		At check-out, the booking's held funds are split
Commission revenue		€90.00	15% under the approved standard rule; €72.00 if the 12% override applies
Partner payable		€510.00	€528.00 if the override applies

Simplified: VAT on the commission and payment fees are omitted; the full entry set for a merchant-model booking is in Accounting for Architects: Ledgers, Journals and Revenue.

An €18 difference per booking is immaterial on its own. Applied by an unreviewed change to thousands of bookings for a quarter, it moves real revenue and real cash, and Finance would find out only when the margin analysis looked odd. That is why the auditor cared about nine undocumented changes even though the sampled payouts were correct: **an unapproved rule change is a revenue decision taken by someone without the authority to take it**.

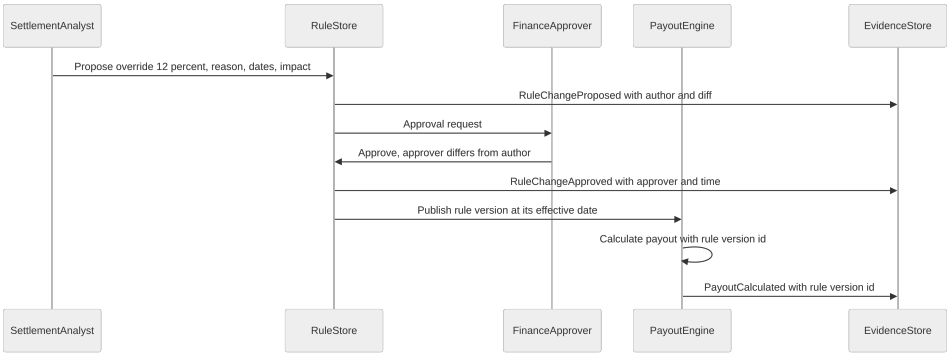


Figure 3: *A payout rule change at Northmere after remediation. The evidence is not collected afterwards; each step emits its own record, and each payout carries the version of the rule that produced it.*

4. Access reviews that cover engineers and machines

Quarterly user access reviews, long run for S/4HANA, now cover SFP and include engineering rights: deployment approvers, pipeline administrators, rule-store approvers, secret-store access and every service account, each with a named owner. The review is generated from the identity provider and the platforms' role data, not from a spreadsheet kept by the team under review, and revocations flow back automatically. Joiners, movers and leavers are driven from Workday HCM, so a transfer out of Stayloft Finance Engineering removes the SFP roles the same day.

5. Logging that an auditor can rely on

Every rule change, deployment, privileged session, approval and payout release writes an **append-only, tamper-evident log entry** (who, what, when, from where, with before and after values) to a store SFP engineers cannot alter, retained as long as the records it supports, and queryable by Mei's team directly. The test Dana uses is blunt: *if Internal Controls needs an engineer to answer an evidence request, the logging is not finished.*

The architecture as a whole

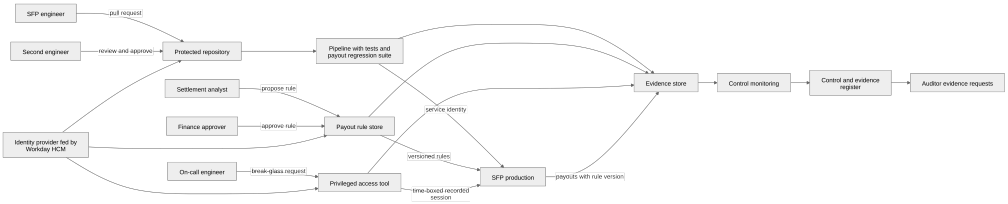


Figure 4: *Remediation as architecture. Three change paths, for code, for rules and for emergencies, each enforce separation of duties and each feed one evidence store; the control register and the auditor read from that store instead of asking engineers.*

Timing the remediation

A deficiency is remediated when the new controls have been designed, implemented and have **operated for long enough to be tested**. Management's assessment is as at 31 December 2027, so the controls must be live well before year-end. Mei's plan, which Dana adopts as a Keystone milestone, works backwards from that date:

Period	Work	Evidence produced
March–April 2027	Design reviewed by Internal Controls; walkthrough of the design with the auditor	Design document mapped to the risk and control matrix
May–June 2027	Pipeline controls, rule store, break-glass and logging built; standing access removed on 15 June	Access removal confirmed from the identity provider
1 July 2027	New controls operating; old configuration-file rules migrated into the rule store with a reconciliation of every active rule	Migration reconciliation signed by Finance
Q3–Q4 2027	Management tests operating effectiveness on samples from the complete populations; continuous monitoring runs in parallel	Test workpapers; monitoring exceptions and their resolution
January 2028	Auditor tests as part of the FY2027 integrated audit	Auditor's conclusion on remediation

Six months of operation is not a rule in any standard; it is a commonly used planning margin that gives management room to test, find an exception, fix it and test again before year-end. **A remediation that goes live in November is a remediation that will probably be reported as still open**.

SaaS Platforms, SOC 1 Reports and the Controls They Hand Back

Much of Northmere's control surface runs on software it does not operate: BlackLine, OneStream, Workday, Stripe, Adyen, and S/4HANA hosted under RISE with SAP. The auditor still needs assurance over the controls inside those services, and the mechanism is the **SOC 1 report**.

A **SOC 1** report is an independent auditor's examination of a service organization's controls that are likely to be relevant to user entities' internal control over financial reporting, performed under the AICPA attestation standards (SSAE 18, AT-C 320). The international equivalent is an **ISAE 3402** report. A **Type I** report covers the design of controls at a point in time; a **Type II** report covers design *and operating effectiveness over a period*, typically six to twelve months. Only Type II gives the user's auditor evidence that controls worked during the year. A **SOC 2** report, against the Trust Services Criteria for security, availability and related areas, serves vendor risk management but **is not a substitute for SOC 1** when the service affects financial reporting.

Every SOC 1 report contains two lists that architects must read:

- **Complementary user entity controls (CUECs)**: controls the *customer* must operate for the vendor's controls to achieve their objectives. Typical examples are granting and reviewing access to the customer's tenant, approving configuration changes, reviewing interface error reports and securing the customer's API credentials. Each CUEC must map to an owned, tested control in the customer's own control framework.
- **Complementary subservice organization controls (CSOCs)**: controls at the vendor's own providers, usually a hyperscaler. Under the **carve-out** method, the vendor's report excludes them, and reliance passes to the subservice provider's own reports.

Two practical issues follow. Reports rarely cover the customer's fiscal year exactly, so a **bridge letter** (the vendor's statement that no significant changes or failures occurred between the report's end and a later date) plus the customer's own monitoring covers the gap. And reports can contain **exceptions**: controls that did not operate in some instances. Each exception must be assessed for its effect on the customer's controls, not filed.

The principle is one sentence: **a SaaS platform transfers the operation of controls, not the accountability for them**. Northmere's configuration, roles, integrations and reports on every one of these platforms remain fully in scope.

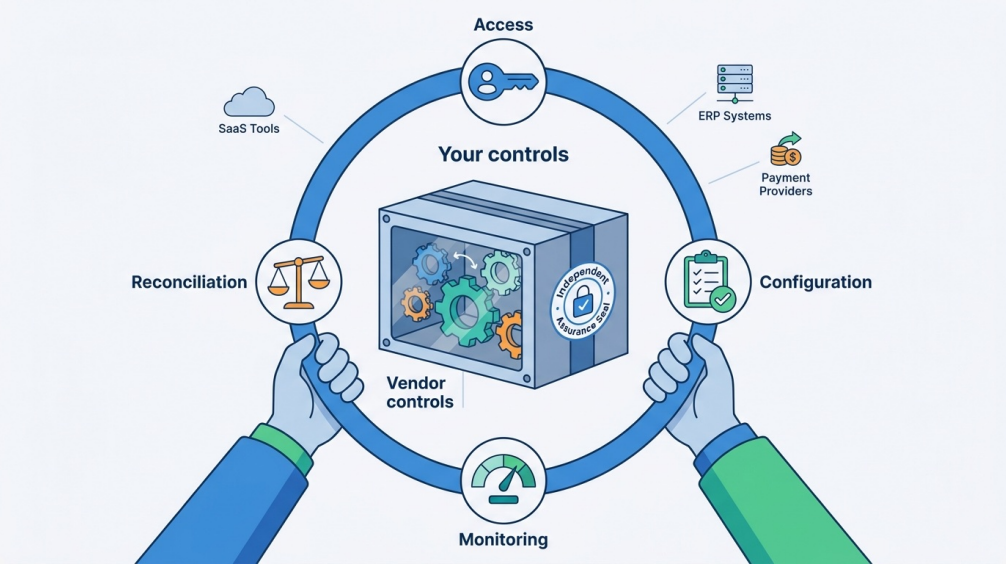


Figure 5: *A SOC 1 report covers the controls the vendor operates; the complementary user entity controls around the platform, such as access, configuration changes, interface monitoring and reconciliation, remain the customer's to run.*

Platform	Role in Northmere ICFR	Assurance evidence (as checked October 2026)	Typical CUECs Northmere must operate	Northmere owner
BlackLine	Account reconciliations, Transaction Matching of PSP settlement, Task Management, Journal Entry	Obtain the current SOC 1 Type II (or ISAE 3402) report and bridge letter under NDA; confirm that the modules Northmere uses are in scope. I could not confirm the report's scope from public sources	User provisioning and review; approval of matching rules and auto-certification criteria; completeness of data loaded from ledgers and banks (IPE); review of failed loads	Leila Haddad's close team; Mei Tanaka for the CUEC map
OneStream	Consolidation, eliminations, currency translation, group mappings	As for BlackLine: request the current SOC 1 Type II and bridge letter and confirm scope; not confirmed from public sources	Mapping and metadata change approval; access reviews; reconciliation of loaded trial balances to source ledgers; review of consolidation journals	Group consolidation team

Platform	Role in Northmere ICFR	Assurance evidence (as checked October 2026)	Typical CUECs Northmere must operate	Northmere owner
Workday (HCM and Tablekin Financials)	Tablekin's ledger; group source of workers, cost-center assignments and approvers	Workday publishes a SOC 1 Type II report issued under ISAE 3402, covering Workday Enterprise Products	Security-group design and review; business-process configuration approval; tenant configuration change control; integration error review	Arjun Rao (Financials); HR systems team (HCM)
Stripe (Billing and Connect)	Tablekin subscription and usage billing; venue payouts; secondary PSP	Stripe states that SOC 1 and SOC 2 Type II reports are produced annually; customers can download them from the Dashboard	Dashboard roles and restricted API keys mapped to owners; review of account security history; approval of price and payout-schedule changes; reconciliation of Stripe balances and payouts to the ledger	Arjun Rao; Ingrid Solberg for the PSP relationship

Platform	Role in Northmere ICFR	Assurance evidence (as checked October 2026)	Typical CUECs Northmere must operate	Northmere owner
Adyen	Primary acquirer and PSP for Stayloft merchant bookings	Adyen's Trust Center lists ISAE 3402 / SOC 1 reports for overlapping twelve-month periods (most recent: April 2025 to March 2026, unqualified opinion) and quarterly bridge letters	Customer-area user administration; API credential management; webhook and settlement-report completeness checks; reconciliation of settlement to bank and to SFP	Ingrid Solberg's Northmere Pay team

The Adyen row illustrates the coverage problem. Northmere's fiscal year runs January to December; a report covering April to March covers only part of it, so Mei's team combines two overlapping reports and the bridge letters to cover the whole year, and documents that judgment. For S/4HANA under RISE, the same exercise applies to the controls SAP operates on Northmere's behalf; SAP S/4HANA: One Journal, Clean Core describes the shared-responsibility split.

The CUEC lists are the most underused architecture input in finance: together they specify what the enterprise must build around every SaaS platform (identity integration, configuration change control, interface monitoring, reconciliation). I recommend reviewing the CUEC map at vendor selection, not after go-live; a vendor whose CUECs assume controls you cannot run is one you will compensate for every year (see Build, Buy or Compose: Choosing What to Own).

Audit Trail Requirements for Custom Ledgers

Vendor ledgers come with an audit trail shaped by thousands of audits; custom ledgers such as SFP's have whatever audit trail their engineers thought of. What auditors and regulators expect is consistent:

- **Posted records are immutable**. Corrections are made by reversal and re-posting, never by update or delete. This is a control concept, not a technology mandate: an append-only table with reversal entries satisfies it, and no auditor requires a blockchain.
- **Who, what and when** is recorded for every creation, approval and change, including by systems and service identities.
- **Before and after values** are preserved for master data and configuration that drive amounts: partner bank details, commission rates, payout rules, account mappings, tax codes, standalone selling price tables.
- **Traceability runs in both directions**: from a financial statement line to the general ledger, to the subledger, to the source event and document, and back. Summarized postings must keep drill-back to their detail (see Reconciled by Design: Traceability and Audit Across Boundaries).
- **Period integrity**: closed periods cannot receive postings; late events post to the open period with their original event date recorded.
- **Retention** matches the statutory period for the underlying records, including the logs that prove who did what.

Jurisdictions add specifics, such as Germany's electronic bookkeeping rules (GoBD), France's standard general ledger export on tax audit (the *fichier des écritures comptables*) and Spain's Verifactu invoicing-software rules; each applies to the custom ledger's export and retention design, not only to the ERP. The ledger design itself belongs to Never Update, Only Append: Designing Custom Ledgers; this chapter adds the test: **could an auditor take any balance in the custom ledger, list every entry that makes it up, and show who or what created each entry under which approved rule?** If not, the ledger is not finished.

Evidence by Design

Evidence by design is the practice of building systems so that the evidence a control has operated is produced automatically, completely and immutably as a by-product of the control operating. It is the difference between a control that generates its own proof and a control whose proof someone has to assemble at quarter-end.

The patterns below are the ones I would require for any finance-relevant system Northmere builds or configures.

Pattern	What it means	Northmere example
Populations from the system of action	The complete list of changes, approvals or transactions comes from the system that performed them, not from a ticket or a spreadsheet	SFP deployment population from the pipeline log, reconciled weekly to production versions
Approvals as data, not signatures	Approvals are records with an approver identity, timestamp and the exact version approved	Rule store approvals; pull request approvals bound to the merged commit
Configuration as approved data	Financially significant configuration lives in a validated, versioned store with maker-checker approval, not in code or files	Payout rules; commission tiers; GL account mappings in the posting rules
Version stamping	Every calculated amount records the version of the logic and rules that produced it	Each payout carries its rule version ID
Key reports as code	Key reports run from reviewed, versioned query definitions with control totals that tie to the ledger	The payout release report; the chargeback reserve query in FinDB
Tamper-evident, self-service evidence store	Evidence, tagged with its control ID, is written to a store the people being controlled cannot alter and control owners can query directly	Central evidence store owned by Internal Controls and Security, not by SFP

These patterns are cheap at design time and expensive afterwards: a rule store with approval is a few weeks of work in a new platform, but retrofitting one into SFP, with the migration and reconciliation of every active rule, takes Jonas's team most of a quarter.

That difference is the economic case against **Audit Archaeology**.

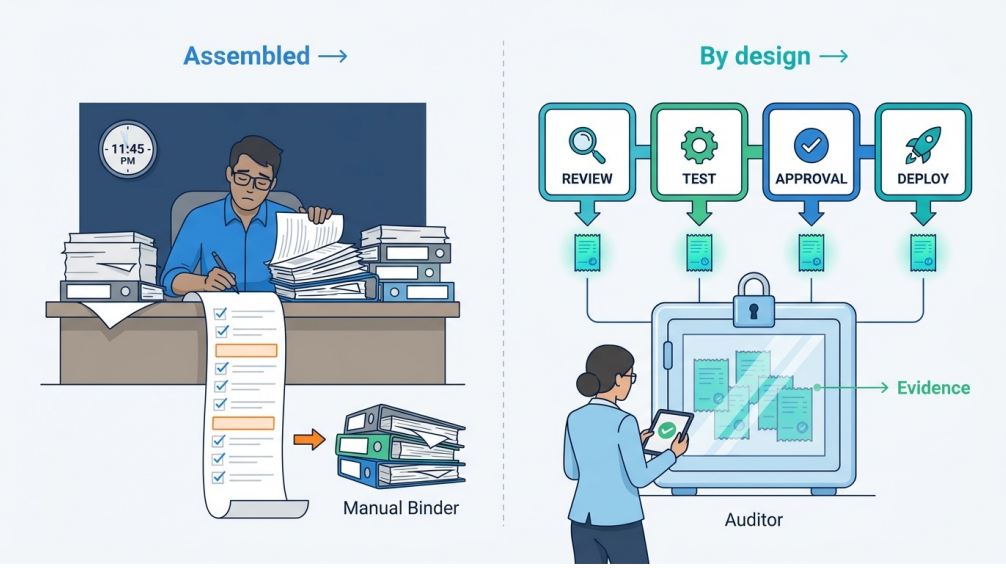


Figure 6: *The cheapest control evidence is a by-product of normal operation: each step of the pipeline writes its own record into a store the controlled team cannot alter, instead of someone assembling proof at quarter-end.*

Continuous Control Monitoring

Continuous control monitoring (CCM) is automated, ongoing testing of control operation, and of the transactions controls govern, across the full population rather than a sample. Once evidence is a complete stream of records, testing becomes a query.

Useful monitors in a finance estate include:

- **Change monitors**: every production deployment has an approved, non-author review; every SOX-relevant change has a code-owner approval; no change reached production outside the pipeline.
- **Access monitors**: no SoD conflicts across systems; no standing privileged access; every break-glass session reviewed within one working day; no active accounts for leavers.
- **Configuration monitors**: no payout rule, posting rule or mapping active without approval; no change to bank details without call-back verification.
- **Transaction monitors**: manual journals above thresholds approved by someone other than the poster; payouts calculated under an active rule version; interface record counts and control totals balanced (see Files, Events and APIs: How Finance Systems Talk).

- **Reconciliation monitors**: unmatched PSP settlement lines and suspense balances against thresholds and aging (see Payments-to-Ledger: Draining the Suspense Swamp).

At Northmere, monitoring results flow into the control and evidence register that Mei's team owns: the single register of key controls mapped to processes, systems and risks. Northmere's FY2026 risk and control matrix held about 610 key controls, of which about 38% were automated; Keystone's target is 60% by 2029. Exceptions become tasks with owners and deadlines, and the trend of exceptions becomes evidence that the control is monitored, which COSO's monitoring component asks for.

Two cautions. **Monitoring is not reliance**: whether the external auditor uses CCM results to reduce their testing is their judgment, and depends on ITGCs over the monitoring platform itself; a monitor anyone can edit proves nothing. And **a monitor that fires constantly gets ignored**: thresholds, exception workflows and owners matter as much as the queries.

Working with Internal Controls and the External Auditor Early

The most effective control practice I know is a meeting before design is finished, in which the architect walks Internal Controls through the design. Northmere makes it mandatory: every architecture decision record for a finance-relevant system includes a **control impact assessment**, completed with Internal Controls before approval (the template is in Architecture Governance That Speeds Decisions and Working Papers: Templates for the Finance Architect). It answers which key controls the change adds, alters or retires; what evidence each will produce and where; and whether the risk and control matrix must change before go-live. A brand payments integration that goes live mid-quarter without updating the control matrix is how a near-miss on revenue completeness happens.

The external auditor must stay independent, so **the auditor cannot design your controls**. What you can do is share designs early and ask how they intend to test: which population, what evidence format, reliance on automated controls or substantive testing. Mei walked the auditor through the SFP design in April 2027, before a line was built. Their questions (how the deployment population is proven complete, whether the pipeline's configuration is under change control, how rule-store administrators are separated from approvers) reshaped three details. Discovered in January 2028, they would have cost a second year of findings.

The governance rule I recommend: **no finance-relevant system goes live without the control owner's sign-off that the controls are designed, implemented and evidenced**. It is the same gate as a security review and should sit in the same place in the delivery process.

The Security Baseline for Finance Systems

Access and privileged-access controls are security controls by another name. The full security architecture, including payments security and recovery, belongs to Recover Before the Close: Resilience and Security for Finance; this section sets the baseline the control framework assumes for every finance-relevant system.

Area	Baseline	Why finance needs it
Identity	Single sign-on through the corporate identity provider for every finance system, including SaaS consoles and custom tools; provisioning via SCIM where supported; joiner-mover-leaver driven from Workday HCM; multi-factor authentication resistant to phishing	Access reviews and SoD analysis are only possible if one system knows every identity and role
Privileged access	No standing administrator or production write access; just-in-time elevation with approval, time limits and session recording; service accounts owned, credential-rotated and non-interactive	Privileged users can bypass application controls; their access is sensitive access in SoD terms
Encryption	Encryption in transit everywhere and at rest for all financial and personal data; customer-managed keys where the platform supports them and the risk justifies the operational cost	Protects confidentiality of financial data and payment information; key access becomes a controlled privilege

Area	Baseline	Why finance needs it
Secrets	Credentials, API keys and signing keys in a secrets manager, never in code or configuration files; scoped keys (for example Stripe's restricted API keys); automated rotation; access logged	A payout API key is as powerful as a bank mandate; leaked keys are a direct path to moving money
Data residency	A register of where each finance dataset is stored and processed, and under which transfer mechanism	Data protection law restricts transfers; some tax and sector rules add location or access requirements
Retention and erasure	Retention schedules per record type and jurisdiction; personal data minimized in ledgers	Statutory retention and privacy rights must both be met, by design

Data residency, precisely

The General Data Protection Regulation (GDPR) does not require personal data to stay in the EU; it restricts **transfers** to third countries, which need an adequacy decision (such as the EU–US Data Privacy Framework for certified US organizations, still subject to litigation as of 2026), standard contractual clauses or binding corporate rules. Separately, some tax laws set conditions on keeping electronic books abroad, and DORA adds location and access-right requirements for Northmere Payments B.V.'s outsourced information and communication technology (ICT) services. The response is a register, not a reflex: know where each dataset lives and why that is lawful, and move no ledger to a new region without Tax and Legal.

GDPR erasure versus statutory retention

The German traveler who booked the Lisbon apartment may later ask Stayloft to erase their data, while tax and accounting law requires the booking, invoice and ledger entries to be kept for years; Germany, for example, requires ten years for books and, since 2025, eight for accounting vouchers including invoices.

GDPR resolves the conflict itself. Article 17(3) disapplies the right to erasure to the extent processing is necessary "for compliance with a legal obligation which requires processing

by Union or Member State law" and "for the establishment, exercise or defence of legal claims". The practical pattern has four parts:

1. **Retention covers only what the law requires**, for the period required, counted usually from the end of the fiscal year: the invoice, the booking record, the ledger entry. It is not a license to keep everything.
2. **Everything else is erased or anonymized on request**: marketing preferences, browsing history, free-text notes.
3. **Retained personal data is restricted**: removed from operational use, access-controlled, deleted at expiry.
4. **Ledgers hold identifiers, not identities**. The canonical financial events carry a `counterparty_id`; the name, address and contact details live in a separate customer or partner store with its own lifecycle. Erasing or anonymizing the identity record does not break the immutable ledger.

Crypto-shredding (encrypting each subject's data with its own key, then destroying the key) is useful, but whether it counts as erasure is a legal judgment for the controller. I would design pseudonymized ledgers first and treat crypto-shredding as a supplement.

Trade-offs: Four Ways to Remediate a Control Finding

The choice Dana faced at Northmere recurs whenever a custom finance platform draws a finding. Four responses are common.

Option	What it is	Cost to build	Cost to run and test	Evidence quality	When it is right
Compensating manual review	Keep the process; add a detective review of changes and payouts after the fact	Lowest	Highest: manual testing every quarter, forever	Weak: depends on the reviewer seeing every-thing	As a bridge for one or two quarters while a real fix is built

Option	What it is	Cost to build	Cost to run and test	Evidence quality	When it is right
Manual gate before change	Remove developer access; require ticketed manager sign-off before each deployment or rule change	Low	High; slows deployment; tested by sampling	Medium: complete only if no change bypasses the ticket	Low-change systems with a few releases a year
Pipeline-enforced controls with configuration as data	Deployment only through an enforcing pipeline; rules in an approved, versioned store; just-in-time privileged access; evidence emitted automatically	Highest up front (Northmere: most of a quarter for one team)	Lowest; tested once plus monitoring	Strong: complete populations from the system of action	Any system that changes often and moves money; the Northmere standard

Option	What it is	Cost to build	Cost to run and test	Evidence quality	When it is right
Move the logic into a vendor platform	Re-implement payout rules in ERP or a payments provider and inherit its controls through SOC 1 and CUECs	Very high; a migration	Medium: CUECs remain	Strong for the vendor's part; your configuration still in scope	When the capability is a commodity anyway and the migration is already justified on other grounds (When the ERP Isn't Enough: Custom Finance Platforms)

My verdict is that the third option is right for any custom platform that deploys weekly or more and computes amounts that reach the ledger or a bank. The first option is a legitimate bridge, not a destination. The fourth should never be chosen *because of* a control finding; controls are a poor reason to migrate and a good reason to fix.

Integration with the Rest of the Estate

Controls cut across the whole book. Ledger immutability, version stamping and drill-back are designed in Never Update, Only Append: Designing Custom Ledgers; reconciliations, the largest family of detective controls, in Reconciled by Design: Traceability and Audit Across Boundaries and BlackLine: Matching, Certifying and Closing; payout, settlement and safeguarding controls in Payments-to-Ledger: Draining the Suspense Swamp and Payments for Finance Architects: Eleven Reasons the Numbers Disagree; ownership of the master data whose changes the audit trail preserves in Finance Data Architecture: Who Owns the Truth. Bots and AI agents that post, approve or change financial data fall under the same ITGCs and evidence rules; Automate the Work, Not the Accountability owns the agent control card (STD-FIN-06) for them.

Failure Modes

These are the mistakes I see most often, each with the warning sign to look for.

- **Controls designed for the auditor, not for the risk.** The control satisfies the wording of a finding and prevents nothing. Warning sign: the control description was written after the audit letter and nobody in the business can explain what it protects.
- **Tickets as the population.** Evidence comes from the change ticketing system, so changes made without a ticket are invisible. Warning sign: nobody has ever reconciled the ticket list to the production deployment history.
- **Configuration treated as code, or as nothing.** Rules that determine amounts live in files reviewed only by engineers, or in admin screens nobody reviews. Warning sign: Finance cannot list the rules that determine what partners are paid.
- **SOC 1 reports filed, not read.** The vendor report is collected annually; nobody maps the CUECs or assesses the exceptions or the coverage gap. Warning sign: no one can name the CUECs for the close platform.
- **Emergency access as the normal path.** Break-glass is used daily because the operational tooling was never built. Warning sign: break-glass sessions outnumber incidents.
- **One remediation per platform.** Each brand fixes its own finding its own way, and the group ends up with three pipelines, three rule stores and three evidence formats. Warning sign: the same deficiency appears on a second payout engine a year later.

The Chief Architect's View

Decide which controls are standards, not choices. At Northmere, the Keystone standard STD-FIN-05, segregation of duties by design, makes five things mandatory for every finance-relevant system, built or bought: identity through the corporate provider with SoD-relevant roles registered; no standing human write access to production financial data; deployment only through an enforcing pipeline; financially significant configuration as approved, versioned data; and evidence emitted to the central evidence store. It is one of the mandatory enterprise-application standards STD-FIN-01 to STD-FIN-06 under Dana's decision rights (Principles That Decide Things), declared mandatory from 1 April 2027 for new builds and changed systems, with dated transition plans for existing ones such as SFP, and its five clauses were agreed with Mei and Marco Bellini, the CISO. STD-FIN-06, the agent control card, extends the same discipline to AI agents from Q3 2027.

Build the control plane once. The pipeline controls, rule-store pattern, privileged access tool and evidence store are shared platforms, not per-team projects. Wayfare's

payout engine adopts the SFP design before it draws its own finding; the retirement of the duplicate payout engine planned for 2029 then inherits a controlled design rather than having to remediate one.

Make the decision rights clear.

Decision	Decides	Consulted	Informed
Which controls are key and how they are tested	Head of Internal Controls (Mei)	Controller, process owners, Chief Architect	Audit committee via CFO
Mandatory control standards for systems	Chief Architect (Dana)	Internal Controls, CISO, CTO	Brand engineering leaders
Design of a system's controls	System owner (e.g., Jonas)	Chief Architect, Internal Controls	Auditor via Internal Controls
Approval of payout rules and other financial configuration	Finance process owner	Engineering owner of the engine	Internal Controls
Severity of a deficiency	Management, with the external auditor's own evaluation	Internal Controls	Audit committee

Watch a small set of measures. The share of key controls that are automated; the share of finance-relevant changes with a completed control impact assessment; the time to answer an auditor's evidence request; the number of SoD conflicts across systems; the number of break-glass sessions and their review time; repeat findings. At Northmere the target for evidence retrieval moves from about four days to one, and repeat findings to zero.

Plan for the long term. Every Keystone migration (the brand ledgers converging on the group core, the SFP merchant subledger moving in 2029) is a significant change under COSO principle 9 and a program-development ITGC in its own right. Migration reconciliations, parallel runs and cut-over evidence are control evidence, and they must be designed with Internal Controls from the first stage gate (Modernizing Without Stopping the Close).

Questions to Ask

Take these into your next design review or audit planning meeting.

1. For this system, which key controls rely on it, and which relevant assertions do they address?
2. Can any human write to production financial data except through an application path that validates and logs?
3. Where does the population of production changes come from, and has it ever been reconciled to what actually runs in production?
4. Which configuration determines amounts, and who approves changes to it, separately from the engineers who build the engine?
5. Which reports does a key control depend on, and how is each proven complete and accurate?
6. Does our SoD ruleset include engineering rights and custom platforms, and does it see conflicts across systems?
7. For each finance-relevant SaaS platform: do we have the current SOC 1 Type II report, have we mapped every CUEC to an owned control, assessed the exceptions and covered the period gap?
8. Can Internal Controls answer an evidence request without an engineer?
9. Where is each personal data element in this system retained, under which legal obligation, and what happens on an erasure request?

Controls depend on data moving between systems completely, once and on time: every interface count, control total and reconciliation in this chapter assumes it. The next chapter, Files, Events and APIs: How Finance Systems Talk, takes that up, starting from Northmere's inventory of 1,100 interfaces.

Sources and Evidence

Vendor documentation

- Workday Compliance, Workday, checked October 2026 — SOC 1 Type II report issued under ISAE 3402, covering Workday Enterprise Products, Adaptive Planning and VNDLY.
- Security at Stripe, Stripe, checked October 2026 — SOC 1 and SOC 2 Type II reports produced annually; restricted API keys, roles and security history audit logs.
- Download SOC reports, Stripe Support, checked October 2026 — SOC reports available from the Dashboard's Compliance & Documents section.
- Adyen Trust Center, Adyen, checked October 2026 — ISAE 3402 / SOC 1 report periods and quarterly bridge letters.

Standards and regulation

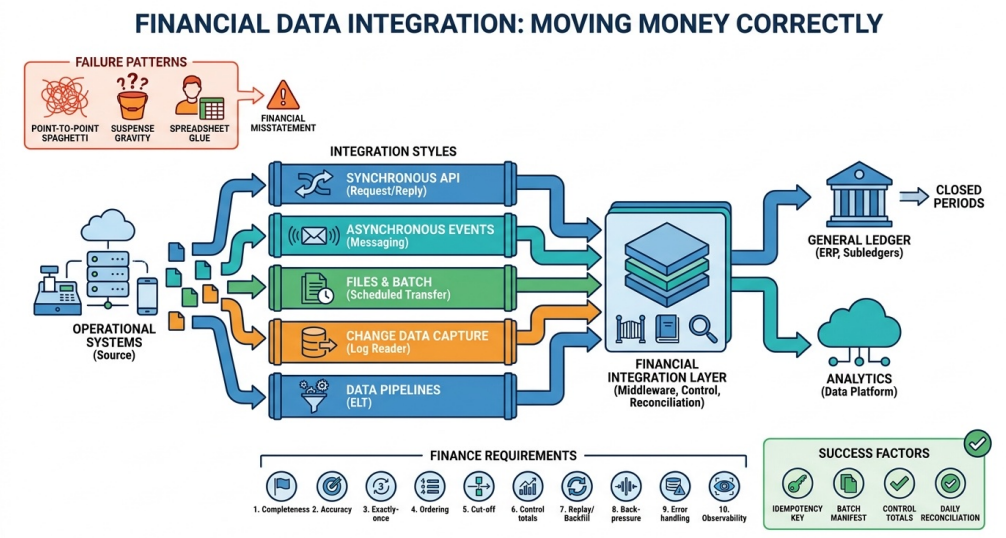
- AS 2201: An Audit of Internal Control Over Financial Reporting That Is Integrated with an Audit of Financial Statements, PCAOB — top-down approach, material weakness and significant deficiency (definitions cited from the standard as generally known; the page could not be fetched during research).
- Commission Guidance Regarding Management's Report on Internal Control Over Financial Reporting, SEC, 2007 — top-down, risk-based management evaluation.
- SOC 1 — SOC for Service Organizations: ICFR, AICPA & CIMA — scope of SOC 1 examinations.
- UK Corporate Governance Code, Financial Reporting Council — 2024 Code dates; Provision 29 from 1 January 2026; auditor scope unchanged; boards decide on external assurance.
- GDPR Article 17, Regulation (EU) 2016/679 — right to erasure and its exceptions for legal obligations and legal claims.
- Digital Operational Resilience Act, EIOPA — scope and application from 17 January 2025.

Practitioner and research sources

- Heads Up: COSO 2013, Deloitte, 2013 — COSO components, principles and transition.
- Provision 29 explainer, KPMG — material controls, comply-or-explain, no mandatory audit sign-off.
- SEC proposes optional semiannual reporting, DLA Piper, May 2026 — status of the semiannual reporting proposal.
- Viertes Bürokratieentlastungsgesetz, Haufe — Germany's shortened retention period for accounting vouchers from 2025.

Northmere Group is fictional; all architectural recommendations are the author's judgment.

12. Files, Events and APIs: How Finance Systems Talk



IN THIS CHAPTER, YOU WILL: Learn the five integration styles a finance estate uses and when each fits, the ten requirements that make an interface fit to carry money, and the transactional outbox and idempotent consumer patterns behind them. You will then see Northmere's daily posting from its custom subledger into SAP S/4HANA redesigned with a batch manifest, idempotency and safe replay.

WHY FINANCE CARES: Every number in the trial balance arrived through an interface. If an interface can lose a record, post it twice or put it in the wrong month, the close becomes a hunt for differences, and the controller cannot say whether a balance is complete without someone checking it by hand.

WHY ARCHITECTS SHOULD CARE: Integration is where finance estates quietly fail: an interface sends a file, gets a timeout and sends it again, or loads half a batch and reports success. These are not defects of one vendor

or one tool. They are design decisions that nobody made, repeated across hundreds of interfaces.

KEY POINTS:

- **Choose the style by the job, not by the platform you own.** Synchronous APIs, events, files, change data capture and data pipelines each fit some finance flows and harm others. A decision table per use case beats a house preference for "everything on events" or "everything through the iPaaS".
- **Exactly-once is an effect you build, not a guarantee you buy.** Messaging gives at-least-once delivery at best. A posting takes effect once only when the consumer is idempotent on a key derived from business identity, when every batch carries a manifest of control totals, and when a reconciliation proves the result.
- **A dead letter is not a suspense account.** A dead letter is a technical failure that an engineer must fix and replay; a suspense account holds money that was posted but not yet explained, which an accountant must clear. If you confuse the two, the books hide integration defects, and engineers end up making accounting decisions.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Point-to-Point Spaghetti** — about 1,100 finance interfaces on three integration technologies, most with their own format, retry logic and error handling, and few with any proof of completeness.
- **The Suspense Swamp** — integration failures that are "resolved" by posting the unexplained amount to a suspense account turn engineering defects into accounting balances that nobody owns.
- **Spreadsheet Glue** — about 70 interfaces have a person and a spreadsheet in the loop, where mappings, corrections and resends happen with no control total and no record.

Most enterprise integration writing assumes that a lost or duplicated message is an inconvenience. In finance it is a misstatement. A booking event that never reaches the subledger is revenue that does not exist in the books; a posting file sent twice is revenue that exists twice. Finance measures integration with a stricter standard than operations does: not "did it arrive", but "**can we prove that everything arrived, once, in the right period**".

That standard changes the design. It adds **control totals** (counts and sums that sender and receiver compare), **idempotency** (the property that processing the same request twice has the same effect as processing it once) and **period cut-off rules** to every interface that carries money. It turns the integration platform from a set of pipes into part

of the control environment the auditor tests. And it makes the most dangerous sentence in integration engineering, "just send it again", into a question about accounting.

This chapter owns four terms that other chapters rely on: the **integration styles**, **idempotency**, **control totals** and the **transactional outbox**. It introduces the canonical financial events that Canonical Events: A Common Language for Money develops in depth, and it gives the vendor integration surfaces at a glance; the platform chapters hold the detail.

Setting the Scene: The Night January Was Posted One and a Half Times

Northmere, the fictional group this book follows, closes January 2027 in the first week of February. On Monday 1 February, at 02:40 Amsterdam time, the Stayloft Finance Platform (SFP), the custom system that holds Stayloft's merchant subledger, writes its daily summary for posting date 31 January: about 38,000 general-ledger lines summarized from about 3 million subledger lines. The file goes by SFTP to an SAP Integration Suite flow, which splits it into journal entries and posts them into SAP S/4HANA through SAP's journal entry posting API.

That night the flow times out after posting about 60% of the entries. The monitoring shows a failed run. At 07:15 an operator on the S/4 platform team restarts it, as the runbook says, from the beginning of the file. S/4HANA accepts every entry again. Nothing in the interface tells S/4 that it has seen these entries before, and SAP documents that its journal entry posting APIs perform no duplicate invoice check. The 31 January summary is now in the ledger one and a half times.

The error surfaces on working day 3, when the BlackLine reconciliation of the traveler-funds control account shows S/4HANA about €38 million higher than SFP. Commission revenue for 31 January is overstated by about €2.9 million (illustrative figures). The fix is a manual reversal journal, approved by the Stayloft entity controller, and a day of the close is lost confirming that nothing else was doubled.

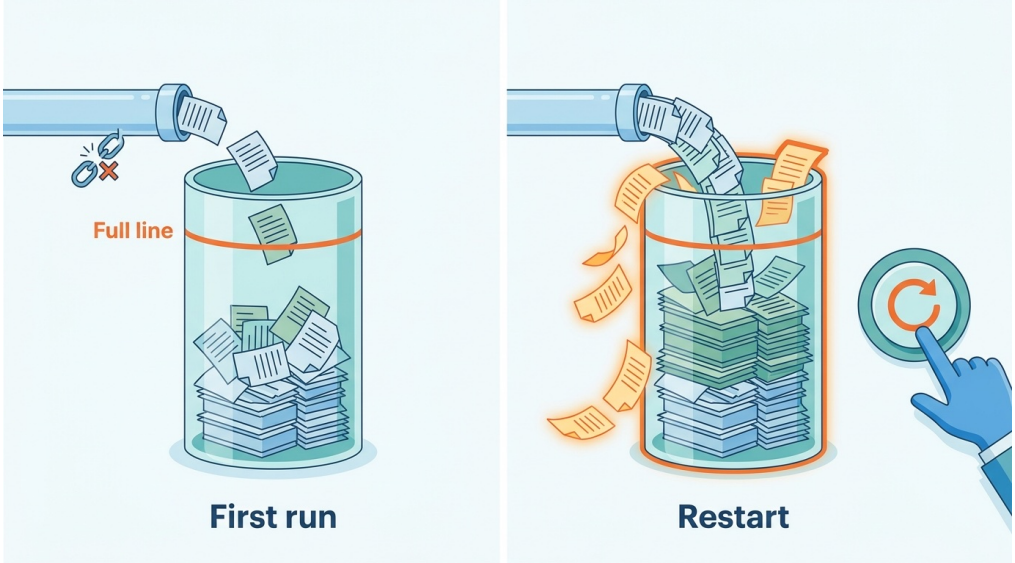


Figure 1: *A flow timed out after posting about 60% of 31 January's entries; restarting from the top of the file posted everything again, and nothing told the ledger it had seen those entries before.*

Dana Whitlock, the Chief Architect, asks Lucía Romero, the lead enterprise architect for integration, how many of the estate's interfaces could do the same thing, and how many could prove they had not. Lucía's inventory, built for Program Keystone (Northmere's finance transformation), counts about **1,100 interfaces** in the finance estate. About **240** are flat-file transfers and about **70** have a spreadsheet and a person in the loop. They run on three integration technologies: **SAP Integration Suite** for SAP-centric flows, **Kafka** as the event backbone owned by Ravi Menon's Platform Engineering, and **MuleSoft** carrying the legacy Wayfare and Tablekin flows.

Her answer to the second question is the uncomfortable one. Of the 1,100, about **310** carry a record count or control total that the receiver checks; the rest assume that a transfer that reported success was complete. Fewer than 100 can be safely re-run (illustrative counts from the 2027 inventory).

The incident did not reveal a bad tool; each component did what it is documented to do. It revealed that nobody had designed what the interface must guarantee. The rest of this chapter is that design.

Five Integration Styles and What Each Is Good For in Finance

An **integration style** is the basic mechanism by which two systems exchange data. Hohpe and Woolf's *Enterprise Integration Patterns* catalogs the patterns within each;

for finance architecture, five styles are enough to classify every interface in the estate.

Style	Mechanism	Strength for finance	Weakness for finance
Synchronous API	Caller sends a request and waits for the answer (REST, SOAP, OData, gRPC)	Immediate answers: tax determination, credit check, validation of an account before posting	Couples availability: if the callee is down or slow, the caller stops. Timeouts leave the outcome unknown, which is exactly the case that produces double postings
Asynchronous events and messaging	Producer publishes a fact; consumers subscribe through a broker (Kafka, event mesh, queues)	Per-transaction detail with low latency; many consumers from one publication; replay from a retained log	Delivery is at-least-once in practice; ordering holds only within a partition or key; completeness must be proven separately
Files and batch	A set of records is produced at a cut-off and transferred whole (SFTP, managed file transfer, vendor bulk-import templates)	Natural unit for period cut-off and control totals; matches how banks, PSPs and SaaS ERPs exchange bulk data	Latency of a day; partial loads; manual resends; a file is easily copied, edited or sent twice
Change data capture (CDC)	A tool reads the source database's transaction log and emits row changes	Captures changes without modifying the source application; useful for legacy systems and analytical replication	Emits table changes, not business facts; at-least-once after failures; couples the consumer to the source's physical schema

Style	Mechanism	Strength for finance	Weakness for finance
Data pipelines and ELT	Scheduled or streaming extraction into a data platform, transformed there	Builds the analytical and reconciliation views (FinDB) across all sources	Not a posting channel: a warehouse table is not a ledger, and its numbers need tie-out to the ledger of record

The styles are not ranked. "Event-driven" is not more modern than "file" in any sense that matters to a controller: banks send statements as files (MT940 or ISO 20022 camt.053), PSPs publish settlement reports as files, and Oracle Fusion's bulk inbound path is a file template. **CDC, in particular, is a technique for integrating legacy systems, not a contract**. Debezium, the widely used open-source CDC platform, reads database transaction logs. It documents that after a failure connectors restart from the last recorded offset, so "these duplicates will always be visible to downstream consuming applications". CDC on a business table also publishes the source's internal schema to every consumer. I use it in two cases: to replicate into the data platform, and as the relay for a transactional outbox, which is described below. I do not use it as the published interface between finance domains.

A decision table for finance use cases

The rule I use is to **choose the style by the unit of completeness the receiver needs**. That might be one transaction, one business day, one settlement batch or one period. Then layer the finance requirements of the next section on top.

Finance use case	Northmere example	Recommended style	Why
Operational financial events to a subledger	Booking, cancellation and payment events into SFP	Events on the backbone, keyed by booking	Per-transaction detail, many consumers, replay from the log

Finance use case	Northmere example	Recommended style	Why
Subledger summary to general ledger	SFP to S/4HANA, ~38,000 lines a day	Batch with manifest, posted through the ERP's asynchronous API or bulk import	A posting date is a natural batch; control totals per batch; the ERP is built for summaries, not streams
PSP settlement to reconciliation and ledger	Adyen settlement details, Stripe balance transactions	Files or report APIs, announced by webhook	The report is the financial record; webhooks are early operational signals (Payments for Finance Architects: Eleven Reasons the Numbers Disagree)
PSP payment status to operations	Adyen and Stripe webhooks into Northmere Pay	Webhook received into a durable store, then events	The provider pushes; the receiver must accept fast and process later
Bank statements and payment files	Kyriba to banks and back	Files in ISO 20022 or SWIFT formats through bank connectivity	The banking standard; each file is a control unit
Real-time decision during a transaction	Vertex tax determination for an SFP commission invoice	Synchronous API with timeout and fallback rule	The answer is needed before the invoice is issued
Master and reference data distribution	Cost centers and approvers from Workday HCM; FX rates from treasury	Events on change, plus a periodic full snapshot for reconciliation	Consumers need changes fast and a way to prove they hold the full set
Trial balances to consolidation	S/4HANA, Fusion and Workday to OneStream	Batch per entity and period, with control totals	Consolidation works on closed periods

Finance use case	Northmere example	Recommended style	Why
Analytics and tie-out	All of the above into FinDB (Snowflake)	Data pipelines and CDC replication	Analytical, not posting; tie-out to the ledger is mandatory (One Number, Many Views: Finance Data from Operations to Analytics)

Lucía's inventory maps the 1,100 interfaces onto these styles (counts rounded, illustrative):

Style	Interfaces	Typical technology at Northmere
Synchronous APIs	~410	SAP Integration Suite, MuleSoft, direct calls
Events and messaging	~180	Kafka
Files and batch	~240	SFTP scripts, Integration Suite, MuleSoft, Kyriba bank connectivity
Spreadsheets in the loop	~70	Email, shared drives, manual upload
Data pipelines and ELT	~150	dbt and loaders into Snowflake
Change data capture	~50	Replication into FinDB

The mix is not the problem; each of the 1,100 making its own decisions about completeness, retries and errors is.

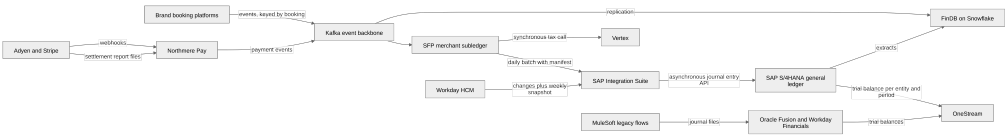


Figure 2: *One finance estate uses all five styles. The architecture question for each arrow is not which technology carries it but what unit of completeness it must prove: one transaction, one batch or one period.*

Ten Requirements for an Interface That Carries Money

An operational integration is judged on latency and availability. A finance integration must also meet the ten requirements below. Together they are the specification Keystone's integration standard imposes on every interface that changes a balance. Controls by Design, Not by Audit Season explains why the auditor cares; Reconciled by Design: Traceability and Audit Across Boundaries turns the same properties into a reconciliation catalog.

1. **Completeness**. Every record the source produced for the period reaches the target, or is visibly accounted for as rejected or pending. Completeness is a property you **prove**, not one you assume. The proof is a comparison of what was sent with what was received and accepted, made by the receiver, on numbers the sender declared independently.

2. **Accuracy**. What arrives means what was sent. Amounts arrive in **minor units with an ISO 4217 currency code**, never as floating-point values, and minor-unit exponents vary (zero for the Japanese yen, three for the Kuwaiti dinar). Mappings from source codes to accounts are versioned master data (Finance Data Architecture: Who Owns the Truth), not logic hidden in an integration flow.

3. **Exactly-once effect**. Each business fact changes the books once. The next section explains why this is a property of the receiver's processing, not of the transport.

4. **Ordering, and its limits**. Some facts depend on others: a refund cannot be accounted for before the capture it refunds. Brokers give narrow ordering guarantees. Apache Kafka guarantees that "any consumer of a given topic-partition will always read that partition's events in exactly the same order", and says nothing about order across partitions. Providers give less: Stripe states that it "doesn't guarantee the delivery of events in the order that they're generated", and Adyen advises checking timestamps and, where present, a `sequenceNumber` (Adyen webhooks). My rule: **key the stream by the business entity whose order matters** (the booking or the payment) so its events share a partition, and **make consumers tolerate the remaining disorder** by parking an event whose parent has not arrived, or by treating the event as a hint and reading current state.

5. **Cut-off and period assignment**. Every record carries enough time information to decide its accounting period without asking when it arrived. That means at least the **event time** (`occurred_at`), the **receipt time** (`recorded_at`) and a **posting date** assigned by a declared rule. The rule names the timestamp, the time zone and the owner. At Northmere, Leila Haddad's team owns the cut-off rules. For SFP, the booking event's `occurred_at` is read in Amsterdam time for Stayloft B.V.; for Adyen events, the rule uses the merchant account's report time zone (Adyen: One Platform from Checkout to Settlement). A record that arrives after its period is locked is posted in the current open period with its original event date kept, so the late arrival can be measured (Accounting

for Architects: Ledgers, Journals and Revenue).

6. **Control totals and batch manifests**. A **control total** is a number computed independently by sender and receiver over the same set of records and then compared: a record count, an amount total per currency, a debit total and a credit total, or a **hash total** (a sum of a field such as account number that has no meaning except as a check). A **batch manifest** is the sender's declaration of a batch: its identifier, its scope (source, entity, posting date), its record count, its control totals, and a cryptographic digest of the content. The receiver checks the manifest before it accepts the batch and records the result. For a stream there is no natural batch, so the producer publishes a periodic **completeness checkpoint** instead, for example per source, per hour and per event type: "between 10:00 and 11:00 I published 88,120 `BookingConfirmed` events totaling €51,302,447.15".

7. **Replay and backfill**. **Replay** re-delivers records already sent, to recover from a consumer defect or outage. **Backfill** sends records for a past period that were never sent, after a new consumer is added or a source defect is found. Both are routine, so both must be safe. Replay must not double-post, which requires idempotency. Backfill must not land silently in a closed period, which requires the cut-off rule. And both need retention: a Kafka topic that keeps seven days of events cannot replay a month-end. Martin Fowler's warning about event sourcing applies in full: during a replay, calls to external systems must be suppressed, or a replay can resend payments.

8. **Back-pressure at month-end**. Finance volume is not flat. On the first working days of a close, accrual runs, settlement files, intercompany postings, trial-balance loads and reconciliation extracts all compete for the same integration runtime and the same ERP posting capacity. At Northmere, the integration runtime carries roughly four times an average day's postings on working day 1 (illustrative). An interface needs a declared behavior when the receiver slows down: it queues, throttles or sheds non-financial load, and it never times out into the unknown-outcome state that caused the January double posting. Vendors document their own limits. Workday, for example, may reject requests with HTTP 429 or 500 under load and recommends exponential back-off.

9. **Error handling: dead letters versus suspense accounts**. This distinction is the most important one in the chapter, and the next section takes it in full.

10. **Observability of money in flight**. Operational observability asks whether the pipes are healthy. Finance asks **how much money is between systems right now, and for how long**. At any moment some value has left one system and not yet been accepted by the next: events consumed but not yet summarized, batches sent but not yet posted, documents parked in a dead-letter queue. I recommend that each finance interface publishes, per source and day, the counts and amounts received, rejected, delivered, replayed and pending. That record is the **delivery ledger**, and it makes "money in flight" a metric with an owner and an aging profile, not a guess on working day 3. Tracing standards such as OpenTelemetry correlate the hops; the delivery ledger counts the money.

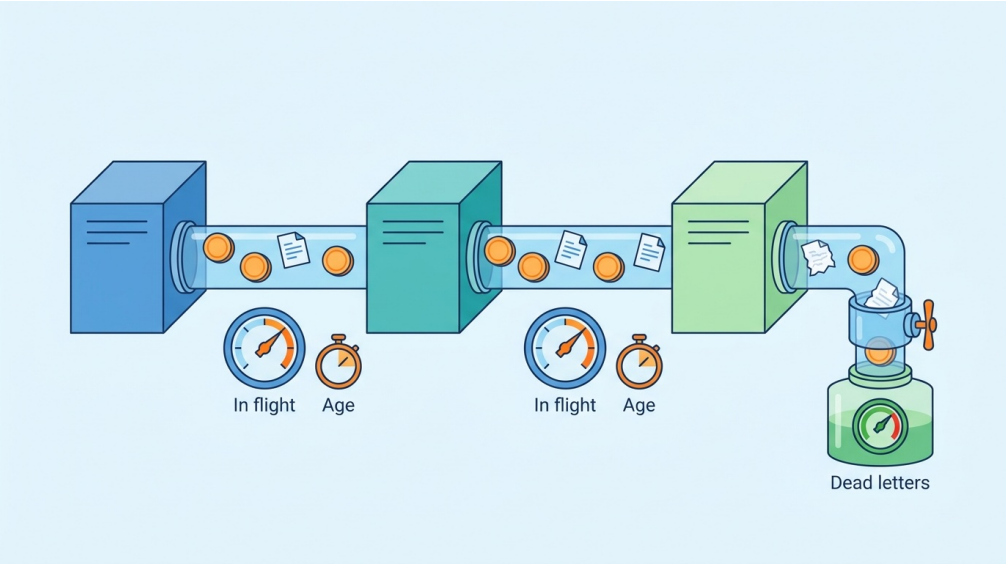


Figure 3: *Finance observability asks how much money sits between systems right now and for how long, including the value parked in dead-letter stores.*

Dead letters and suspense accounts are different things

A **dead letter** is a message the integration layer could not process: malformed, failing validation, referencing an account that does not exist, or rejected by the target. Hohpe and Woolf's Dead Letter Channel is where "a messaging system … may elect to move the message" when it "cannot or should not deliver" it. A dead letter has not reached the books. Fixing it is an engineering task, and the fix is a correction followed by a replay.

A **suspense account** (or clearing account) is a general-ledger account that holds amounts that have been **posted** but whose final destination is not yet known: a settlement line that cannot be matched to a booking, a receipt without a remittance reference. A suspense item is in the books, it affects the balance sheet, and clearing it is an accounting decision with an approver.

	Dead letter	Suspense account
Where it lives	Integration layer: a queue, topic or error store	General ledger: a balance-sheet account
Has it reached the books?	No	Yes

	Dead letter	Suspense account
Typical cause	Technical: schema error, invalid master data reference, target unavailable, duplicate rejected	Business: unmatched or unexplained money, missing reference, timing difference
Who resolves it	The interface's engineering owner	An accountant, with approval for write-offs
How it is resolved	Fix the data or the code, then replay	Match, reclassify by journal, or write off under policy
What it must report	Count, amount and age per interface	Balance, aging and items older than policy, reconciled monthly
Failure if confused	Engineers "park" bad records by posting them to suspense, so defects become balances	Accountants fix balances by journal, so the defect that caused them recurs

The anti-pattern is easy to slip into. A deadline approaches and someone configures a flow to post anything unmappable to a suspense account "so the batch balances". From then on every mapping defect becomes an accounting balance: the dead-letter count drops to zero and the suspense balance grows. That is how **The Suspense Swamp** starts. Northmere's PSP clearing and suspense balances averaged **€14 million** at month-end in 2026, with about €2.1 million older than 90 days. Not all of that came from integration defects, but Lucía's sample found mapping failures among the oldest items (Payments-to-Ledger: Draining the Suspense Swamp owns the fix).

The rule I use: technical failures stay out of the ledger. An interface may post to a suspense account only where an accounting policy defines that account for a specific business situation, such as an unmatched PSP settlement line. It may never use one as an error handler. Dead letters carry money too, so they are reported in the delivery ledger with amounts and ages. An interface with zero dead letters and a growing suspense account is not healthy.

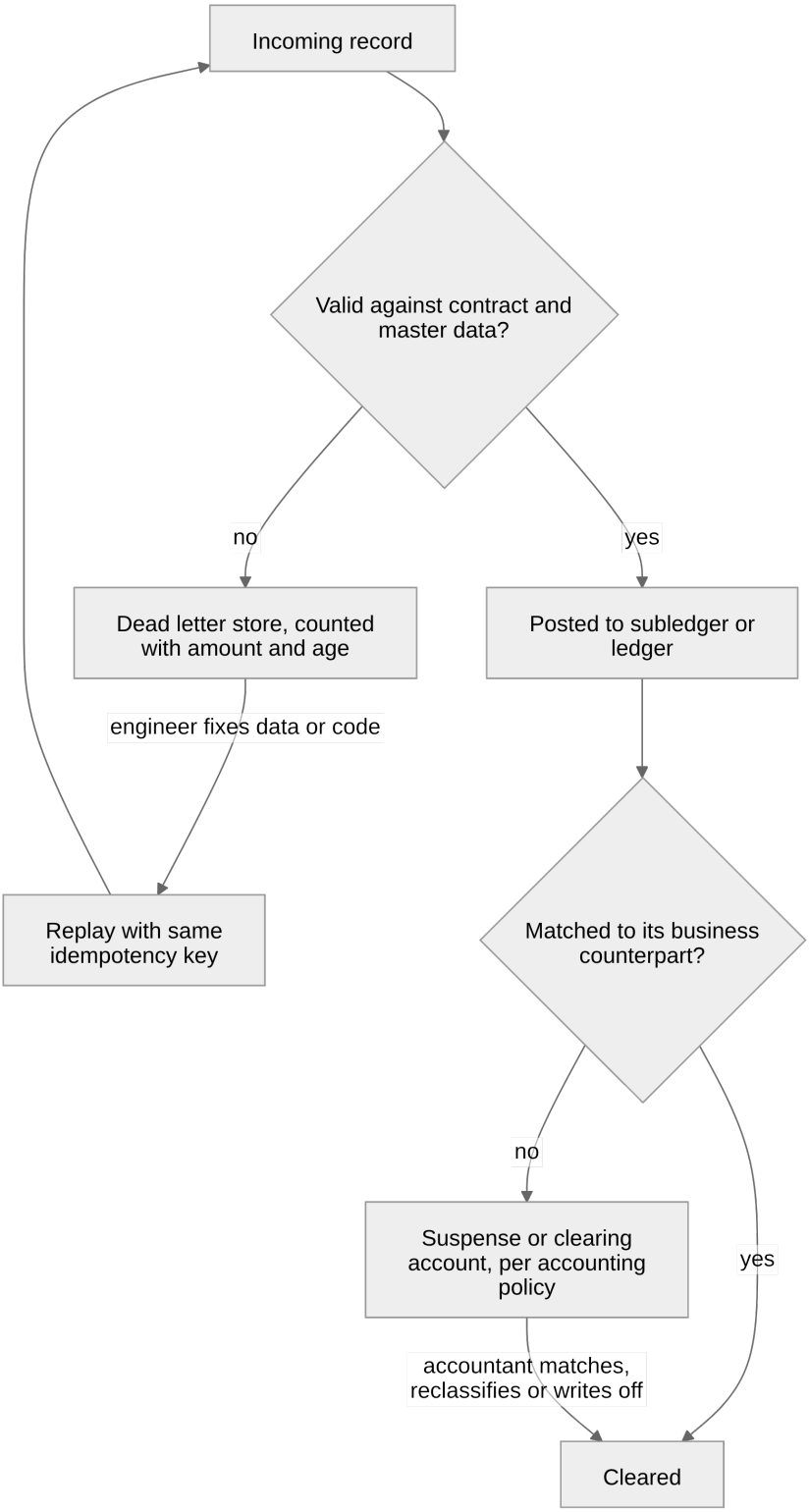

Incoming record
Valid against contract and master data?
no
yes
Dead letter store, counted with amount and age
Posted to subledger or ledger
engineer fixes data or code
Replay with same idempotency key
Matched to its business counterpart?
no
yes
Suspense or clearing account, per accounting policy
accountant matches, reclassifies or writes off
Cleared

Figure 4: *Two error paths with two owners. Technical failures stop before the books and are replayed after a fix; business exceptions are posted to accounts defined by policy and cleared by accounting decisions. The dangerous arrow is the one this diagram leaves out: from dead letter straight to suspense.*

Exactly-Once Is an Effect, Not a Delivery Guarantee

A sender that gets no acknowledgment cannot tell whether the message or the acknowledgment was lost. It can retry and risk a duplicate, or give up and risk a loss. Finance cannot accept a loss, so **at-least-once delivery** is the realistic baseline.

Middleware marketing blurs this. Kafka does offer "exactly-once semantics", but Confluent's own explanation (Narkhede and Wang, 2017) limits it to read-process-write **within Kafka**. Side effects outside Kafka, such as a call to an ERP or a write to another database, are not covered, and rewinding offsets cannot roll back external state. An ERP posting is precisely such a side effect. **No middleware delivers exactly-once postings**.

What finance can have is an **exactly-once effect**: each business fact changes the books once, however many times it is delivered. It is built from three parts:

1. **At-least-once delivery**, so nothing is lost: durable publication, retries with exponential back-off and jitter.
2. **Idempotent processing**, so repeats change nothing: the consumer recognizes a fact it has already applied and returns the earlier result.
3. **Reconciliation**, so any residual error is caught: control totals per batch and a subledger-to-GL comparison per day.

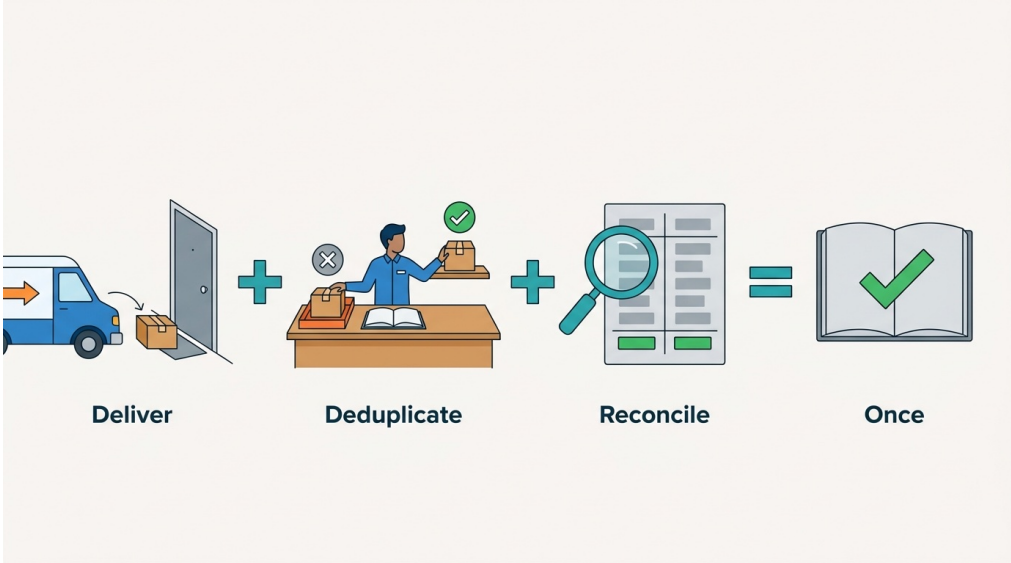


Figure 5: *Exactly-once is an effect, not a delivery guarantee: at-least-once delivery, an idempotent receiver and reconciliation together make each business fact change the books once.*

Hohpe's short essay "Your Coffee Shop Doesn't Use Two-Phase Commit" makes the same point from the other direction. Loosely coupled systems handle errors by write-off, retry or compensating action. Retry needs idempotent receivers, and the write-off strategy depends on "reconciliation reports" run periodically. In finance, all three of those are architecture.

Idempotency keys derived from business identity

An **idempotency key** identifies the business operation a request represents, so that the receiver can detect a repeat. Stripe's implementation is the reference most engineers know. A client sends an `Idempotency-Key` of up to 255 characters on a POST; Stripe stores the result and returns it for a repeated key, and rejects a repeat whose parameters differ. Keys can be pruned after at least 24 hours. Brandur Leach's essay on idempotency adds the retry discipline: exponential back-off with jitter.

Three design rules follow for finance.

- **Derive the key from business identity, never from the attempt.** A random identifier generated per attempt satisfies the API and defeats the purpose, because the retry gets a new identifier. Northmere's keys look like `SFP:NL01:2027-03-12:run-1:doc-0042` for a summary document or `ADYEN:{pspReference}:CAPTURE` for a payment modification. A reviewer can read the key and tell which business fact it represents.

- **Same key, different content, is an error, not an update.** A corrected batch gets a new run number and is preceded by a reversal of the run it replaces. A silent overwrite would destroy the audit trail.
- **Hold keys for at least as long as you can replay.** Stripe's 24-hour minimum protects a caller's retries, not a replay a week later. The receiver's own processed-key store must cover the longest replay or backfill window the interface supports. For a ledger posting, that window runs to period close and beyond.

Most ERP posting interfaces will not do this for you. SAP's knowledge base states that there is no duplicate invoice check for the Journal Entry Post API, synchronous or asynchronous. The duplicate checks that exist sit in the payables application, not in the generic posting service. The idempotency gate therefore belongs in front of the ERP, under the integration owner's control, with the key also written into a reference field of each posted document so that a reconciliation can find duplicates that slipped through.

The transactional outbox and the idempotent consumer

The producer side has its own version of the problem, called the **dual write**. A service that updates its database and then publishes an event can crash between the two, leaving a change with no event or, if it publishes first, an event with no change. The **transactional outbox** (Richardson) removes the gap. The service writes the business change and an outbox row in the same database transaction; a separate **relay** reads the outbox and publishes. Debezium's Outbox Event Router is a common relay. It routes rows to topics by aggregate type and keys messages by aggregate identifier, which preserves per-aggregate order. The relay can publish a row twice after a crash, so the outbox always pairs with an **idempotent consumer**: Hohpe and Woolf's Idempotent Receiver, "one that can safely receive the same message multiple times". In practice the consumer records each processed event identifier **in the same transaction as its own effect**.

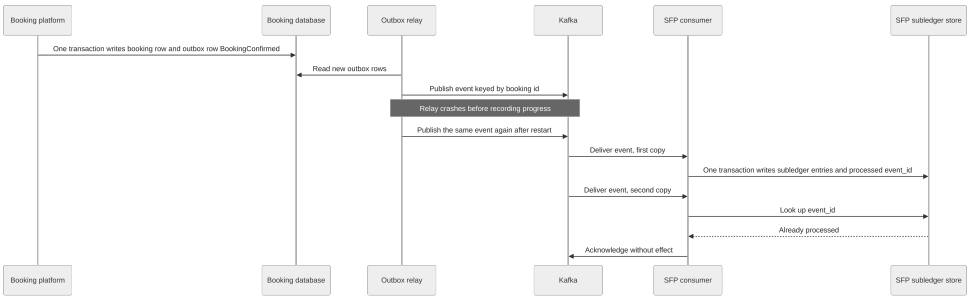


Figure 6: *The outbox closes the gap between changing state and announcing it; the idempotent consumer closes the gap the relay reopens. Neither guarantees exactly-once delivery. Together they give an exactly-once effect on the subledger, which reconciliation then confirms.*

Two details make or break this in finance. The processed-event record and the accounting effect must commit **atomically**. A consumer that posts and then records the key, in two steps, recreates the dual write it was meant to remove. And when the effect lands in a system the consumer does not control, such as an ERP or a PSP, atomicity is impossible. Then the key must travel with the request and be stored on the target document, and a reconciliation must look for duplicates by that key.

Integration Platforms: iPaaS, ESB, Event Backbone or Code

Styles describe how data moves; platforms describe where integration logic runs and who owns it. Four options recur: an **integration platform as a service (iPaaS)**, a vendor-run platform with connectors, mapping, API management and monitoring (SAP Integration Suite, MuleSoft Anypoint, Oracle Integration, Boomi, Workato), commonly organized in layers such as MuleSoft's widely cited system, process and experience APIs; an **enterprise service bus (ESB)**, its usually self-hosted predecessor; an **event backbone**, a durable, partitioned log or broker (Kafka, SAP Integration Suite's advanced event mesh) that moves and retains events but does not transform them; and **integration in code**, adapters and services run by the domain team that owns the data.

Option	Who usually owns it	Fits	Does not fit	The finance risk
iPaaS	A central integration team or a platform team (at Northmere: Sven Aalto's SAP team for Integration Suite)	Connecting SaaS and ERP surfaces through delivered adapters; API management; moderate-volume, mapping-heavy flows	Per-transaction streams at marketplace volume; logic that needs domain tests	**Posting logic creeps into mappings.** Account derivation, summarization and tax rules end up in flows no accountant reviews and no unit test covers

Option	Who usually owns it	Fits	Does not fit	The finance risk
ESB	Central middleware team	Legacy estates already built on it	New finance designs	Becomes the place every rule lives, and the bottleneck every change waits on
Event backbone	Platform engineering (at Northmere: Ravi Menon's team)	Canonical financial events; many consumers; replay; decoupled domains	Request-reply; bulk vendor imports; anything needing transformation in the pipe	Ownership of the platform without ownership of the contract. The topics run reliably while their meaning drifts
Code in the domain	The domain team that owns the data	Idempotent consumers, posting services, PSP adapters, anything with financial logic	Commodity SaaS-to-SaaS connections a delivered adapter already handles	Every team writes its own retry, idempotency and control-total code, differently. A shared library and standard fix this

The real trade-off is **where the financial logic is allowed to live**. My position: integration platforms **move and validate** data and never **decide accounting**. Account determination, summarization rules and posting rules belong in the subledger or the accounting engine, under the change controls the auditor tests (Never Update, Only Append: Designing Custom Ledgers). If an iPaaS mapping decides which account a commission lands in, that mapping is an application control in an unlikely place. It needs versioning, approval and a test the auditor can rely on, and it rarely has any of them.

Three technologies: a Keystone decision

Northmere runs three integration technologies because of its history, not because of a design. SAP Integration Suite arrived with S/4HANA in 2021. Kafka grew in brand engineering and is run by Platform Engineering. MuleSoft came with the Wayfare and Tablekin acquisitions. Each has a different owner, a different monitoring console and a different idea of an error.

Lucía's recommendation to the Keystone design authority, which Dana adopted as the 2027 position, assigns each technology a lane rather than picking a winner:

Lane	Technology	Scope	Direction
Event lane	Kafka	Canonical financial events between domains; retained for replay through period close plus one month	Invest. Platform Engineering runs it; Lucía's team owns the event contracts
ERP and SaaS lane	SAP Integration Suite	Surfaces of S/4HANA, Workday HCM, Coupa, Concur, Kyriba and other vendors; API management	Invest, under a rule that it moves and validates but does not derive accounting
Legacy lane	MuleSoft	Existing Wayfare and Tablekin flows	Contain: no new flows from 2027; flows retire with the ledger convergence from 2029
Domain adapters	Code, on a shared library	Posting services, PSP adapters, webhook receivers	Standard library for idempotency keys, manifests and delivery-ledger reporting

The lane model is a transition state, not a target. It accepts two technologies' overlap

in the ERP lane until 2029 because migrating working MuleSoft flows ahead of the ledger convergence would mean rebuilding each twice. Duplicates and Dependencies: The Hidden Costs of a Grown Estate treats the general question of when duplication is acceptable.

Vendor Integration Surfaces at a Glance

Each platform exposes its own surfaces, with its own semantics for bulk, events and errors. The table compares them on the questions this chapter asks. It includes only documented facts and limits; the platform chapters (SAP S/4HANA: One Journal, Clean Core, Oracle Fusion: An Accounting Hub for High-Volume Events, Workday: The Org Chart Behind the Ledger, Adyen: One Platform from Checkout to Settlement, Stripe: Every Movement Is a Balance Transaction) hold the detail.

Platform	Inbound for bulk financial data	Inbound for transactions	Outbound and events	Documented limits and behaviors that matter
SAP S/4HANA	Asynchronous Journal Entry Post API; IDoc for classic integrations	Released OData and SOAP APIs (synchronous Journal Entry Post among them); BAPIs as "classic APIs"	Business events (RAP events in recent releases) through SAP Integration Suite and its event mesh	SAP's clean-core levels rate released APIs as Level A and BAPIs as Level B. Large synchronous postings can time out; SAP advises the asynchronous API. No duplicate invoice check in the journal entry APIs

Platform	Inbound for bulk financial data	Inbound for transactions	Outbound and events	Documented limits and behaviors that matter
Oracle Fusion Cloud ERP	FBDI (file-based data import): files uploaded to Universal Content Management (UCM), then load and import jobs; Accounting Hub's Import Accounting Transactions	REST APIs per resource; ERP Integration Service to automate FBDI	Business events through the Oracle Integration ERP adapter; BI Cloud Connector (BICC) for bulk extraction, with a newer extraction tool since 26A	Oracle publishes volume mechanisms (split files, separate upload, import and accounting processes) but **no throughput number**. Imports are scheduled jobs, so completeness is checked per import run

Platform	Inbound for bulk financial data	Inbound for transactions	Outbound and events	Documented limits and behaviors that matter
Workday	Enterprise Interface Builder (EIB) file loads; Workday Web Services (SOAP); Accounting Center for external accounting	REST and versioned SOAP APIs; Studio and Orchestrate for complex flows	RaaS (custom reports as web services) for extracts; outbound integrations from business processes	Published limits: up to 5 concurrent EIBs; HTTP 429 or 500 throttling; RaaS quotas by tenant size (a tenant under 100,000 workers, like Northmere's, gets 250,000 extracts per 24 hours and 75,000 per hour); datetimes returned in Pacific Time
Adyen	Settlement details and payment accounting reports, announced by a `REPORT_AVAILABLE` webhook	API requests for payments and modifications; modifications are asynchronous and confirmed by webhook	Webhooks, HMAC-signed, at-least-once	Acknowledge within 10 seconds; retries from 30 seconds up to 8 hours for up to 30 days; duplicates share `eventCode` and `pspReference`; reports follow the account's time zone

Platform	Inbound for bulk financial data	Inbound for transactions	Outbound and events	Documented limits and behaviors that matter
Stripe	Payout reconciliation and balance reports (API and Dashboard)	API with `Idempotency-Key` on POST requests	Webhooks, signed, at-least-once, unordered; thin or snapshot event payloads	Retries for up to three days in live mode; no ordering guarantee; keys up to 255 characters, prunable after 24 hours; API and event shapes versioned

Three patterns run through the table. **The SaaS ERPs are batch-shaped for volume**. Their documented bulk paths are files or asynchronous jobs, and the vendors document throttling rather than promising throughput. Marketplace-scale detail is therefore summarized before it reaches them (When the ERP Isn't Enough: Custom Finance Platforms). **The PSPs are event-shaped for operations and report-shaped for money**. Webhooks are fast, unordered and at-least-once, while the reports are the financial record. **Time zones differ by surface**. Workday returns Pacific Time, Adyen's reports follow the account setting, and Stripe's reporting views differ. No interface should rely on a timestamp without declaring the time zone in its contract.

Canonical Financial Events, Introduced

An estate of 1,100 interfaces cannot be governed one interface at a time. Keystone's answer is a **canonical financial event**: a business fact with financial consequence, published once by the domain that owns it, in a shared envelope that every consumer understands. Lucía's team drafted version 1 in early 2027; the design authority approved it in June 2027, with the schema registry live from 1 July 2027. The event names include `BookingConfirmed`, `BookingCancelled`, `StayCompleted`, `PaymentCaptured`, `PaymentSettled`, `RefundIssued`, `ChargebackReceived`, `PSPFeeCharged`, `PartnerPayoutInstructed`, `PartnerPayoutSettled`, `CommissionInvoiced`, `SubscriptionInvoiced`, `UsageRated`, `AdClickBilled` and `FXRateApplied`.

The envelope is where this chapter's requirements become fields. Here it is for the

capture of the Lisbon booking, the €600.00 Stayloft stay paid by card through Adyen on 12 March 2027:

Envelope field	Value for the Lisbon capture	Requirement it serves
`event_id`	Derived from the Adyen modification: `ADYEN:{pspReference}:CAPTURE`	Exactly-once effect: the idempotency key
`event_type`, `event_version`	`PaymentCaptured`, `1`	Contract versioning
`occurred_at`	2027-03-12, time of capture, with offset	Cut-off and period assignment
`recorded_at`	Time Northmere Pay recorded it	Late-arrival measurement
`brand`, `legal_entity`, `business_model`	Stayloft; Northmere Payments B.V. for the funds; merchant	Routing and account derivation without lookups in the pipe
`source_system`	Northmere Pay	Completeness per source
`correlation_id`	The booking id	Ordering key and traceability
`amount`	`60000` minor units, `EUR`	Accuracy: no floating point
`counterparty_id`, `tax_jurisdiction`	The traveler's tokenized id; PT	Downstream tax and partner logic
`payload`	Event-specific detail	Kept versioned and validated

Three properties of the model matter for integration design: events are **immutable** (corrections are new events that reference the original), **keyed by business identity**, and **accompanied by completeness checkpoints** per source. Everything else, including the adapter kit that lets a brand publish canonical events, the schema registry, consumer-driven contract tests and the governance of the model, belongs to Canonical Events: A Common Language for Money.

Worked Example: Redesigning the SFP-to-S/4HANA Daily Posting

The January incident gave Keystone its first integration pattern to standardize. Lucía, Jonas Berg (who owns SFP) and Sven Aalto (who owns S/4HANA and Integration Suite) redesigned the SFP-to-S/4HANA posting between February and April 2027. The design

changes the interface, not the systems. SFP still summarizes, S/4HANA still owns the balances, and Integration Suite still carries the traffic.

What was wrong

Property	Before	Consequence
Batch identity	A file name with a timestamp	A re-sent file looked like a new batch
Completeness	Integration Suite logged "success" when it finished	A partial post reported as a failure, and a full re-run reported as a success
Idempotency	None. The S/4 API performs no duplicate check and nothing in front of it did either	Re-run doubled what had already posted
Recovery procedure	Restart from the beginning of the file	Turned a partial load into a double posting
Error handling	Rejected documents written to a log file; the batch "balanced" by an operator's manual journal	Defects turned into ledger adjustments
Reconciliation	Monthly, in BlackLine, on working day 3	Found on day 3 what the interface could have known in minutes

The new design

1. **A batch manifest that SFP produces before it sends anything.** For each legal entity and posting date, SFP writes a manifest that declares the batch. Using the posting date 12 March 2027, the date When the ERP Isn't Enough: Custom Finance Platforms uses to illustrate SFP's summary controls:

```yaml
batch_id: SFP:NL01:2027-03-12:run-1
source: SFP
legal_entity: NL01            # Stayloft B.V.
posting_date: 2027-03-12      # Amsterdam-time cut-off rule v2
period: 2027-03
events_consumed: 2114880
detail_lines: 3061442
summary_lines: 38210
documents: 77                 # summary lines chunked into journal entries
totals:
  - {currency: EUR, debit_minor: 18240633117, credit_minor: 18240633117}
  - {currency: GBP, debit_minor: 1102448902, credit_minor: 1102448902}
content_digest: sha256:9f2c…
supersedes: null
```

Debits equal credits per currency, the detail lines and the summary lines agree to the cent, and the manifest is signed by SFP's posting service (amounts illustrative). The manifest is the sender's independent declaration. Everything downstream is checked against it.

2. **Deterministic chunking and document keys.** The 38,210 summary lines are chunked into journal entries of at most 500 lines each, a Northmere design choice that keeps each request small after document splitting. SAP warns that document splitting can multiply item counts and slow large postings. Chunking is deterministic: the same batch always yields the same 77 documents, each with a key such as `SFP:NL01:2027-03-12:run-1:doc-0042`. The key is written into the journal entry's reference field in S/4HANA.

3. **An idempotency gate in front of S/4HANA.** The Integration Suite flow checks a **posting registry**, a small store owned by Sven's team, before it sends each document:

- key not seen → send to the asynchronous Journal Entry Post API and record "sent";
- key seen and posted → return the stored S/4 document number and send nothing;
- key seen and pending → wait for the asynchronous confirmation instead of resending;
- key seen with a different content digest → reject the batch as a contract violation and alert both owners.

The asynchronous API removes the timeout that left January's outcome unknown. Each document is confirmed or rejected individually, and the registry records which.

4. **A completion check against the manifest.** When every document is confirmed, the flow compares the posted documents, lines and debit and credit totals per currency with the manifest and writes the result to the delivery ledger. Only then is the batch marked complete. A mismatch raises an alert to the S/4 platform team within minutes, not to a controller on working day 3.

5. **Rejected documents go to a dead-letter store, never to a suspense account.** A document that S/4HANA rejects, for example because it references a blocked account, is held with its amount and age, visible in the delivery ledger. Jonas's team fixes the cause and the same key is replayed. The batch stays incomplete until the dead letters are resolved. The controller sees "€1.2 million of 12 March not yet posted, one document, rejected for a blocked account", not a balancing journal.

6. **Corrections as superseding runs.** If SFP finds an error in a posted batch, it does not resend. It produces `run-2` with `supersedes: run-1`; the flow first posts a reversal of every document in `run-1`, keyed by the original keys with a reversal suffix, and then posts `run-2`. Both runs stay visible in S/4HANA and in the registry.

7. **Daily subledger-to-GL reconciliation.** BlackLine compares SFP's control-account balances with S/4HANA's each day, per entity and currency, instead of once a month. It is the safety net for everything the first six steps miss: a manual journal on an SFP control account, a posting into a closed period, a document posted outside the gate.

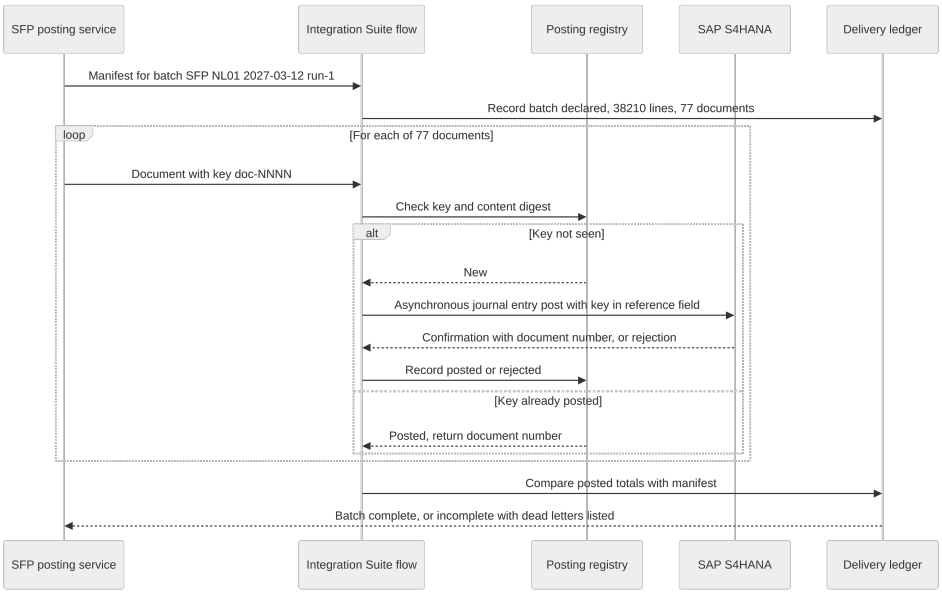


Figure 7: *The redesigned daily posting. The manifest declares what should arrive, the registry makes every resend harmless, and the delivery ledger compares what arrived with what was declared before anyone calls the batch complete.*

The January night, replayed under the new design

The redesign was tested by replaying the January failure in a test environment. The flow posts 46 of 77 documents and then loses its connection. The operator re-runs the batch, as before. The registry returns the stored document numbers for documents 1 to 46 and the flow sends only 47 to 77. The delivery ledger shows the batch complete at

38,210 lines with debit and credit totals equal to the manifest. Nothing was doubled, and nobody had to know which documents had posted.

A second test sends a hand-edited copy under the same `run-1` key; the registry rejects it on the content digest. A changed file with an old identity is exactly what an auditor wants an interface to refuse.

What it cost and what it bought

The redesign took about eleven engineer-weeks across three teams (illustrative). It made duplicate postings from retries structurally impossible through the gate, made partial loads visible within minutes, replaced "restart from the top" with "re-run; the registry decides", and gave Mei Tanaka's team an automated interface control with a complete population of batches and results (Controls by Design, Not by Audit Season).

It did not fix everything. The summary is still a summary: drill-back from an S/4HANA line to bookings still goes through SFP's batch index, and the convergence of the merchant subledger into the group core remains a 2029 program. The pattern, though, became the reference implementation of the **idempotent posting** standard Dana had proposed in November 2026, cataloged as STD-FIN-02 and enforced from 1 July 2027 (Principles That Decide Things). By the end of 2027 it applied to the Northmere Ads summary postings to S/4HANA and to Tablekin's journal loads into Workday.

Integration Governance: Catalog, Contracts, Versions and Retirement

Lucía's team built the inventory in six weeks from LeanIX, integration-platform configuration, Kafka topic metadata and interviews for the spreadsheets. Keeping it true is the governance problem, and four instruments do the work.

An interface catalog with finance fields. Every interface that changes a balance or feeds a key report is registered with more than its endpoints:

Field	Example for SFP-to-S/4HANA
Identifier and owner (sender side, receiver side)	INT-0412; Jonas Berg (sender), Sven Aalto (receiver)
Style and technology	Batch with manifest; SAP Integration Suite
Financial relevance	Posts to GL; in scope for internal control over financial reporting (ICFR); feeds key reconciliation R-117
Unit of completeness and control totals	Entity and posting date; lines, documents, debit and credit per currency
Idempotency key and retention	Batch and document key; registry retained 13 months
Cut-off rule	Amsterdam-time rule v2, owner Leila Haddad's team
Error path	Dead-letter store; suspense posting not permitted
Replay procedure	Re-run under same key; corrections as superseding runs
Contract and version	Data contract `sfp-daily-gl-summary` 3.1.0
Retirement	Planned replacement by canonical events into the group core, 2029

Contracts as the unit of change. A **data contract** is the agreed, versioned specification of what an interface carries, including completeness and semantics as well as schema (Finance Data Architecture: Who Owns the Truth shows SFP's posting contract in the Open Data Contract Standard format). A change to an interface that is not a change to its contract is not reviewable.

Versioning that follows the vendors' own discipline. The vendors' practices are a reasonable model. Workday's SOAP API introduces a new version only for breaking changes and lets the caller pin it. Stripe ships monthly non-breaking versions and twice-yearly breaking releases, with requests pinned by header. The same rules suit canonical events: additive changes in place, breaking changes as a new major version run in parallel, and consumers pinned and tested at upgrade.

Deprecation with a date and a census. An interface or contract version is deprecated with an end date, a list of its consumers taken from the catalog and from runtime traffic, and a named owner for each migration. Lucía's rule is that **nothing is switched off that still carries traffic, and nothing is left running after its date without a waiver that itself expires.** The expiring waiver comes from the standards process Dana set up in November 2026.

Failure Modes

Point-to-Point Spaghetti. Each of the 1,100 interfaces was the cheapest answer to one project's need. Lucía found booking cancellations represented five different ways across the three technologies. The signs are an inventory nobody trusts, failures found by downstream users, and replacement projects that discover their interface count only during testing. The remedy is not a big rewrite. Canonical events and a few governed patterns become mandatory for new interfaces, and old ones migrate when their systems change.

Silent partial loads. A batch loads 80% of its records and reports success because the loader counts only rejected records it saw, or because a file was truncated in transfer. Without a manifest, the receiver has no independent number to compare with. Watch for interfaces whose only completeness check is "the job finished". At Northmere, about 790 of 1,100 were in that state at the start of 2027.

Double posting on retry. The January incident is the textbook case: a timeout leaves the outcome unknown, a well-meaning retry restarts from the beginning, and a target with no duplicate check accepts everything. It is caused by random or attempt-based keys, by synchronous calls for bulk work, and by runbooks that say "re-run" without saying how the interface knows what already happened.

Time-zone cut-off errors. A payment at 23:30 in Lisbon on 31 March is 00:30 on 1 April in Amsterdam and 15:30 on 31 March in Workday's Pacific Time. Interfaces that assign periods by server time or arrival time put the same transaction in different months in different systems. Every close then opens with a "timing difference" that hides real breaks. The signs are recurring unexplained differences at month boundaries that "reverse next month". The remedy is a declared cut-off rule per source, with event time and time zone carried in the contract (Global by Design: Entities, Currencies, Jurisdictions and Brands takes up the multi-time-zone, multi-entity version).

Spreadsheet relays. The 70 spreadsheets-in-the-loop are interfaces with no control totals and a human as the retry mechanism, and often where mapping decisions are made invisibly. Each is a candidate for automation or, where the human judgment is real, for a workflow with an approval record.

The Chief Architect's View

Decide the guarantees, not the tools. The first standard I would issue: every interface that changes a balance declares its unit of completeness, control totals, idempotency key, cut-off rule, error path and replay procedure, and the receiver verifies them. Tool choices matter less than they seem.

Separate platform ownership from contract ownership. At Northmere, Platform Engineering runs Kafka and the SAP team runs Integration Suite, and both run them

well. Neither owns what a `PaymentSettled` event means, how complete a batch must be, or who clears a dead letter at month-end. Lucía's team owns the contracts and the standard; domain teams own their interfaces; Leila's team owns the cut-off rules. Write this down in the decision-rights table (The Chief Architect's Job: Making Many Systems Tell One Story), because each owner will otherwise assume one of the others has it.

Keep financial logic out of the pipes. The cheapest-looking place to put a mapping is an integration flow. It is also the place where the auditor is least likely to find it and the accountant least likely to review it. I treat any account derivation found in an integration platform as technical debt with a control risk attached.

Fund the boring parts centrally. The idempotency library, the manifest format, the posting registry and the delivery ledger benefit every team and are funded by none. Unfunded, every team rebuilds them slightly differently, which is how an estate gets 1,100 interfaces with 1,100 retry strategies.

Measure money in flight. A few metrics belong on Leila's and Dana's joint dashboard: value and age of dead letters per interface; share of financially relevant interfaces with verified control totals; batches complete by cut-off on working day 1; duplicate postings found by reconciliation (target zero); integration-related items in suspense. Each moves before the close does.

Watch the vendors' surfaces as part of the roadmap. API versions, event formats, throttling limits and extraction tools change with vendor releases. Oracle introduced a new extraction capability in 26A, Stripe publishes breaking versions twice a year, and SAP renames and repackages its integration products from time to time. The interface catalog should record which interfaces depend on which vendor surface version, so a release note becomes a query, not a surprise.

Questions to Ask

1. For this interface, what is the unit of completeness, and which control totals does the receiver check against an independent declaration from the sender?
2. What happens, step by step, if the sender retries after a timeout? Where is the idempotency key stored, how is it derived, and how long is it kept?
3. Which timestamp and which time zone decide the accounting period, who owns that rule, and what happens to a record that arrives after the period is locked?
4. Where do rejected records go, who is alerted, and can any rejected record end up in a suspense account?
5. How far back can we replay, and what prevents a replay from re-triggering payments or other external effects?
6. How will this interface behave on working day 1, at four times its normal volume, when the target system throttles?

7. Is any account derivation, summarization or tax logic implemented inside the integration platform? Who reviews it, and how is it tested?
8. Which vendor surface and version does this interface depend on, and what is the plan when that version is deprecated?
9. How much money is in flight on this interface right now, and how old is the oldest item?

Next: Many Entities, Currencies and Jurisdictions

This chapter treated an interface as a pipe between two systems. At Northmere, almost every pipe also crosses a boundary between legal entities, currencies or jurisdictions. The Lisbon booking's €600 moves from a Dutch e-money institution's safeguarding account to a Portuguese property, its commission settles intercompany, and its period depends on whose clock is read. The next chapter, Global by Design: Entities, Currencies, Jurisdictions and Brands, takes up the legal-entity, intercompany, multi-currency and multi-jurisdiction design that the integration layer has to carry correctly.

Sources and Evidence

Vendor documentation

- No Duplicate Invoice Check for API Posting (KBA 2817042), SAP — states that the Journal Entry Post APIs, synchronous and asynchronous, have no duplicate invoice check.
- Time out error occurs in Journal Entry – Post (Synchronous) API (KBA 2830584), SAP — advises the asynchronous API for large postings and notes the effect of document splitting on item counts.
- Extend SAP S/4HANA Cloud the right way, SAP News, 2025 — clean-core levels A to D; released APIs as Level A and BAPIs as Level B.
- File-Based Data Import (FBDI) process, Oracle — upload to UCM, load and import jobs.
- Upload transaction data for Accounting Hub and advanced features for importing transaction data, Oracle, 26C — Accounting Hub import mechanics and volume mechanisms with no published throughput figure.
- REST API for Oracle Fusion Cloud Financials and BICC large-scale extraction, Oracle — transactional and bulk outbound surfaces.
- Integration and Web Service Limits, Workday — throttling responses, RaaS quotas by tenant size, EIB concurrency, Pacific Time datetimes.
- Workday Web Services versioning, Workday — new versions only for breaking changes; caller-pinned versions.

- Handle webhook events and Troubleshoot webhooks, Adyen — 10-second acknowledgment, duplicates, ordering hints, retry schedule up to 30 days.
- Webhooks, Idempotent requests and API versioning, Stripe — retries up to three days, no ordering guarantee, idempotency-key behavior, version cadence.
- Introduction to Apache Kafka, Apache Software Foundation — ordering guaranteed within a topic-partition.
- Debezium FAQ and Outbox Event Router, Debezium project — at-least-once delivery after failures; outbox routing keyed by aggregate.

Standards and regulation

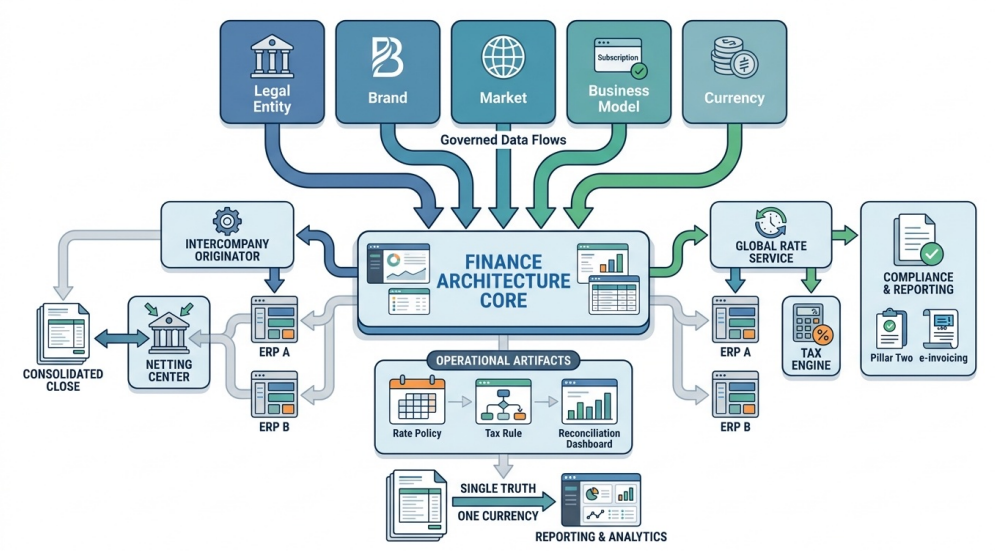
- Open Data Contract Standard, Bitol — the data contract format referenced for interface contracts.
- What is OpenTelemetry?, OpenTelemetry project — vendor-neutral traces, metrics and logs used to correlate hops.

Practitioner and research sources

- *Enterprise Integration Patterns*, Gregor Hohpe and Bobby Woolf, 2003 — the pattern language, including Idempotent Receiver and Dead Letter Channel.
- "Your Coffee Shop Doesn't Use Two-Phase Commit", Gregor Hohpe, *IEEE Software*, 2005 — write-off, retry and compensation, and the dependence of write-off on reconciliation.
- "Exactly-Once Semantics Are Possible: Here's How Kafka Does It", Neha Narkhede and Guozhang Wang, Confluent, 2017 — the scope of Kafka's exactly-once guarantee and its exclusion of external side effects.
- "Pattern: Transactional outbox", Chris Richardson — the outbox pattern and the duplicate-publication caveat.
- "Designing robust and predictable APIs with idempotency", Brandur Leach, Stripe, 2017 — idempotency keys and jittered exponential back-off.
- "Event Sourcing", Martin Fowler, 2005 — replay hazards with external systems.

Northmere is fictional, and its interfaces, incidents and figures are illustrations. The architectural recommendations are the author's judgment.

13. Global by Design: Entities, Currencies, Jurisdictions and Brands



IN THIS CHAPTER, YOU WILL: Learn how a multinational finance estate represents legal entities, intercompany trade, currencies, accounting standards and local tax and regulatory rules, and how to keep brand, entity, market and business model apart so that each can change without breaking the others.

WHY FINANCE CARES: The controller signs statutory accounts per legal entity, tax files per jurisdiction, and the group reports one consolidated number in one currency. When entity, currency and jurisdiction data are wrong at the source, each of those outputs becomes a reconciliation project at period end.

WHY ARCHITECTS SHOULD CARE: Entity structures, functional currencies and ledger designs are among the hardest ERP decisions to reverse. Intercompany, exchange rates and tax attributes cross every system boundary,

so each needs one owner, one source and one contract, or it becomes the estate's most expensive integration problem.

KEY POINTS:

- **Brand, legal entity, market, business model and currency are five dimensions, not one.** The legal entity is the unit of law, tax and statutory books, not the unit of the business. Model each dimension independently, master legal entities in one register, and derive every system's entity representation from it.
- **Intercompany is a product to design, not a by-product to clean up.** Every intercompany flow needs an agreement, a pricing rule, a counterparty on both sides, one exchange rate, a settlement route and an elimination rule. Most elimination differences found at close were designed in months earlier.
- **Currency and jurisdiction are governed data with owners and calendars.** One rate service with named rate types, one month-end currency calendar, and tax and regulatory attributes carried on every event, separate from the accounting treatment, turn mandates such as ViDA, DAC7 and Pillar Two into data work instead of re-platforming.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Doppelgänger Record** — legal entities, their currencies and their tax registrations are created separately in each ERP, the consolidation tool, the tax engine and the treasury system, with no agreed register.
- **Spreadsheet Glue** — transfer-pricing calculations, exchange-rate tables and intercompany mappings live in spreadsheets the close cannot run without.
- **The Permanent Exception** — each brand or new market gets its own entity conventions, rate sources and tax logic, and each becomes a permanent fork.

In a single-country company, brand, legal entity, market and currency are the same thing, so nothing forces anyone to tell them apart. A group that trades through dozens of entities in dozens of countries cannot. Its brands do not map one-to-one to entities, it earns in currencies other than the one that books the revenue, and it pays tax under rules that change every year. **Every one of those mismatches is represented somewhere in the systems, and where it is represented decides whether the close is a computation or an investigation.**

This chapter covers that representation: the **legal entity model** and how SAP S/4HANA, Oracle Fusion Cloud ERP and Workday encode it; **intercompany** trade, transfer pricing and netting; the **multi-currency model**; **multi-GAAP** and statutory reporting; local

requirements such as SAF-T, e-invoicing and fiscal numbering; indirect tax at platform scale; the data Pillar Two demands; and the constraints a licensed payments entity places on everything else. The argument throughout: **treat each of these as a governed dimension or a designed flow**, not as a property each system configures for itself.

Setting the Scene: Seventy-One Entities and Three Requests

Northmere, the fictional group this book follows, is a New York–listed digital commerce group headquartered in Amsterdam. It reports in US dollars under US GAAP, trades through **71 legal entities in 38 countries**, takes customer payments in **46 currencies**, settles with partners in **31 currencies**, and files local GAAP or IFRS statutory accounts for each entity. Meet Northmere: Five Brands, One Set of Books introduces its brands and its licensed e-money institution, **Northmere Payments B.V.**

In the first week of April 2027, three requests reach Dana Whitlock, the Chief Architect. Kofi Asante, Group Head of Tax and Treasury, needs a **Pillar Two dataset** per entity and jurisdiction, reconciled to consolidation; last year his team built it from 38 spreadsheet extracts. Leila Haddad, the Chief Accounting Officer, reports that at working day 4 of the March close, intercompany balances in **41 of about 140 active entity pairs** disagreed by a gross **$3.8 million**, mostly because the two sides had been converted at different rates, in different systems, on different days. And the Keystone design authority has put the **2028 Brazil launch** on its agenda. The commercial plan says "configure Stayloft for Brazil". Nobody has written down which entity will contract with Brazilian hosts, which ledger it will live in, or who sets its exchange rates.

The three requests share a root. Northmere has no single register of legal entities, no single source of exchange rates, and no consistent way of carrying entity, jurisdiction and currency from the brand platform to the tax return.

Five Dimensions That Must Not Collapse Into One

Most estates start from an implicit equation: one brand, one country, one company, one currency. It fails gradually. A brand enters a second country through the first country's company; a second brand borrows the first brand's company; a payments entity holds money for all brands; an acquisition brings an entity serving three markets. Each step is reasonable; together they leave systems keyed on a dimension that no longer means what the key assumes. The fix is to define five dimensions separately, each with an owner and a code list:

- **Legal entity** — a company with its own legal personality, statutory accounts, tax registrations and contracts. The unit of law and of the books.

- **Brand** — a commercial identity customers and partners recognize. The unit of product and, often, management reporting.
- **Market** — where the customer or the supplied service is located. The unit of tax jurisdiction, consumer law and local payment methods.
- **Business model** — agency, merchant, subscription, usage, advertising, payments facilitation. The unit that decides posting rules (see Accounting for Architects: Ledgers, Journals and Revenue).
- **Currency** — the denomination of an amount. One transaction can carry four currencies, as shown below.

If you treat…	…as the same as…	What breaks
Brand	Legal entity	A second brand in an existing entity has no home for its P&L; a brand carve-out cannot find its own assets and contracts.
Market	Legal entity	Cross-border sales are taxed by the entity's country instead of the customer's; DAC7 and digital services tax reports, which are by user or property location, cannot be produced.
Market	Currency	A traveler in Germany paying in dollars breaks the assumption; nobody can say which rate applied to which amount.
Business model	Brand	A brand that runs agency and merchant bookings, as Stayloft does, gets one set of posting rules for two accounting patterns.
Legal entity	Ledger	One ledger per entity multiplies configuration and close effort; a shared ledger without an entity balancing segment cannot produce a balance sheet per entity.

At Northmere the matrix is real. Stayloft trades through nine operating entities, but its contracts with European hosts are signed by **Stayloft B.V.**, its US marketplace by **Stayloft Inc.**, and merchant-booking funds are held by Northmere Payments B.V., which serves no brand of its own. Tablekin has four entities but bills venues in 11 countries. "Revenue by country" means three different things depending on whether the entity, the customer or the property is meant.

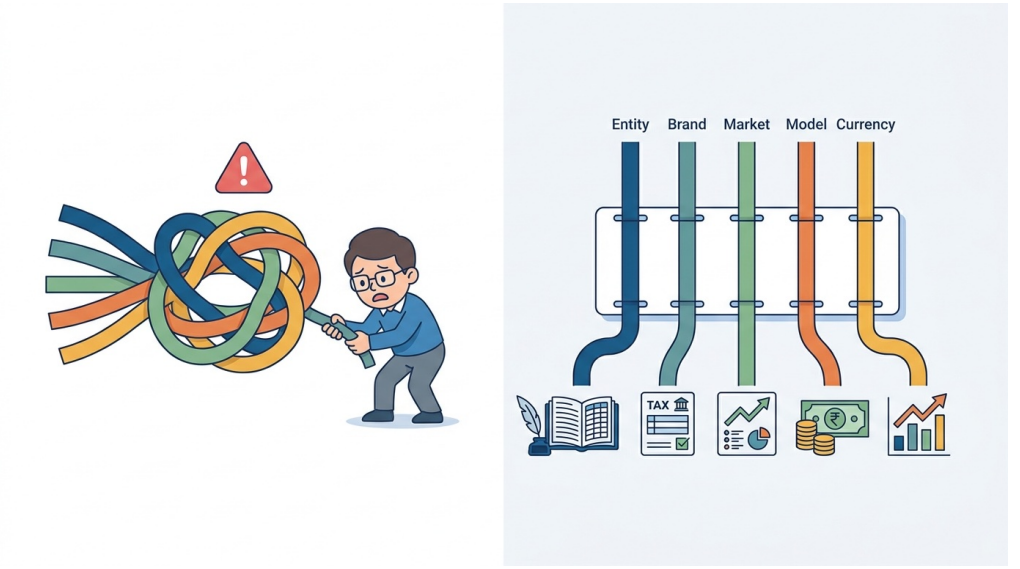


Figure 1: *Brand, legal entity, market, business model and currency are five dimensions, not one: every financial event carries all five explicitly, so each can change without breaking the others.*

The rule I use: every financial event carries all five dimensions explicitly, and no brand platform derives one from another in code. The derivation rules (which entity contracts for which brand in which market under which model) belong in governed configuration, managed by the onboarding kit in The Next Brand Should Be Boring: Onboarding as Configuration. Northmere's canonical event envelope (Canonical Events: A Common Language for Money) already has separate `brand`, `legal_entity`, `business_model` and `tax_jurisdiction` fields, with the currency inside every `amount`. This chapter is about making sure the values are right.

The Legal Entity Model

A **legal entity model** is the group's authoritative description of its companies: which exist, what each does, who owns it, where it is resident, which currency and accounting standards apply, which ledgers it books in, and how it is registered for tax and regulation. It is master data in the sense of Finance Data Architecture: Who Owns the Truth, and the most cross-cutting master data a finance estate has.

Attribute group	Examples	Owner
Identity and ownership	Legal name, registration number, lifecycle status, shareholdings and their effective dates, ultimate parent	Legal
Accounting	Functional currency and its evidence, statutory GAAP, fiscal calendar, consolidation method	Group controller
Tax and regulatory	Tax residence, VAT and state registrations, Pillar Two classification, licenses and safeguarding duties	Tax; compliance
Banking	Bank accounts, cash-pool and netting participation	Treasury
System representation	SAP company code, Oracle legal entity and balancing segment value, Workday company, OneStream entity member, tax-engine company	Finance systems

Two attributes deserve emphasis. **Functional currency**, the currency of the entity's primary economic environment, is determined by economic facts under IAS 21 and its US GAAP counterpart ASC 830; it is not a configuration choice. **System representation** is the attribute most estates never record. Without it, every interface that crosses an ERP boundary carries its own mapping table, the classic home of Spreadsheet Glue.

Northmere's 71 entities fall into five types, and the type decides most of the design: 6 **holding and financing** entities; 24 **brand operating** entities that contract with customers and partners; 36 **local service** entities (marketing support, customer service, engineering hubs, finance shared services) paid cost plus a markup and never selling to third parties; 3 **intellectual property owners**; and 2 **licensed payments** entities. The distinction between service and operating entity matters. São Paulo is already a Northmere hub, through an engineering service entity. "Using the Brazilian company we already have" for the 2028 launch would turn a cost-plus service provider into a company contracting with hosts and travelers, changing its tax profile, transfer-pricing position and regulatory obligations at a stroke. Recording the type makes that a visible decision rather than a configuration shortcut.

In early 2027 Northmere creates entities in SAP S/4HANA, Oracle Fusion, Workday, OneStream, Vertex, Kyriba and Workday HCM, each from a different document. That is **The Doppelgänger Record** in its purest form, and the reason Kofi needed 38 extracts. Keystone's response is a **legal entity register** as golden source, owned by Legal for

identity and ownership and stewarded by Leila's team for accounting attributes. The register does not replace ERP configuration; it decides it.

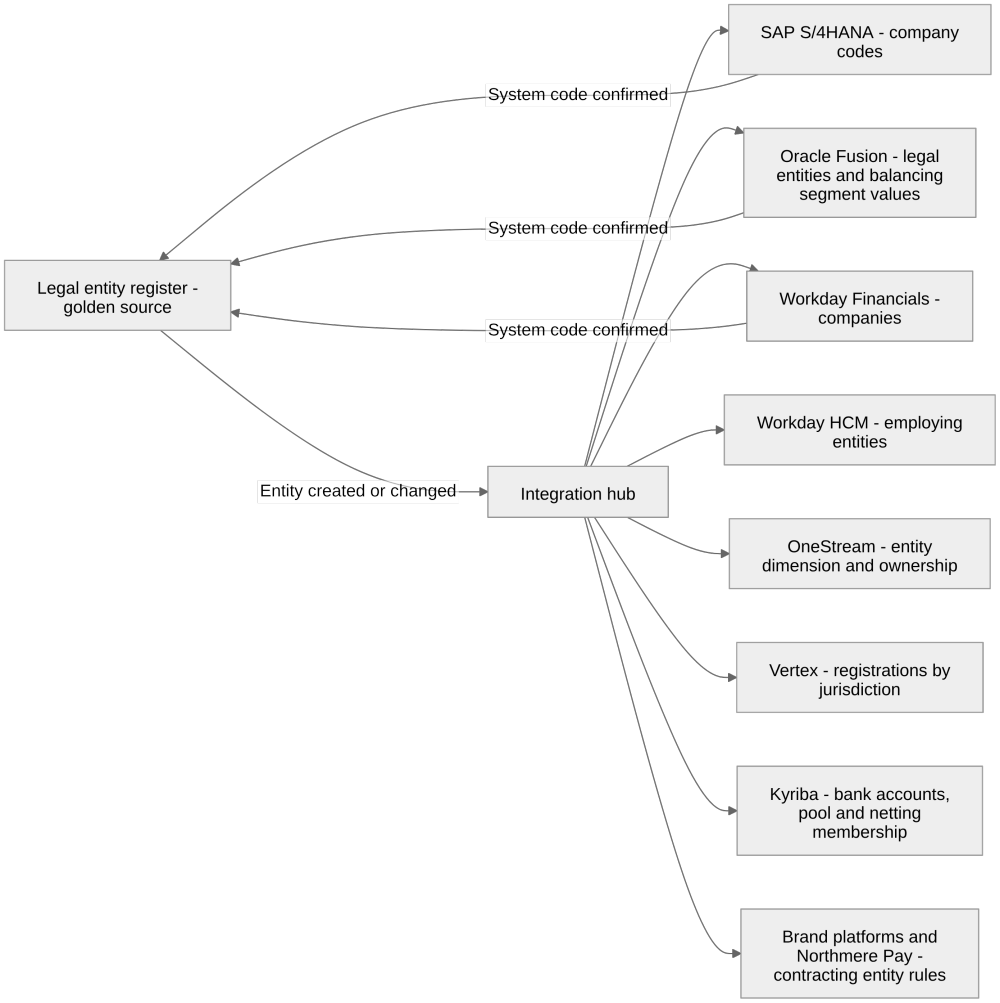


Figure 2: *The legal entity register decides how every system represents an entity, and each system confirms its code back, so the register holds the full cross-system mapping that interfaces, consolidation and tax reporting rely on.*

Creating an entity in an ERP is configuration under change management; the hub publishes a change request to the platform team, and the confirmation closes the loop. What matters is not automation but **authority**: no system creates an entity the register does not know, and the cross-system mapping is maintained in one place.

How Each ERP Represents a Legal Entity

The three ERPs encode the legal entity differently. The detail belongs to SAP S/4HANA: One Journal, Clean Core, Oracle Fusion: An Accounting Hub for High-Volume Events and Workday: The Org Chart Behind the Ledger; the comparison below is what an architect needs when designing across them.

Concern	SAP S/4HANA	Oracle Fusion Cloud ERP	Workday Financial Management
Entity object	**Company code**, "an independent legal accounting entity… with independent accounts"	**Legal entity**, assigned to a ledger, with **primary balancing segment** values assigned to it	**Company**, the primary balancing worktag
Ledgers	All company codes in the **leading ledger (0L)**; further standard ledgers and delta-only **extension ledgers**	A **primary ledger** combines chart of accounts, calendar, currency and accounting method; many legal entities can share one	One structure; **book codes** separate book-specific entries
Currencies	Document, company code, global, plus up to eight freely defined currencies per ledger	Entered and ledger currency; **reporting currencies** at balance, journal or subledger level	Transaction and company currency; translation for consolidation
Multi-GAAP	Parallel ledgers; extension ledgers for adjustments	**Secondary ledgers** at a chosen data conversion level	Multibook

Concern	SAP S/4HANA	Oracle Fusion Cloud ERP	Workday Financial Management
Intercompany	Cross-company-code postings with automatic clearing lines; intercompany billing and reconciliation	Intercompany balancing rules and module (detail to verify per release)	**Intercompany profiles** define which companies may transact; eliminations when both are in the reporting hierarchy
Irreversibility	Currency, fiscal-year variant and chart assignments constrain controlling-area design	Chart, calendar and currency are fixed once a ledger is saved	Company hierarchy and worktag design underpin every report

Sources: SAP Learning on organizational units, ledgers and currency types; Oracle's ledgers, legal entities and balancing segments (26C); Workday's Financial Management datasheet and consolidations and multicurrency guide. Oracle's intercompany functions and SAP's intercompany app names were not verified for this edition and are described at capability level only.

Three consequences follow. **First, "one ledger per entity" is a modeling choice, not a law.** Oracle recommends limiting primary ledgers and distinguishing entities by balancing segment; SAP puts every company code in the same leading ledger; Workday's company is a worktag. Entities that share currency, calendar and chart can share a ledger. **Second, the balancing unit is the legal entity.** Whatever the product, the system must produce a balanced trial balance per entity, because statutory accounts and tax returns are built from it. **Third, each ERP is internally consistent and externally blind.** Northmere's 55 SAP company codes, 12 Oracle legal entities (in three primary ledgers, one per functional currency) and 4 Workday companies meet only in OneStream. That federated design is legitimate for now, and the convergence question belongs to Converge or Coexist: Living with More Than One ERP; but it means no ERP can see a cross-ERP intercompany flow, so those flows must be designed to meet cleanly at consolidation.

Intercompany by Design

Intercompany transactions are trades between entities of the same group: royalties, service fees, cost recharges, loans and interest, dividends. Each side records its half; consolidation **eliminates** the matched balances, income and expense so that the group presents only third-party dealings. Accounting for Architects: Ledgers, Journals and Revenue covers the accounting; this section covers the architecture.

Northmere's flows follow from how it is legally organized and taxed:

Flow	From → to	Basis
Brand IP royalty	Stayloft's 8 other operating entities → Stayloft B.V., which owns the Stayloft trademarks	Percentage of the payer's net revenue, monthly
Payment service fees	Brand operating entities → Northmere Payments B.V. and Northmere Payments UK Ltd	Payment-provider costs passed through, plus a service margin
Shared services recharges	Operating entities → service entities (finance shared services in Dublin, engineering in São Paulo and Singapore)	Cost plus markup; allocation keys from Workday HCM headcount

Treasury flows (cash pool, intra-group loans, dividends) run in Kyriba on top of these.

Transfer pricing decides the prices. Tax authorities require intercompany prices to follow the **arm's length principle**, the price unrelated parties would agree in comparable circumstances, as set out in the OECD Transfer Pricing Guidelines and national law. Setting the rates is a tax and legal judgment; the rates here are illustrative. Where the pricing rules run, what data they read, and how their results reach both ledgers at the same amount, period and rate, is the architecture decision. Every flow needs six things:

1. **An agreement**, referenced on every transaction it generates.
2. **A pricing rule** that runs in one place on governed inputs. At Northmere today, royalty bases are pulled from three ERPs into a spreadsheet, multiplied and keyed back: Spreadsheet Glue on a line that moves tens of millions of dollars a month.
3. **A counterparty dimension on both sides**, set at posting time, or matching becomes a manual exercise.

4. **One transaction currency and one rate**, from the same source on the same date for both sides. Most of Leila's $3.8 million came from violating this.
5. **A settlement route**: invoiced and paid, netted, or booked to an in-house bank account.
6. **An elimination rule**, defined when the flow is designed, not met for the first time at quarter-end.

Invoice-based intercompany runs through receivables and payables with a legal invoice, which is required wherever the charge is a taxable supply, as most cross-border services between EU entities are for VAT. **Journal-based** intercompany posts paired journals with due-to and due-from lines, common for allocations. Within one ERP both are well supported. Across ERPs, the estate needs an **intercompany originator**: one component that creates both halves from one calculation with a shared transaction identifier. It can be a close platform (BlackLine markets an Intercompany Create product), an ERP acting as hub, or a small integration service; which one is a question for Build, Buy or Compose: Choosing What to Own. The requirement is that **exactly one component creates both sides**.

Matching pairs the halves by counterparty and transaction identifier and routes differences to a dispute workflow; Northmere matches today in OneStream, in Oracle Fusion for Wayfare and by email, and Keystone's 2027 target is one matcher fed continuously during the month. **Netting** settles many intercompany positions with one net payment per entity per cycle: a netting center collects positions in their transaction currencies, converts them at one cycle rate, and instructs one payment to or from each participant. It cuts payment volume, bank fees and FX conversions, but some countries restrict netting or require central-bank reporting of it, and licensed entities may be excluded. Northmere runs netting monthly in Kyriba; BlackLine also offers intercompany netting, an entry for the overlap register in Duplicates and Dependencies: The Hidden Costs of a Grown Estate.

Elimination happens in the consolidation layer, OneStream at Northmere, using the counterparty dimension to pair the sides (OneStream: Consolidating Actuals, Planning the Future covers the mechanics). It works cleanly only if both halves arrive with the same counterparty codes and, after translation, the same amounts. **Any difference that survives elimination must go to a defined account under a documented rule**, never to a plug someone clears by hand. The worked example shows where such differences come from.

Worked Example: A Royalty Crosses Two Currencies

The figures are fictional and rounded to two decimals at each step; rates are US dollars per euro.

The facts. Stayloft Inc. (US dollar functional currency) licenses the Stayloft brand and platform from Stayloft B.V. (euro functional currency) for a royalty of **6% of net revenue**, invoiced monthly in US dollars. March 2027 net revenue is **$25,000,000**, so the royalty is **$1,500,000**, invoiced on 31 March. Both entities book in their own S/4HANA company codes. Rates from the group rate service: 31 March closing **1.0900**; March average **1.0860**; 15 April netting cycle **1.0760**.

Step 1: each side books its half (31 March)

Entity	Account	Debit	Credit	Note
Stayloft Inc. (USD)	Royalty expense – intercompany, counterparty Stayloft B.V.	1,500,000.00		Created by the intercompany originator
Stayloft Inc. (USD)	Intercompany payable – Stayloft B.V.		1,500,000.00	No FX in this entity
Stayloft B.V. (EUR)	Intercompany receivable – Stayloft Inc.	1,376,146.79		USD 1,500,000.00 ÷ 1.0900; USD amount and rate ID stored on the line
Stayloft B.V. (EUR)	Royalty income – intercompany, counterparty Stayloft Inc.		1,376,146.79	Same agreement and transaction ID

The FX exposure sits in exactly one entity, Stayloft B.V., which holds a dollar receivable. The invoice currency decides which entity carries the risk, and that choice belongs in the license agreement, not in whichever system issued the invoice.

Step 2: consolidation in US dollars (March close)

OneStream translates Stayloft B.V. into dollars: balance sheet at the closing rate, income at the average rate.

Item	EUR	Rate	USD
B.V. intercompany receivable	1,376,146.79	1.0900 closing	1,500,000.00
B.V. royalty income	1,376,146.79	1.0860 average	1,494,495.41
Inc. payable and expense	—	—	1,500,000.00 each

Entity	Account	Debit	Credit	Note
Consolidation	Intercompany payable (Inc.)	1,500,000.00		
Consolidation	Intercompany receivable (B.V., translated)		1,500,000.00	Balance sheet eliminates exactly
Consolidation	Royalty income – intercompany (B.V., translated)	1,494,495.41		
Consolidation	Intercompany elimination difference – translation	5,504.59		Routed by rule to a designated account
Consolidation	Royalty expense – intercompany (Inc.)		1,500,000.00	

Neither entity made a mistake: **the $5,504.59 is produced by the translation policy**, because a transaction dated on the last day of the month is translated at a monthly average. There are two clean designs. Translate large discrete intercompany flows at their transaction-date rate, which IAS 21 permits (an average is only a practical approximation of transaction-date rates), so they eliminate exactly; or keep average-rate translation and route the difference by rule to a reviewed translation account. Discovering it at working day 4 and clearing it to suspense is not a design. Leila's team

chose the first option for royalties and service fees from Q2 2027 and the second for high-volume recharges.

Step 3: settlement through netting (15 April)

The April cycle includes three March invoices, converted to the euro netting base at 1.0760:

Invoice	Payer → receiver	Amount	EUR at 1.0760
Royalty	Stayloft Inc. → Stayloft B.V.	USD 1,500,000.00	1,394,052.04
Finance shared services	Stayloft Inc. → Northmere Services Ireland Ltd	USD 380,000.00	353,159.85
Payment service fees	Stayloft B.V. → Northmere Payments B.V.	EUR 2,350,000.00	2,350,000.00

Participant	Net position (EUR)	Settlement
Stayloft Inc.	−1,747,211.89	Pays USD 1,880,000.00, the only currency conversion
Stayloft B.V.	−955,947.96	Pays EUR 955,947.96
Northmere Payments B.V.	+2,350,000.00	Receives into its own-funds operating account
Northmere Services Ireland Ltd	+353,159.85	Receives EUR 353,159.85

Gross invoices of EUR 4,097,211.89 settle with EUR 2,703,159.85 of net movements and one conversion instead of two. Settlement realizes the exchange difference in the entity that carried it:

Entity	Account	Debit	Credit	Note
Stayloft B.V. (EUR)	Netting center clearing	1,394,052.04		USD 1,500,000.00 ÷ 1.0760
Stayloft B.V. (EUR)	Intercompany receivable – Stayloft Inc.		1,376,146.79	Clears the original amount
Stayloft B.V. (EUR)	Realized FX gain		17,905.25	The dollar strengthened after invoicing
Stayloft Inc. (USD)	Intercompany payable – Stayloft B.V.	1,500,000.00		
Stayloft Inc. (USD)	Netting center clearing		1,500,000.00	

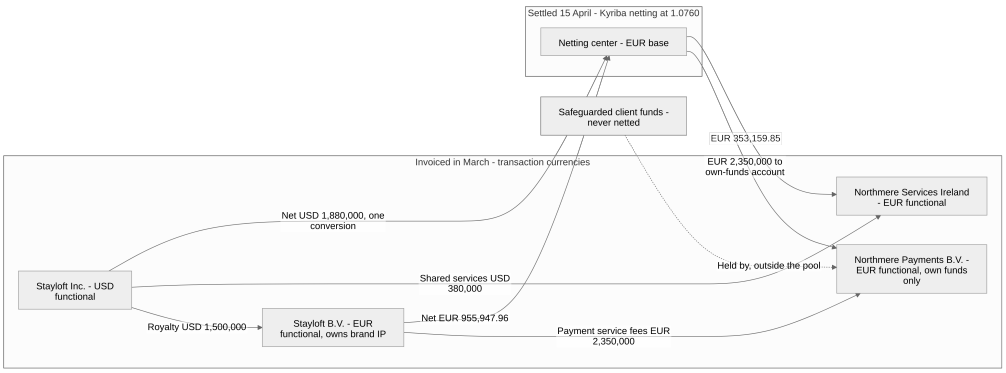


Figure 3: *Three intercompany invoices in two currencies settle as four net movements with one currency conversion; the licensed entity takes part only with its own funds, never with safeguarded client money.*

The example works only because one originator created both halves from governed data, both sides used one rate, every line carried a counterparty, the translation rule was decided in advance, and the netting center knew which of Northmere Payments B.V.'s balances it could touch. **Remove any one and the $5,504.59 becomes one of forty-one breaks on Leila's working-day-4 list**.

The Multi-Currency Model

A multi-currency estate needs an explicit model of which currency an amount is in, which rate converted it, who decided that rate, and when. Most estates have the first;

few have the rest.

Role	Definition	Northmere example
Transaction currency	The currency a transaction is denominated in	A traveler's payment in Swedish kronor
Settlement currency	The currency a payment provider or bank settles in	Adyen settles to Northmere Payments B.V. in euros
Functional currency	The currency of the entity's primary economic environment (IAS 21, ASC 830)	Euro for most EU entities; dollar for Stayloft Inc.
Group currency	The presentation currency of the consolidated statements	US dollar

Northmere's 71 entities use **19 functional currencies**. Three families of rates must be kept apart. **Commercial rates** convert customer prices: when an American traveler pays in dollars for the €600.00 Lisbon apartment from the book's recurring example, Stayloft quotes **$654.00**, a rate of 1.0900 against a market rate of 1.0850 (at which €600.00 is $651.00); the **$3.00** margin is a commercial decision owned by the brand and Northmere Pay. **Provider rates** are applied by payment providers and banks when they convert at settlement; they are reconciled from settlement reports, not assumed (see Payments-to-Ledger: Draining the Suspense Swamp). **Accounting rates** are what the ledgers use: **daily spot** for initial recognition, **period closing** for revaluation and balance-sheet translation, **period average** for income translation, and **budget** and **forecast** rates for planning.

Ownership follows the families. Treasury owns market-rate sourcing: provider, fixing time, publication schedule and feed controls. The group controller owns **rate policy**: which rate type applies to which process, how averages are calculated, how rates are estimated for currencies that cannot be exchanged (IAS 21's *Lack of Exchangeability* amendments apply from 2025), and how hyperinflationary economies are handled. Brands own commercial rates within a group margin policy. Finance systems own **distribution**. At Northmere in early 2027, S/4HANA, Oracle Fusion, Workday, OneStream and the Stayloft Finance Platform each take rates from a different source, which is the structural cause of Leila's breaks. Keystone's 2027 standard is a **single group rate service**, fed by treasury from one market-data source, publishing each rate type with an identifier, effective date and version to every ledger, subledger and the consolidation engine. Every converted amount stores the rate identifier it used, so a difference can be

traced to its cause rather than argued about.

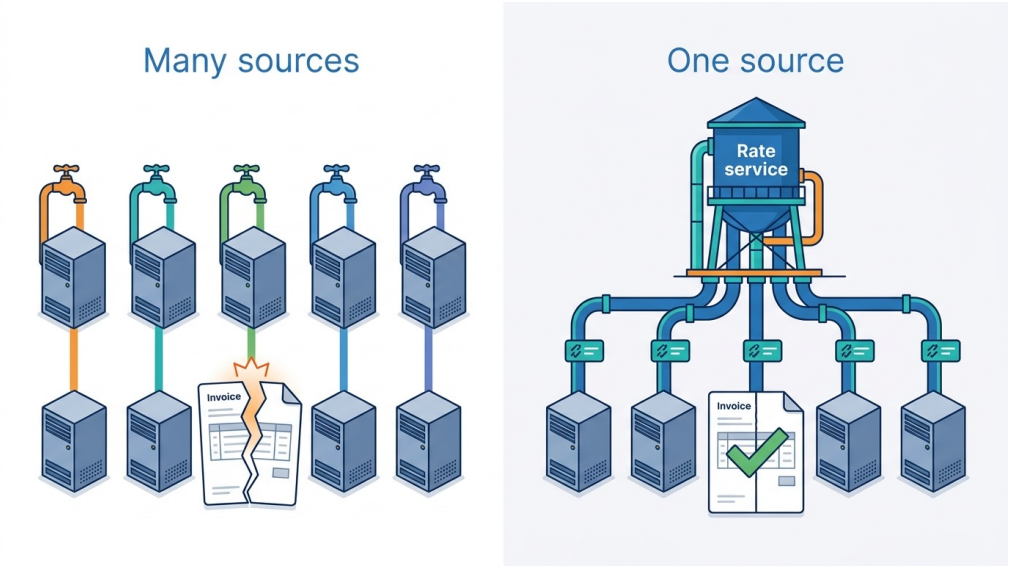


Figure 4: *When each ledger takes rates from its own source, the two halves of an intercompany trade are converted differently and break at close; one treasury-sourced rate service, with a rate identifier on every converted amount, lets them meet.*

Two month-end processes are routinely conflated. **Revaluation** restates foreign-currency **monetary items** (cash, receivables, payables, loans) at the closing rate in each entity's books, with the difference in profit or loss; it runs **in each ledger**. **Translation** converts each entity's functional-currency statements into the group currency (assets and liabilities at closing, income and expense at transaction-date or average rates, equity at historical rates), with the balancing difference in other comprehensive income as the **cumulative translation adjustment** (CTA); it runs **once, in consolidation**. The order matters.

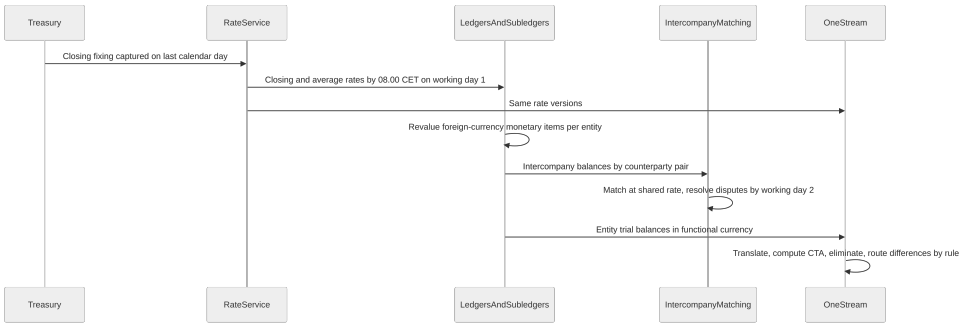


Figure 5: *The month-end currency calendar: one rate publication feeds every ledger and the consolidation engine, revaluation runs per entity before matching, and translation*

runs once, centrally, after it.

One consequence is easy to miss: **a subledger that holds foreign-currency open items must revalue them, or hand them to a ledger that will**. The Stayloft Finance Platform holds partner payables in 31 settlement currencies inside euro entities. If it posts only daily euro summaries to S/4HANA, those payables are never revalued unless the platform does it or posts open items in transaction currency. Revaluation responsibility must be assigned per open-item population, explicitly (Never Update, Only Append: Designing Custom Ledgers covers how a custom ledger carries currency).

Rounding causes a steady trickle of breaks unless three things are written down. Precision per currency: ISO 4217 minor units differ (two decimals for the euro, none for the yen, three for the Kuwaiti dinar), and money is never stored as binary floating point. Where conversion rounds: converting each line and summing differs from summing and converting once, and the subledger and the ledger receiving its summaries must use the same rule or their reconciliation never ties to the cent. Where residue goes: a designated rounding account with a tolerance, or rounding becomes suspense.

Many GAAPs, One Set of Events

Northmere reports to the SEC under US GAAP and files statutory accounts under local GAAP or IFRS. **Multi-GAAP** is accounting for the same events under more than one standard at once. The differences touch a minority of transactions but can be material: lease accounting (ASC 842 keeps a lessee distinction between finance and operating leases that IFRS 16 does not), some capitalization policies, and presentation, which diverges further as IFRS 18 replaces IAS 1 for annual periods from 1 January 2027.

Pattern	How it works	Strength	Cost
Parallel ledgers	Every event posts to a ledger per standard (SAP parallel ledgers, Oracle journal- or subledger-level secondary ledgers, Workday book codes)	Full trail per standard	Volume and configuration multiply; Oracle warns journal- and subledger-level secondary ledgers are much more expensive than balance-level ones

Pattern	How it works	Strength	Cost
Adjustment layer	The leading ledger follows one standard; differences post to a delta layer (SAP extension ledgers, Oracle adjustments-only secondary ledgers)	Low volume; explicit list of differences	Works only where differences are few and well understood
Consolidation-level adjustment	Entities book local GAAP; group adjustments are made in the consolidation tool	Simple for entities	Group-GAAP numbers exist only at consolidation, with weak drill-back: effectively a shadow ledger

The hardest decision to reverse is **which standard the leading or primary ledger follows**. Northmere's S/4HANA has used US GAAP as its leading ledger since 2021, because the group close is the binding deadline, with local GAAP or IFRS in a parallel ledger for statutory accounts (SAP S/4HANA: One Journal, Clean Core). Wayfare's Fusion ledgers are the reverse, inherited from before the acquisition: IFRS primary ledgers, with US GAAP differences in Adjustments Only secondary ledgers added in 2025 (Oracle Fusion: An Accounting Hub for High-Volume Events). That inconsistency is tolerable while the ERPs are federated and is a question for Converge or Coexist: Living with More Than One ERP. **The rule I recommend: systematic GAAP differences are posting rules, not people.** A lease or a capitalization policy belongs in the accounting engine as a per-ledger rule, which is what Oracle's per-ledger accounting methods and SAP's ledger-specific postings exist for; manual GAAP adjustments are listed, owned exceptions.

Local Requirements That Reach Into the Ledger

Statutory and tax rules increasingly demand **data in a prescribed form, on demand or in near real time**. Each is a country variation on a few patterns.

Standard audit files. **SAF-T** (Standard Audit File for Tax), an OECD-originated XML format, is mandated in national variants in several countries (commonly cited: Portugal, Norway, Poland, Romania, Lithuania; check per country). France requires a standard ledger export, the FEC (*fichier des écritures comptables*), on tax audit. These are extracts

from the ledger and subledgers mapped to standard account lists, and they fail on missing master data and local mappings kept in spreadsheets.

E-invoicing and continuous transaction controls. Invoices are moving from documents audited later (**post-audit**) to structured data a government or accredited platform must accept before the invoice is valid (**clearance**). Italy and Poland run clearance systems; Belgium mandated domestic B2B e-invoicing over Peppol from 1 January 2026; France's receiving obligation and its issuing obligation for large and mid-sized companies began on 1 September 2026; Germany phases in issuing in 2027 and 2028. These dates, reported by law firms and tax-technology vendors, keep moving; Around the Core: Where Specialist Platforms Earn Their Place covers the architecture. For this chapter the point is that **invoice issuance joins the critical path of order-to-cash and procure-to-pay in each affected entity**, with rejections, outages and clearance latency to handle. Stayloft's commission invoices to Belgian and Polish hosts are already in scope.

Fiscal numbering. EU VAT rules require "a sequential number, based on one or more series, which uniquely identifies the invoice" (VAT Directive, Article 226), and several countries add rules on series and gaps. A sequence is a per-entity, per-series property; a brand platform issuing invoices for several entities from one global counter creates findings in every country that checks.

Retention and residency. Retention varies by country, record type and sector, usually counted from fiscal year-end; Germany, for example, cut voucher retention to eight years from 2025 while keeping ten for books. GDPR restricts transfers of personal data outside the EU rather than mandating EU storage, but some tax laws require books to be kept in, or accessible from, the country, and supervisors add requirements for licensed entities. The pattern: append-only ledgers with reversal-based correction (Controls by Design, Not by Audit Season), minimal personal data in ledgers, and a per-entity record of where its books may legally live.

Indirect Tax at Platform Scale

Indirect tax is where the five dimensions meet most visibly: the treatment depends on the supplying entity, the market, the business model and the counterparty's status. A marketplace adds a twist: it is party to supplies it does not itself make. The central idea is that **accounting, tax and regulatory treatments of the same transaction are separate determinations** and must be carried as separate attributes. Take the Lisbon booking: a German traveler pays Stayloft €600.00 for a Lisbon apartment under the merchant model, with a 15% commission.

Question	Rulebook	Answer (simplified)	Where it lives
How is revenue presented?	ASC 606 / IFRS 15, principal versus agent	Agent: €90.00 commission, net	Accounting engine posting rules
What VAT applies?	EU VAT; from July 2028, ViDA platform rules	Today VAT on the commission (place-of-supply rules for accommodation-related intermediation are a recurring dispute, so the rule belongs in the tax engine). From July 2028, if the host is not VAT-registered, possibly VAT on the full price as deemed supplier	Tax engine, with the determination stored
What is reported about the host?	DAC7	Host identity and tax number, consideration credited and fees withheld, by quarter	Seller master and transaction ledger
Are the funds regulated?	E-money safeguarding	€600.00 is client money at Northmere Payments B.V. until paid out or earned	Payments ledger and safeguarding reconciliation

None of these answers determines another. ViDA's **deemed-supplier** status does not make Stayloft a principal for revenue; holding the cash does not make it the supplier of the room. A billing engine that switches to gross revenue when it starts charging VAT on the full price will misstate the accounts the day the tax rule changes.

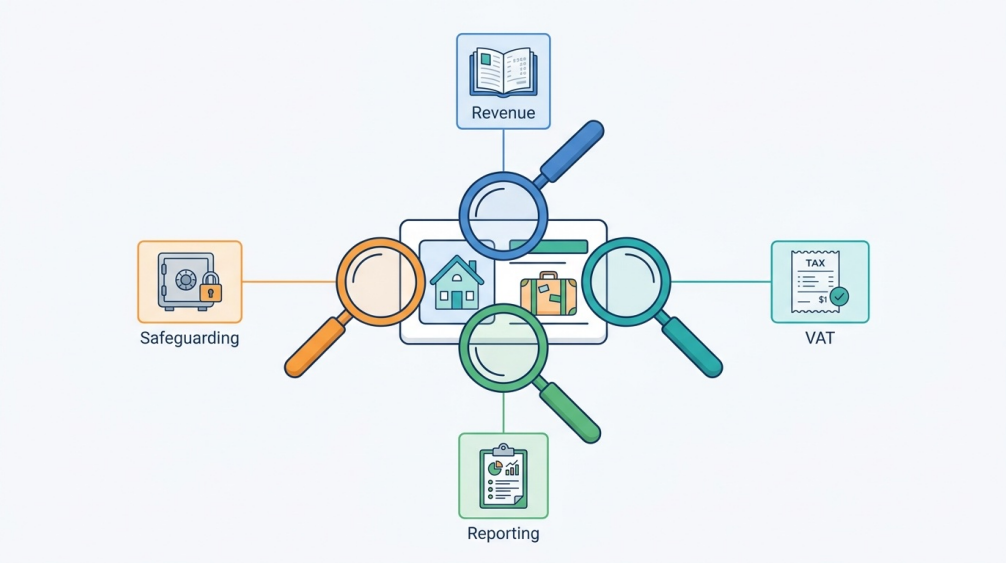


Figure 6: *One booking, four independent determinations: revenue presentation, VAT, seller reporting and safeguarding are carried as separate attributes, so a change in one rulebook never silently changes another.*

ViDA. The EU's VAT in the Digital Age package entered into force on 14 April 2025 (European Commission). From **1 July 2028**, platforms facilitating **short-term accommodation rental** and **passenger transport by road** become deemed suppliers where the provider does not charge VAT, with member states allowed to delay until **1 January 2030**; the platform is relieved where the provider gives a valid VAT number and declares it will account for the VAT. Single VAT registration measures start the same day, and intra-EU B2B e-invoicing with digital reporting follows on **1 July 2030**. For Stayloft (and possibly Wayfare's road transfers), the host's VAT status and declaration become **tax-determining master data with an effective date**, and the rule must be configured by country and date, because member states may start at different times.

DAC7 and digital services taxes. DAC7 (Council Directive (EU) 2021/514) requires platform operators to report seller information, including hosts renting property, annually by 31 January for activity from 2023; similar OECD-model rules apply in the UK. DAC7 and ViDA describe the same hosts and transactions, so **both must come from one seller master and one transaction ledger**, reconciled to the books before filing. **Digital services taxes** are unilateral levies on revenue from services such as online intermediation or advertising, attributed to users in the taxing country; scope and rates differ and move with trade policy (Canada rescinded its DST in June 2025). Stayloft and Northmere Ads are both potentially in scope, and the data they need, **in-scope revenue by user location**, cannot be reconstructed from entity or currency.

US sales tax. Since *South Dakota v. Wayfair* (2018), states may impose collection duties on economic presence, and states with a sales tax have adopted **marketplace facilitator**

laws that make marketplaces collect and remit for third-party sellers. Thresholds and scope differ by state, and lodging and occupancy taxes often follow separate rules, frequently at city or county level. For Stayloft Inc. this means determination at the property's address across thousands of jurisdictions and remittance on behalf of hosts, distinct from revenue recognition.

The response to all of it is Keystone's year-3 target: **tax determination and e-invoicing as shared services**. One engine called at every taxable document event by every brand and ERP; tax master data mastered once; every determination stored with its inputs, rule version and result, because tax disputes routinely outlast the systems that created the data; and tax accounts reconciled to the engine before any return is filed. Northmere's split between Vertex and Oracle's tax engine for some Wayfare markets is Capability Creep that Around the Core: Where Specialist Platforms Earn Their Place resolves.

Pillar Two: The Group's Data Model Becomes a Tax Return

The OECD's **Pillar Two** (GloBE) rules apply to groups with consolidated revenue of at least €750 million in two of the four preceding years. They compute an **effective tax rate per jurisdiction** and collect top-up tax to 15%, through a qualified domestic minimum top-up tax in the source country, an income inclusion rule at the parent, or an undertaxed profits rule as backstop. Northmere, at $8.2 billion of revenue, is in scope. The first GloBE Information Returns for 2024 fell due on **30 June 2026** in many jurisdictions (Ireland's Revenue confirms the date). A January 2026 "side-by-side" package, as widely reported, relieves US-parented groups from the parent-level rules from 2026 while leaving domestic top-up taxes in place; it is unlikely to help Northmere, whose ultimate parent, Northmere Group N.V., is Dutch (illustrative), and the data obligations remain either way. All of this is time-sensitive as of October 2026.

For the architect, what matters is the **data shape**: per constituent entity, grouped into 38 jurisdictions, accounting income **as used for consolidation**, **before intercompany eliminations**; current and deferred taxes; **payroll** and **tangible assets** for the substance-based exclusion; intercompany data; and lineage for every adjustment, retained for years.

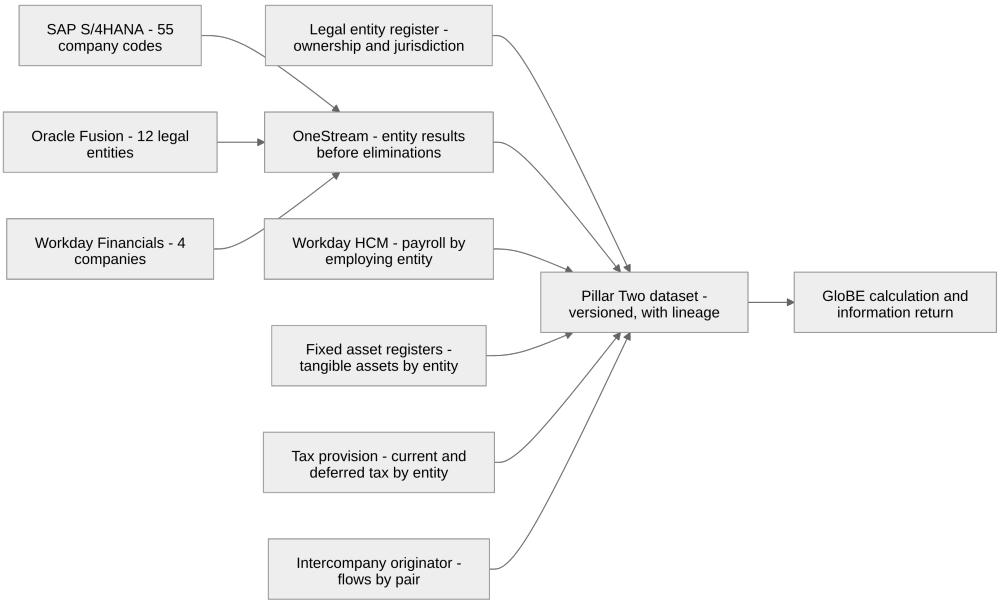


Figure 7: *Pillar Two turns entity and jurisdiction data into a tax return; the calculation is only as good as the entity register, the pre-elimination entity results, and the payroll, asset and intercompany data behind them.*

Every input depends on something argued earlier: the entity register for keys and jurisdictions, clean intercompany for pre-elimination results, Workday HCM's employing entity agreeing with the register, and reconciliation to consolidation rather than private extracts. Kofi's request therefore lands not as a new tax system but as **jurisdiction and entity dimensions on every finance dataset**, and an entity-tagged tax data mart in the finance data platform that both the provision and the GloBE calculation read (see One Number, Many Views: Finance Data from Operations to Analytics).

Licensed Entities: Where the Group Cannot Simply Move Money

Northmere Payments B.V. is an e-money institution supervised by De Nederlandsche Bank; **Northmere Payments UK Ltd** is authorized by the FCA. Together they hold Stayloft travelers' money between payment and payout. **Safeguarding** protects those funds from the institution's creditors. Under PSD2 Article 10, applied to e-money institutions through the E-Money Directive, funds still held at the end of the business day after receipt are segregated in a separate account at a credit institution or in secure low-risk assets, or covered by insurance or a comparable guarantee (PSD2). The FCA's new UK safeguarding regime, widely reported to apply from 7 May 2026, requires daily reconciliations, monthly returns and, above a threshold, an annual audit. PSD3 and the

Payment Services Regulation, politically agreed in November 2025 and not yet in force as of October 2026, are expected to tighten safeguarding (Payments for Finance Architects: Eleven Reasons the Numbers Disagree). Five architectural constraints follow:

1. **Client money and own money are separate in the chart of accounts and the bank.** The commission that becomes revenue is released from client funds through a defined event, under the safeguarding policy agreed with the regulator, before it can move.
2. **Safeguarded funds stay out of group cash management:** no cash pool, no netting, no intra-group lending. Figure 3's dotted line is enforced by account in the netting configuration, not by a note in a procedure.
3. **Reconciliation becomes a regulatory control:** owed versus held, daily, with SOX-grade completeness and evidence.
4. **Intercompany pricing is scrutinized twice:** by tax authorities for transfer pricing, and by the supervisor for whether the entity is adequately funded by its group.
5. **Resilience obligations attach to its systems.** As a financial entity, Northmere Payments B.V. is subject to DORA, so the ICT services supporting its critical functions fall under DORA's contract, register and testing requirements.

The modeling rule: **regulatory status is an entity attribute, and every flow touching a licensed entity carries it.** Netting exclusions, reconciliation scope and resilience scope derive from the register, so a third licensed entity, which Brazil may require, extends them by configuration.

Opening Brazil: A Preview of the Launch Checklist

The 2028 Brazil launch tests everything above. Treated as "configure Stayloft for Brazil", it produces exactly the forks this chapter warns against. Treated as a change to the entity, currency, tax and regulatory model, it becomes a checklist that The Next Brand Should Be Boring: Onboarding as Configuration turns into a reusable market-launch kit:

Dimension	Decision required	Owner
Legal entity	A new operating entity, not the São Paulo engineering service entity; recorded in the register before any system is configured	Legal, group controller

Dimension	Decision required	Owner
Ledger and GAAP	S/4HANA company code in the group core, mapped to the group chart from day one; local statutory GAAP in a parallel or extension ledger	Group controller, SAP platform lead
Currency	Brazilian real as evidenced functional currency; added to the rate service; real-denominated pricing rules	Controller, treasury, Stayloft
Indirect tax	Brazilian consumption taxes and electronic fiscal documents in the shared tax engine, including the transition rules of Brazil's ongoing indirect-tax reform, verified against current law at design time	Group tax
Payments	Pix and other local methods via a local provider; Adyen's documentation states its pass-through payout model is the only one available in Brazil, which changes settlement timing and reconciliation	Northmere Pay
Regulatory	Whether holding Brazilian travelers' funds needs local authorization, or whether the merchant model launches later; local FX and cross-border payment rules	Compliance, Northmere Pay

Dimension	Decision required	Owner
Intercompany	Royalty to Stayloft B.V., payment and shared-service fees; whether local rules allow netting	Group tax, treasury
Pillar Two and data	Brazil as a 39th jurisdiction; where books may legally be kept; local data-protection law	Tax, finance systems

None of this is a Brazil-specific build. Each item is a register entry, a code-list value or a shared-service rule. **If launching a market requires code changes in a brand platform, the dimensions were not separated well enough.**

Trade-Offs

Entity granularity in the ledger was settled above: share a ledger where currency, calendar and chart are shared, and never let a brand ledger stand in for entity books. Rate sourcing has only one answer that scales, the treasury-sourced, versioned group rate service. The open trade-off is where intercompany originates and settles.

Intercompany origination and settlement	Strengths	Weaknesses	Best fit
Inside each ERP	Native and well controlled	Blind to flows between ERPs	Single-ERP groups
Close platform as originator and matcher	Cross-ERP; dispute workflow; strong evidence	Another critical-path system; overlaps ERP and treasury	Federated estates, as Northmere is until convergence
Treasury in-house bank and netting	Fewest payments and conversions; cash visibility	Tight reconciliation with every ERP; country and licensed-entity limits	Many entity pairs and currencies

Intercompany origination and settlement	Strengths	Weaknesses	Best fit
Consolidation-only matching	Cheap to start	Breaks found at period end; no prevention	Small groups, or a temporary state

For Northmere I would combine a continuously fed intercompany originator and matcher with Kyriba netting, keeping consolidation matching as the final check. Whether the originator is BlackLine, SAP's own capability or a small service is a portfolio decision for Build, Buy or Compose: Choosing What to Own.

Integration With the Rest of the Estate

These models belong to no single platform, so they need explicit owners. The entity register, rate service and tax master data are master-data domains under Finance Data Architecture: Who Owns the Truth, with authority recorded in Systems of Record: Ending the Doppelgänger Record; the event envelope that carries the five dimensions belongs to Canonical Events: A Common Language for Money; translation and elimination run in OneStream: Consolidating Actuals, Planning the Future, intercompany and FX account reconciliation in BlackLine: Matching, Certifying and Closing, tax, e-invoicing and netting in Around the Core: Where Specialist Platforms Earn Their Place; and intercompany, rate and safeguarding reconciliations join the catalog in Reconciled by Design: Traceability and Audit Across Boundaries, to the evidence standards of Controls by Design, Not by Audit Season.

Failure Modes

- **The entity created three times**. The ERP, the consolidation tool and the tax engine each set up a new entity from a different document; two disagree on functional currency. The first sign is an unexplained translation difference at quarter-end.
- **Intercompany keyed from a spreadsheet**. Two teams key their halves into two ERPs on different days; matching fails, and the auditor asks for a calculation that lives on a laptop.
- **The missing counterparty**. Intercompany lines post without a counterparty because an interface never carried one; elimination depends on a person's memory.
- **The subledger that never revalues**. Foreign-currency payables live in a subledger that posts functional-currency summaries; month-end revaluation misses them until a currency moves sharply.

- **Tax logic in brand code**. Each mandate, from ViDA to a national e-invoicing phase, becomes a development project per brand, on dates the tax authority sets and the brand roadmap ignores.
- **The licensed entity in the cash pool**. Pooling or netting includes the payments entity's accounts without separating client funds; the first sign may be a supervisor's question.

The Chief Architect's View

Functional currencies, ledger structures and entity representations outlive several generations of applications; tax and regulatory rules change every year. The architecture must get the slow-changing structures right and make the fast-changing rules configurable.

Decision	Decides	Architect's role
Legal entity register, attributes and workflow	Group controller and Legal	Define the register and its publication contract; standard: no entity outside the register
Entity representation and ledger granularity	Group controller, platform owners	Set the modeling standard; review each new entity
Rate service and rate policy	Treasury (sourcing), controller (policy)	Make the single rate service mandatory; require rate IDs on converted amounts
Intercompany originator, matcher, settlement	Controller and treasury	Name one capability owner; stop overlapping tools acquiring it
Multi-GAAP pattern and leading ledger	Chief Accounting Officer	Ensure GAAP differences are posting rules
Tax determination and e-invoicing as shared services	Head of tax	Make them mandatory for new brands and markets; plan migration

Decision	Decides	Architect's role
Licensed-entity constraints	Licensed-entity management, compliance	Enforce regulatory status as an entity attribute in netting, pooling and reconciliation scope

What to watch: systems creating entities outside the register; the share of intercompany created by the originator rather than keyed; intercompany differences at the matching cut-off, by cause; the number of rate sources in use; manual GAAP adjustments per period; and the lead time to make a new market or tax rule live as configuration. That last one is the measure I would show the CFO: the cost of global scale, in weeks.

What to resist: "just use the entity we already have", which changes an entity's tax and regulatory profile without a decision; and the brand that wants its own tax logic, rate source or entity conventions because the shared service is not ready. The answer to both is a time-boxed, recorded exception with a retirement date, which is how an exception is kept from becoming **The Permanent Exception**.

Questions to Ask

1. Where is our legal entity register, who owns it, and how many systems create entities without it?
2. For each entity, can we name its functional currency, statutory GAAP, representation in each system and tax registrations from one place?
3. Which component creates both halves of a cross-ERP intercompany transaction, with a shared identifier?
4. How many exchange-rate sources do our ledgers, subledgers and consolidation use, and does every converted amount record its rate?
5. Which foreign-currency open-item populations are revalued at month-end, by which system, and which are not?
6. Are GAAP differences produced by posting rules or by manual adjustments, and how many manual ones did we post last quarter?
7. Does one engine determine tax for every brand and ERP, storing each determination with its inputs and rule version?
8. Can we produce DAC7, digital services tax and Pillar Two data from entity- and jurisdiction-tagged datasets reconciled to the books?
9. Where is the exclusion of safeguarded funds from pooling and netting enforced?
10. What would it take, in weeks and in code changes, to launch our next market?

Each of these questions ends in the same decision: which capabilities the group builds, which it buys and which it composes, from the intercompany originator to the rate

service to the tax engine. That decision, with its tests for differentiation, volume and regulation, is the subject of Build, Buy or Compose: Choosing What to Own.

Sources and Evidence

Vendor documentation

- Managing Organizational Units in Financial Accounting, SAP Learning — company code as an independent legal accounting entity; controlling-area constraints.
- Checking the Basic Settings in General Ledger Accounting, SAP Learning — leading ledger 0L, standard and extension ledgers, ledger groups.
- Managing Currency Types, SAP Learning — company code, global and freely defined currencies in the Universal Journal.
- Ledgers, Legal Entities, Balancing Segments, and Business Units, Oracle, 26C — primary and secondary ledgers, entity and balancing-segment assignment, guidance to limit primary ledgers.
- Primary Ledgers, Secondary Ledgers, and Reporting Currencies, Oracle — data conversion levels and their relative cost.
- Workday Financial Management datasheet, Workday, 2024 — multibook; accounting generated with intercompany and translation.
- Consolidations and Multicurrency Guide, Workday — intercompany profiles, hierarchies, eliminations, translation and revaluation rules.
- Getting paid, Adyen — pass-through payout as the only model in Brazil.
- BlackLine acquires FourQ, BlackLine, 2022 — origin of BlackLine's intercompany products.

Standards and regulation

- IAS 21 The Effects of Changes in Foreign Exchange Rates, IFRS Foundation — functional and presentation currency; Lack of Exchangeability amendments.
- VAT Directive 2006/112/EC, consolidated text, EUR-Lex — Article 226 sequential invoice numbering.
- VAT in the Digital Age, European Commission — ViDA dates, including the platform deemed-supplier rule from 1 July 2028 with optional delay to 2030.
- Directive (EU) 2015/2366 (PSD2), EUR-Lex — Article 10 safeguarding.
- Pillar Two key dates, Irish Revenue — 30 June 2026 first filing date; side-by-side package dates.
- Canada rescinds digital services tax, Department of Finance Canada, 2025 — DST rescission.
- South Dakota v. Wayfair, Inc., US Supreme Court, 2018 — economic nexus.

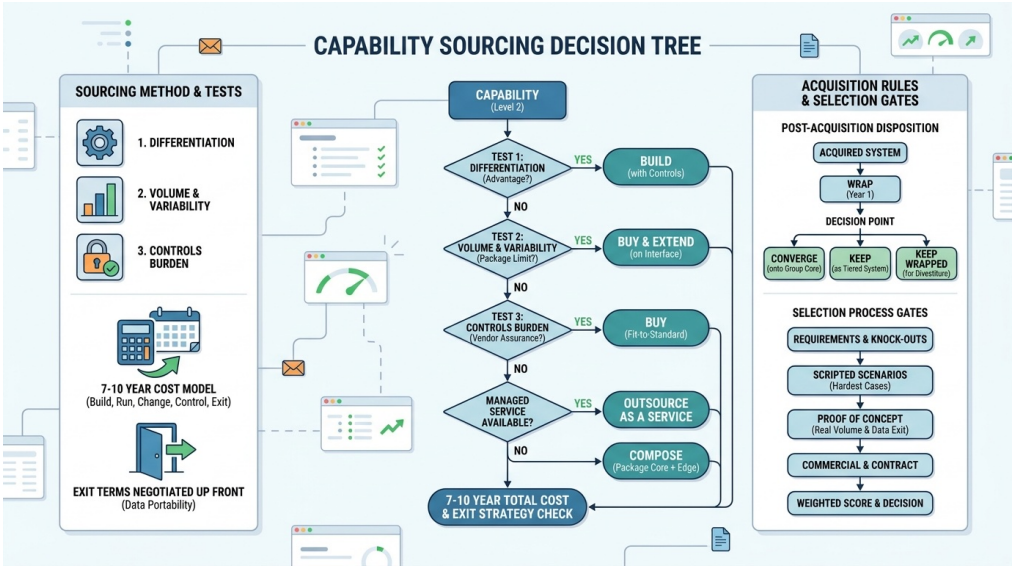
- Digital Operational Resilience Act, EIOPA — DORA scope and application from January 2025.
- About Peppol, OpenPeppol — the four-corner model behind Belgian and other mandates.

Practitioner and research sources

- OECD releases guidance on side-by-side system, BDO, 2026 — side-by-side content and safe harbors (secondary).
- Pillar Two filings due 30 June 2026, EY Global Tax News, 2026 — first GloBE return deadlines (secondary).
- VAT in the Digital Age, PwC Luxembourg, 2025 — deemed-supplier mechanics and relief (secondary).
- Deadline for first DAC7 reporting, PwC Belgium — DAC7 scope and 31 January deadline (secondary).
- Electronic invoicing: what does the reform entail?, CMS France — French phases from September 2026 (secondary).
- Belgium's 2026 e-invoicing regulations, Vertex — Belgian Peppol mandate (vendor source).
- Germany e-invoicing, Avalara — German phases 2025–2028 (vendor source).
- Marketplace facilitator sales tax laws, Avalara — state variation (vendor source).
- CASS 15 safeguarding, SW Group, 2026 — UK regime from 7 May 2026 (secondary).
- Impact of PSD3: are you ready?, Clifford Chance, 2026 — PSD3/PSR status and safeguarding (secondary).
- Viertes Bürokratieentlastungsgesetz, Haufe — German voucher retention from 2025 (secondary).
- Money, Martin Fowler, *Patterns of Enterprise Application Architecture*, 2003 — currency-aware money and rounding.

Northmere Group, its entities, figures and decisions are fictional; transfer-pricing and exchange rates in the examples are illustrative, not tax or accounting advice. Architectural recommendations are the author's judgment.

14. Build, Buy or Compose: Choosing What to Own



IN THIS CHAPTER, YOU WILL: Learn a repeatable method for sourcing finance capabilities (three tests, a decision tree and a seven-to-ten-year cost of ownership that includes control and exit), a selection process that tests vendors on your hardest scenarios rather than theirs, a weighted scoring model that does not lie to you, and the rules for converging, keeping or wrapping the finance systems that arrive with an acquisition.

WHY FINANCE CARES: Every sourcing decision fixes, for a decade, how much of the close is automated, who carries the audit burden and how much Finance must change its own processes. A bought system Finance will not adopt as designed, or a built one nobody can evidence, costs more than either side of the original business case admitted.

WHY ARCHITECTS SHOULD CARE: Sourcing is the architecture decision with the longest half-life and the strongest commercial pressure on it. If the enterprise has no method of its own, vendors' sales cycles and acquisitions' accidents decide the shape of the estate.

KEY POINTS:

- **Source capabilities, not systems.** Ask three questions of each capability. Is it a source of advantage or a commodity? Can a package carry its volume, latency and business-model variety? Who will carry its control and audit burden? Most "build versus buy" arguments are two people answering different questions about different parts of the same system.
- **Cost the decision over seven to ten years, including control and exit.** Build, run, change, control and exit costs decide more cases than license prices do, and the comparison usually disqualifies options rather than picking a winner. When the cost ranges overlap, say so, and let the tests and the risks decide.
- **Select on your hardest scenarios, with the exit negotiated up front.** Capability-based requirements, scripted demonstrations of your worst cases, a proof of concept on real volume and a tested data exit beat any feature matrix. Exit assistance and data portability are cheapest to win before signature, and the EU Data Act and DORA now give them legal weight.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Borrowed Roadmap** — vendor-led selections write the requirements, choose the scenarios and bundle modules nobody asked for; the 2022 consolidation purchase brought a reconciliation module that duplicated an existing platform.
- **The Permanent Exception** — Tablekin's Workday Financials was inherited in 2023 and never sourced at all: no tests, no tier, no exit condition.
- **Customization Concrete** — a package bought for its standard but configured to mimic the old process gives the enterprise the cost of buying and the cost of building at the same time.

Finance architects inherit most of their sourcing decisions. The ERP was chosen a decade ago, the close platform came with a previous controller, and two of the ledgers came with acquisitions. The decisions that remain open are the expensive ones: whether to keep an acquired ledger, whether to replace a custom platform, which vendor gets the next decade of a capability. They are usually argued in the wrong terms. Engineering argues that the capability is special and should be built; Finance argues that it is standard and should be bought; Procurement argues about price; and the vendor's account team argues, persuasively, that its platform does all of it.

This chapter replaces that argument with a method. The unit of decision is the **capability**, not the system, because one system usually contains both the most differentiating logic an enterprise owns and the most commodity logic it runs (capabilities are

defined in Map What Finance Does Before What Runs It). Each capability faces **three tests** (differentiation, volume and variability, and the controls burden) and a **cost of ownership over seven to ten years** that includes the two costs business cases forget, control and exit. The output is one of five sourcing choices on a spectrum: build, buy, buy-and-extend, compose, or outsource as a service.

The second half is about buying well: **fit-to-standard**, a selection process built on the enterprise's hardest scenarios, a scoring model that survives scrutiny, exit terms under the EU Data Act and DORA, and the rules for rationalizing finance systems after an acquisition. This chapter owns the build-or-buy criteria, fit-to-standard and the selection process. Custom platforms in depth belong to When the ERP Isn't Enough: Custom Finance Platforms, vendor tiering and exit plans across the portfolio to Own Your Roadmap: Vendor Strategy Without Lock-In Panic, and running several ERPs side by side to Converge or Coexist: Living with More Than One ERP.

Setting the Scene: Three Sourcing Decisions at Northmere

Northmere, the fictional multi-brand digital commerce group this book follows (see Meet Northmere: Five Brands, One Set of Books), carries three sourcing decisions that show the whole problem between them.

A build from 2012. Stayloft, Northmere's accommodation marketplace, built the **Stayloft Finance Platform (SFP)** between 2012 and 2020, because the ERP the group ran in 2012 could not hold per-booking subledger volume and no package represented the merchant model, in which Stayloft collects the traveler's money and pays the property after the stay. SFP now handles about 3 million subledger lines a day and posts about 38,000 summarized general-ledger (GL) lines a day into SAP S/4HANA. It also costs about $18.5 million a year to own, and it earned the group a **significant deficiency** in the FY2026 audit (a control weakness serious enough to report to the audit committee, though short of a material weakness) for change management and access on its payout engine.

A buy from 2022. Group Finance replaced Oracle Hyperion Financial Management and a separate planning tool with **OneStream** in 2022, a selection that was run well in most respects. The contract also included a OneStream reconciliation module, added late in the commercial negotiation, which duplicates BlackLine and today reconciles 22 corporate-entity accounts.

An inheritance from 2023. Northmere acquired **Tablekin**, a software company for restaurants and venues, in 2023. Tablekin earns subscription fees, a usage fee per seated cover, and a margin on payments it facilitates through Stripe Connect, about 11% of group revenue in 2026. It ran **Workday Financial Management** before the acquisition

and still does, in its own Workday tenant, federated into the group only at consolidation in OneStream. Nobody decided that it should stay; nobody decided that it should go.

In January 2027 Tomás Ferreira, the Group CFO, asks Dana Whitlock, the Chief Architect, to bring the Tablekin question to the Keystone design authority in February. Program Keystone, the group's five-year finance transformation, aims for one group finance core on S/4HANA by 2029, and the choice between SAP Central Finance and an accounting-hub pattern for converging the brand ledgers is due in the fourth quarter of 2027. Arjun Rao, Tablekin's Head of Finance Systems, wants to keep Workday; Sven Aalto, the SAP Platform Lead, wants Tablekin on the group template. Both vendors' account teams have already sent offers. Tomás's question is short: *stay, move, or something else?* Dana answers that she wants one method, applied first to SFP, which was built, and OneStream, which was bought, so the design authority can see whether it would have caught what went wrong. The platform-level comparison of Workday against the group S/4HANA template belongs to Workday: The Org Chart Behind the Ledger; this chapter builds the method that produced it.

The Sourcing Spectrum and When Each Choice Fits

"Build or buy" is a false binary. Five sourcing choices recur in finance, and most estates use all of them.

Choice	What the enterprise owns	Right when	Wrong when	Finance examples
Build	Code, data model, operations, every control and its evidence	The capability differentiates, a package fails volume or variability, and the controls burden is accepted by name	The real reason is that the team prefers building	Commission rating, payout orchestration, a traveler-funds ledger

Choice	What the enterprise owns	Right when	Wrong when	Finance examples
Buy	Configuration, master data, process adoption, user-side controls	Commodity capability that a package carries at the enterprise's volume	The business model needs invented meanings for standard fields	General ledger, payables, fixed assets, account reconciliation
Buy and extend	Configuration plus extensions *outside* the package, on released interfaces	A bought core with a few genuinely distinctive steps	Extensions creep into the core and upgrades become projects	An ERP with a side-by-side app for a brand-specific approval
Compose	The seams: integration, reconciliation between components, the data contract	Strong integration and data-contract discipline exists	Seams are spreadsheets, or nobody owns end-to-end reconciliation	Billing platform + revenue engine + accounting hub + ERP ledger
Outsource as a service	Oversight, policy, the controls that stay with the enterprise	Commodity process, modest volume, provider assurance available	Significant accounts at an SEC registrant without strong oversight	Payroll in smaller countries, accounts-payable processing

Two terms need precision. **Buy and extend** is only buying if the extensions sit outside the package core, on interfaces the vendor supports across releases; modifying the package itself is a build hidden inside a buy, and it produces Customization Concrete (SAP's version of this discipline, the **clean core**, is owned by SAP S/4HANA: One Journal, Clean Core). **Compose** is the choice the industry has been describing for a decade. Gartner introduced **postmodern ERP** around 2013 or 2014 (sources differ) to describe the break-up of the monolithic ERP into loosely coupled, often cloud-based applications, and around 2020 moved its narrative to **composable ERP**, a foundation of administrative and operational capabilities assembled from composable applications and configurable services, as summarized by TechTarget; I cite the secondary account

because Gartner's notes are behind its paywall.

The architectural reading of composability is the part the vendor decks leave out: **composition moves cost from licenses to seams**. Every boundary between two composed finance components is an interface to operate, a reconciliation to perform and a control to evidence. A composed estate with five components and clean contracts between them can be better than a suite; a composed estate with five components and spreadsheets between them is Point-to-Point Spaghetti with better logos.

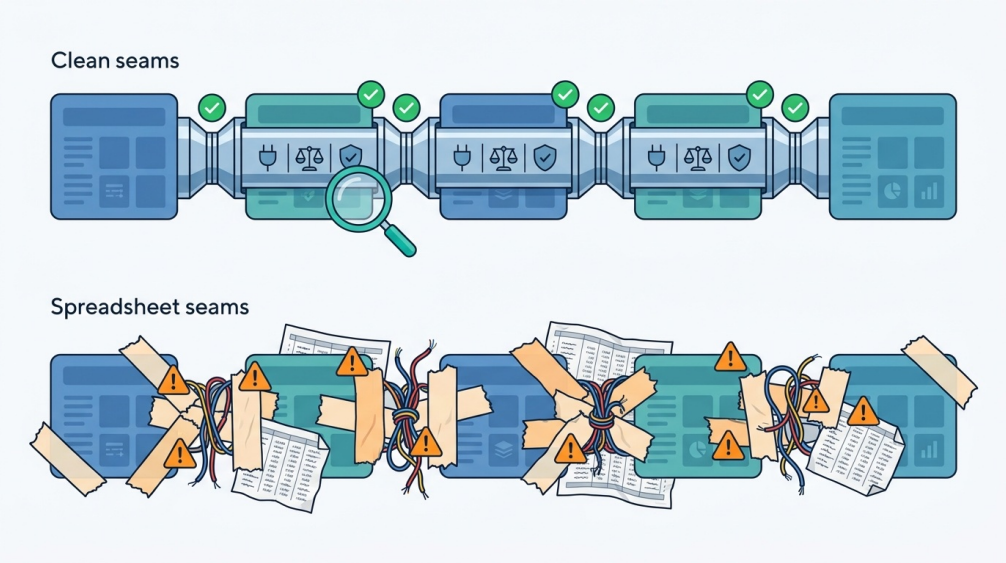


Figure 1: *Composition moves cost from licenses to seams: every boundary between composed finance components is an interface to operate, a reconciliation to perform and a control to evidence, so clean contracts decide whether a composed estate beats a suite.*

Outsource as a service differs from buying SaaS in who runs the process: the provider's people, not the enterprise's. The enterprise still owns the outcome and the controls over the provider, so the provider's assurance report and the controls it expects the customer to run matter as much as for any software (see Controls by Design, Not by Audit Season for SOC 1 reports and complementary user entity controls, or CUECs).

The Three Tests

The tests are asked **per capability**, at the second level of the capability map, not per system. For each, the table gives the question, the evidence that answers it, and the direction each answer points.

Test	The question	Evidence that answers it	Points toward building or composing when…	Points toward buying when…
1. Differentiation	Is the capability a source of advantage, or a commodity the business must do correctly?	Who asks for changes and how often; whether customers, partners or regulators see its output; whether competitors could copy it by buying the same package	Product or commercial teams change it monthly or faster, and its behavior is part of what the business sells	Changes come from accounting standards, tax law or audit, and nobody outside Finance would notice a package's standard behavior
2. Volume and variability	Can a package carry the volume, the latency and the variety of business models?	Peak-day transaction counts, latency a user or partner needs, number of distinct business models, how many standard objects must be given invented meanings	Proven package limits fall short of three times today's peak, or the model needs fields the package does not have	A proof of concept on real volume passes, and the business models map to standard objects

Test	The question	Evidence that answers it	Points toward building or composing when…	Points toward buying when…
3. Controls burden	Who will design, operate and evidence the controls, and who carries the audit burden?	Number of key controls, whether a vendor assurance report covers the processing, the complementary controls the enterprise must run, the team's record on access and change controls	The enterprise already runs strong change and access controls on its own platforms and the regulator requires positions no package keeps	A vendor's assurance report covers the processing, and the enterprise's remaining controls are configuration and access

Differentiation is the test people answer from pride. The question I use to cool it down is: *if a competitor bought the best package for this capability tomorrow, what would they not be able to do that we can?* For Stayloft's commission rating the answer is concrete: weekly commercial programs and promotional overrides, evaluated at booking time. For the general ledger it is nothing; nobody books a hotel because the group's trial balance is elegant. The test mirrors the pace layers of Manage the Application Portfolio as an Investment: systems of record are bought, systems of differentiation are often built or composed.

Volume and variability is the test people answer from vendor slides. *Volume* means transactions per peak day and period-end, not per year. *Latency* means how soon someone outside Finance needs the number: a partner app showing "you will be paid €510 on 22 April" needs it at booking time. *Variability* means business models: Northmere runs six, and a package that handles four natively and two through invented field meanings has failed for those two. The evidence is a proof of concept, never a reference architecture.

The controls burden is the test nobody asks until the audit. A bought platform comes with the vendor's assurance over its processing and a list of controls the customer must run around it. A built platform comes with nothing: every automated calculation is an

application control the enterprise designs and evidences, resting on access and change controls it must prove every period (When the ERP Isn't Enough: Custom Finance Platforms costs the burden). The test is not "can we write the code?" but "**can we prove**, **every quarter**, **for a decade**, **that the code did what we said?**"

Two rules govern how the tests combine. First, **a build needs a pass on differentiation plus at least one of the other two**, and the controls burden must be accepted explicitly, by name, in the decision record; volume alone argues for a high-volume package, not for a custom one. Second, **a commodity capability is bought even when it is hard**: if the general ledger cannot carry the volume, the answer is to summarize before the ledger, not to write a ledger.

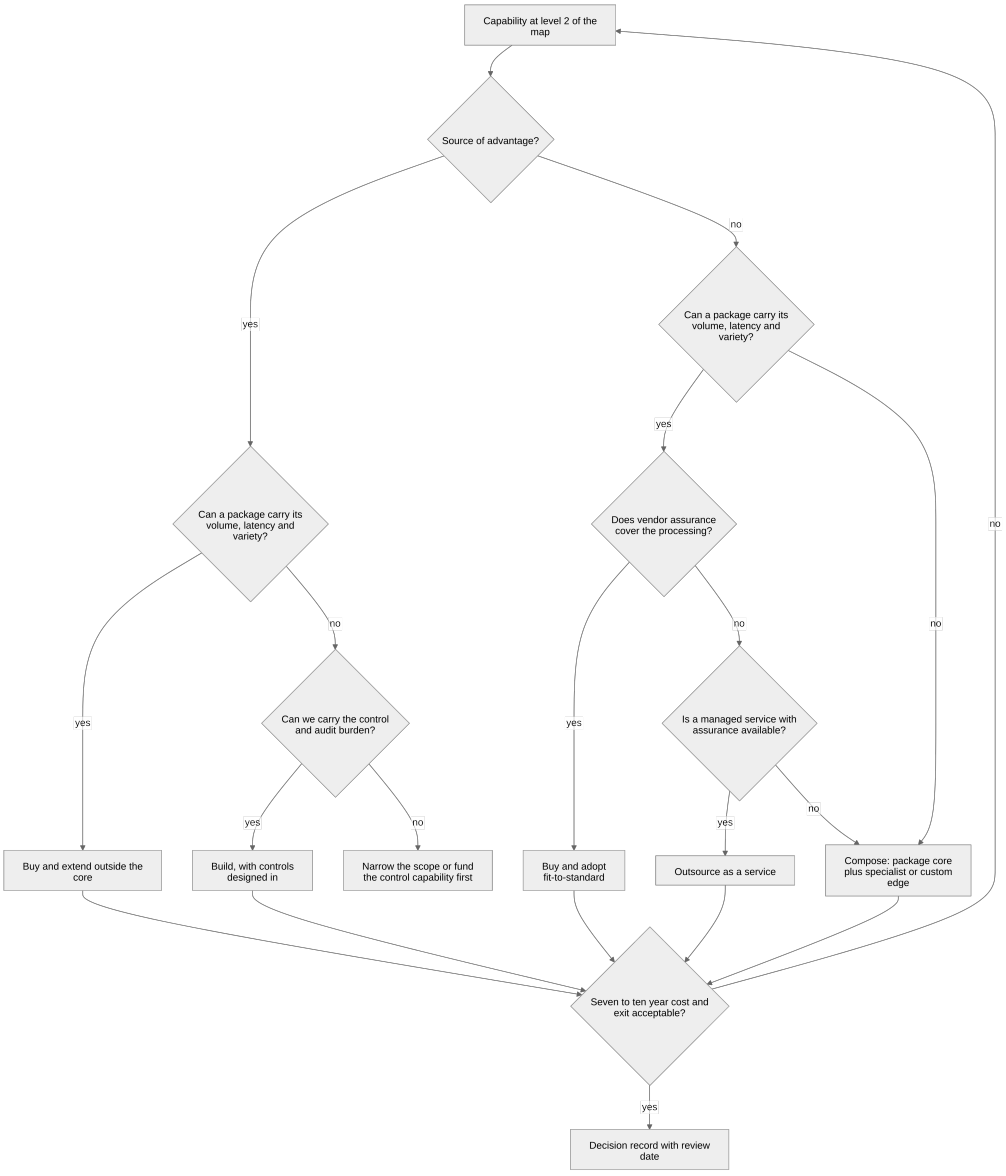


Figure 2: *The sourcing decision tree. Differentiation decides whether building is even on the table; volume decides whether a package can carry the load; the controls burden decides who proves the result; and every path ends in a cost and exit check that can send the capability back to the start.*

The tree is deliberately asymmetric. A commodity capability that a package cannot carry goes to **compose**, not to build: the package keeps the book of record and a specialist or custom edge carries the volume, which is exactly the pattern of a high-volume subledger summarizing into a general ledger (see Anatomy of a Finance Estate: Layers, Planes and Boundaries for the volume gradient). And a differentiating capability whose controls

the enterprise cannot carry does not go to build with a promise to fix the controls later; that promise is how Audit Archaeology starts.

Cost of Ownership Over Seven to Ten Years

The tests decide what kind of answer is admissible. The cost decides whether it is affordable, and it must be counted over the life of the decision, not the life of the business case.

Why seven to ten years. Finance platforms outlive the contracts that buy them: a SaaS term is commonly three to five years, a ledger usually runs a decade, SAP gives each S/4HANA release seven years of mainstream maintenance (SAP, 2022), and financial records commonly stay under retention for seven to ten years. A shorter horizon flatters options with low entry and high run costs; a longer one pretends to precision nobody has.

Five lines, every time. The six-component total cost of ownership (TCO) model of Manage the Application Portfolio as an Investment (license, hosting, run team, change, integration upkeep, audit and control) measures a system's annual cost. A sourcing decision needs two more things: the one-time cost of getting there and the cost of leaving. I group them into five lines:

Line	What it covers	What business cases usually miss
Build (or implement)	Design, build or configuration, data migration, testing, parallel runs, training, the integration work to connect it	Parallel running of the old system; Finance's own time in design and testing
Run	License or subscription, hosting, the run team, integration upkeep	Non-production environments; the share of middleware and monitoring
Change	Enhancements, regulatory change, upgrades, regression testing for vendor releases	SaaS release regression: twice a year for Workday, quarterly for Oracle Fusion, permanently

Line	What it covers	What business cases usually miss
Control	Operating and evidencing key controls, access reviews, internal and external audit testing, remediation	Almost everything; it is spread across Finance, internal controls and the audit fee
Exit	Data extraction in a usable form, archive with a read path, parallel run on the successor, contract termination costs	All of it; it is incurred by a future team under a future budget

Staff time is valued at Northmere's planning rate of **$165,000 per full-time-equivalent (FTE) year**, and capacity freed is kept separate from cash saved, as Manage the Application Portfolio as an Investment requires.

A worked comparison: Tablekin's ledger, 2028 to 2034

Dana's portfolio team costs the three options that the design authority will consider (detailed in Workday: The Org Chart Behind the Ledger): **A**, keep Tablekin on Workday Financials in its own tenant with the planned improvements; **B**, migrate Tablekin's four legal entities onto the group S/4HANA core during the 2029 convergence; **C**, move Tablekin's Financials into the group Workday tenant beside HCM as a governed second-tier ERP, with group-facing intercompany posted into S/4HANA through a defined interface. All figures are illustrative, in millions of US dollars, summed over seven years from 2028.

Line ($M, 2028–2034)	A: Stay as is	B: Move to S/4HANA	C: Workday as second tier
Build (one-time)	0.6 (worktag redesign, Accounting Center proof of concept)	8.6 (18-month migration: open items, revenue schedules, contracts, two parallel closes, retraining)	3.1 (tenant merge, worktag redesign, data conversion, intercompany interface to S/4HANA)

Line ($M, 2028–2034)	A: Stay as is	B: Move to S/4HANA	C: Workday as second tier
Run: subscription and license	8.4 (Workday Financials, $1.2M a year)	4.3 (Workday for 18 months of dual running, then incremental S/4HANA capacity and users)	7.0 (Financials in the group tenant, $1.0M a year)
Run: team and integration upkeep	6.5 (3.5 FTE; worker copy feed, Stripe flow, OneStream load and mapping)	4.0 (1.5 FTE after cut-over; Stripe postings into S/4HANA)	4.4 (2.5 FTE; Stripe flow, intercompany interface)
Change (including release regression)	2.3 (two Workday tenants, two releases a year each)	1.6 (S/4HANA template changes; Workday during dual running)	1.4 (one Workday tenant)
Control	2.8 (two tenants' security, a copied approver set, a manual mapping control)	1.7 (Tablekin inside the group SAP control set; dual controls during migration)	1.5 (one tenant, tested once; interface completeness control)
Exit (at the end of the horizon)	0.8 (extract and archive two tenants)	0.3 (archive Workday history with a read path)	0.8 (extract and archive one tenant)
Seven-year total	**21.4**	**20.5**	**18.2**
Estimate range	17–27	17–29 (migration estimates skew upward)	14–23
Share spent in 2028–2029	30%	61%	41%
Confidence	Medium	Low on the migration line	Medium; depends on the proof of concept

Four conclusions come out of the table, and none of them is "C wins on cost".

The ranges overlap. The widest gap between the options, $3.2 million over seven years, sits inside ranges of $10 million or more, and that is not a decision. I tell design authorities this plainly: **a TCO comparison usually disqualifies options rather than choosing one**. Here it disqualifies nothing, which means the tests and the risks must decide.

Option A is not the cheap option. Staying looks free because its costs are already in someone's budget, but it is the most expensive in run and control: two Workday tenants, a nightly copy of workers and approvers, and a mapping control performed by hand every month.

Option B's cost lands at the worst time. Sixty-one percent of it falls in 2028 and 2029, when Keystone is already converging SFP's merchant subledger and launching Brazil, and the same few controllers and SAP architects are needed for all three.

The control line moves the comparison. It accounts for $1.3 million of the $3.2 million difference between A and C, about 40%. This is the line that disappears from most business cases, and in finance it is rarely small.

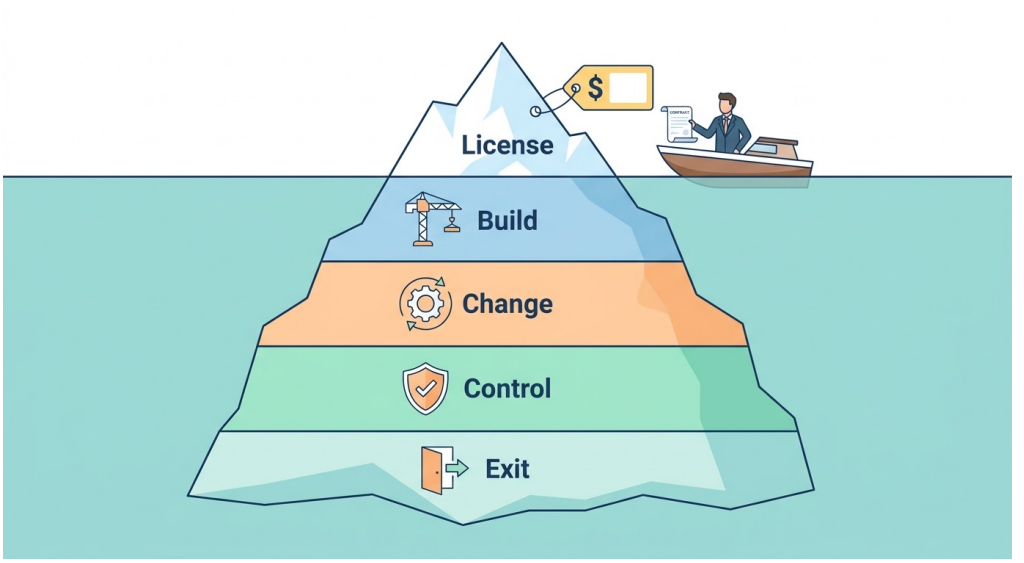


Figure 3: *The license is the visible tip: over seven to ten years the build, change, control and exit costs decide more sourcing cases than price does, and the control line is the one most business cases leave out.*

Applying the Tests to Tablekin

Before any of the options were costed, Dana's team applied the three tests to Tablekin's capabilities, one by one. The result reframed the question.

Tablekin capability	Differentiation	Volume and variability	Controls burden	Sourcing result
Usage rating and subscription billing	Partly: pricing plans are product decisions	Per-cover usage for thousands of venues; Stripe Billing carries it today	Stripe's assurance plus Tablekin's configuration controls	**Buy** (already bought: Stripe Billing); keep rating outside any ledger
Venue payouts	Partly: payout timing is part of the product	About 1.4 million venue payouts a month	Stripe's assurance; Tablekin approves payout rules	**Buy** (Stripe Connect)
Accounting for Stripe activity	No	About 5.5 million Stripe balance transactions a month (March 2027)	Completeness of the summary is a key control	**Buy or compose**: accounting engine in whichever ledger receives Tablekin
General ledger, payables, expenses, assets	No	Small: four legal entities, about 1,900 workers	Standard ERP controls	**Buy**: the only question is *which* bought ledger
Revenue recognition	No: policy is the group's	Subscription in advance, usage in arrears, one invoice	Revenue is a significant account	**Buy**: configured in the receiving ledger or in a revenue engine

Tablekin capability	Differentiation	Volume and variability	Controls burden	Sourcing result
Consolidation into the group	No	Monthly trial balance	Mapping is a key control	**Buy** (OneStream); replace the mapping spreadsheet with rules

Nothing in Tablekin's finance estate passes the differentiation test except the pricing and payout rules that already live in Stripe. **The Tablekin decision is therefore not a build-or-buy decision at all. It is a rationalization decision: which of the group's bought ledgers should hold a commodity capability**, and on what terms. That changes who should decide it (the design authority, against group criteria, not Tablekin against its own preferences) and which rules apply (the post-acquisition rules later in this chapter).

The tests also narrowed the options. A **compose** option, with Stripe Billing and Stripe Revenue Recognition feeding an accounting hub that posts into S/4HANA, is the route Option B would take if the fourth-quarter decision chooses a hub, so it is assessed with that decision. An **outsource** option fails the controls test for a significant account at an SEC registrant building one finance core; the decision record says so in one line.

That left the three options costed above. The decisive differences between B and C are the ones the platform chapter examines: whether Tablekin's worktags map to the group chart entirely by rule, whether Workday's Accounting Center carries Tablekin's Stripe volume, and whether the fourth-quarter convergence pattern accommodates a governed second tier. Workday: The Org Chart Behind the Ledger records Dana's recommendation, Option C under those three conditions with B as the fallback, and the design authority's decision to record it as conditional with a review in the first quarter of 2028. The method's contribution was to make the conditions explicit: **every one of them is a test result still to be obtained, not an opinion**.

Two Earlier Decisions, Read Through the Method

A method earns trust when it explains decisions already made. Dana's team applies it retrospectively to the two contrasting cases.

SFP: the right build, scoped wrongly and costed partly

In 2012 Stayloft's case for building would have passed the tests. Commission rating was coupled to the product and changed weekly (differentiation); the group's ERP could not hold per-booking merchant positions and no package represented the merchant model

(volume and variability). The controls burden was never assessed, because in 2012 the merchant model was small.

The method finds two errors. **The first is scope**. The build decision was made for a platform, not for capabilities, so commodity components (commission invoicing, GL summarization and account determination, finance reports) were built alongside the differentiating ones and inherited the same justification. In May 2027 the design authority's component-by-component classification, recorded in When the ERP Isn't Enough: Custom Finance Platforms, keeps three of SFP's nine components custom and converges or retires the other six. A per-capability decision in 2012 would have bought most of those six.

The second is cost. The 2012 case counted engineers and infrastructure. The 2027 estimate of SFP's annual cost of owning adds about $1.5 million a year of controls and audit work and about $4.1 million of finance operations, together more than half of the $9.9 million engineering line and about 30% of the roughly $18.5 million total. The FY2026 significant deficiency is the controls test failing in production fourteen years after it was skipped at design. **A build decision that does not name who will evidence the controls is not a build decision; it is a deferral of one**.

OneStream: a good selection, with a gravitational accident

The 2022 consolidation selection did three things right. It wrote requirements by capability (consolidation, translation, eliminations, planning on the same model as actuals), not from Hyperion's feature list. It scripted demonstrations on Northmere's own hard cases, including a trial balance from a non-SAP ledger with a different chart of accounts, because group strategy anticipated acquisitions. And it ran a proof of concept on a full quarter of data. Tablekin arrived on Workday in 2023 and Wayfare on Oracle Fusion in 2024; both load into OneStream today, which is the selection paying off.

It did two things wrong, and both are the commercial phase overruling the architecture. The **reconciliation module** was added in the final negotiation at a steep bundle discount, without the capability-owner check that would have noticed BlackLine already reconciled almost every balance-sheet account. It has cost about $448,000 a year since and is now being retired (Manage the Application Portfolio as an Investment has the retirement case). And **the exit terms were thin**: the contract specified data export in general terms but not the consolidation journals, mapping history and audit trail in a form another platform could load. When OneStream's ownership changed in April 2026 (OneStream: Consolidating Actuals, Planning the Future covers the take-private), the group negotiated from a weaker position than it should have, because leaving was harder than it needed to be.

Neither error came from the vendor doing anything improper. Both came from letting the vendor's commercial process set the last word on scope and terms, which is the mild, everyday form of **The Borrowed Roadmap**.

Fit-to-Standard and What It Asks of Finance

Buying a finance package is only half a decision. The other half is whether the enterprise will adopt the package's processes or bend the package to its own.

Fit-to-standard is the implementation approach in which the enterprise starts from the package's delivered processes and configuration, demonstrates them to the business on a working system, and records only the genuine gaps, instead of writing requirements from the current process and configuring the package to meet them. SAP's Activate methodology builds its Explore phase around fit-to-standard workshops that review standard functionality with the customer, identify delta requirements and capture them in a backlog (SAP Learning); SAP's material for the public-cloud edition advises pursuing only critical requirements, because changes to standard processes add upgrade maintenance (SAP Learning). Workday and Oracle implementations commonly run comparable design workshops under their own methods. For multi-tenant SaaS (Workday, Oracle Fusion, SAP's public-cloud ERP) there is little alternative: the vendor upgrades every customer on its own schedule, and anything bent too far breaks at the next release.

The approach is simple to state and hard to live with, because **the demands fall on Finance, not on IT**. Process owners with authority must decide each gap in the room; a project analyst who writes "user prefers current layout" into the gap register has decided by default. Every gap carries a ten-year price, and a gap nobody will fund is not a gap. Accounting policy is in scope, because many "system gaps" are conventions made years ago for a different system (a cut-off rule, an approval threshold, an intercompany settlement rhythm), and the controller must be ready to revisit them. Finance owns release adoption permanently: someone must decide on each newly delivered feature, every cycle (Workday: The Org Chart Behind the Ledger and Oracle Fusion: An Accounting Hub for High-Volume Events cover the cadences). And controls that lived in the old system's custom code must be re-designed in the package's configuration and security model and tested before go-live, not after the first audit.

Gap disposition	When it is right	Ten-year consequence
Adopt the standard	The difference is habit or layout	Training once; nothing after
Configure	The package supports the variant through settings	Regression test each release

Gap disposition	When it is right	Ten-year consequence
Extend outside the core	Genuinely differentiating, and released interfaces exist	An owned component with its own run, change and control costs
Change the policy	The gap is an old accounting or approval convention	A policy change approved by the controller and the auditor
Modify the package	Never by default; only with a design-authority waiver that expires	Customization Concrete: every upgrade becomes a project

The publicly documented counter-example is Birmingham City Council's Oracle Fusion program, which went live in April 2022 and contributed to the council's financial crisis. Reporting describes an intention to use the software largely out of the box, followed by customizations including a bank-reconciliation design that did not work properly, and an auditor's view that the business case and program governance were not adequately designed; a re-implementation on the same platform went live in August 2026 (The Register). I cite it as a governance case, not a product case: the same software works when the standard is adopted deliberately.

Northmere's implication. If Option C goes ahead, Tablekin's move into the group Workday tenant is a fit-to-standard exercise on the group's Workday configuration, not a copy of Tablekin's old tenant. Leila Haddad, the Chief Accounting Officer, owns the policy decisions; Arjun owns the gap register; and the design authority sees every gap whose disposition is "extend" or "modify".

A Selection Process That Finds the Truth

When the tests say buy and the rationalization rules do not already name the platform, the enterprise selects a vendor. The process below is the one Dana makes mandatory for any finance-relevant selection above $1 million over the contract term. It has seven stages, each of which can exclude a vendor.

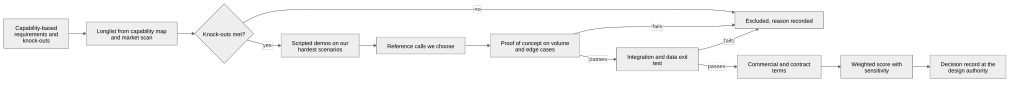


Figure 4: *The selection process as a series of gates. Knock-outs, the proof of concept and the exit test can each exclude a vendor regardless of its score; the weighted score ranks only the vendors that survive them.*

1. Requirements by capability, with knock-outs

Requirements are outcomes the capability must deliver, not features of the incumbent. "Post a summarized journal per legal entity per day with drill-back to every source transaction" is a requirement; "support 40-character reference fields like today" describes the past. Each is a **knock-out** (failure excludes the vendor), **scored** or **noted**. Knock-outs are few, testable and set before any vendor is seen: for a finance platform at Northmere they typically include a SOC 1 Type 2 report covering the relevant processing, data residency options for EU entities, the ability to hold 71 legal entities in 46 transaction currencies, and released interfaces for the integration patterns in Files, Events and APIs: How Finance Systems Talk.

2. Scripted demonstrations on the hardest scenarios

A vendor's standard demonstration shows its best path through its best data. A **scripted demonstration** gives every shortlisted vendor the same script of the enterprise's hardest cases, with the enterprise's data, two weeks in advance. Northmere's library of hard scenarios for finance selections includes:

- **The €420 chargeback**, received 40 days after the stay, after the property has been paid: recover €357 from the property's next payout, book €63 as contra-revenue in the current period, keep the closed May period closed (the accounting is in Accounting for Architects: Ledgers, Journals and Revenue).
- **The Tablekin invoice**: one $1,029 invoice dated 1 April that carries April's $249 subscription in advance (a contract liability until April) and March's $780 usage in arrears (March revenue).
- **The Lisbon booking**: €600 collected, €598.26 settled by Adyen after a €1.74 fee, €600 safeguarded, €510 paid out after the stay, €90 commission recognized on completion.
- **The period-end peak**: a day of about 3 million subledger lines summarized to about 38,000 GL lines, with control totals that prove completeness.
- **The failed day**: reprocess a day's postings after a forced failure with no duplicates.
- **A new market as configuration**: create a Brazilian legal entity with its currency, tax and e-invoicing settings without a code change.
- **The auditor's question**: trace a GL balance back to its source transactions and to the version of the rule that produced each entry.

A vendor that cannot show a scenario says so, and that is useful information. A vendor that wants to substitute its own scenario is showing you who is running the selection.

3. Reference calls the enterprise chooses

Vendor-supplied references are useful only with specific questions: what volume do you run at period-end, how long did your last release regression take, what did you build outside the package and why, how did the vendor behave when something broke at quarter-end? The more valuable calls are the ones the enterprise finds itself through peers and user groups, including at least one customer who left the vendor.

4. Proof of concept on volume and edge cases

A **proof of concept** (PoC) runs the candidate on the enterprise's real data, at real volume, on the scenarios most likely to fail, under conditions the vendor does not control. It is the only stage that tests the volume-and-variability test's claims, and for platforms that publish no throughput figures it is the only evidence there is. The Accounting Center PoC that Option C depends on is typical: load March 2027's 5.5 million Stripe balance transactions in daily batches and time each stage, reprocess a failed day without duplicates, and account for April refunds of March charges with drill-back (Workday: The Org Chart Behind the Ledger specifies it). Success criteria are written before the PoC starts, and a PoC that ends with "promising, needs tuning" has failed.

5. Integration and data exit testing

Two tests most selections skip. The **integration test** connects the candidate to the enterprise's real middleware and identity provider and pushes the canonical financial events through it, including malformed ones. The **data exit test** asks the vendor to export a complete period, including posted journals, open items, master data with its history, the audit trail and the configuration, and then loads the export into something else, even a database, to prove it is complete and usable. Exporting is easy to promise; an export that reproduces a trial balance and its audit trail is not.

6. Commercial and contract terms, including exit

Exit terms are cheapest before signature, when the vendor still wants the deal. Two regulations now give them legal force in Europe.

The **EU Data Act** (Regulation (EU) 2023/2854) applies generally from 12 September 2025 and regulates switching between **data processing services**, defined as digital services that give on-demand network access to a shared pool of configurable, scalable and elastic computing resources (EUR-Lex). The definition is broad, and practitioners commonly treat cloud ERP and finance SaaS as in scope, subject to an exemption for services whose main features were mostly custom-built for one customer (Article 31). Article 25 requires the contract to include, among other things: switching or porting on request within a **maximum transitional period of 30 calendar days** (extendable where technically

unfeasible, to at most seven months), an obligation on the provider to **support the customer's exit strategy**, a **maximum notice period of two months** to start switching, an **exhaustive specification of the categories of data and digital assets that can be ported**, including at minimum all exportable data, and a **data retrieval period of at least 30 days**. Article 26 obliges providers to publish the data structures and formats of exportable data, and Article 29 provides that **from 12 January 2027 providers may not impose switching charges**. Switching charges are not the same as every egress or service fee, and practitioners note that existing contracts need amendment (Latham & Watkins).

The Data Act gives the mechanism; it does not say what complete finance data is. I write the specification in finance terms: posted journals with all dimensions, subledger detail with drill-back keys, open items, effective-dated master data, the approval audit trail and the posting-rule configuration, in documented formats, for every period under retention.

The **Digital Operational Resilience Act (DORA)** (Regulation (EU) 2022/2554) applies from 17 January 2025 to financial entities, which at Northmere means **Northmere Payments B.V.**, the group's Dutch e-money institution (EUR-Lex). Before contracting for information and communication technology (ICT) services, the entity must assess whether the service supports a **critical or important function**, along with concentration risk and the provider's due diligence (Article 28(4)); it must keep a register of all its ICT contractual arrangements (Article 28(3)); and for services supporting critical or important functions it must have **exit strategies** (Article 28(8)), backed by contract terms that include a **mandatory adequate transition period** during which the provider keeps providing the service (Article 30(3)(f)). DORA does not require exit plans for every vendor; it requires them where the function is critical or important, and the entity makes that assessment.

The architectural consequence for Northmere is a routing rule: **any sourcing decision that touches Northmere Pay's payment, safeguarding or safeguarding-reconciliation processing goes through the DORA path**, with Ingrid Solberg's risk team assessing criticality before the shortlist, not after the choice. A group platform that Northmere Pay merely shares, such as the reconciliation platform that evidences daily safeguarding, may still fall in scope. Northmere Payments UK Ltd sits under the UK's own operational-resilience regime, which this book does not cover. Exit plans across the whole vendor portfolio belong to Own Your Roadmap: Vendor Strategy Without Lock-In Panic.

Beyond exit, the terms that matter most to architecture are price protection on growth metrics (per-transaction pricing is the volume test in commercial form), the scope and timing of assurance reports, non-production environments sized for regression testing, and a named module list, so nothing arrives in the bundle that the capability map did not ask for.

7. Score, record, decide

Only vendors that pass the knock-outs, the PoC and the exit test are scored. The score informs a decision record at the design authority, which states the decision, the conditions, the review date and the control impact assessment every finance-relevant record carries at Northmere.

A Weighted Scoring Model and Its Pitfalls

Weighted scoring is useful for one thing: forcing a group of people to state what they value before they see the candidates. It is dangerous when it is mistaken for measurement.

Northmere's model scores each surviving vendor 0 to 5 against **anchored criteria**, where each score level has a written description, so that a 3 means the same to every scorer. Weights are set and signed off before the demonstrations. The illustration below is an anonymized 2027 selection of an e-invoicing and continuous-transaction-controls gateway (the service that exchanges invoices with tax authorities in mandatory e-invoicing countries; see Around the Core: Where Specialist Platforms Earn Their Place) for Keystone's shared tax services. Vendor names are withheld; the scores are illustrative.

Criterion	Weight	Vendor P	Vendor Q	Vendor R
Country coverage in production for Northmere's markets, incl. Brazil	Knock-out	Pass	Pass	**Fail** (Brazil on roadmap only)
Fit to scripted scenarios	25%	4	4	5
PoC on volume and failure recovery	20%	4	3	4
Integration with the canonical events and the tax engine	15%	3	5	4

Criterion	Weight	Vendor P	Vendor Q	Vendor R
Controls and assurance (SOC 1 scope, CUECs, audit trail)	15%	4	3	4
Data exit test and contract terms	10%	4	3	3
Seven-year cost (5 = lowest)	15%	3	4	4
Weighted score	100%	**3.70**	**3.70**	(4.15, excluded)

The table shows the three pitfalls I see most often.

Compensatory scoring hides knock-outs. Vendor R has the highest weighted score and cannot serve Brazil, which launches in 2028. In a model without explicit knock-outs, strength elsewhere would have compensated for a fatal gap, and the selection would have bought a roadmap. Knock-outs are evaluated first and are never weighted.

False precision. P and Q tie at 3.70. Shift five points of weight from cost to integration and Q leads 3.75 to 3.70; shift five from integration to exit and P leads 3.75 to 3.60. **A ranking that flips under a five-point weight change is a tie**, and the honest output is "P and Q are equivalent on our criteria; the decision rests on the remaining PoC risk and the commercial terms". I require a sensitivity check on every score sheet for this reason.

Weights reverse-engineered from a favorite. When weights are set after the demonstrations, they drift toward whatever the favorite does well. Signing them off beforehand, in the decision record, is the only defense.

Three more are worth naming. **Demonstration halo**: polish scores higher than the PoC justifies, which is why scenarios and PoC carry 45% of the weight here. **Price overweighted**: at 40%, price decides everything, and the seven-year cost already includes what makes the cheapest license expensive. **Averaged scorers**: averaging hides disagreement; when scorers differ by two points or more, they discuss the evidence before a score is recorded.

Rationalization After M&A: Converge, Keep or Wrap

Acquisitions bring most finance estates their second and third ledgers, usually without a sourcing decision. The question changes from "which product?" to "how long does the acquired system stay, and in what role?"

Three dispositions cover almost every case. **Converge**: move the acquired entity onto the group platform and retire the acquired system. **Keep**: retain the acquired system

in a defined role, such as a governed second-tier ERP, with entry criteria, a template and an interface into the group core. **Wrap**: keep the acquired system temporarily but surround it with the group's standards, so that it behaves like a group system to everything around it (canonical events out, the group chart of accounts by rule, group identity and access, the group's close calendar), while a dated plan converges or keeps it. Wrapping is the default for the first year after almost any acquisition, because it buys time without creating a permanent exception.

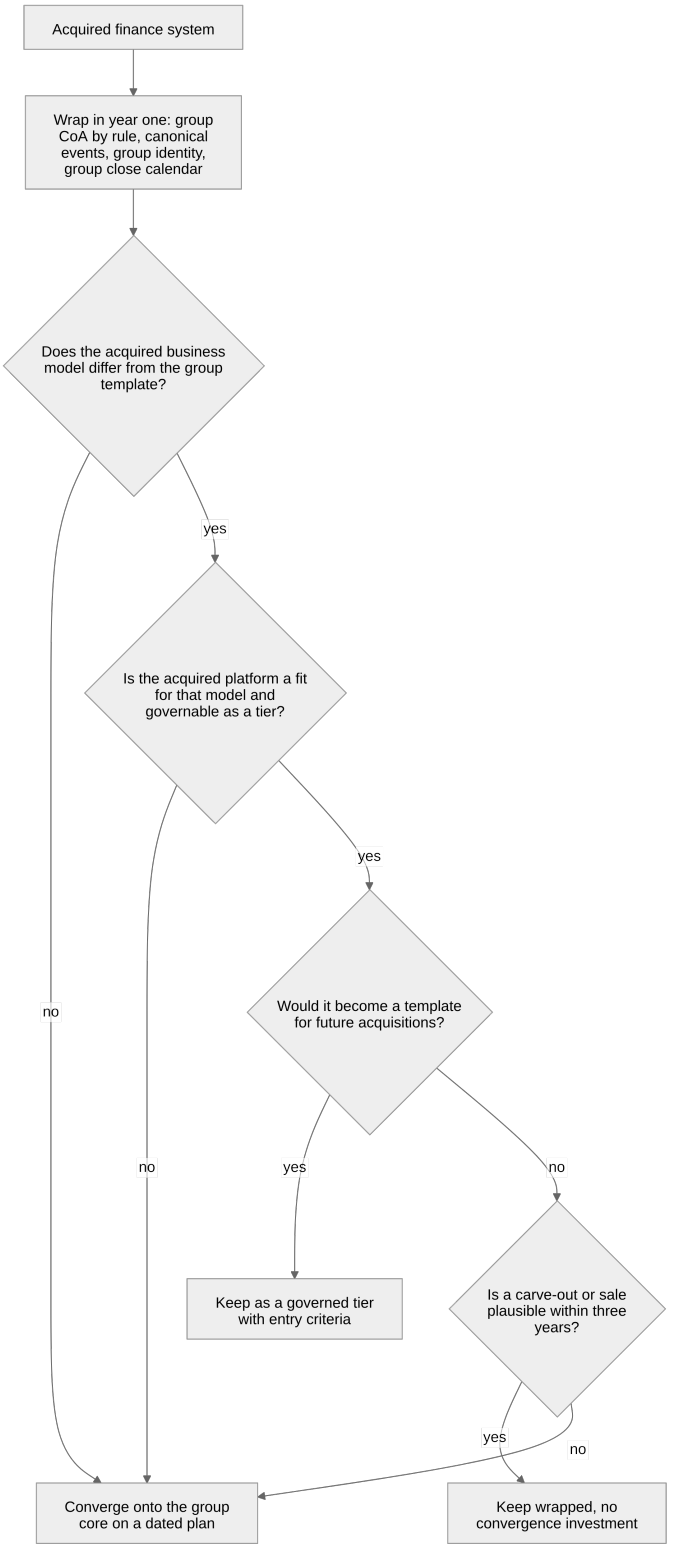

Acquired finance system
Wrap in year one: group CoA by rule, canonical events, group identity, group close calendar
Does the acquired business model differ from the group template?
yes
no
Is the acquired platform a fit for that model and governable as a tier?
yes
no
Would it become a template for future acquisitions?
yes
no
Keep as a governed tier with entry criteria
Is a carve-out or sale plausible within three years?
yes
no
Converge onto the group core on a dated plan
Keep wrapped, no convergence investment

Figure 5: *Post-acquisition decision rules. Every acquired finance system is wrapped first; it is kept only if its business model differs from the group's, the platform fits that model, and keeping it creates a reusable tier rather than a one-off exception.*

The rules behind the figure, which Northmere adopted in its decision-record template in 2027:

Rule	Reason
Wrap within twelve months, always	Consolidation from day one with the group chart applied by rule; no mapping spreadsheets; group access controls before the first audit cycle
Converge when the business model matches the group template	Two ledgers for one kind of business is duplication with a consolidation cost attached
Keep only as a tier, never as an exception	A kept system has entry criteria, a template, an interface and an owner; otherwise it is The Permanent Exception
Keep wrapped, without convergence investment, if a sale or carve-out is plausible	Converging a business the group may sell destroys value twice
Decide by the next major contract or release event	The renewal date or a forced upgrade is the cheapest moment to move; missing it costs a full term
Price the acquired system's control environment at due diligence	The acquired company's SOX readiness is often the largest hidden cost; it belongs in the deal model, not the integration budget

Applied to Northmere: **Tablekin** was never properly wrapped in 2023 (its chart is mapped in a spreadsheet, its workers copied nightly), and its business model genuinely differs from the group template, which is why it reaches the "keep as a tier" question at all. **Wayfare**, acquired in 2024 on Oracle Fusion, is closer to the group template; its future belongs to the fourth-quarter convergence decision and Oracle Fusion: An Accounting Hub for High-Volume Events. Coexistence patterns and convergence economics belong to Converge or Coexist: Living with More Than One ERP.

Integration With the Rest of the Estate

Sourcing decisions start from the **capability map** (Map What Finance Does Before What Runs It), whose level-2 capabilities are the unit of the three tests, and from the **portfolio**

fact base (Manage the Application Portfolio as an Investment), whose dispositions and six-component costs are the baseline for every option. Controls by Design, Not by Audit Season supplies the controls-burden evidence, and Files, Events and APIs: How Finance Systems Talk sets the interface knock-outs; the canonical financial events are what make a composed or wrapped estate reconcilable at all. Downstream, a sourcing decision becomes a migration (Changing the Engine in Flight: Modernization Patterns for Finance), a dependency to manage (Own Your Roadmap: Vendor Strategy Without Lock-In Panic) and a step in the roadmap (The Middle Matters: Transition States and the Roadmap). The decision record links in both directions, so that when a PoC condition fails or a vendor repackages, the dependent decisions can be found.

Failure Modes

1. **Vendor-led selection**. The vendor's discovery workshop writes the requirements, its solution engineers pick the scenarios, its references are the only references, and its commercial team adds modules at signature. Each step looks helpful; together they are **The Borrowed Roadmap**. The countermeasure is four artifacts the vendor never touches: the requirements, the scenario scripts, the PoC success criteria and the module list.
2. **Deciding per system instead of per capability**. A platform is kept because one capability in it differentiates, or replaced because one does not. SFP's 2012 scope is the example.
3. **Building on the volume test alone**. "The package cannot handle our volume" justifies a high-volume package or a summarizing edge, not a custom ledger.
4. **Counting license and engineering only**. Control and exit are missing, the build looks cheap, and the true cost arrives as an audit finding.
5. **Fit-to-standard in name only**. Every gap is approved because nobody priced them: Customization Concrete in a product sold as standard.
6. **A PoC on the vendor's data**. It proves the demonstration again.
7. **Exit left to the renewal**. By then the vendor knows how hard leaving is.
8. **The acquisition that was never sourced**. A tolerated ledger with no disposition, review date or owner hardens by inertia into The Permanent Exception.
9. **The scoring model as a verdict**. A 0.05-point lead is announced as a winner, a knock-out failure is averaged away, or the weights changed after the demonstrations.

The Chief Architect's View

What to decide. First, that the method is mandatory above a threshold: at Northmere, any finance-relevant decision above $1 million over the contract term, or one that

touches a significant account or Northmere Pay's regulated processing. Second, a default sourcing posture for each capability group on the map (bought by default, candidate to build, composed), so every decision starts from a stated position. Third, the post-acquisition rules, written before the next deal, because the deal timetable will not wait for them.

Who decides. The design authority, chaired by Dana with Leila, decides sourcing for capabilities that bind more than one brand; the capability owner, a Finance leader, is accountable for the requirements and for fit-to-standard decisions; Procurement owns the commercial process but not the scope; the Head of Internal Controls signs the controls-burden assessment; and for anything in Northmere Pay's regulated perimeter, Ingrid's risk function decides criticality under DORA. The vendor decides nothing, which sounds obvious and is the rule most often broken.

What to watch. The share of selections with a PoC on real volume and with exportable finance data specified in the contract; modules licensed but mapped to no capability owner; acquired systems without a disposition and review date; and, in every implementation in flight, a rising count of "extend" and "modify" gaps, the earliest warning of Customization Concrete.

The long view. Products change names, owners and packaging several times a decade; OneStream changed owner in 2026 and SAP renamed most of its cloud ERP portfolio in 2025. **The method should outlive all of them**. An enterprise that owns its capability map, hardest scenarios, cost model and exit terms can change vendors when it must; one that outsourced its thinking to a vendor's discovery workshop cannot.

Questions to Ask

1. At which level of the capability map is this decision being made, and have the three tests been applied to each capability separately?
2. What can we do with this capability that a competitor buying the best package tomorrow could not?
3. What volume, at what latency, across which business models, has a package been *proven* to carry, on our data?
4. Who will design, operate and evidence the controls, and is that named in the decision record?
5. What do build, run, change, control and exit cost over seven to ten years, and how wide is the range?
6. Which of our hardest scenarios did each vendor demonstrate, and which did they decline?
7. Has the data exit been tested by loading a complete period, with its audit trail, into something else?
8. Do the contract's data-portability terms specify finance data in finance terms, and does DORA apply to any entity using the service?

9. Which modules in the proposal are mapped to a capability owner, and which arrived with the discount?
10. For an acquired system: has it been wrapped, what is its disposition, and when is the next contract or release event that makes moving cheap?

From Choosing to Changing

A sourcing decision answers *what* should hold a capability. It does not answer *how* the enterprise gets there from what it runs today, without stopping the close. Tablekin's Option C is a tenant merge and a worktag redesign; Option B is an 18-month migration with two parallel closes; SFP's converging components move one by one into the group core while the platform keeps paying partners. Each is a different modernization pattern with its own risks at cut-over. Changing the Engine in Flight: Modernization Patterns for Finance catalogs those patterns, from rehosting and replacing to strangler-fig convergence, parallel runs and cut-over at period boundaries, and shows which fits which kind of sourcing decision.

Sources and Evidence

Vendor documentation

- Explore Phase Overview (SAP Activate), SAP Learning — fit-to-standard workshops in the Explore phase, delta requirements captured in a backlog.
- Gathering Data in Fit-to-Standard Workshops, SAP Learning — the guidance to pursue only critical requirements because changes to standard add upgrade maintenance.
- New SAP S/4HANA release and maintenance strategy, SAP, 2022 — seven years of mainstream maintenance per release, used for the decision horizon.

Standards and regulation

- Regulation (EU) 2023/2854 (Data Act), European Union, 2023 — definition of data processing service (Art. 2(8)), contractual switching terms (Art. 25), information obligations (Art. 26), switching charges and the 12 January 2027 date (Art. 29), custom-built exemption (Art. 31), application dates (Art. 50).
- Regulation (EU) 2022/2554 (DORA), European Union, 2022 — register of information (Art. 28(3)), pre-contract assessment (Art. 28(4)), exit strategies (Art. 28(8)), contractual transition period (Art. 30(3)(f)), application from 17 January 2025.

Practitioner and research sources

- Evolution of postmodern ERP to composable ERP explained, TechTarget — secondary account of Gartner's postmodern (2013–2014) and composable (around 2020) ERP framing.
- EU Data Act: Significant New Switching Requirements Due to Take Effect for Data Processing Services, Latham & Watkins, 2025 — practitioner reading of switching charges versus other fees, and the need to amend existing contracts.
- Birmingham City Council Oracle reimplementation coverage, The Register, 2026 — the 2022 go-live, customizations including bank reconciliation, program governance findings and the August 2026 re-implementation; used as a governance case.

Northmere Group and every figure attributed to it are fictional; the sourcing method, cost model, scoring model and post-acquisition rules are the author's architectural judgment, offered with the conditions under which they hold.